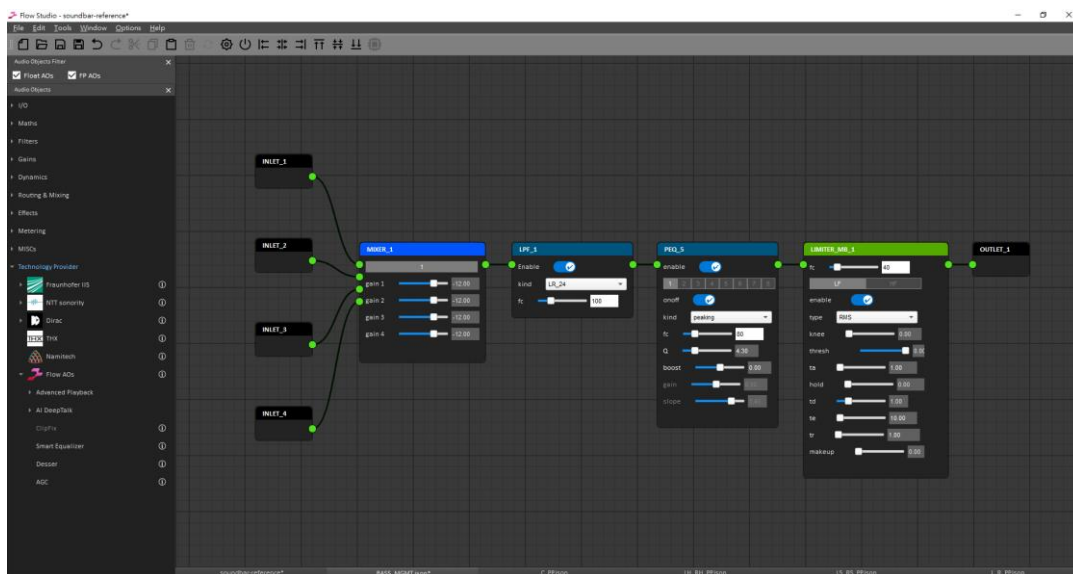




Flow DSP



Flow Studio User Guide

V1.29

Table of Contents

Table of Contents	2	objects or sub-patch	20
Revision History	5	4.4.5 Redo/undo	20
1 Overview	8	4.4.6 Encryption of sub-patch.....	20
2 Getting Started	9	5 Workflow	22
2.1 Setup	9	5.1 Local Target.....	22
2.2 Activation	10	5.1.1 Design Signal Flow on Design Canvas	22
2.3 Create your first signal flow	10	5.1.2 Setup Soundcard.....	22
2.4 Important Concepts	11	5.1.3 Connect to local target.....	23
2.4.1 What is Audio Object?.....	11	5.1.4 Tune Parameters	23
2.4.2 What is Flow Engine?	11	5.1.5 Save Project	23
2.4.3 Local Target and Remote Targets.....	11	5.2 Remote Target – TCP	24
2.4.4 Edit Mode	12	5.2.1 Design Signal Flow on Design Canvas	24
2.4.5 Tuning Mode.....	12	5.2.2 Setup Soundcard.....	24
3 Main Screen Tour	12	5.2.3 Connect to remote target	24
3.1 Main Screen Area.....	12	5.2.4 Tune Parameters	25
3.1.1 Menu toolbar.....	12	5.2.5 Save Project	25
3.1.2 Main toolbox	11	5.3 Remote Target - USB.....	25
3.1.3 Audio Object pane.....	11	5.3.1 Setup Soundcard.....	25
3.1.4 Audio Object property pane.....	11	5.3.2 Connect to remote target	25
3.1.5 Signal flow design canvas	12	5.4 Remote Target - UART	26
3.1.6 Status display bar.....	12	5.4.1 Setup Soundcard.....	26
4 Design Canvas.....	13	5.4.2 Connect to remote target	26
4.1 Audio Object	13	5.4.3 Upload Design to Flash (Flow EVK target only).....	27
4.1.1 Audio Object node.....	13	6 Audio Objects (AOs).....	29
4.1.2 Audio Object node with Control channel.....	13	6.1 I/O.....	30
4.1.3 Audio Object Context Menu.....	14	6.1.1 Input, Input (fixed-point)	31
4.1.4 Input / Output port Tooltip.....	15	6.1.2 Output, Output (fixed-point)	32
4.2 Canvas Basic Operation.....	15	6.1.3 Wav Player	33
4.2.1 Add/delete audio object.....	15	6.1.4 Sine Tone Generator	34
4.2.2 Copy/cut/paste audio objects	16	6.1.5 Noise Generator.....	35
4.2.3 Connection between the audio objects	17	6.1.6 Chirp Generator	36
4.2.4 Redo/undo.....	17	6.1.7 Audio Dump	37
4.3 Multi-Channel Audio Object Capabilities.....	17	6.1.8 Signal Visualizer	38
4.4 Sub-Patch feature.....	18	6.2 Maths	39
4.4.1 Add/delete sub-patch.....	18	6.2.1 Adder, Adder (fixed-point)	40
4.4.2 Enter sub-patch	19	6.2.2 Negator	41
4.4.3 Copy/cut/paste sub-patches	19	6.2.3 Abs, Abs (fixed-point).....	42
4.4.4 Connection between the sub-patch and audio		6.2.4 Sqrt, Sqrt (fixed-point).....	43
		6.2.5 Multiplier, Multiplier (fixed-point)	44
		6.2.6 Subtractor	45
		6.3 Filters.....	46

6.3.1	Biquad filter	47	6.8	Metering.....	112
6.3.2	Parametric EQ, Parametric EQ (fixed-point).....	49	6.8.1	Level Meter, Level Meter (fixed-point)	113
6.3.3	Parametric EQ V2.....	52	6.8.2	Spectrum Analyzer	114
6.3.4	Generic IIR filter.....	55	6.8.3	Real Time Analyzer.....	116
6.3.5	FIR filter, FIR filter (fixed-point)	56	6.9	Technology Provider	117
6.3.6	Low Pass Filter/High Pass filter, Low Pass Filter/High Pass filter (fixed-point)	58	6.9.1	Fraunhofer IIS.....	118
6.3.7	2-way crossover, 2-way crossover (fixed-point)	60	6.9.1.1	Cingo	119
6.3.8	Game EQ.....	62	6.9.1.2	upHear VQE.....	122
6.4	Gains	65	6.9.1.3	uphear Virtualizer	124
6.4.1	Gain, Gain (fixed-point)	66	6.9.2	NTT Sonority	125
6.4.2	Smart Gain	67	6.9.2.1	Intelligent Microphone Voice	126
6.4.3	Mute, Mute (fixed-point).....	68	6.9.2.2	Noise Level AGC	128
6.4.4	Attenuator	69	6.9.3	Dirac	130
6.4.5	Loudness Volume	70	6.9.3.1	Dirac Live Processor	131
6.4.6	Polarity.....	72	6.9.4	Aizip.....	134
6.4.7	Quick Gain	73	6.9.4.1	Vocal Removal.....	135
6.5	Dynamics.....	75	6.9.4.2	Dialogue Enhancement.....	136
6.5.1	Limiter, Limiter (fixed-point).....	76	6.9.5	Flow DSP	137
6.5.2	2 Band Limiter	78	6.9.5.1	Advanced Playback	138
6.5.3	3 section Compressor, 3 section Compressor (fixed-point) 80		6.9.5.1.1	Dynamic Filter	139
6.5.4	Compressor, Compressor (fixed-point).....	83	6.9.5.1.2	Dynamic Bass	141
6.5.5	Smart Compressor	86	6.9.5.1.3	Virtual Bass	143
6.5.6	Clipper	89	6.9.5.1.4	Dynamic Loudness	145
6.5.7	Gate	90	6.9.5.1.5	Dynamic EQ.....	147
6.5.8	Smart Gate.....	92	6.9.5.1.6	Spatializer.....	149
6.6	Routings & Mixing.....	94	6.9.5.1.7	Crossfeed	150
6.6.1	MERGER.....	95	6.9.5.1.8	CTC	152
6.6.2	Multiplexer, Multiplexer (fixed-point)	96	6.9.5.1.9	AFS	154
6.6.3	Dry wet fader	97	6.9.5.2	AI DeepTalk	155
6.6.4	Mixer, Mixer (fixed-point).....	98	6.9.5.2.1	AI Noise Reduction (16k)	156
6.7	Effects.....	99	6.9.5.2.2	AI Noise Reduction (48k)	157
6.7.1	Delay, Delay (fixed-point)	100	6.9.5.2.3	Voice Enhancement Package (VEP)	158
6.7.2	Fractional Delay	101	6.9.5.2.4	DSP Noise Reduction	160
6.7.3	Multitap Delay	102	6.9.5.2.5	2-4 Mic Beamforming	161
6.7.4	Low Pass Comb Filter	103	6.9.5.2.6	Coherence Beamforming.....	163
6.7.5	Long All Pass Filter	104	6.9.5.2.7	AI BEAMFORMING (16k).....	164
6.7.6	Reverb V2	105	6.9.5.2.8	AEC.....	165
6.7.7	Chorus.....	109	6.9.5.2.9	AI VAD	166
6.7.8	IR Convolver.....	111	6.9.5.2.10	VAD 167	
			6.9.5.3	De-esser	168
			6.9.5.4	Smart Equalizer	169

6.9.5.5	CLIPFIX	171
6.9.5.6	AGC	173
6.9.5.7	Sample Rate Converter (SRC)	174

Revision History

Date	Revision	Note	Author
2020.12.7	V1.0	- Initial document	Ned Yang
2021.9.27	V1.1	- Update Algorithm guide - Update screen shot of the newer UI and activation process	Ned Yang
2022.4.30	V1.2	- Update Algorithm guide - Update menu tool bar - Add Sub-Patch guide - Add Inter-AO capabilities	Peter Chen Jason Zhong
2022.4.30	V1.3	- Update Algorithm guide - Add audio processing objects: Proprietary - Add VBASS, AGC capabilities	Clark Chen
2022.6.30	V1.4	- Update Algorithm guide - Add DBASS, AUTO_COMP - Remove LMS - Add UART / USB interface - Add Heap Monitor	Clark Chen Tofu Tsai
2022.7.22	V1.5	- Add DLOUDNESS	Tofu Tsai
2022.8.17	V1.6	- Add Platform specific AO handling in rule check - Enable Inter-AO - Add VAD - Add Smart Gain - Add Smart Gate	Tofu Tsai Peter Chen Jason Zhong
2022.9.30	V1.7	- Add CROSSFEED	Ned Yang
2023.1.16	V1.8	- Update Flow Studio registration process - Add BEAMFORMING - Add AI_NR - Add AEC - Add NOISEREDUCTION - Add SUBTRACTOR - Add VAD_IABSE - Add AIROHA Tuning interface (UART)	Clark Chen Tofu Tsai
2023.3.27	V1.9	- Add SPECIALIZER - Add Flow APO connection - Add AIROHA Tuning interface (USB)	Ned Yang Tofu Tsai Manson Chen
2023.4.18	V1.10	- Add Fixed Point AO – AB1565 - Refactor connect settings	Ned Yang Tofu Tsai
2023.7.12	V1.11	- Add Audio Dump - Add Signal Visualizer	Tofu Tsai Jason Zhong

		-Add strength param for Noise Reduction -Audio UI adjustment: MUX, GAIN -Add template function in menu.	
2023.8.14	V1.12	-Add Dynamic Filter	Ned Yang
2023.10.13	V1.13	- Add SMART EQ - Add Fraunhofer Cingo - Add CLIPFIX - Add NTT Sonority AEC	Tofu Tsai
2023.11.24	V1.14	- Add uphear Virtualizer - Add uphear VQE	Tofu Tsai Scott Song
2024.01.03	V1.15	- Adjust uphear Virtualizer - Adjust CLIPFIX	Tofu Tsai
2024.03.28	V1.16	- Re-arrange AO guide contents - Correct some error information - Add AI Noise Reduction (48k) - Add AI Beamforming - Add Voice Enhancement Package	Tofu Tsai Ned Yang
2024.5.24	V1.17	- Update AGC description	Ned Yang Rowan Williams
2024.6.24	V1.18	- Add PEQ V2 - Add NTTS AGC - Update AGC with new parameter	Clark Chen
2024.7.12	V1.19	- Adjust AGC loThres parameter minimum value	Clark Chen
2024.8.1	V1.20	- Add Dirac Live Processor description	Scott Song
2024.10.11	V1.21	- Add Upload to Flash feature for Flow EVK	Tofu Tsai
2025.1.3	V1.22	- Add CTC - Add CohBF	Clark Chen Tofu Tsai
2025.3.21	V1.23	- Optimize AO descriptions and format	Vincent Luo Ned Yang Tofu Tsai Clark Chen Oran Huang Jeanie Chiou Kieran Chang
2025.4.14	V1.24	- Add AFS - Add AIZip Human Vocal Removal	Vincent Luo Oran Huang Clark Chen
2025.6.6	V1.25	- Update Beamforming UI	Tofu Tsai Hazel Lin Jeanie Chiou

2025.6.22	V.126	- Add Dynamic EQ	Jeanie Chiou Tofu Tsai
2025.7.31	V1.27	- Add SRC (Sample rate converter)	Vincent Luo Tofu Tsai
2025.9.4	V1.28	- Add Quick Gain - Add Dialog Enhancement - Update CINGO	Tofu Tsai Vincent Luo Jason Zhong Clark Chen Jeanie Chiou Oran Huang
2025.11.13	V1.29	- Update CINGO - Add Gain EQ	Clark Chen Tofu Tsai

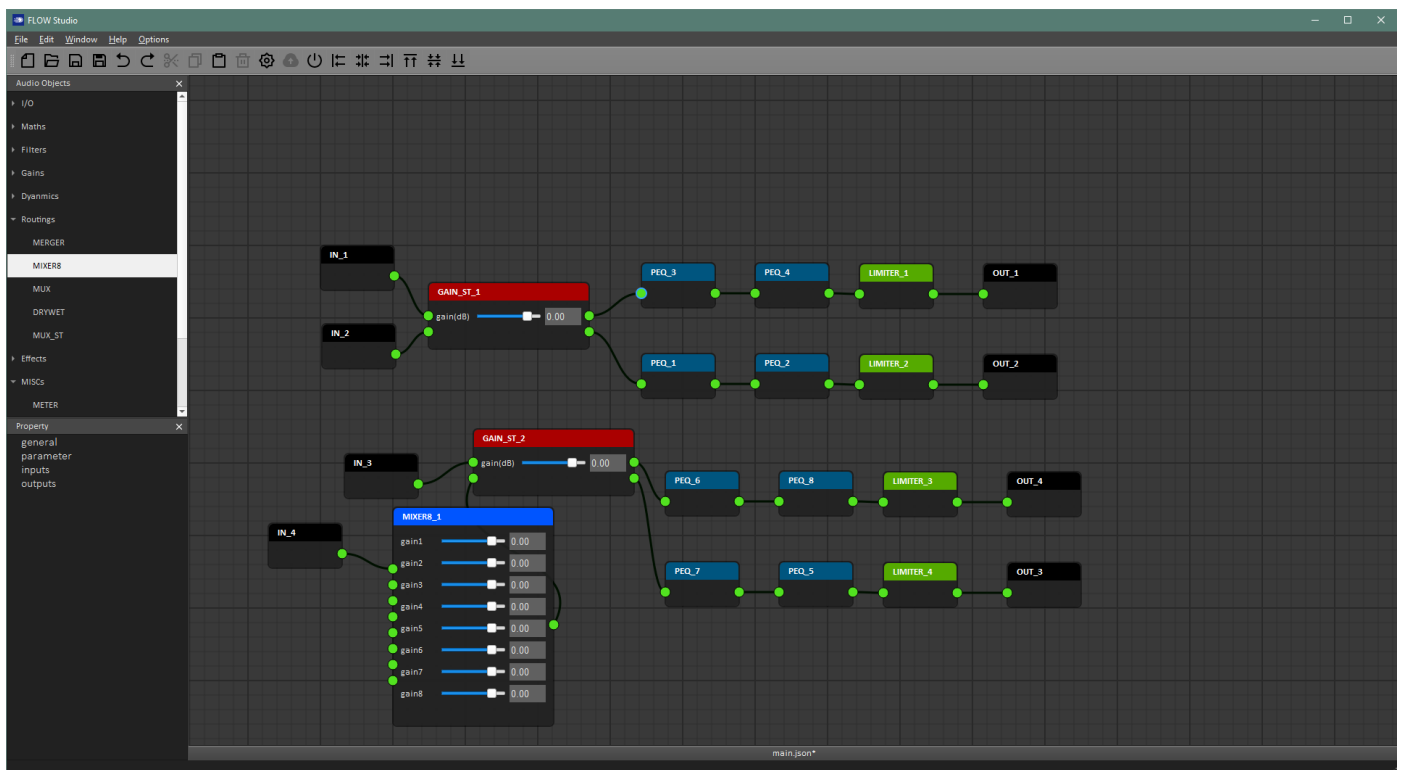
1 Overview

Flow Studio is a powerful and intuitive graphical user interface designed specifically for the **Flow Engine**, an advanced embedded audio processing system. It offers a comprehensive solution for designing, developing, and integrating high-performance audio processing workflows with exceptional efficiency. From basic filters and dynamic equalizers to sophisticated AI-driven voice enhancement algorithms, Flow Studio simplifies the entire process, enabling users to prototype, test, and refine cutting-edge audio solutions seamlessly.

The platform's drag-and-drop interface eliminates the need for complex programming, allowing users to construct signal flows visually rather than writing intricate code. Developers can leverage Audio Objects (AOs)—modular building blocks that can be arranged in a flowchart-style layout—to create structured, logical representations of audio algorithms. This intuitive approach makes audio processing design accessible to both seasoned professionals and newcomers to the field.

This user guide provides detailed explanations of the various Audio Objects available in Flow Studio. Each AO serves a specific purpose in signal processing, from noise suppression and automatic gain control to adaptive equalization and beyond. By mastering these components, users can design highly efficient audio workflows that enhance project quality while significantly reducing development time. Flow Studio creates a powerful, flexible, and user-friendly environment that fundamentally transforms how audio processing systems are designed and implemented.

The remained sections of this guide are as follows. In Section 2, basic instructions will be covered. Section 3 provides a Main Screen Tour, while Section 4 explores the Design Canvas in detail. Section 5 covers Workflow processes and best practices. The guide concludes with a comprehensive examination of Audio Processing Objects.



2 Getting Started

Most PCs will be able to run Flow Studio. Follow the instructions below to install Flow Studio and operation tutorial step by step, we can start building audio algorithm with a faster and smarter solution.

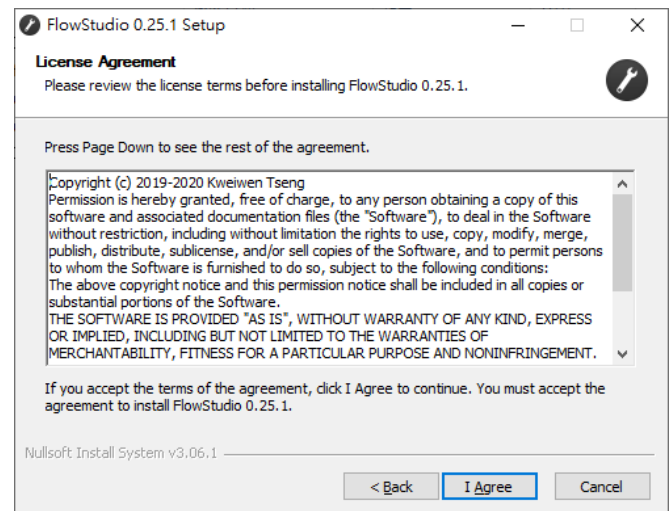
2.1 Setup

Quit any applications you are running. This is not strictly required for Flow Studio but is generally good practice while installing new software.

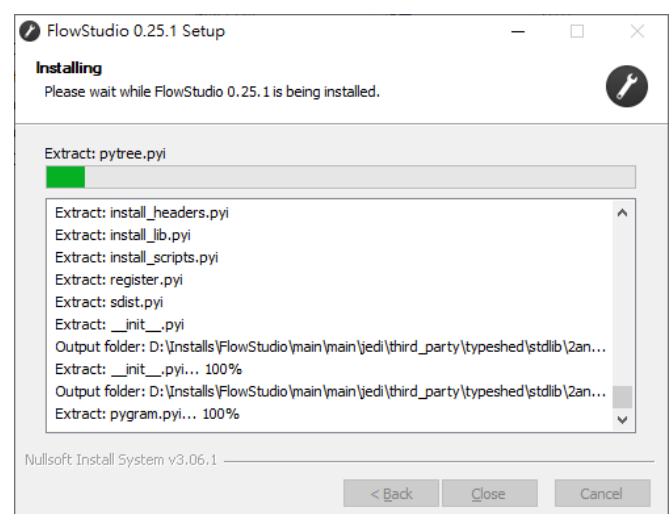
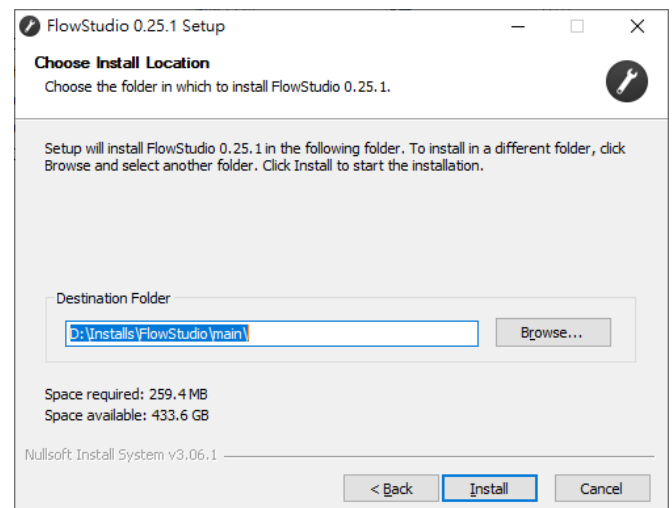
Once Flow Studio installer executable has been downloaded, double click on it and click 'Accept' when Windows asks if you want to allow the program to make changes to your computer. The first installation screen will look like this:



Once all other applications are closed, click 'Next' to continue to the license agreement step.



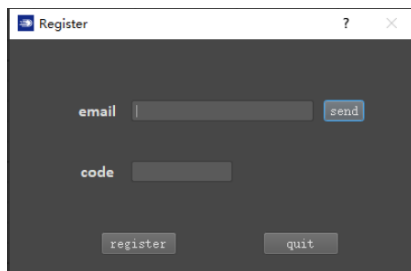
Read over the terms in the FLOW Studio license agreement. When you have read it and are satisfied with its contents, click the **I accept the agreement** button, and click **Next**.



Wait for the installation process to finish. The installer will place a shortcut on your desktop.

2.2 Activation

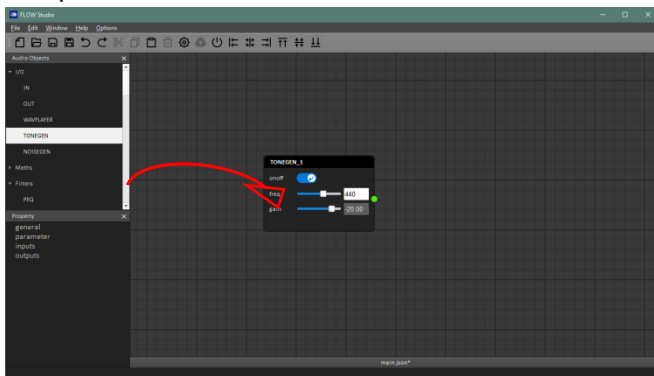
It will pop up an activation window when executing Flow Studio at the first time. Enter your email then press **send** you will receive a set of 4-digit activation code via your email for 30 days trial. After 30 days period, if you need to extend the usage, please contact Tymphany.



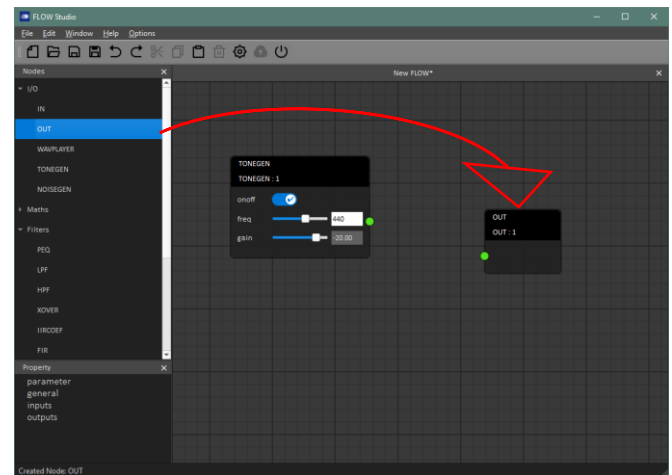
2.3 Create your first signal flow

The following steps will allow user to create a simple signal processing flow using your own PC's default soundcard output:

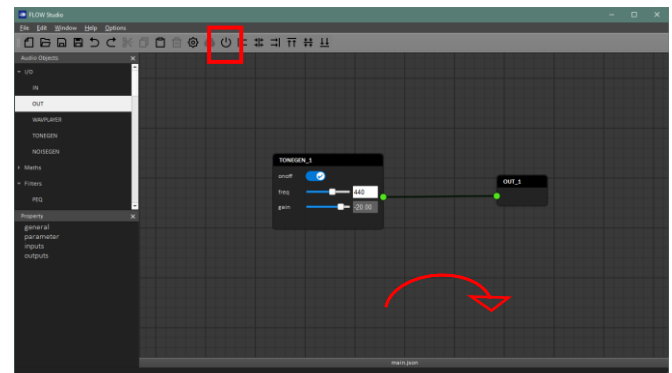
1. Drag a TONEGEN audio object from left audio object pane to canvas.



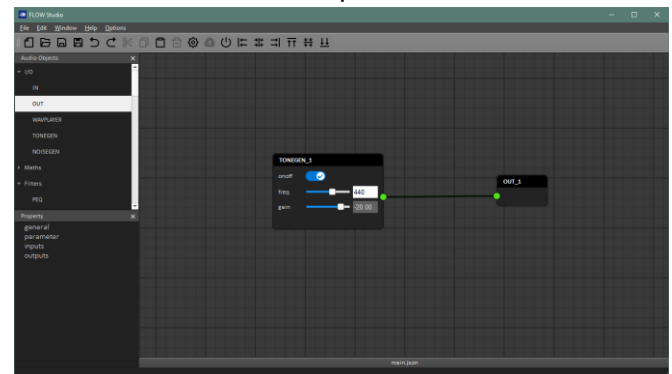
2. Drag an OUT audio object from left audio object pane to canvas.



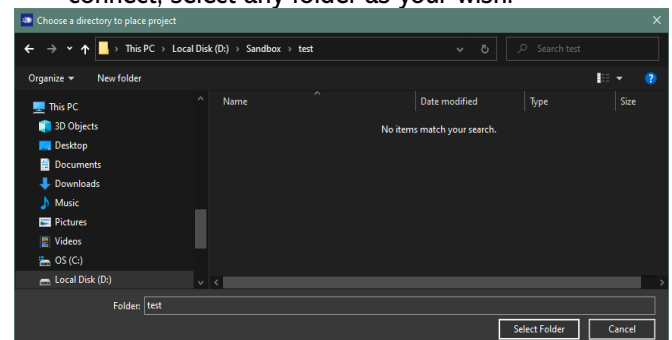
3. Connect TONEGEN and OUT by left click the output port of the TONEGEN then drag the wire to the input port of OUT. Then



4. Click **Connect** on the top toolbox

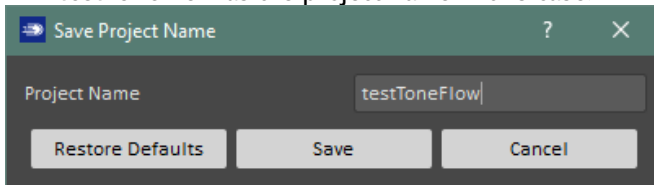


5. Flow Studio will prompt you to select the current design folder as workspace folder after you click connect, select any folder as your wish.

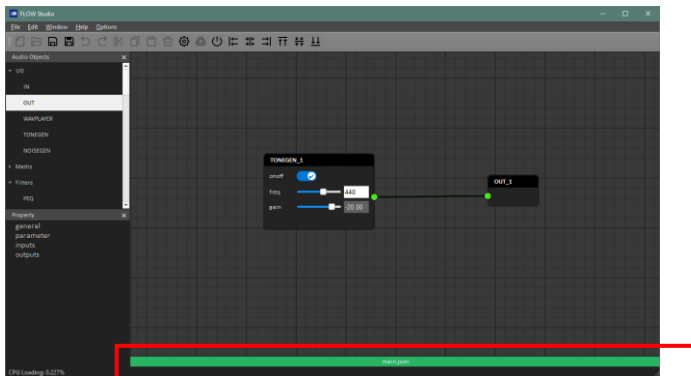


6. After you select workspace folder, Flow Studio will

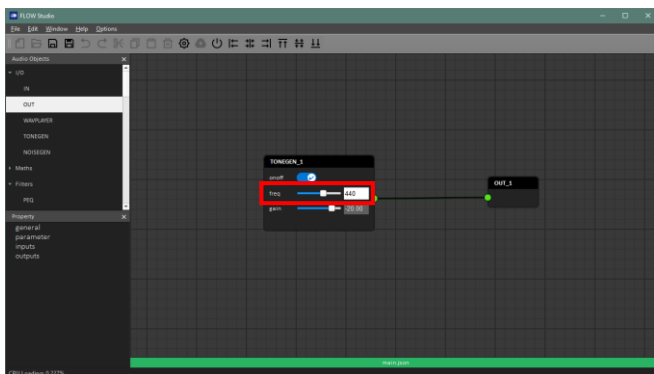
prompt you to save as a project name, we use **testToneFlow** as the project name in this case:



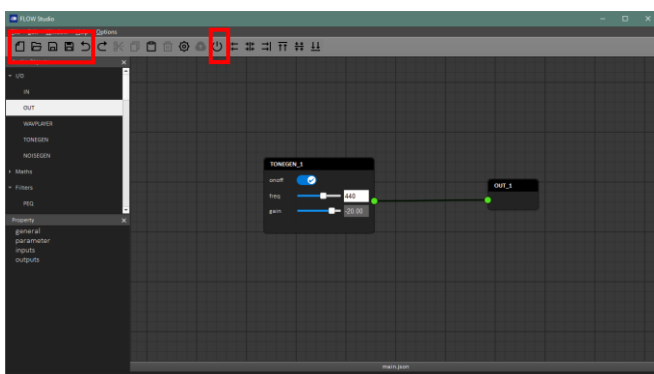
- Once the signal flow has been created, the tab of the canvas will become green. It means that the current signal flow has been started to process and ready for tuning.



- The 440Hz sine tone will come out to the first channel of your soundcard.
- Adjust the parameter on the TONEGEN to hear the change of the sine tone. E.g. adjust the freq from 440Hz to 1kHz



- Save the design by clicking the save or save as on the top toolbox or **File->save/save as** once you are done.



- Click the **connect** again to stop the audio processing.

2.4 Important Concepts

2.4.1 What is Audio Object?

The audio processing module that constructs the signal flow.

2.4.2 What is Flow Engine?

Flow Engine is a runtime library or application that reside on the target, it will do the real audio processing on target. You need Flow Studio to provide a configuration to Flow Engine.

2.4.3 Local Target and Remote Targets

The **target** denotes the audio devices that Flow Studio interact with. You design a signal flow in Flow Studio and push the signal flow information to target, then the run time audio processing engine (Flow Engine) on target will parse the signal flow information (configuration) then create the audio processing on target.

Typically, local target is your PC + soundcard (Fig. 1), your PC's CPU would be the audio processor. Remote target may be your audio device that needs audio signal processing capabilities (Fig. 2) (e.g. Wi-Fi speakers, soundbars, etc...), typically those audio devices will process audio using on board SoC, MCU or DSP.

The benefit of design a signal flow on local target is you can verify the final audio result before you apply the design to real audio device.

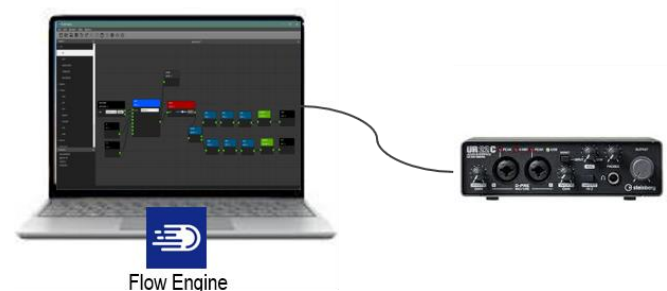


Fig. 1 Local target

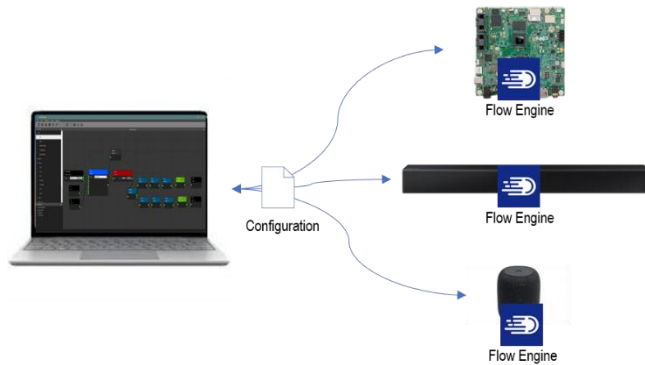


Fig. 2 Remote target

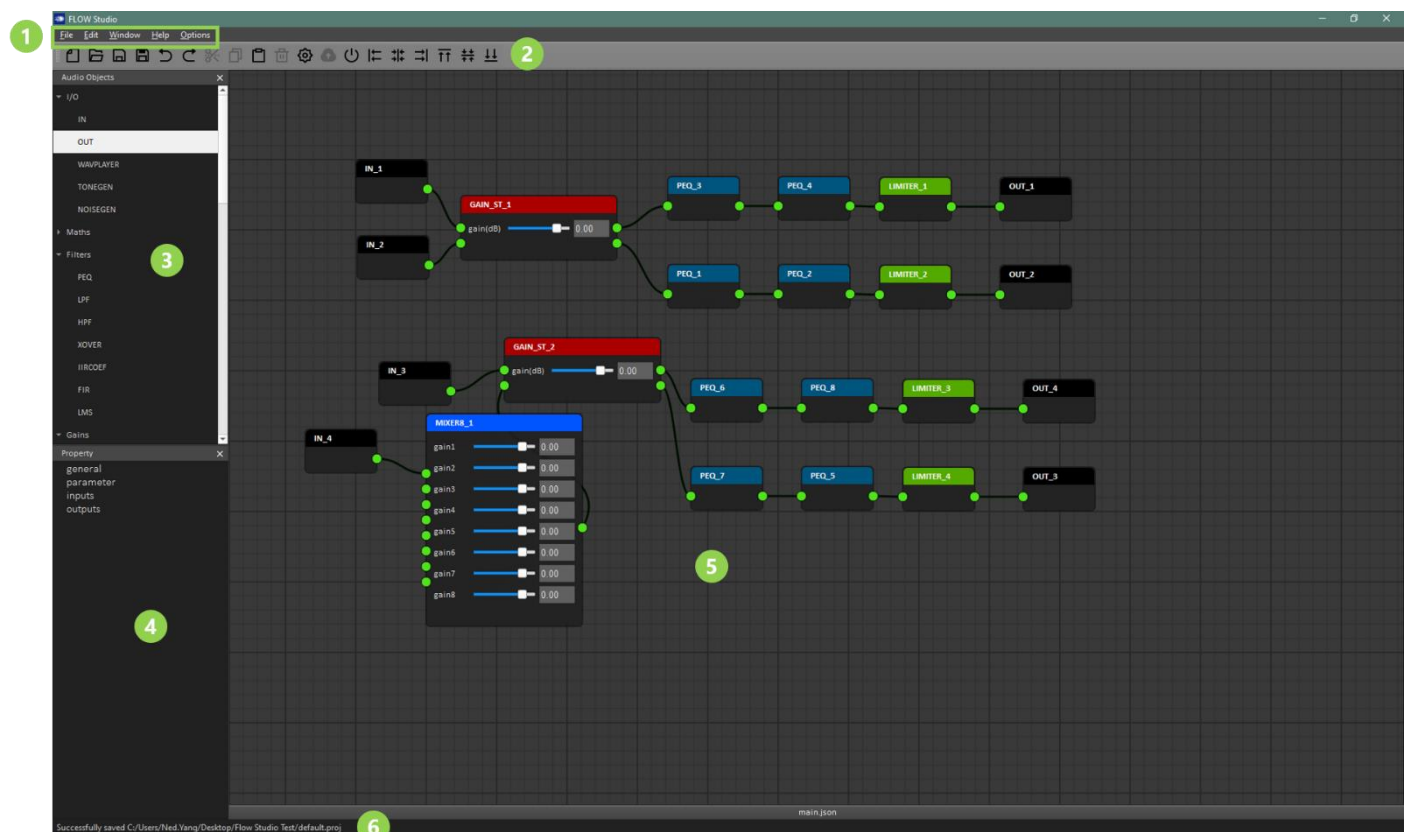
2.4.4 Edit Mode

This is the mode when you place any audio object into canvas, connect them together, set the default parameters on each audio object.

2.4.5 Tuning Mode

This is the mode after you connect to the local or remote target. In tuning mode, you can tune all of the parameters in real time. But you can't change the signal flow before you disconnect from the target. The reason FLOW Studio not allowed this is to make sure anytime when connected to the target, the design between the target and FLOW Studio is synchronized.

3 Main Screen Tour



3.1 Main Screen Area

1. Menu toolbar
2. Main toolbox for quick access of the functionalities
3. Audio Objects pane

4. Audio Object Property pane
5. Design canvas
6. Status display bar

3.1.1 Menu toolbar

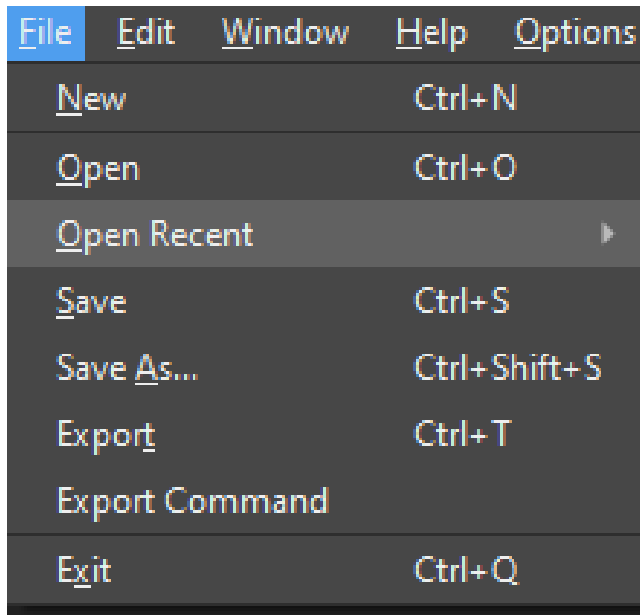
File Edit Window Help Options

- File

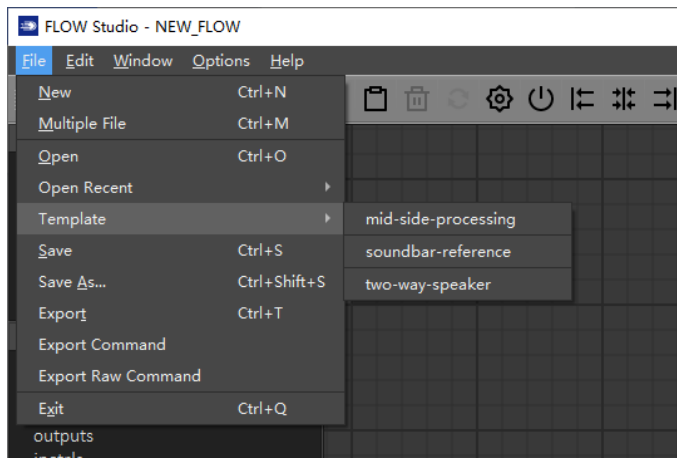
New – Creates an empty new design, unsaved old design will be prompted to save to somewhere in your disk.

Open – Open an existing design, unsaved old design will be prompted to save to somewhere in your disk.

Open Recent– Open a recent design, unsaved old design will be prompted to save to somewhere in your disk.



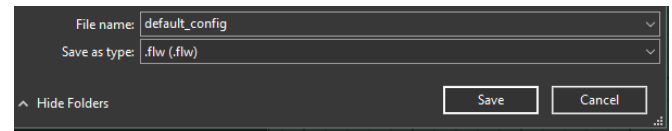
Template – Flow Studio provides design templates for users to reference, accelerating the design and development of signal paths. Additional templates will be continuously added as well.



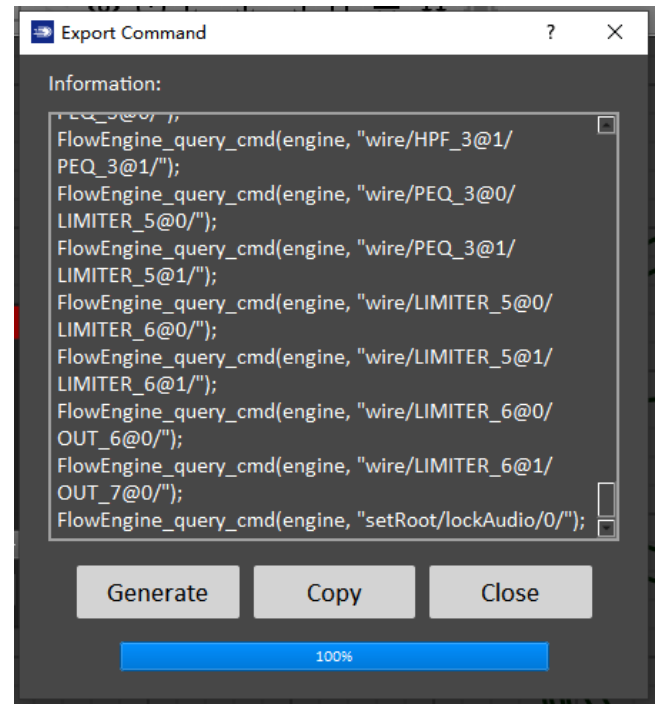
Save: Save the design to previous saved location.

Save As: Save the design to new location.

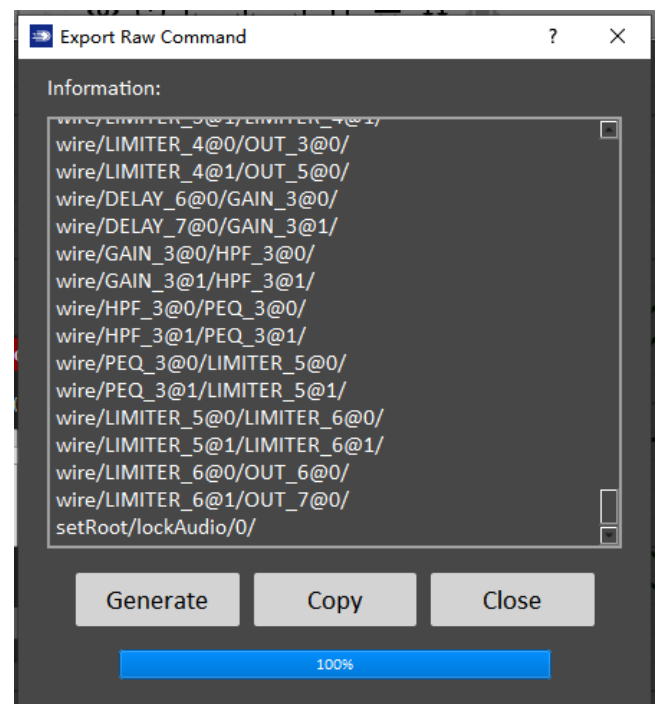
Export: Export Flow Engine configuration file (.flw) to disk.



Export Command – Export Message Command for programmer



Export Raw Command – Export Message Raw Command for programmer

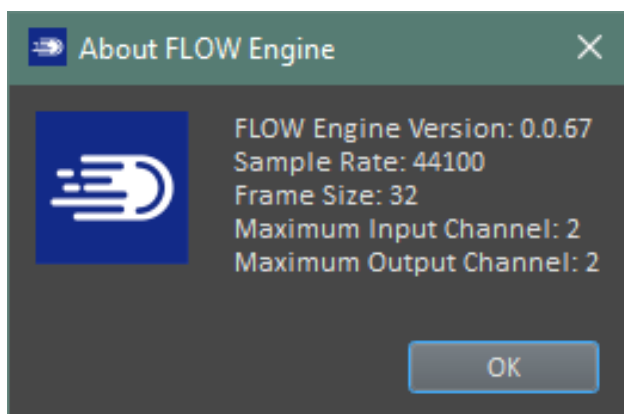


Exit – Exits the application

- **Edit**
Provides basic operation on audio object nodes. Support operations – undo/redo/copy/cut/paste/select all
- **Windows**
Provides basic operation to turn off/on each pane.
- **Help**

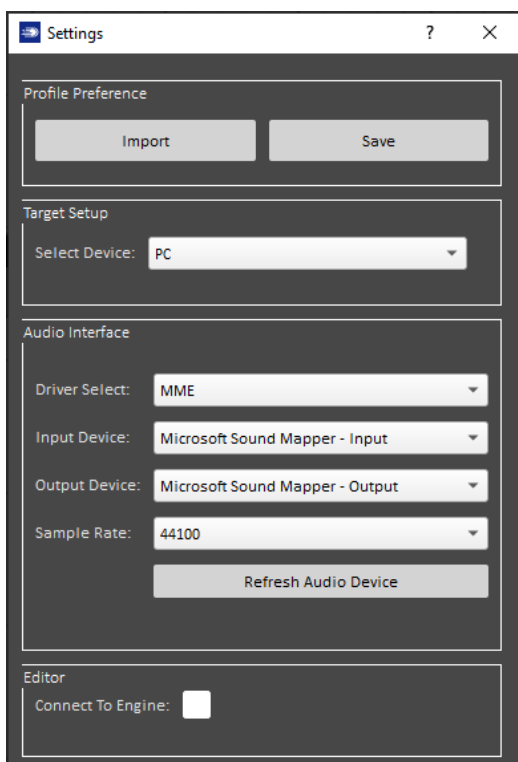
About– Basic information of current FLOW Studio

Info – Display the FLOW Engine information, this will valid when FLOW Studio has been connected to FLOW Engine.



- **Options**

Connect Setting – Open connect setting dialog box (see Workflow section for more details)



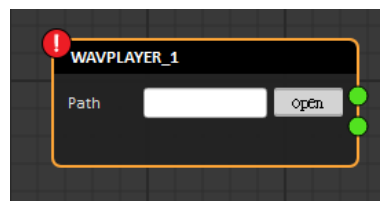
Edit Mode – Lock the current design

Rule Check – Display connection check between audio objects nodes, if the design has issues, the nodes will display as follows:



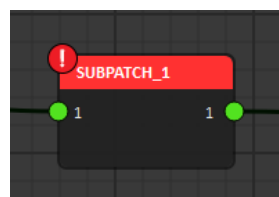
PEQ_5 has output connected but without input connection, which is invalid design.

If audio objects which are not supported by current selected target device exist, the nodes will display as follows:

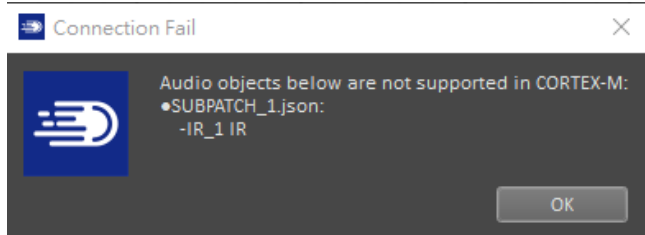


For example, Wav Player object is not supported by CORTEX-M, which is invalid design.

Here is another example, if subpatch object contains audio objects which are not supported by current selected target device, the subpatch will also display as follows:



And once clicking connecting, if audio objects which are not supported by current selected target device exist, popup window will show up and tell the location of unsupported audio objects, like below.(audio object “IR” in SUBPATCH_1.json is not supported in CORTEX-M)



Scan Audio Preference – Refresh connected soundcard list (only valid in PC target)

Upload Design – Upload current design to target (only valid on remote target)

Download Design – Download design from target (only valid on remote target)

3.1.2 Main toolbox



For quick access to any often use functions.

New (Edit Mode only)

Open (Edit Mode only)

Save

Save As

Undo (Edit Mode only)

Redo (Edit Mode only)

Cut (Edit Mode only)

Copy (Edit Mode only)

Paste (Edit Mode only)

Delete (Edit Mode only)

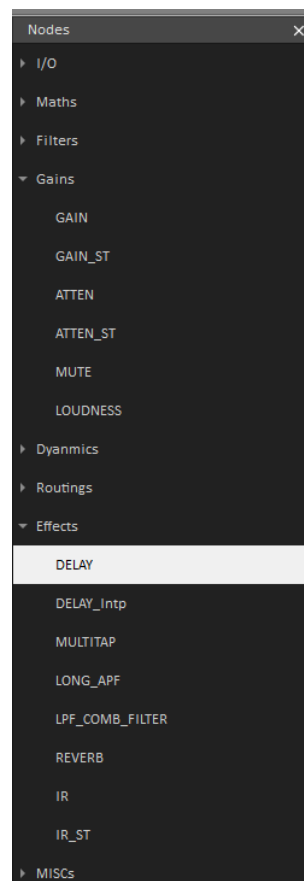
Dump (Edit Mode only)

Connect Settings

Connect

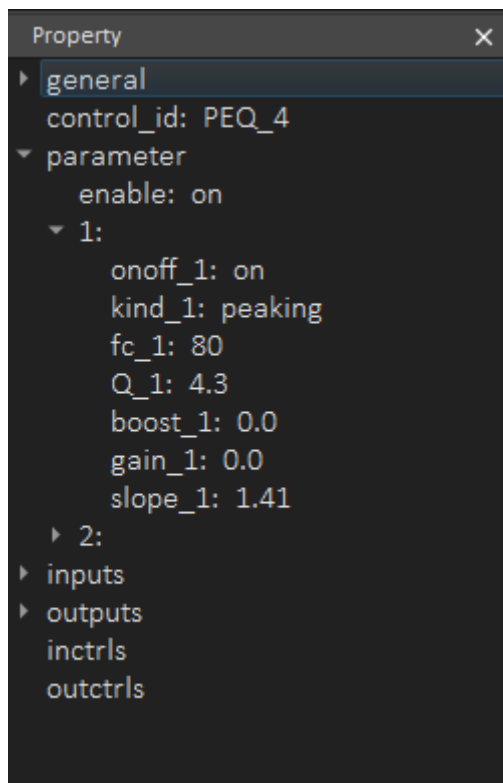
3.1.3 Audio Object pane

The list box of the collection of all available audio objects, user could drag the selected audio object then drop to the canvas.



3.1.4 Audio Object property pane

Displays detail information when select one of the audio objects on canvas.



2. Current memory usage of the target
3. Memory pool allocated in the target
4. Other message describing an action is completed, such as
 - A. “Successfully saved as D:/demo.json”
 - B. “FLOW Engine Connected”
 - C. “Disconnect from FLOW Engine”
 - D. “Export Configuration File”

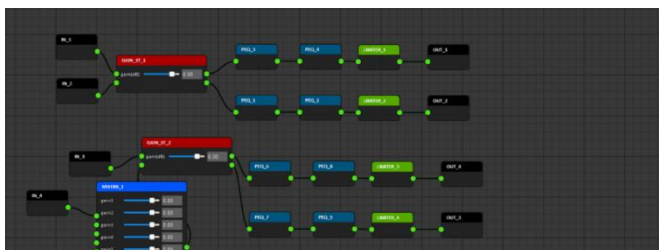
Successfully saved as D:/demo.json

While connecting to target. It shows the CPU loading and Memory usage in target. For example, the figure below shows that the CPU loading is 11.250%, Memory usage is 17.265 KB among the memory pool allocated in the target with total 447.992 KB.

CPU Loading: 11.250% Memory usage: 17.265 KB / 447.992 KB

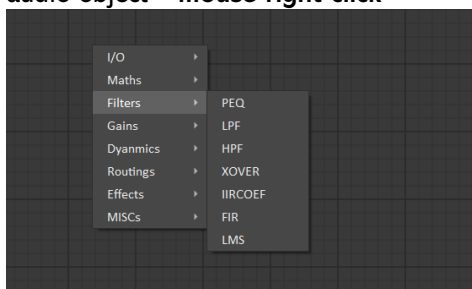
3.1.5 Signal flow design canvas

The main area that arranges the signal flow.



It comes with several useful mouse actions to help user design signal flow smoothly:

- Zoom in/out – **mouse scroll**
- Drag view area – **mouse middle click + left click** or **shift + left click**
- Add audio object – **mouse right click**



3.1.6 Status display bar

Some useful information will be displayed at the Status display bar, such as:

1. CPU loading of the target

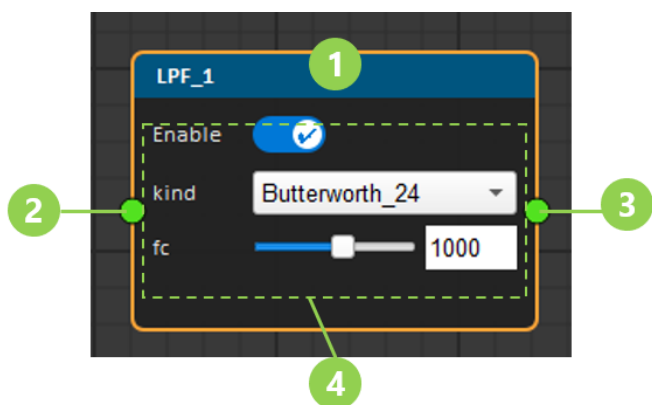
4 Design Canvas

4.1 Audio Object

The audio object is the smallest unit to construct a signal flow, here describes the functionality of audio objects, we take LPF audio object as an example:

4.1.1 Audio Object node

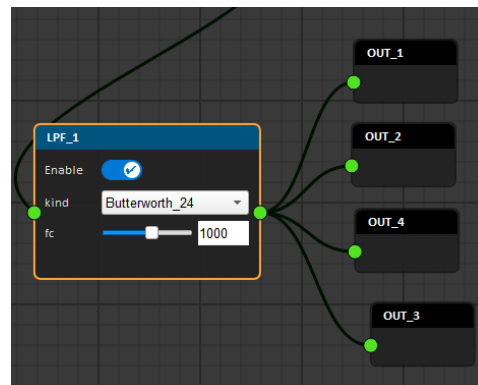
We call each added audio object in canvas “node”, every different node has different specifications. e.g., different parameters, different input output ports.



1. **Information Tab** -- The Information Tab provides essential details about each Audio Object (AO) in Flow Studio. Every AO follows a standardized naming convention: A_B, where A represents the AO's nickname or abbreviation, and B denotes its instance number. In this example, LPF_1 means the first Low Pass Filter object. This structured naming ensures clarity and consistency when managing multiple instances of the same AO, making it easier for users to track and configure their audio processing workflow.
2. **Input Port:** The **input ports** of an audio object receive signals from the **output ports** of other audio objects within the signal flow. These ports serve as entry points, allowing the AO to process incoming audio data before passing it

through its internal processing algorithms.

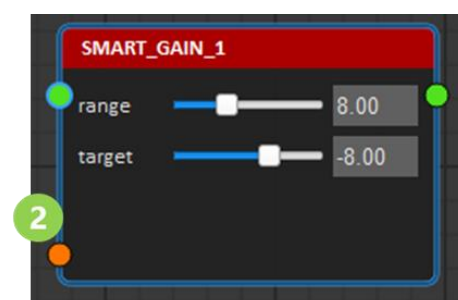
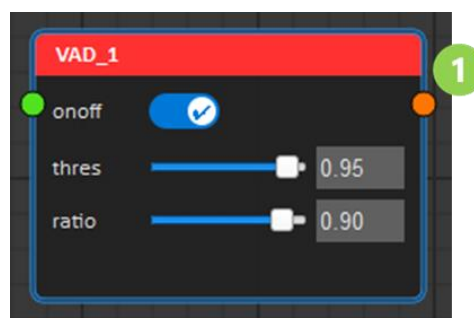
3. **Output Port:** The output ports of an audio object deliver the processed audio signal to the next stage in the signal chain. If the signal flow requires multiple destinations, the output port can **fan out**, sending identical copies of the signal to multiple nodes, ensuring flexibility in routing and signal distribution.



4. **Live Parameter Control** – you can set the parameters of the audio object directly on node or open up detailed UI (see next section) to control the parameters.

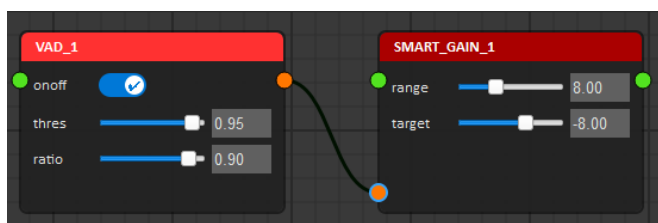
4.1.2 Audio Object node with Control channel

Control channel is color in orange, which provide non audio data transmission.



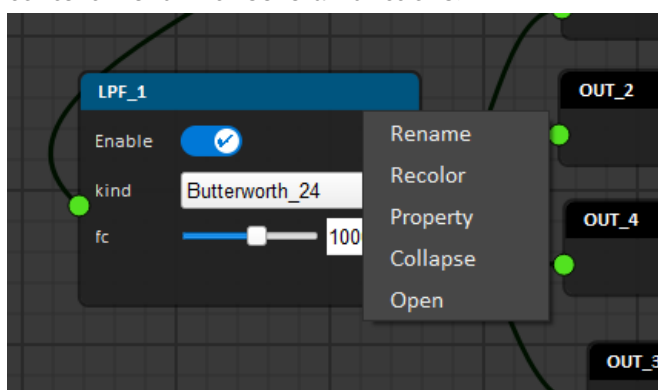
1. **Control output port** - could **fan out** multiple signals if signal flow needs to have same copy of current Port's signal to next node.
2. **Control Input Port** - only ONE channel signal could be connected into.

Notes: Control output port can only be connected to control input port, and vice versa

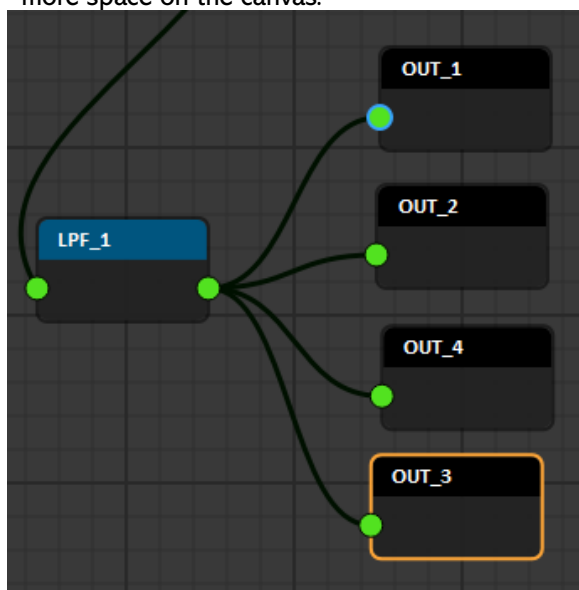


4.1.3 Audio Object Context Menu

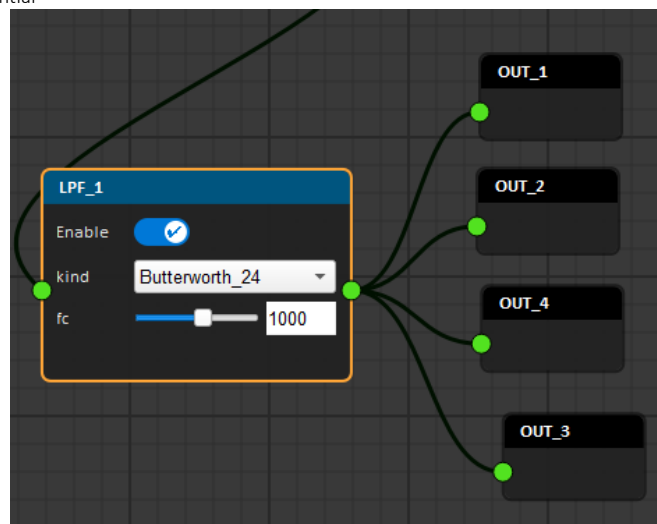
Right click on the selected node will show up a context menu with several functions:



- **Expand/Collapse**– You can expand to show up the live parameter controls or collapse the node to have more space on the canvas.

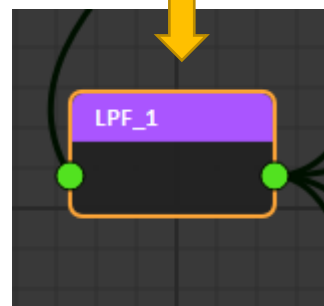
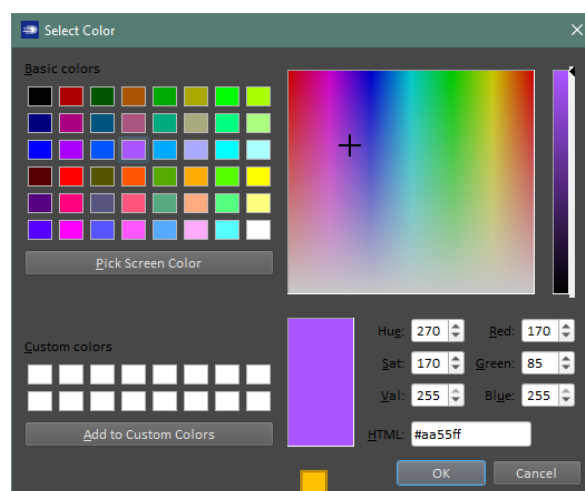


Collapsed mode

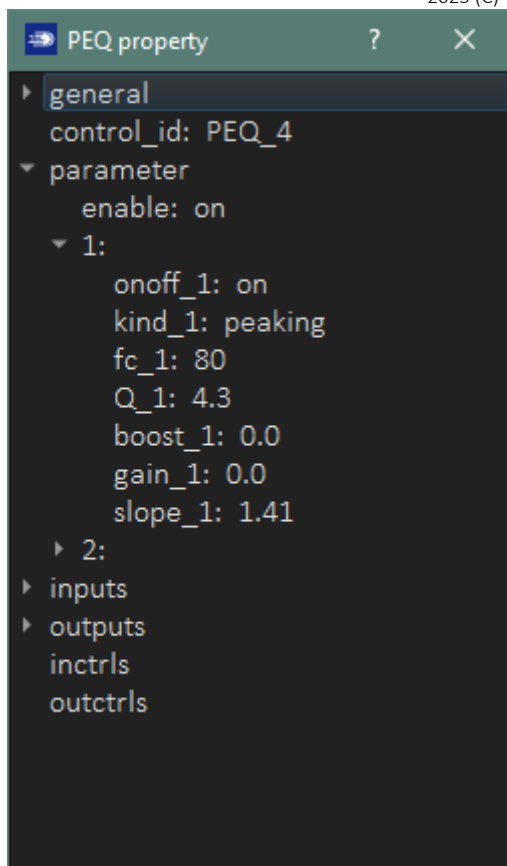


Expanded mode

- **Recolor** – Open up color palette to change the Information Tab color



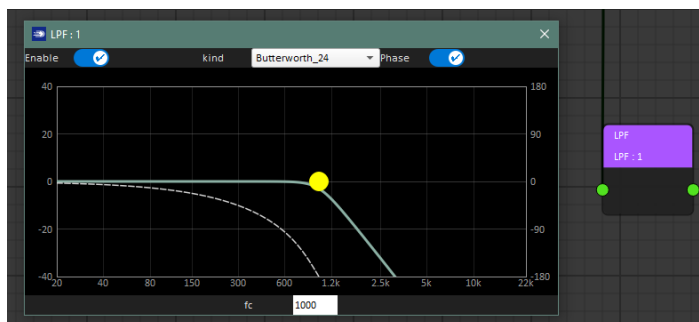
- **Property** – Open up detailed property window to display its properties.



- **Open** – Opened up separate detailed UI window for tuning, note that the node will automatically collapse.

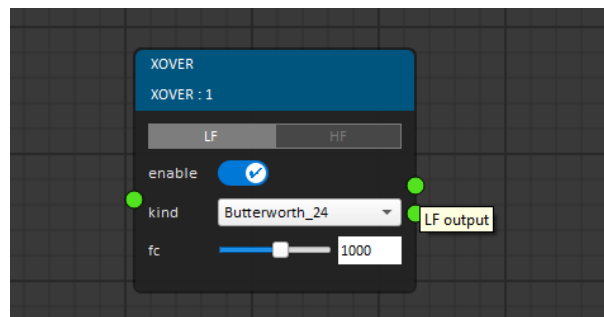
◆Tips:

To quick switch between expand/collapse/open mode on node, use mouse double click on node.



4.1.4 Input / Output port Tooltip

Some input or output port has its function, mouse hover on the input or output port will show up the port name. e.g XOVER:



4.2 Canvas Basic Operation

Here describes the operation on the Design Canvas for signal flow:

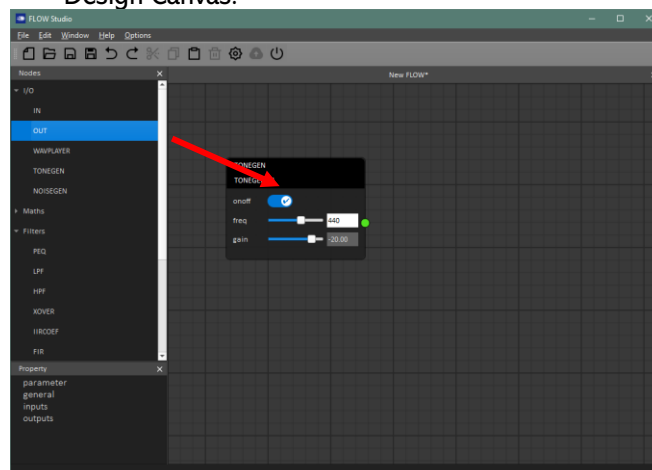
◆Note:

The follow operation only valid in Edit Mode.

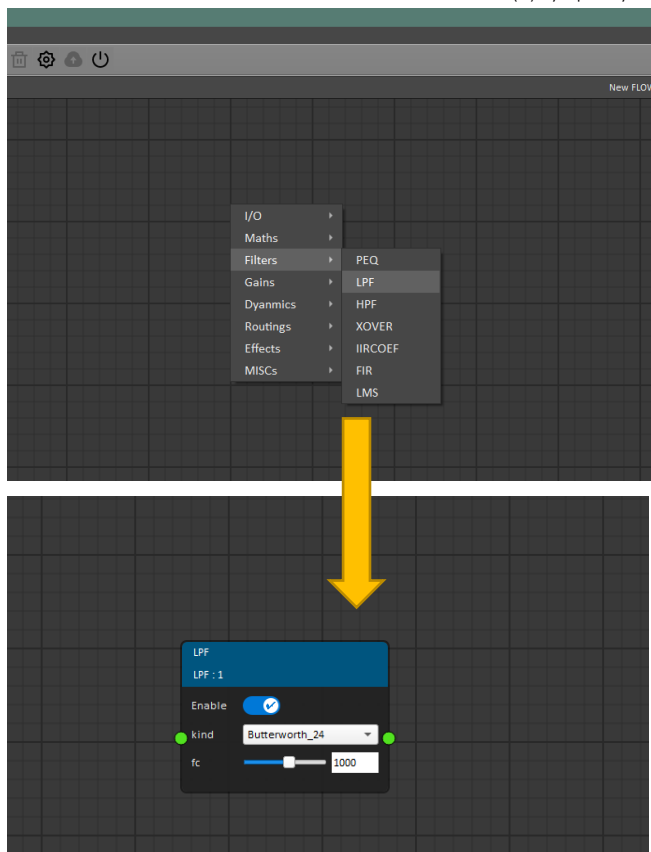
4.2.1 Add/delete audio object

There are two way to add the audio object:

- **By drag and drop from the Audio Objects pane** – select one of the audio objects from the Audio Objects pane hold mouse left click then drag into the Design Canvas:



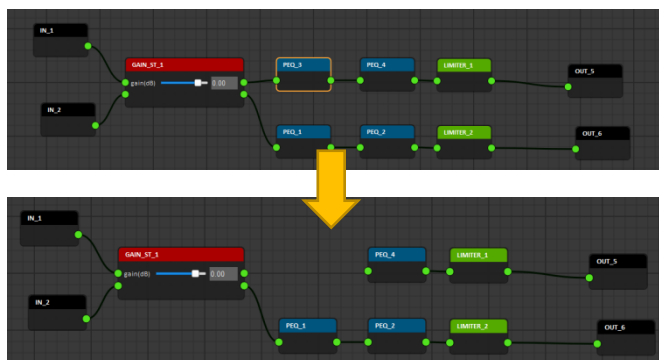
- **By mouse right click** – when mouse cursor inside the Design canvas, mouse right click will show up a context menu for selecting the audio object. Select the audio object the audio object will add to the canvas.



To delete the audio object mouse left click on one of the audio objects (the selected audio object will be highlighted) in canvas then press **del** or click the



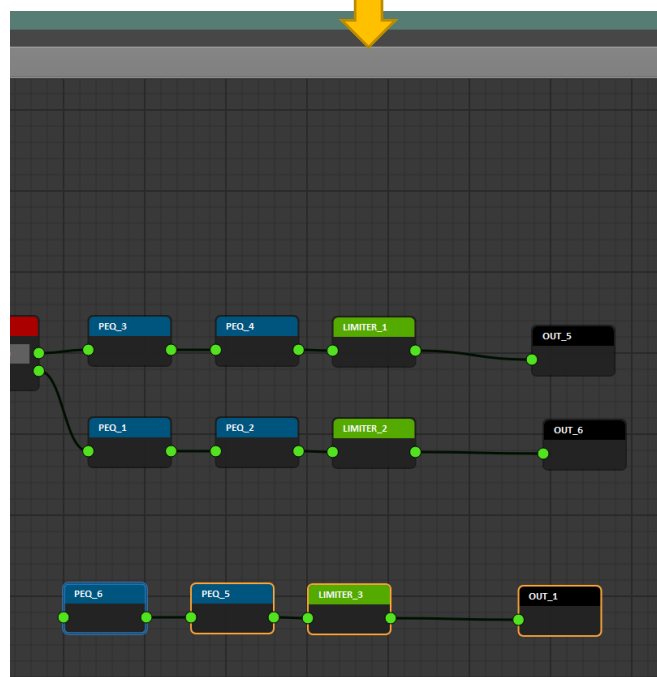
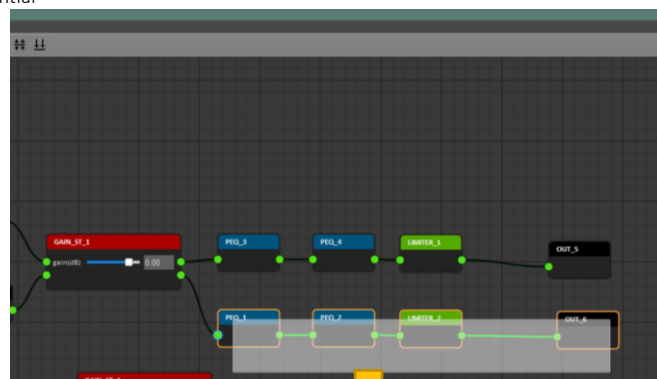
in main toolbox. You can also delete it from menu toolbar **Edit->delete**.



4.2.2 Copy/cut/paste audio objects

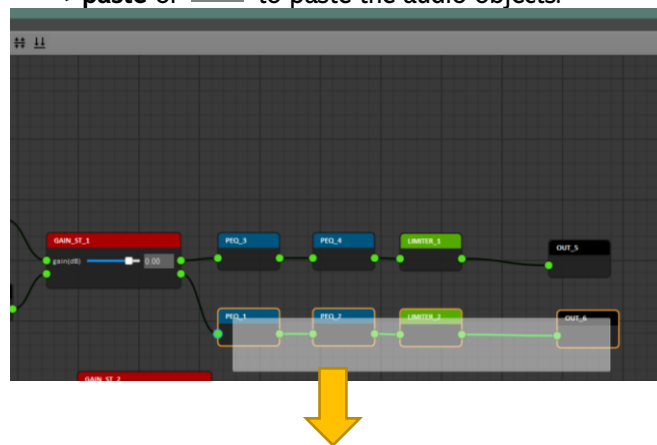
You can select one or multiple audio objects then doing cut or copy on canvas.

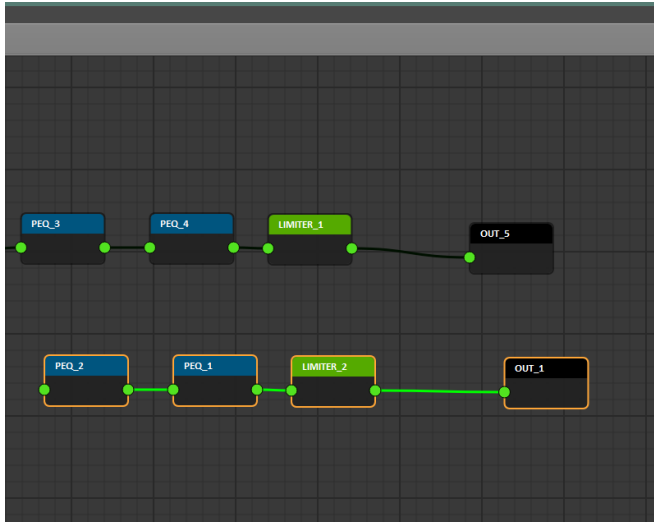
- **Copy and paste** – Select one or multiple audio objects by mouse then press **ctrl + c** or use **Edit->copy** or  form Main Toolbox. Press **ctrl + v** or use **Edit->paste** or  to paste the audio objects.



Note that the id number of the newly pasted audio objects will automatically be increased.

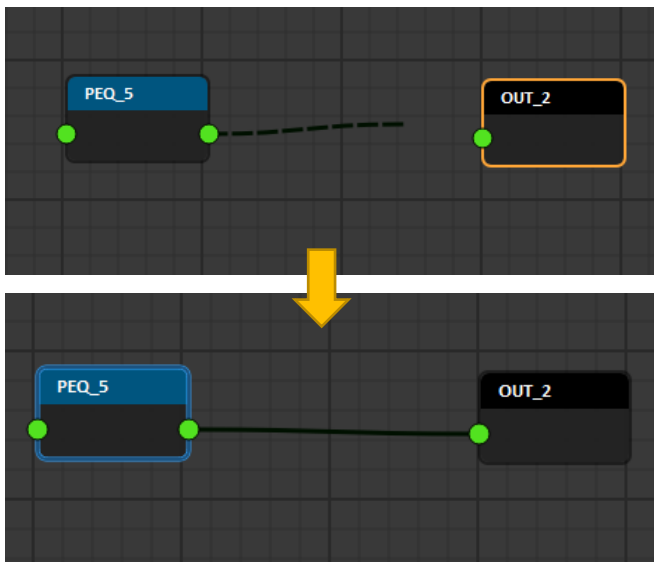
- **Cut and paste** – Select one or multiple audio objects by mouse then press **ctrl + x** or use **Edit->cut** or  form Main Toolbox. Press **ctrl + v** or use **Edit->paste** or  to paste the audio objects.





4.2.3 Connection between the audio objects

The signal flow direction is from left to right in FLOW studio. To connect the audio objects with each other, hover the mouse to the output port of the first node then hold the left click to the input port of the next node. The wire will turn from dashed line to solid line once the connection made. Note that Flow Studio provides port snap feature to make faster connection.



4.2.4 Redo/undo

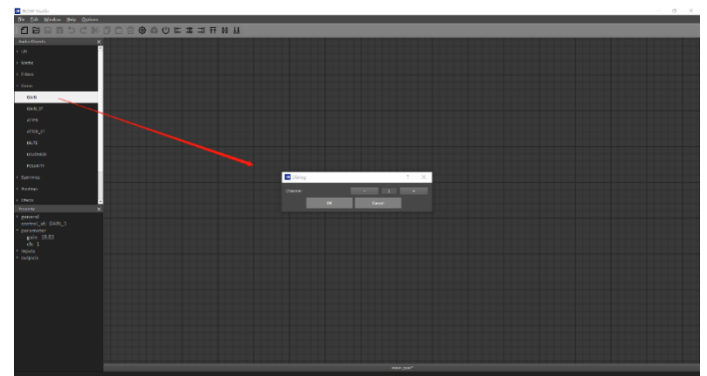
Use **ctrl + z** or **Edit->Redo** or  to back to the previous operation, **ctrl + y** or **Edit->Undo** or  to move one step further.

4.3 Multi-Channel Audio Object Capabilities

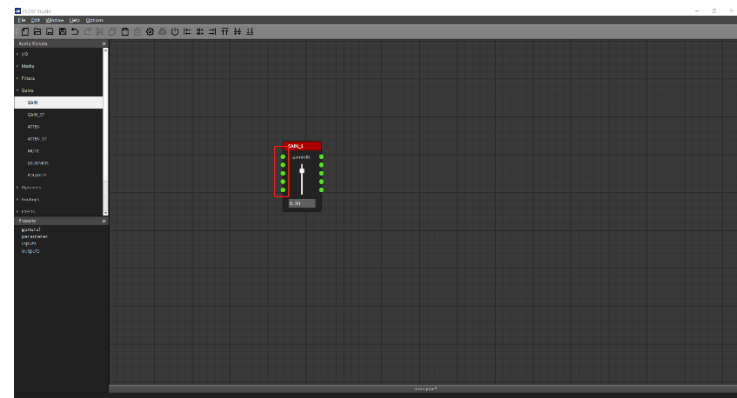
Multi-channel capabilities is a function that can provide one or more channels on the audio objects with linked parameters and can be added or reduced during designing signal flow.

Add Multi-Channel AO

Take GAIN AO as example: By drag and drop from the Audio Objects pane – select GAIN from Gains category of Audio Objects pane, hold mouse left click, drag into the Design Canvas, then it will pop up the dialog to let users input the number of channels.



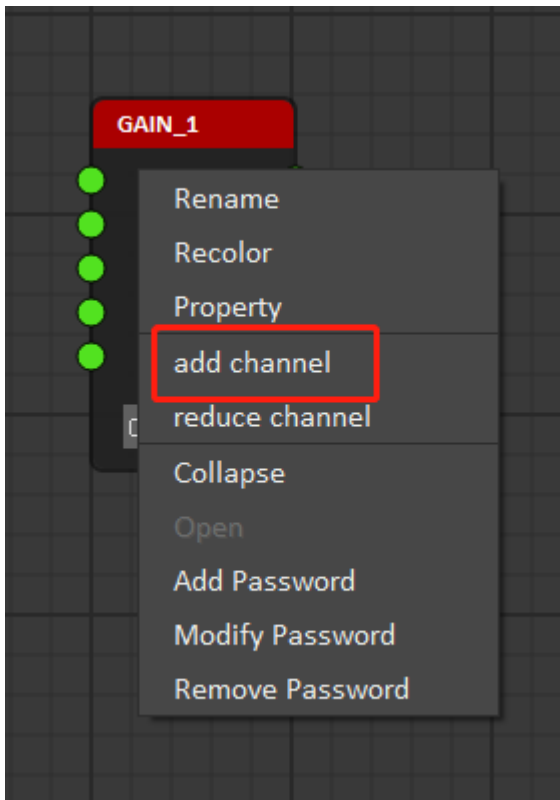
If we enter 5 (range 1-32), the AO of 5 channels will appear



Add Channel or Reduce Channel

Right click GAIN AO to display the context menu and

select Add channel to add a channel, select reduce channel, the number of channels will be reduced



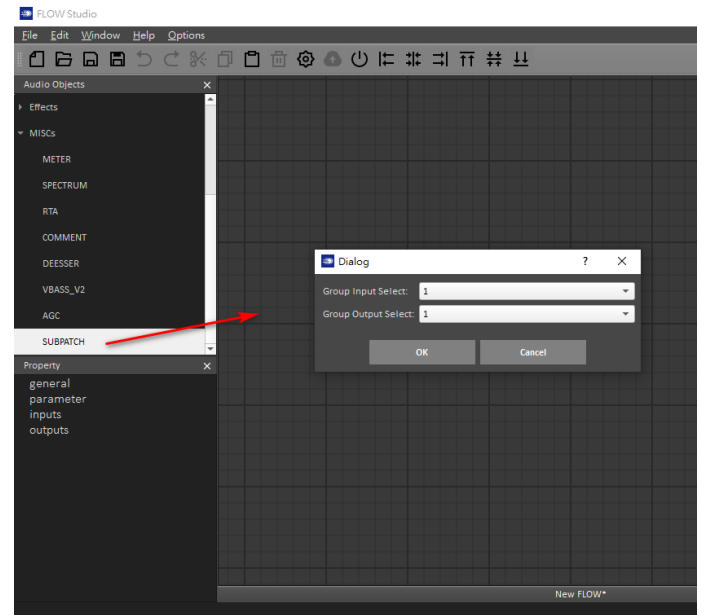
4.4 Sub-Patch feature

Sub-patch feature provides audio objects grouping capability on a newly created canvas. By using sub-patch, you can create a clean-looking signal flow in main canvas by placing reusable sub-signal flow inside the sub-patch. This is super useful when you need to design complex signal flow. And there are also ports on sub-patch, which we could connect to other audio objects or sub-patches. Sub-patch also could contain sub-patches.

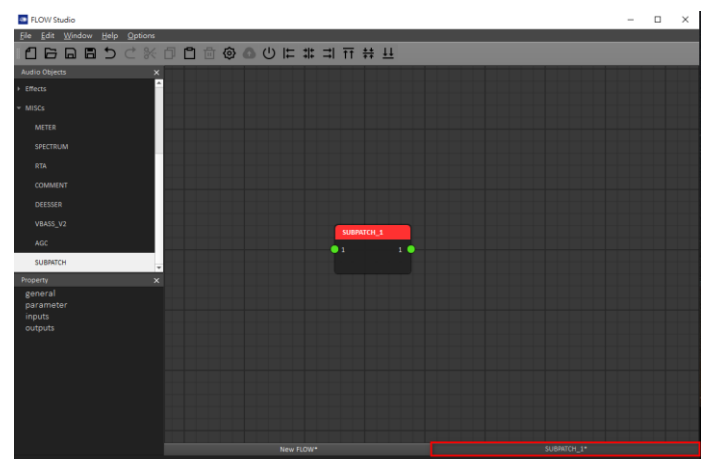
4.4.1 Add/delete sub-patch

There are two ways to add the sub-patch:

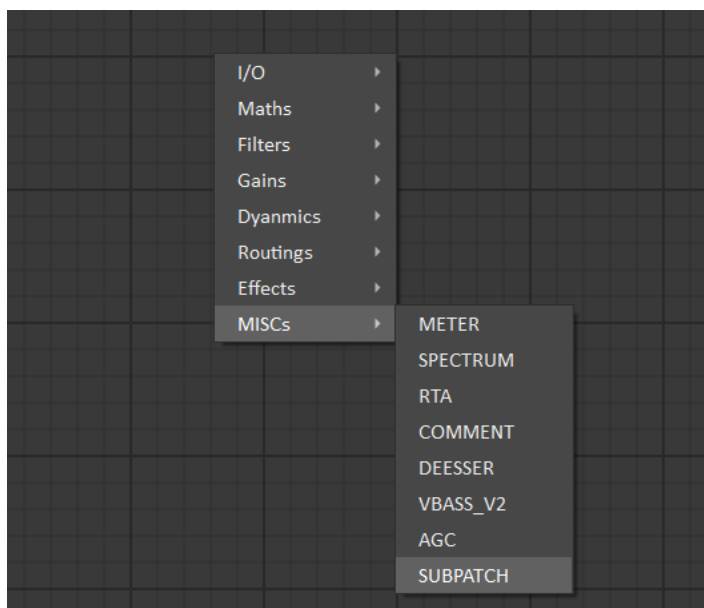
- **By drag and drop from the Audio Objects pane** – select sub-patch from MISCs category of Audio Objects pane, hold mouse left click, drag into the Design Canvas, then it will pop up the dialog to let users select the number of inputs and outputs:



After selecting the number of inputs and outputs, please click OK, then it will create new design canvas:



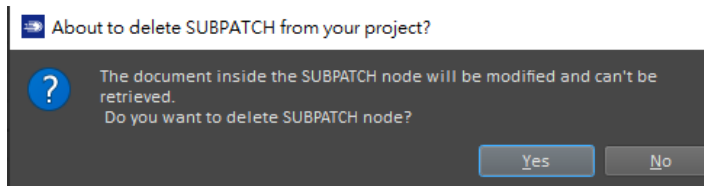
- **By mouse right click**
when moving mouse cursor inside the design canvas, mouse right click will show up a context menu for selecting the audio object. Select the sub-patch, then the subsequent process is the same as first method.



To delete the sub-patch, mouse left click on it (the selected sub-patch will be highlighted) in canvas

then press del or click the  in main toolbox. You can also delete it from menu toolbar Edit->delete.

When deleting sub-patch, it will pop up this dialog to ask you whether to delete the subpatch for sure or not, if you are for sure to delete the sub-patch, just click Yes.

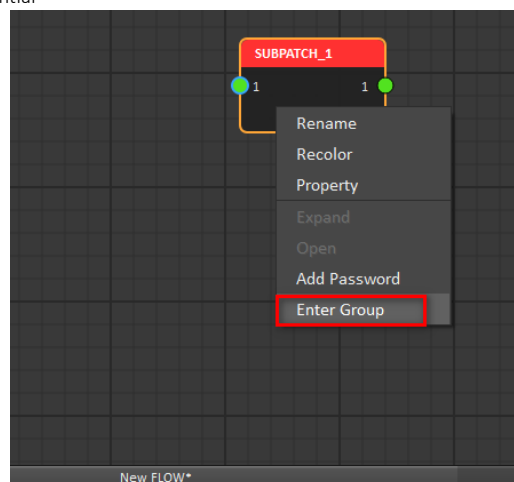


4.4.2 Enter sub-patch

There are two ways to enter the sub-patch:

- Double-click (left mouse button) on sub-patch
- By mouse right click

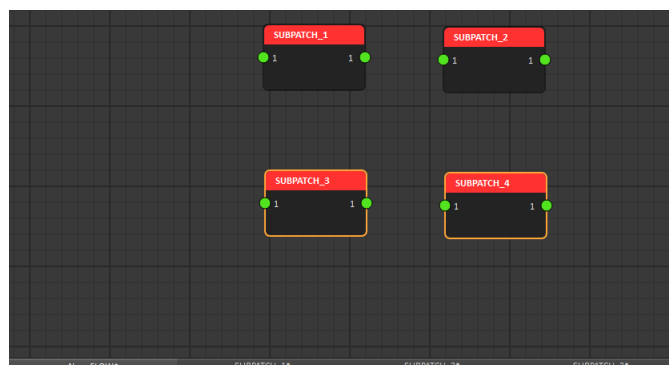
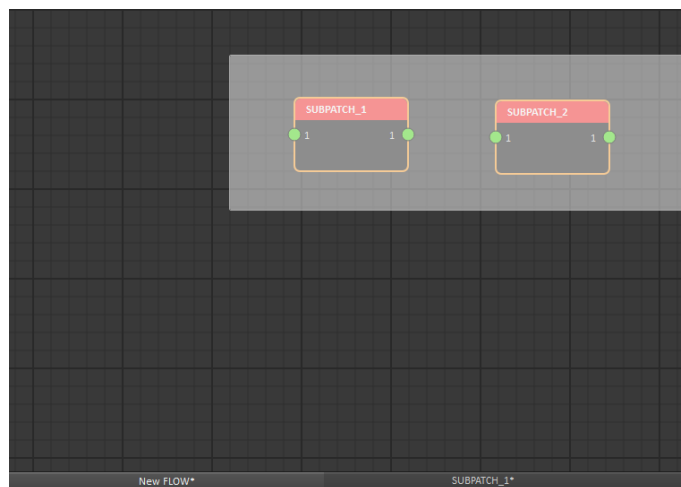
Mouse right click on sub-patch, click "Enter Group", then it will switch to corresponding sub-patch design canvas.



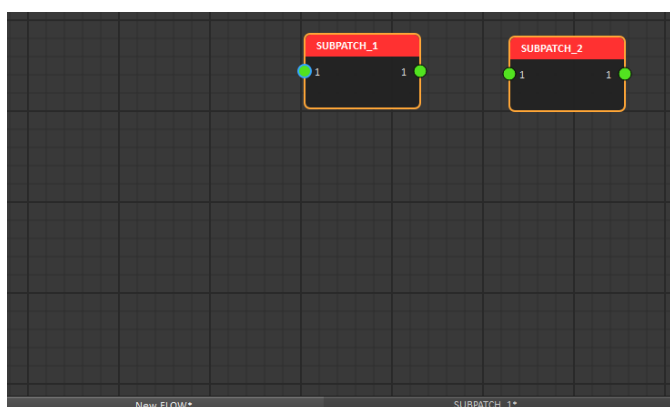
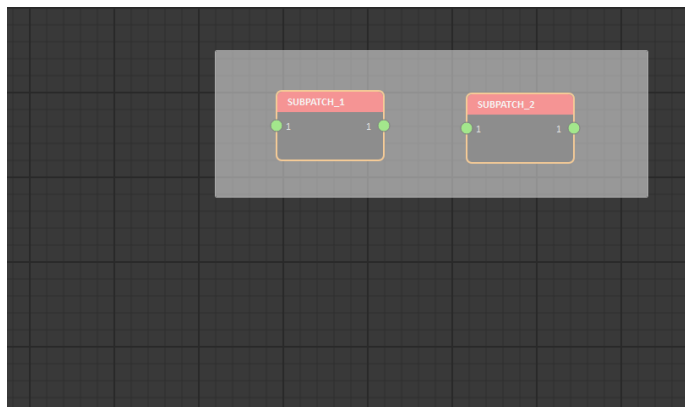
4.4.3 Copy/cut/paste sub-patches

You can select one or multiple sub-patches, then doing copy or cut on canvas.

- **Copy and paste** – Select one or multiple sub-patches by mouse then press `ctrl + c` or use **Edit->copy** or  from Main Toolbox. Press `ctrl + v` or use **Edit->paste** or  to paste the sub-patches.

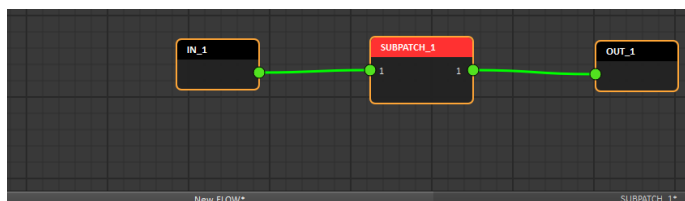


- **Cut and paste** – Select one or multiple sub-patches by mouse then press **ctrl + x** or use **Edit->cut** or  form Main Toolbox. Press **ctrl + v** or use **Edit->paste** or  to paste the sub-patches.

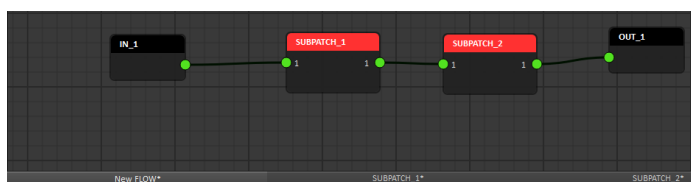


4.4.4 Connection between the sub-patch and audio objects or sub-patch

Sub-patch could connect to other audio objects like this:



Or connect to other sub-patch like this:



4.4.5 Redo/undo

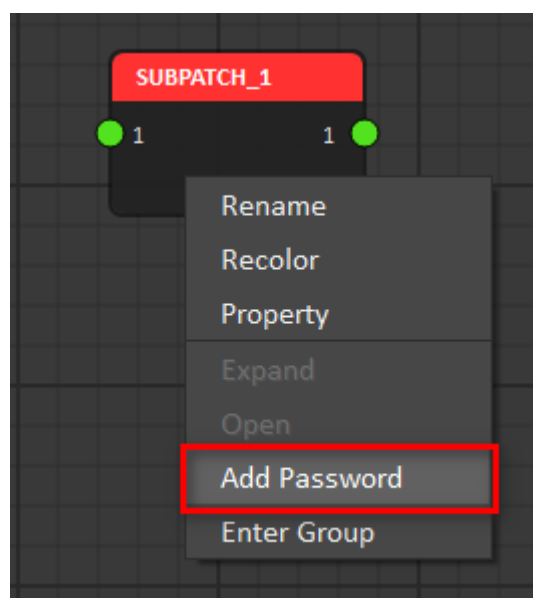
Use **ctrl + z** or **Edit->Undo** or  to go back to the previous operation, use **ctrl + y** or **Edit->Redo** or  to move one step further.

4.4.6 Encryption of sub-patch

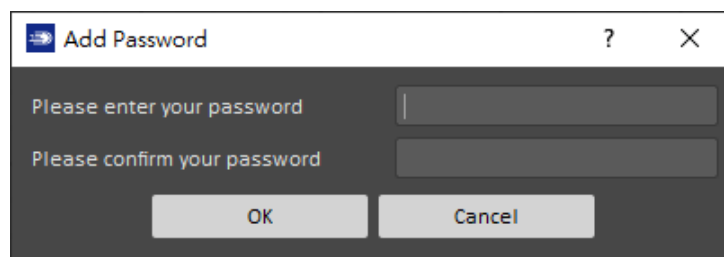
If you would like to encrypt your sub-patch file, we also provide this functionality.

- **Add password**

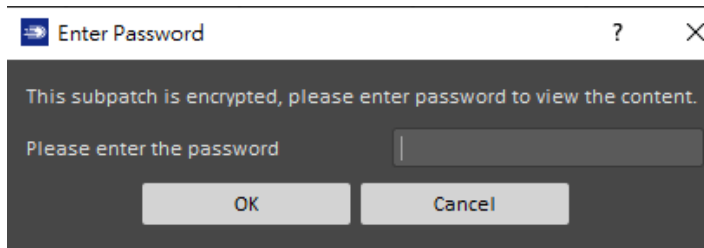
Right click on sub-patch, choose “Add Password”:



Then it will pop up the dialog to ask to input the disired password:



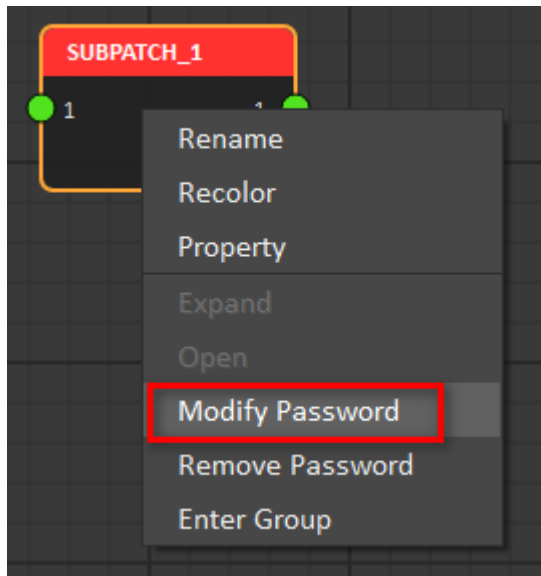
After inputting the password, click OK, then save the files. Once opening project file, if you would like to enter that encrypted sub-patch, system will ask you to enter the password:



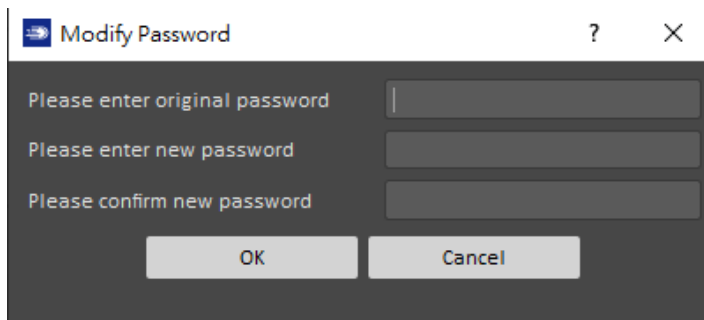
Note: if sub-patch1 contains sub-patch2, and sub-patch is not encrypted, when adding password on sub-patch1, the sub-patch2 will be encrypted by using same password as sub-patch1.

- **Modify password**

If you would like to modify the password, right click on the sub-patch, choose “Modify Password”

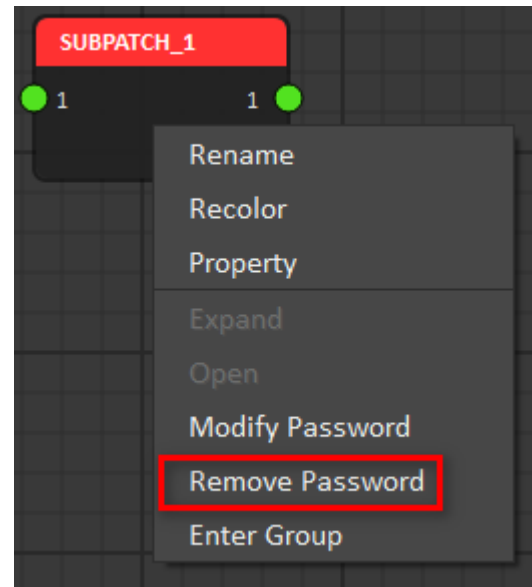


Then it will pop up the dialog to ask to input the original and new desired password. Once finishing inputting, click OK, save it, the password will be changed.

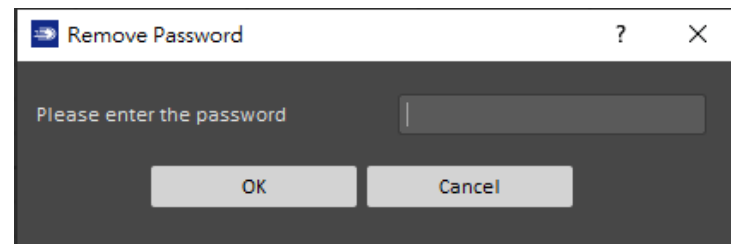


- **Remove Password**

If you would like to remove the password, right click on the sub-patch, choose “Remove Password”.



Then it will pop up the dialog to ask to input the password. Once inputting correctly, click OK, save it, the password will be removed.



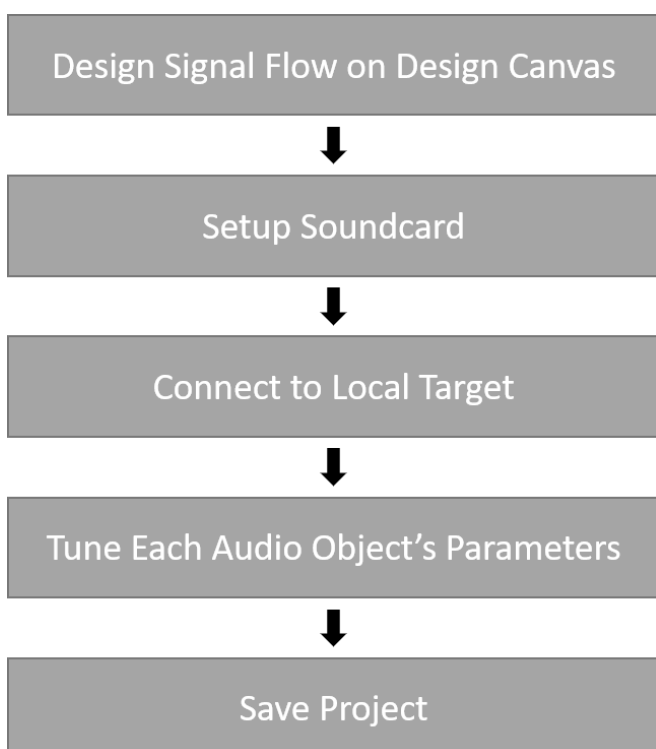
5 Workflow

◆Note:

Please read through the section : **Getting Started-
>Important Concepts**

5.1 Local Target

Here is the typical workflow to design the signal flow on your PC + soundcard:

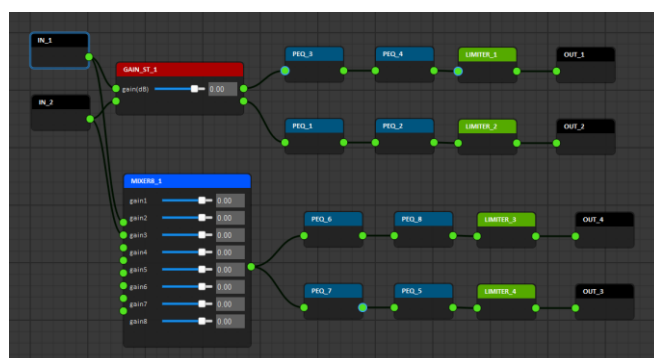


5.1.1 Design Signal Flow on Design Canvas

Design a preference signal flow in Design Canvas.
Please refer to Design Canvas chapter for more detail.

◆Note:

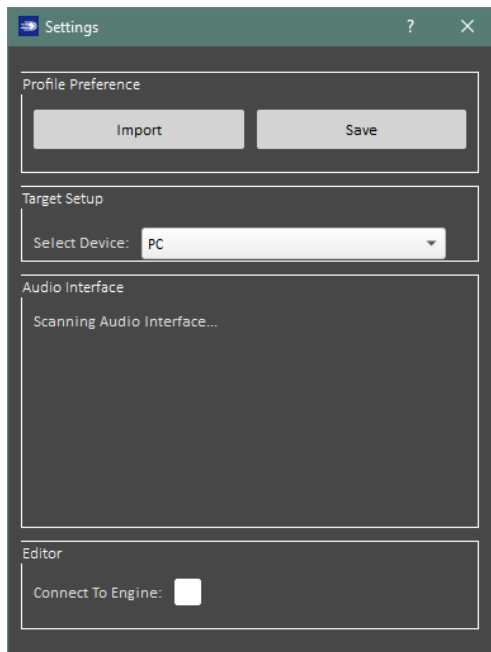
Using **IN** audio object to map the input channel of your soundcard, and use **OUT** audio object to map the output of your soundcard



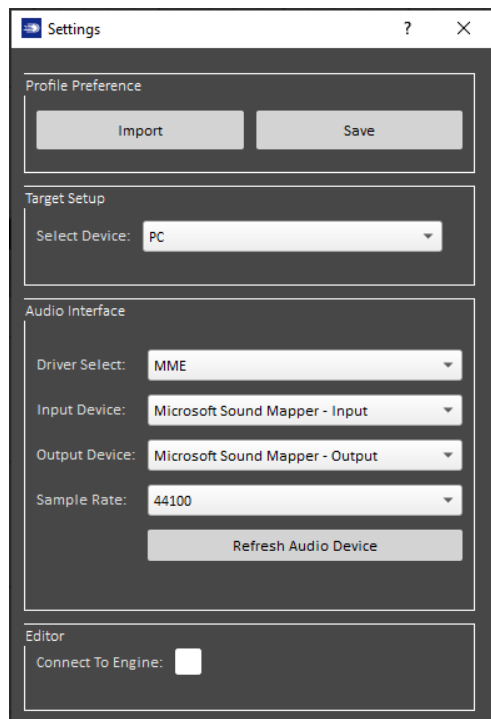
5.1.2 Setup Soundcard

By clicking Connect Settings  or **Options->Connect Settings** to open the control setting dialog window:

The Connect Setting will first scan the available audio interfaces on your PC.

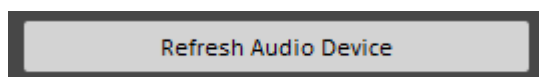


Once the scanning is done, it will show up the available audio devices:



Select preference driver/input device/output device and sample rate you want to stream the audio in or out to your soundcard.

If device is not in devices list, please check the device driver and refresh devices list.



Save profile if you need to reload the Connect Setting next time you open FLOW Studio.



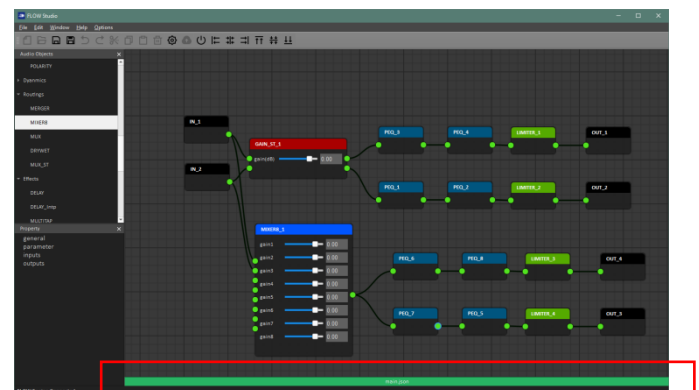
5.1.3 Connect to local target

In Connect Setting, check the Connect to Engine check box.



Alternatively, click  or **Option->Connect** to connect to local target.

Once the local target is successfully connected, the design canvas tab on top will turn into green.



5.1.4 Tune Parameters

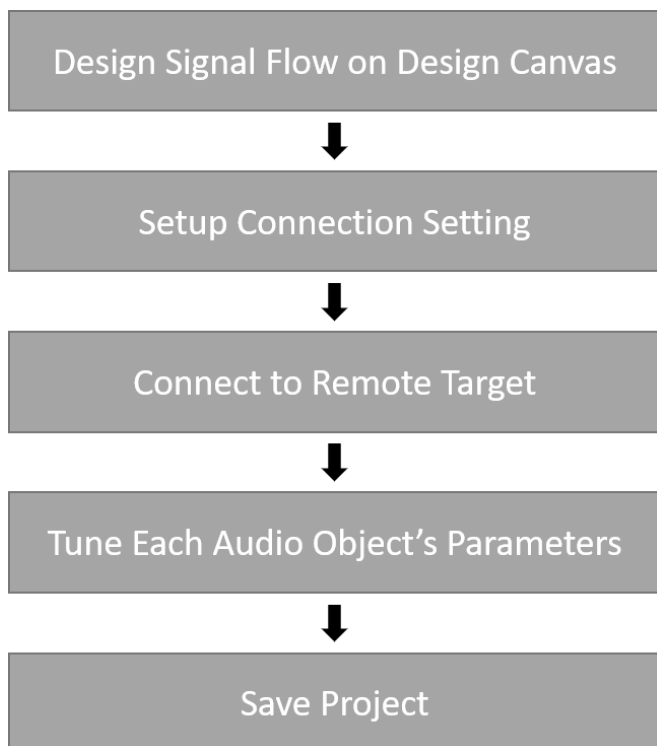
Adjust each parameter on each audio object, use live parameter on node, or detailed UI as you wish.

5.1.5 Save Project

Use File->save/save as to save your current settings for next use.

5.2 Remote Target – TCP

This is the typical workflow on remote target.



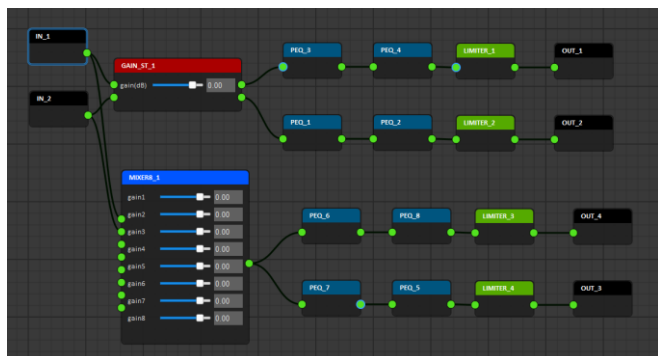
5.2.1 Design Signal Flow on Design

◆Note:

Contact the local FW or DSP team for the above information. Different remote target has different settings.

Canvas

Design a preference signal flow in Design Canvas. Please refer to Design Canvas chapter for more detail.



5.2.2 Setup Soundcard

By clicking Connect Settings  or Options-

>Connect Settings to open the control setting dialog window.

Select the target from Target Setup

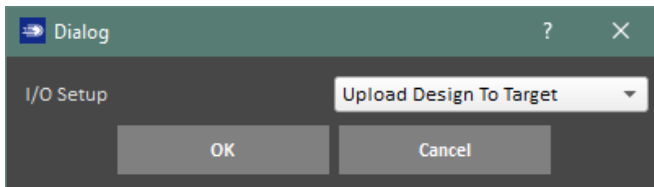
Fill in the Config Path, Host Name, Port to needed by the remote target.

5.2.3 Connect to remote target

In Connect Setting, check the Connect to Engine check box.

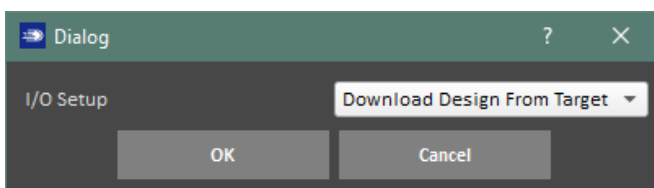
Alternatively, click  or Option->Connect to connect to local target.

Once the device is connected, there will show up a dialog window



This will upload the current signal flow design configuration to the target. Note that the FLOW Engine will parse the configuration and create the signal processing flow inside the target in real time.

Another option is **Download Design From Target** to FLOW Studio.



This mechanism will make sure the Design between target and FLOW Studio is synchronized.

5.2.4 Tune Parameters

Adjust each parameter on each audio object, use live parameter on node, or detailed UI as you wish.

5.2.5 Save Project

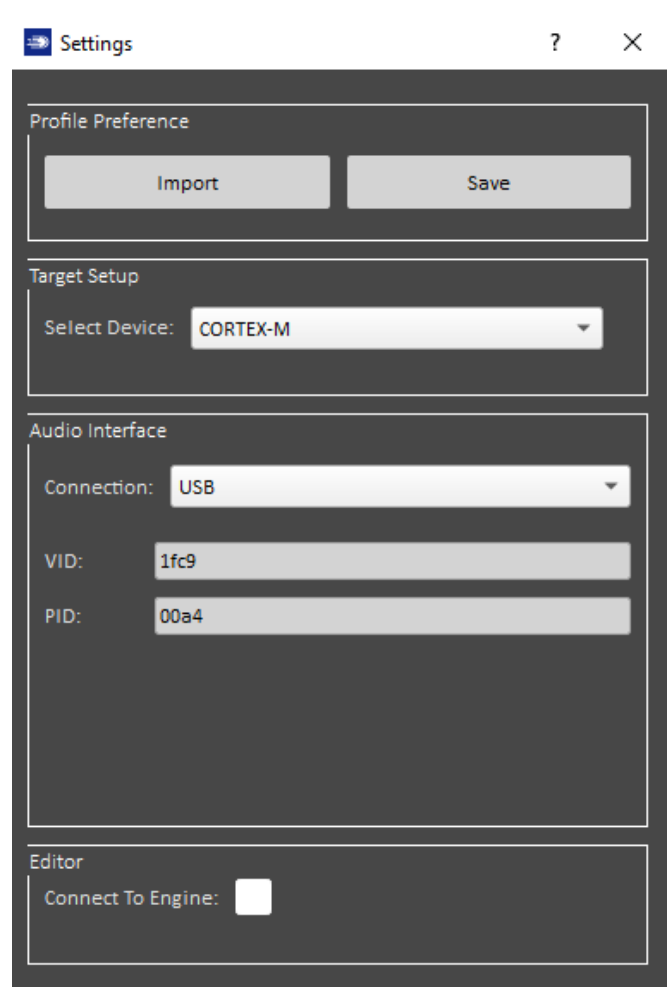
Use File->save/save as to save your current settings for next use.

5.3 Remote Target - USB

By clicking Connect Settings  or **Options->Connect Settings** to open the control setting dialog window.

5.3.1 Setup Soundcard

Select the target from Target Setup



Select Device : CORTEX-M

Connection : USB

Fill in the VID , VIP needed by remote target

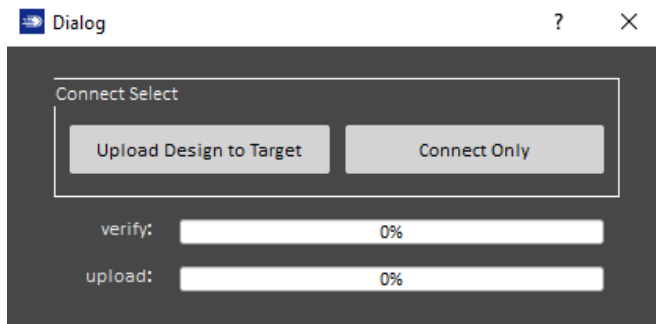
5.3.2 Connect to remote target

In Connect Setting, check the Connect to Engine check box.



Alternatively, click  or **Option->Connect** to connect to local target.

Once the device is connected, there will show up a dialog window

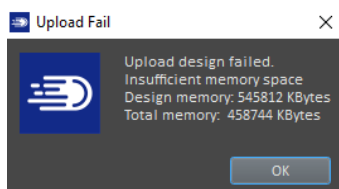


Upload Design to Target

Before upload signal design to target, FlowStudio will check memory space is feasible to upload. Then upload the current signal flow design configuration to the target. Note that the FLOW Engine will parse the configuration and create the signal processing flow inside the target in real time.

*upload failed message

If signal flow design memory larger than target memory pool. FlowStudio will terminate upload process. Please check the design again.



Connect Only

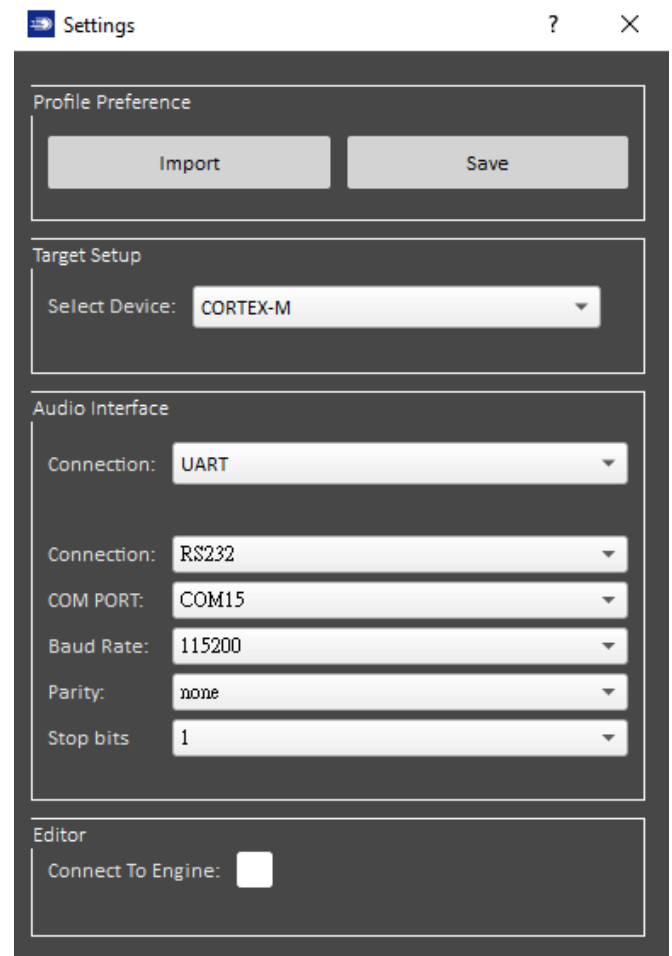
If the signal flow design between Flow Studio and target is the same, it didn't need to upload design every time. This mechanism will connect to target directly.

5.4 Remote Target - UART

By clicking **Connect Settings**  or **Options->Connect Settings** to open the control setting dialog window.

5.4.1 Setup Soundcard

Select the target from Target Setup



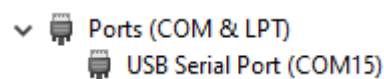
Select Device : CORTEX-M

Connection : UART

Fill in the COM PORT number needed by remote target

To find the number of port.

Windows -> Devices Manager -> Ports

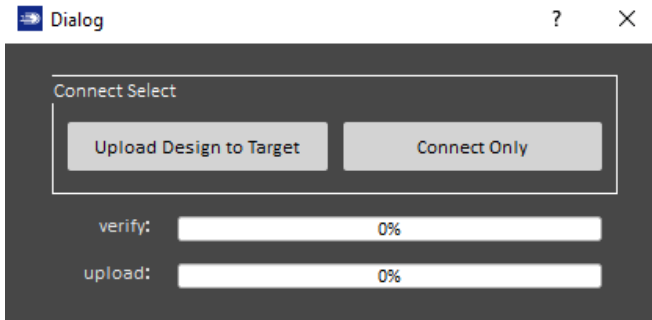


5.4.2 Connect to remote target

In Connect Setting, check the Connect to Engine check box.



Alternatively, click  or **Option->Connect** to

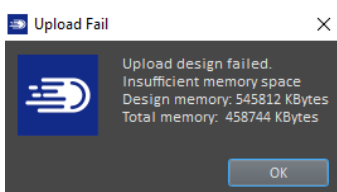


Upload Design to Target

Before upload signal design to target, FlowStudio will check memory space is feasible to upload. Then upload the current signal flow design configuration to the target. Note that the FLOW Engine will parse the configuration and create the signal processing flow inside the target in real time.

*upload failed message

If signal flow design memory larger than target memory pool. Flow Studio will terminate upload process. Please check the design again.

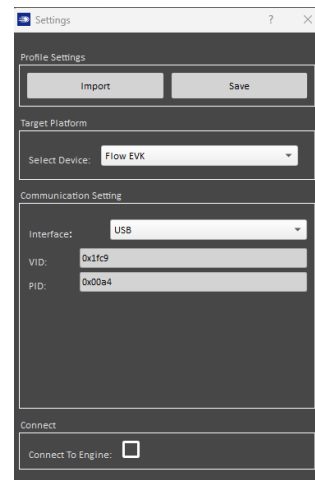


Connect Only

If the signal flow design between Flow Studio and target is the same, it didn't need to upload design every time. This mechanism will connect to target directly.

5.4.3 Upload Design to Flash (Flow EVK target only)

This feature currently supports the Flow EVK, allowing users to update the Signal Design and save it to Flash without the need for additional flashing software. Once stored in Flash, the Signal Design remains intact even after a power cycle.

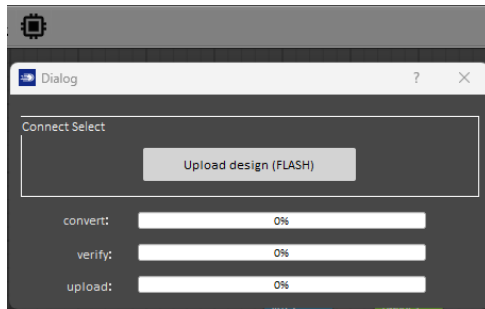


The **Upload to Flash** function operates as shown in the

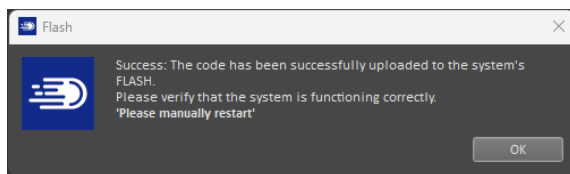
diagram . Please note that the option to upload to Flash will be disabled if you haven't executed **Upload Design** first. This is to ensure that the Signal Design being updated has already been verified.

Operation Steps

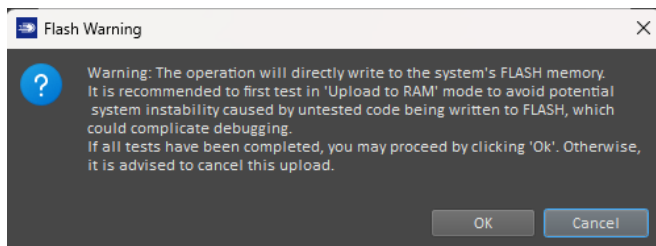
1. First, ensure that you are using Flow EVK, and that your **DSP firmware version is 1.8.1 or higher**.
2. Verify your design and execute **Upload Design to Target**.
3. After that, you will see the **Upload to Flash** option become available. Once clicked, a progress indicator window will appear, showing the upload options and the progress of the update.



4. Upon a successful update, a confirmation window will inform you that the update has completed.



Important: At this stage, the design is not yet running from Flash. To execute the design from Flash, you will need to reset the Flow EVK. After the reset, your design will be loaded and executed from Flash.



6 Audio Objects (AOs)

Audio objects (AOs) or **audio processing objects** are the fundamental units used to construct signal flows in an audio processing module. Each audio object represents a self-contained processing unit that performs a specific function, such as filtering, gain control, voice detection, or dynamic range compression. In Flow Studio, these AOs are visualized as nodes within the graphical interface. Each node has its own specifications, including unique parameters and input/output port configurations. By interconnecting nodes, users can design and refine complex audio processing workflows with flexibility and precision.

Each audio processing node typically consists of the following key elements:

- **Information Tab:** The Information Tab displays the name and unique identifier of each audio object, allowing users to quickly identify and organize nodes within the signal flow. For instance, the first created input object may be labeled as **IN_1**, while a second parametric equalizer could be denoted as **PEQ_2**. This structured naming convention helps users track different nodes, understand their roles in the processing chain, and manage complex workflows with ease.
- **Input Port:** The input port serves as the entry point for an audio signal into the node. Some nodes may not require an input port if they function as signal sources (e.g., noise generators or tone generators).
- **Output Port:** The output port transmits the processed audio signal from the node to the next stage in the signal flow. An output port can fan out multiple signals, meaning the same processed audio can be sent to multiple downstream nodes simultaneously. Similar to input ports, not all nodes require output ports. Some may simply modify parameters or control other nodes without directly processing audio.
- **Live Parameter Control:** Each node features a live parameter control panel, which allows users to modify key processing settings directly on the node itself within the graphical canvas.

In this section, we categorized all the AOs into several types: **I/O** (for handling audio input and output), **Maths** (for mathematical operations on signals), **Filters** (for frequency shaping and equalization), **Gains** (for volume adjustments), **Dynamics** (for compression, expansion, and limiting), **Routing & Mixing** (for signal distribution and blending), **Effects** (for creative and spatial processing), **Metering** (for signal analysis and visualization), and **Technology Provider**, which include both third-party integrations and proprietary algorithms developed by our team.

It is worth noting that not all AOs are algorithmic processing units. Some perform basic operations or visualization, such as basic gain control, metering, and signal generation. These essential building blocks do not rely on complex DSP techniques. In contrast, **algorithm-driven AOs**—found in the **Effects, Dynamics, and Technology Provider** categories—incorporate advanced DSP and AI-based processing for tasks like noise reduction, beamforming, and echo cancellation. These AOs address challenging audio scenarios, enhancing real-time communication, recording, and playback. Each category plays a vital role in constructing efficient audio workflows, which will be explored in the following sections.

6.1 I/O

Overview

The AOs related to input/output source are the essential components that manage signal flow within the system. These **I/O AOs** allow users to import, generate, monitor, and analyze audio signals, providing the foundation for designing and testing high-quality audio processing pipelines.

Available I/O AOs:

- **Input & Output:** Serve as the primary input and output points for external audio sources and processed signals. They define how audio enters and exits the system, making them crucial for integrating Flow Studio with real-world applications.
- **Wav Player:** Allows users to load and playback pre-recorded WAV files, enabling quick testing and debugging of audio processing chains using real-world samples.
- **Sine Tone Generator:** Generates pure sine waves of adjustable frequencies and amplitudes, commonly used for testing filters, calibrating audio systems, or measuring harmonic distortion.
- **Noise Generator:** Produces white or pink noise, useful for testing equalizers, dynamic range processors, and noise reduction algorithms.
- **Chirp Generator:** Generates frequency sweeps across a defined range, often used for system response testing and tuning frequency-based effects.
- **Audio Dump:** Captures and stores audio signals at different points in the processing chain, allowing for offline analysis and debugging.
- **Signal Visualizer:** Provides real-time graphical feedback on signal waveforms, levels, and frequency content, making it easier to monitor and fine-tune audio performance.

These I/O AOs are fundamental to the workflow in Flow Studio, enabling users to efficiently design, evaluate, and refine their audio processing systems. By offering a variety of input sources, test signal generators, and analysis tools, this section equips users with the necessary components to build and optimize highly precise and effective signal processing solutions.

6.1.1 Input, Input (fixed-point)

Functional description of Input object

The **Input object** (displayed as **IN** or **IN_FP** for the fixed-point version in the information tab) serves as the primary audio capture interface in Flow Studio, allowing users to bring external audio signals into their processing pipeline. This input can come from various sources depending on the system setup:

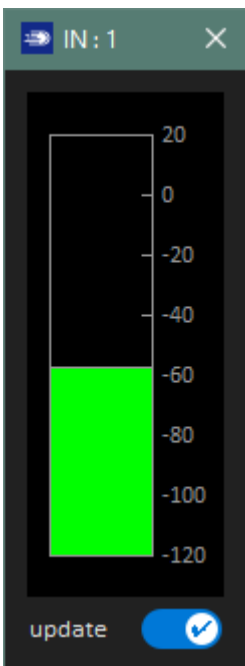
- **Soundcard Input** (when running Flow Studio on a PC).
- **Analog-to-Digital Converter (ADC) Output** from embedded systems.
- **Internal Audio Pipeline** from a previous processing stage within an audio system.

Both floating-point and fixed-point versions of the Input object are available for the user and the number of available Input objects depends on the hardware's physical inputs. Additionally, each Input object is equipped with a level meter, providing real-time visual feedback to indicate whether an input signal is detected. This makes it easy for users to monitor signal presence and troubleshoot connectivity issues.

Input & Output Configuration of Input object

- **Audio inputs:** N/A
- **Audio Outputs:** 1

Flow Studio GUI of Input object



6.1.2 Output, Output (fixed-point)

Functional description of Output object

The **Output object** (displayed as **OUT** or **OUT_FP** for the fixed-point version in the information tab) acts as the primary audio playback interface in Flow Studio, allowing users to route processed audio signals to external hardware.

Depending on the system configuration, the output can be directed to:

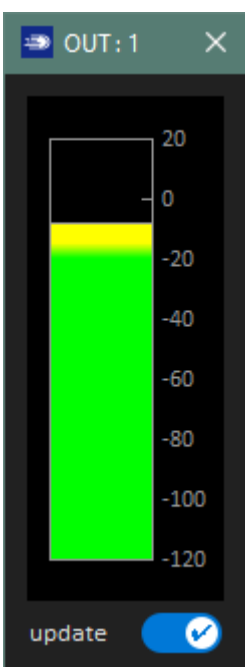
- **Soundcard Output** (when running Flow Studio on a PC).
- **Digital-to-Analog Converter (DAC) Input** in embedded systems.
- **Next-Stage Audio Pipeline** within an audio processing system.

Same as Input object, both floating-point and fixed-point versions of the Output object are available, allowing users to select the format that best suits their system's processing architecture and performance requirements. The maximum number of Output objects depends on the system's available physical outputs. Each Output object also features a level meter, providing real-time visual feedback on the output signal. This ensures that users can easily monitor and verify if the audio signal is correctly routed and processed.

Input & Output Configuration of Output object

- **Audio inputs:** 1
- **Audio Outputs:** N/A

Flow Studio GUI of Output object



6.1.3 Wav Player

Functional description of Wav Player object

The **Wav Player** object (displayed as **WAVPLAYER** in the information tab) provides a convenient way to play stereo audio from a WAV file, making it an essential tool for testing and evaluating audio processing operations. It allows users to inject test tones, music, or any pre-recorded content into their signal flow, which can be mixed with live physical inputs. This is especially useful for debugging, prototyping, and validating audio effects in real-world scenarios.

There are several things need to be noticed:

- **File Placement:** The WAV file must be stored on the same device running the Flow Engine. For example, if using Flow DSP on a PC, place the WAV file somewhere on the PC's hard drive. If running on an embedded Linux target (e.g., Raspberry Pi), store the WAV file in the device's flash memory.
- **File Selection:** On a PC, users can manually specify the file path or use the GUI to browse and select a file. On embedded systems, users must enter the exact file path where the WAV file is stored.
- **Sample Rate Compatibility:** Flow Studio's Wav Player object does not perform sample rate conversion. If the WAV file's sample rate does not match the system's, it will result in playback speed mismatches and pitch shift artifacts.
- **Mono File Handling:** If a mono WAV file is used, the first two output channels will contain identical audio content.

Input & Output Configuration of Wav Player object

- **Audio inputs:** N/A
- **Audio Outputs:** 1~8

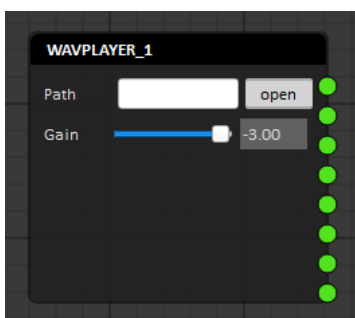
Adjustable Parameters of Wav Player object

- **Path:** The audio path
- **Gain:** Controls the output gain level, ranging from -120 dB to 0 dB. (default: 3 dB)

Notes: Supported WAV Formats:

- **Bit Depths:** 8-bit, 16-bit, 24-bit, 32-bit, unsigned 8-bit, floating-point
- **Channels:** Mono to 8-channel audio
- **Sample Rates:** Any (but must match system sample rate for proper playback)

Flow Studio GUI of Wav Player object



6.1.4 Sine Tone Generator

Functional description of Sine Tone Generator object

The **Sine Tone Generator** (displayed as **TONEGEN** in the information tab) is a fundamental tool that generates a pure sine wave that can be used for calibrating, debugging, and evaluating audio processing systems. By adjusting the frequency and gain, users can simulate different test conditions, making it an essential component for audio engineers, developers, and researchers working with signal processing.

This object is particularly useful for:

- **System Calibration:** Ensuring that audio hardware and software components function as expected.
- **Frequency Response Testing:** Checking how different audio algorithms or systems react to specific frequencies.
- **Gain Structure Validation:** Testing the dynamic range and behavior of signal paths at different levels.

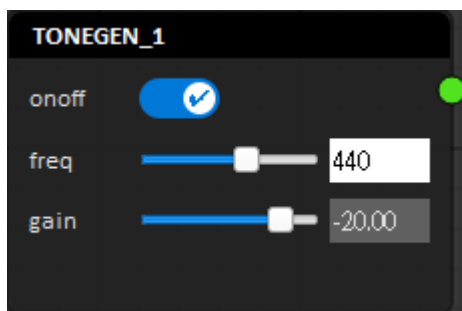
Input & Output Configuration of Sine Tone Generator object

- **Audio inputs:** N/A
- **Audio Outputs:** 1

Adjustable Parameters of Sine Tone Generator object

- **onoff:** Enables or mutes the Sine Tone Generator. (Default: on)
- **freq:** Sine wave frequency in Hertz, ranging from 1 Hz to 20 kHz. (Default: 440 Hz)
- **gain:** Adjusts the noise signal level, ranging from -120 dB to 0 dB. (Default: -20 dB)

Flow Studio GUI of Sine Tone Generator object



6.1.5 Noise Generator

Functional description of Noise Generator object

The **Noise Generator** (displayed as **NOISEGEN** in the information tab) is a crucial tool for audio testing, calibration, and performance evaluation. It provides a tunable noise signal source, allowing users to generate either white noise or pink noise based on their testing needs. White noise has a flat frequency spectrum, making it useful for system diagnostics and general-purpose testing, while pink noise has equal energy per octave, making it ideal for acoustic measurements and audio equalization.

This object is particularly useful for:

- **System Stress Testing:** Evaluating how an audio system performs under continuous random signal excitation.
- **Equalization & Calibration:** Using pink noise for room tuning and frequency response adjustments.
- **Noise Floor & Dynamic Range Analysis:** Measuring how noise affects audio signals in various processing environments.

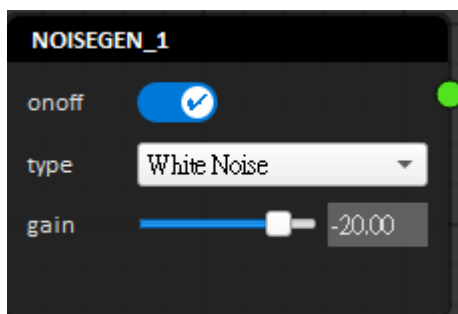
Input & Output Configuration of Noise Generator object

- **Audio inputs:** N/A
- **Audio Outputs:** 1

Adjustable Parameters of Noise Generator object

- **onoff:** Enables or mutes the Sine Tone Generator. (Default: on)
- **type:** Specifies the noise type. Options: White Noise, Pink Noise. (Default: White Noise)
- **gain:** Controls the output level of the noise signal, ranging from -120 dB to 0 dB. (Default: -20 dB)

Flow Studio GUI of Noise Generator object



6.1.6 Chirp Generator

Functional description of Chirp Generator object

The **Chirp Generator** (displayed as **CHIRP** in the information tab) is a powerful tool for audio system testing, speaker calibration, and frequency response analysis. It generates a sweep signal that moves from a defined start frequency to an end frequency over a specific time interval. This sweeping tone helps in analyzing how an audio system responds to different frequencies, making it ideal for:

- **Speaker and Microphone Testing** – Measuring how a system reproduces sound across its frequency range.
- **Room and Acoustic Analysis** – Evaluating reflections, resonances, and standing waves.
- **Filter and Equalizer Tuning** – Checking the performance of signal processing algorithms.

The **Chirp Generator** supports both logarithmic (Log) and linear (Linear) sweep types, giving users flexibility in testing different acoustic scenarios.

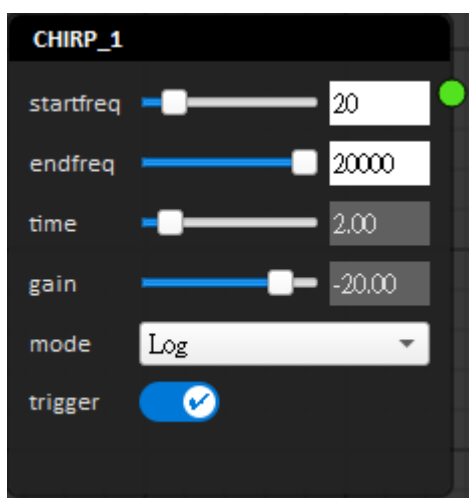
Input & Output Configuration of Chirp Generator object

- **Audio inputs:** N/A
- **Audio Outputs:** 1

Adjustable Parameters of Chirp Generator object

- **startfreq:** Defines the starting frequency (in Hertz) of the sweep. (Default: 20 Hz)
- **endfreq:** Defines the frequency (in Hertz) at which the sweep stops. (Default: 20 kHz)
- **time:** Duration of the frequency sweep ranging from 1 sec to 10 sec. (Default: 2 sec)
- **gain:** Adjustable output level, ranging from -120 dB to 0 dB. (Default: -20 dB)
- **mode:** Specifies the sweep type. Options: Log or Linear. (Default: Log)
- **trigger:** Activates the frequency sweep when turned on.

Flow Studio GUI of Chirp Generator object



6.1.7 Audio Dump

Functional description of Audio Dump object

The **Audio Dump** object (displayed as **DUMP** in the information tab) allows users to capture and save audio at any point within the signal flow as a WAV file. This feature is invaluable for debugging, enabling users to analyze how the signal behaves at different processing stages. Whether troubleshooting an issue or verifying processing accuracy, Audio Dump object simplifies the debugging workflow by providing direct access to recorded audio.

Usage Steps

1. Drag & Drop the Input object (or Wav Player object) as input source into the workspace.
2. Drag & Drop the DUMP audio object into the workspace and connect with input source.
3. Start the Flow Engine to begin audio processing.
4. Click the Dump icon to open the settings.
5. Set Dump ID, duration, and file directory, then start recording by clicking “DUMP” in the UI window.

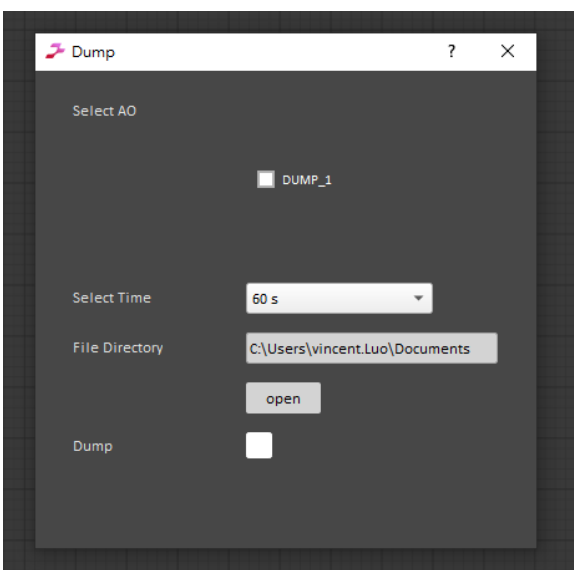
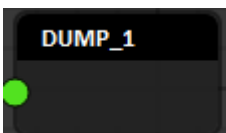
Input & Output Configuration of Audio Dump object

- **Audio inputs:** 1
- **Audio Outputs:** N/A

Adjustable Parameters of Audio Dump object

- **Select AO:** Assigns a specific DUMP ID to differentiate multiple audio dumps.
- **Select Time:** Defines the recording duration. Options: 60, 120, 180, 240, or 300 sec. (Default: 60 sec)
- **File Directory:** Specifies the storage path for the recorded file.

Flow Studio GUI of Audio Dump object



6.1.8 Signal Visualizer

Functional description of Signal Visualizer object

The **Signal Visualizer** object (displayed as **VISUALIZER** in the information tab) provides a real-time graphical representation of the audio signal in the time or frequency domain. This tool helps users analyze waveform characteristics, detect distortions, and monitor audio behavior at any point in the signal chain. It is especially useful for debugging, tuning, and verifying audio processing performance within Flow Studio.

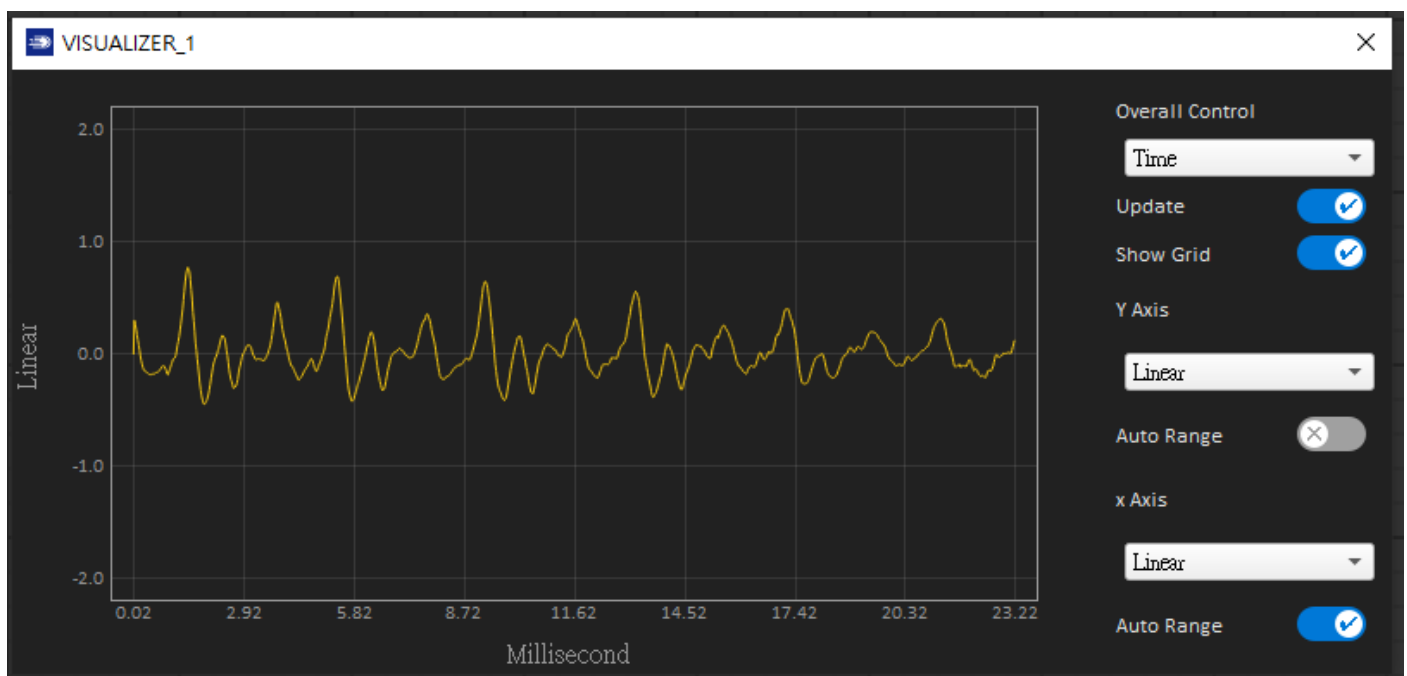
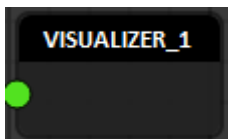
Input & Output Configuration of Signal Visualizer object

- **Audio inputs:** 1
- **Audio Outputs:** N/A

Adjustable Parameters of Signal Visualizer object

- **Representation:** time domain or frequency domain representation of the signal. (Default: Time)
- **Update:** Starts the signal visualization, continuously updating in real time. (Default: False)
- **Show Grid:** Enable/Disable a grid overlay in the GUI. (Default: Enabled)
- **Auto Range:** Adjust the scale of Y axis or X axis automatically. (Default: Disabled)

Flow Studio GUI of Signal Visualizer object



6.2 Maths

Overview

Math-related audio objects (AOs) provide essential arithmetic operations for manipulating audio signals in real-time.

Maths AOs allow users to perform fundamental mathematical functions such as addition, subtraction, multiplication, and more, enabling precise control over signal processing. They are particularly useful in scenarios such as gain scaling, signal inversion, and custom DSP (Digital Signal Processing) implementations.

Available Maths AOs:

- **Adder & Adder (Fixed-Point):** The Adder AO sums multiple audio signals, combining them into a single output. This is useful for mixing signals or blending different sound sources. Fixed-point version is available.
- **Negator:** The Negator AO inverts the polarity of an audio signal by multiplying it by -1. This is useful in phase cancellation techniques and signal correction.
- **Abs & Abs (Fixed-Point):** The Abs AO computes the absolute value of an audio signal, eliminating negative values. This can be beneficial for amplitude modulation and envelope detection. Fixed-point version is available.
- **Sqrt & Sqrt (Fixed-Point):** The Sqrt AO calculates the square root of an audio signal, which can be used for dynamic range adjustments and specialized DSP operations. Fixed-point version is available.
- **Multiplier & Multiplier (Fixed-Point):** The Multiplier AO performs element-wise multiplication of two signals, useful for amplitude modulation, gain scaling, and envelope shaping. Fixed-point version is available.
- **Subtractor:** The Subtractor AO calculates the difference between two audio signals. This can be applied to noise cancellation, differential signal processing, or creating contrast between waveforms.

By integrating these Maths AOs into their signal flows, users gain fine-grained control over audio manipulation, allowing for both fundamental operations and complex DSP techniques. Whether it's blending signals, adjusting gain dynamically, or implementing custom processing algorithms, these AOs provide the essential mathematical foundation needed for precise and flexible audio engineering.

6.2.1 Adder, Adder (fixed-point)

Functional description of Adder object

The **Adder** audio object (displayed as **ADDER** or **ADDER_FP** for the fixed-point version in the information tab) sums two input signals and outputs the combined result. It is a fundamental operation in digital signal processing (DSP) and is commonly used in applications such as audio mixing, signal summation, and effect processing. The mathematical representation of the adder is:

$$y(n) = x_1(n) + x_2(n)$$

- $y(n)$ is the output signal at time n .
- $x_1(n)$ and $x_2(n)$ are the input signals at time n .

There are several examples including that the Adder can be used to combine multiple microphone signals after phase-aligned filtering, improving the signal-to-noise ratio (SNR) for better speech clarity. Or the Adder can be used to combine the original dry signal with the effect-processed signal to create a balanced wet/dry mix.

Input & Output Configuration of Adder object

- **Audio inputs: 2**
- **Audio Outputs: 1**

Flow Studio GUI of Adder object



6.2.2 Negator

Functional description of Negator object

The **Negator** audio object (displayed as **NEGATOR** in the information tab) inverts the polarity of the input signal, effectively flipping its phase by 180 degrees. The mathematical operation can be expressed as:

$$y(n) = -x(n)$$

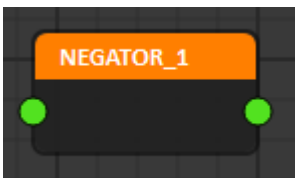
- $y(n)$ is the output signal at time n .
- $x(n)$ is the input signal at time n .

This operation results in that all positive values in the waveform become negative and vice versa. While this does not change the perceived loudness of the signal, it can significantly impact how signals interact when combined, particularly in multi-microphone and stereo applications. For example, when multiple microphones capture the same sound source, phase cancellation may occur due to different distances between the microphones. The Negator can be used to align signals by inverting the phase of one microphone signal before summing them. Or if a noise profile has been captured separately, inverting it and mixing it back with the original signal can help cancel out unwanted noise.

Input & Output Configuration of Negator object

- **Audio inputs: 1**
- **Audio Outputs: 1**

Flow Studio GUI of Negator object



6.2.3 Abs, Abs (fixed-point)

Functional description of Abs object

The **Abs** audio object (displayed as **ABS** or **ABS_FP** for the fixed-point version in the information tab) outputs the absolute value of the input signal, effectively removing the polarity (negative sign) of any negative samples. The mathematical operation is:

$$y(n) = |x(n)|$$

- $y(n)$ is the output signal at time n .
- $x(n)$ is the input signal at time n .
- $|\cdot|$ indicates the absolute value operation.

This operation ensures that all signal values remain non-negative. The Abs (Fixed-Point) variant performs the same operation using fixed-point arithmetic, which is beneficial for DSP systems with limited floating-point support. The Abs object is commonly used in amplitude envelope detection for applications like automatic gain control (AGC) and dynamic range processing. Alternatively, in speech and audio analysis, the absolute value is useful for extracting features like energy levels and zero-crossing rates.

Input & Output Configuration of Abs object

- **Audio inputs:** 1
- **Audio Outputs:** 1

Flow Studio GUI of Abs object



6.2.4 Sqrt, Sqrt (fixed-point)

Functional description of Sqrt object

The **Sqrt** audio object (displayed as **SQRT** or **SQRT_FP** for the fixed-point version in the information tab) computes the square root of the input signal. This is a fundamental mathematical operation often used in signal processing for various tasks, such as feature extraction, scaling, and energy calculation. The mathematical operation is represented as:

$$y(n) = \sqrt{x(n)}$$

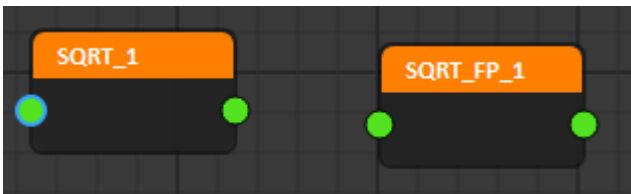
- $y(n)$ is the output signal at time n .
- $x(n)$ is the input signal at time n .
- $\sqrt{\cdot}$ indicates the square root operation.

In dynamic range compression, the square root operation is used to calculate the Root Mean Square (RMS) of an audio signal. The RMS value is crucial for determining the average power of the signal, which helps in applying gain reduction or amplification based on signal levels. On the other hand, square roots operations are often part of non-linear processing chains where the signal needs to be scaled or transformed to create specific audio effects.

Input & Output Configuration of Sqrt object

- **Audio inputs:** 1
- **Audio Outputs:** 1

Flow Studio GUI of Sqrt object



6.2.5 Multiplier, Multiplier (fixed-point)

Functional description of Multiplier object

The **Multiplier** audio object (displayed as **MULTIPLIER** or **MULTIPLIER_FP** for the fixed-point version in the information tab) calculates the multiplication of two input signals. This operation is often used in audio processing for tasks such as dynamic range compression, modulation, and signal mixing. The mathematical operation is represented as:

$$y(n) = x_1(n) \times x_2(n)$$

- $y(n)$ is the output signal at time n .
- $x_1(n)$ and $x_2(n)$ are the two input signals at time n .

In amplitude modulation, the amplitude of a carrier signal is varied in proportion to a modulating signal. The multiplication operation is used to combine the carrier and modulating signals. On the other hand, multiplying a signal with a filter or shaping function can modify its spectral content. This is often used in noise shaping to reduce quantization noise in digital systems.

Input & Output Configuration of Multiplier object

- **Audio inputs:** 2
- **Audio Outputs:** 1

Flow Studio GUI of Multiplier object



6.2.6 Subtractor

Functional description of Subtractor object

The **Subtractor** audio object (displayed as **SUBTRACTOR** in the information tab) calculates the subtraction of two input signals. This operation is frequently used in various audio processing tasks, such as differential signal processing, audio effects, and noise cancellation. The mathematical operation is represented as:

$$y(n) = x_1(n) - x_2(n)$$

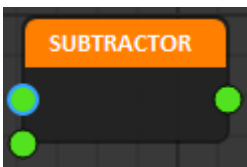
- $y(n)$ is the output signal at time n .
- $x_1(n)$ and $x_2(n)$ are the two input signals at time n .

This operation is essential in contexts where one signal needs to be removed or isolated from another, such as in noise subtraction or signal differencing. For example, in a speech-enhancement system, the noise captured by a reference microphone is subtracted from the main microphone signal, removing background noise and improving speech clarity.

Input & Output Configuration of Subtractor object

- **Audio inputs:** 2
- **Audio Outputs:** 1

Flow Studio GUI of Subtractor object



6.3 Filters

Overview

Filtering is a fundamental aspect of audio processing, used to shape frequency content, remove unwanted noise, and enhance specific audio characteristics. Flow Studio provides a range of **Filters AOs** that allow users to precisely control the spectral balance of their signals. These AOs include both Infinite Impulse Response (IIR) and Finite Impulse Response (FIR) filters, offering flexibility for different processing needs. Whether users require precise parametric equalization, simple low-pass filtering, or advanced crossover design, Flow Studio's Filters AOs provide the necessary tools.

Available Filters AOs:

- **Biquad Filter:** A versatile IIR filter that can implement various filter types, including low-pass, high-pass, band-pass, and notch filtering. It is widely used due to its efficiency and smooth frequency response.
- **Parametric EQ & Parametric EQ (Fixed-Point):** A parametric equalizer (EQ) allows users to control the gain, center frequency, and bandwidth of specific frequency bands, providing precise tonal shaping.
- **Parametric EQ V2:** An enhanced version of the Parametric EQ, offering improved processing capabilities and additional control options for refined equalization.
- **Generic IIR Filter:** A customizable IIR filter that allows users to design and implement various filter types using user-defined coefficients, providing maximum flexibility for advanced users.
- **FIR Filter & FIR Filter (Fixed-Point):** FIR (Finite Impulse Response) filters provide linear-phase filtering, making them ideal for applications where phase distortion must be minimized. Fixed-point version is also available.
- **Low-Pass Filter / High-Pass Filter & Fixed-Point Versions:** These dedicated low-pass and high-pass filters provide efficient frequency separation, removing unwanted high-frequency or low-frequency content as needed. Fixed-point versions ensure optimized performance on hardware platforms.
- **2-Way Crossover & 2-Way Crossover (Fixed-Point):** A crossover filter divides an audio signal into two separate frequency bands, typically used in multi-way speaker systems for directing different frequencies to the appropriate drivers (e.g., tweeters and woofers).
- **GAME EQ:** Provides a collection of popular equalizer presets tailored for today's most popular games. Users can quickly switch between different EQ profiles to enhance their gaming experience, helping them achieve convenient and optimized gaming audio.

Filters AOs play a crucial role in shaping the tonal balance and clarity of an audio signal. Whether refining a voice recording, designing a multi-speaker system, or implementing sophisticated DSP techniques, these AOs offer the necessary precision and flexibility. By integrating efficient IIR filters, phase-accurate FIR filters, and highly configurable parametric equalizers, Flow Studio empowers users to design and optimize audio processing workflows for a wide range of applications. With these tools, users can achieve the desired spectral characteristics and ensure high-quality audio output across different use cases.

6.3.1 Biquad filter

Functional description of Biquad Filter object

The **Biquad Filter** object (displayed as **BIQUAD** in the information tab) is a second-order **infinite impulse response (IIR) filter**, named after its characteristic **biquadratic** transfer function in the **Z-domain**. This means that both the numerator and denominator of the transfer function are represented by quadratic polynomials. Due to its versatility and efficiency, the Biquad filter is widely used in **digital signal processing (DSP)** for audio applications. By adjusting the filter coefficients to configure different filter types, users can fine-tune the audio signal to achieve the desired frequency response, making the Biquad filter a fundamental tool in **audio processing and equalization**.

Specifically, the Biquad filter object allows filter coefficients parameterization for different filter types, including:

- **Low-pass filter (LPF)** – Attenuates frequencies above the cutoff while allowing lower frequencies to pass.
- **High-pass filter (HPF)** – Blocks lower frequencies and permits higher frequencies to pass.
- **Band-pass filter (BPF)** – Passes frequencies within a specific range while attenuating those outside the range.
- **Notch (band-stop) filter** – Suppresses a narrow band of frequencies while allowing others to pass.
- **Peaking EQ** – Boosts or attenuates frequencies within a specific band.
- **Shelving filter** – Adjusts gain at either the high or low end of the frequency spectrum.

User Cases include:

1. Speech Enhancement in Communication Systems

In **teleconferencing applications**, background noise and low-frequency hum can interfere with speech clarity. A **high-pass filter (HPF)** can be applied to remove low-frequency noise such as air conditioning hum, while a **peaking EQ** can be used to enhance mid-range frequencies where human speech is most intelligible.

2. Music Production and Equalization

In **audio mixing and mastering**, engineers often use biquad filters to shape the frequency response of different instruments. For example, a **low-pass filter (LPF)** can be applied to a bass guitar track to remove high-frequency noise, while a **band-pass filter (BPF)** can isolate and emphasize specific frequency ranges of a vocal recording.

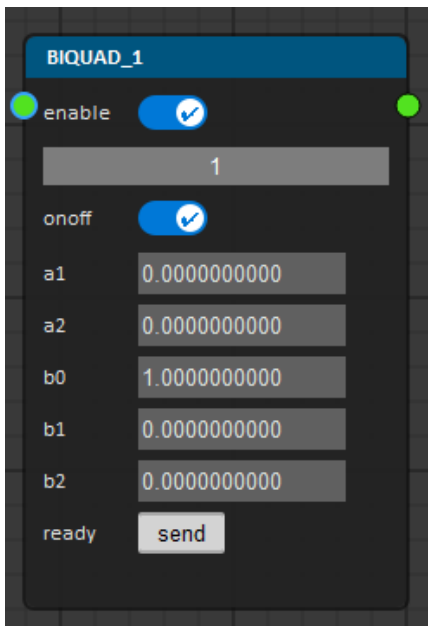
Input & Output Configuration of Biquad Filter object

- **Audio inputs:** 1~32
- **Audio Outputs:** 1~8

Adjustable Parameters of Biquad Filter object

- **enable:** Enables or bypasses the Biquad filter object. (Default: “Enable”)
- **onoff:** Enable or bypasses the corresponding biquad band
- **a1:** filter coefficient
- **a2:** filter coefficient
- **b0:** filter coefficient
- **b1:** filter coefficient
- **b2:** filter coefficient
- **ready:** Trigger button to load coefficient into the Biquad filter.

Flow Studio GUI of Biquad Filter object



6.3.2 Parametric EQ, Parametric EQ (fixed-point)

Functional description of PEQ object

A **Parametric Equalizer (PEQ)** object, displayed as **PEQ** or **PEQ_FP** for the fixed-point version in the information tab, is a powerful and precise audio processing tool that allows fine-tuned control over frequency response. It is based on **Infinite Impulse Response (IIR) biquad filters**, making it highly efficient for **real-time audio applications**. In **digital signal processing (DSP)**, the **PEQ block** integrates multiple biquad filter types, enabling the design of complex equalization curves. By adjusting the **EQ type, center frequency, gain, and bandwidth (Q factor)**, users can selectively boost or attenuate specific frequency ranges. The system dynamically calculates the **filter coefficients**, ensuring the desired **frequency response** and **phase characteristics** are accurately achieved.

As shown in **Fig. 1**, the **transfer function** of the PEQ object is derived from the selected filter parameters. The implementation follows the **Direct Form II Transposed structure (Fig. 2)**, which is preferred for its numerical stability and computational efficiency in embedded DSP systems. This approach makes parametric equalization highly effective for various applications, including speech enhancement, music production, hearing aids, and automotive audio systems.

$$H(z) = \frac{b_0 + b_1 * z^{-1} + b_2 * z^{-2}}{a_0 + a_1 * z^{-1} + a_2 * z^{-2}}$$

Fig.1 transfer function

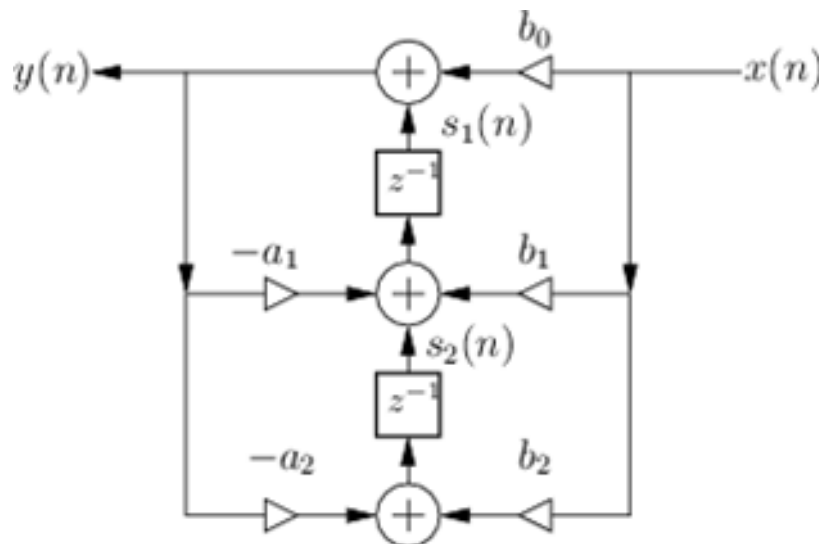


Fig.2 Direct Form II Transposed

A single PEQ block can configure up to 10 cascaded biquad filters, providing flexible and powerful equalization capabilities. The Flow Studio GUI enables real-time visualization of both Magnitude response (how much a frequency is boosted/attenuated) and Phase response (how the phase of signals is altered)

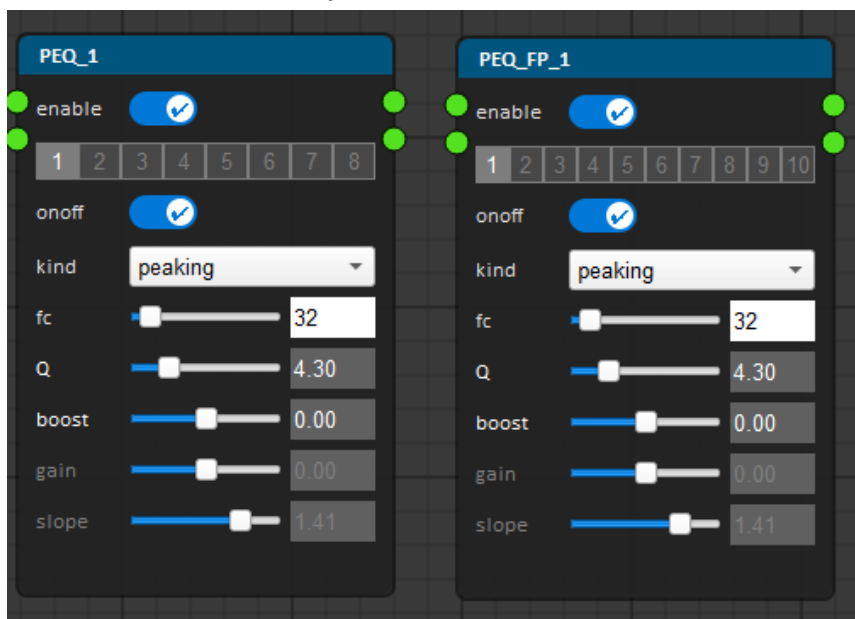
Input & Output Configuration of PEQ object

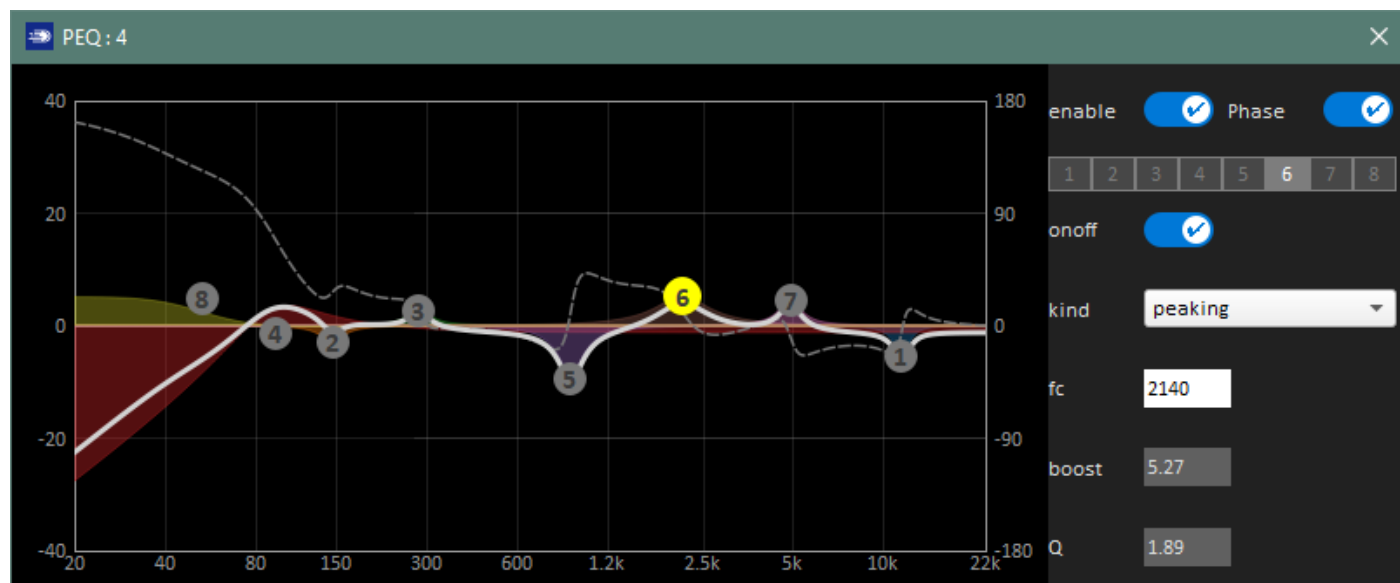
- **Audio inputs:** 1~32
- **Audio Outputs:** 1~32
- **Band Counts:** 1~10

Adjustable Parameters of PEQ object

- **enable:** Enable or bypass whole PEQ audio object
- **onoff:** Enable or bypasses the corresponding EQ band
- **type:** Specifies the filter type for the corresponding EQ band, including the following options:
 - **low-pass 1 / low-pass 2:** first-order/ second-order low pass filter
 - **high-pass 1 / high-pass 2:** first-order / second-order high pass filter
 - **all-pass 1 / all-pass 2:** first-order / second-order high pass filter
 - **low-shelf / high-shelf:** Boosts or attenuates frequencies below / above a specified cutoff point
 - **parametric:** A fully adjustable band filter for precise frequency control
 - **band-pass:** Allows frequencies within a certain range to pass while attenuating others
 - **peaking:** Boosts or attenuates a selected frequency band
 - **notch:** Attenuates a narrow frequency range to remove unwanted noise or interference
- **frequency:** Defines the cutoff or center frequency for the selected filter type.
- **boost:** Sets the boost or attenuation gain (in dB) at the specified frequency. (*Applicable only to shelving and peaking/parametric filters.*)
- **gain:** Controls the overall gain of the current EQ band.
- **Q:** Sets the Q factor for the filter.
- **slope:** Defines the slope factor for the shelving filter. (*Applicable only when the filter type is set to shelving.*)

Flow Studio GUI of PEQ object





6.3.3 Parametric EQ V2

Functional description of PEQ V2 object

The Parametric EQ (PEQ) V2 object, displayed as **PEQ_V2** in the information tab) is an improved version of the original PEQ audio object, offering enhanced flexibility and stability in equalization processing. While retaining all the core functionalities of original PEQ, the V2 update introduces key improvements that refine the filter's behavior and prevent signal distortion due to excessive gain adjustments.

The PEQ V2 block introduces two key improvements over the original version:

1. **Shelf Filter Definition Enhancement** – The Q factor is now used to define shelf filters. The slope parameter remains available for shelf filter configuration in the original PEQ AO.
2. **Auto Rescale Feature** – A new Auto Rescale function has been added to prevent headroom issues caused by EQ boost. It adjusts the highest peaking gain as the rescale gain to reduce the overall gain. Refer to Fig. 1 and Fig. 2 below for the frequency response after rescale gain adjustment.

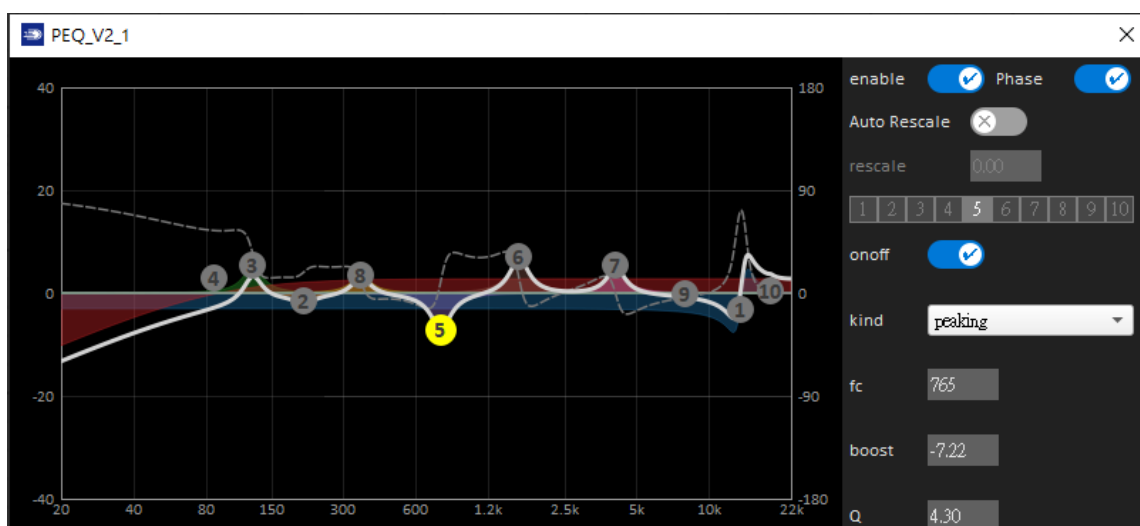


Fig.1 Before Auto Rescale

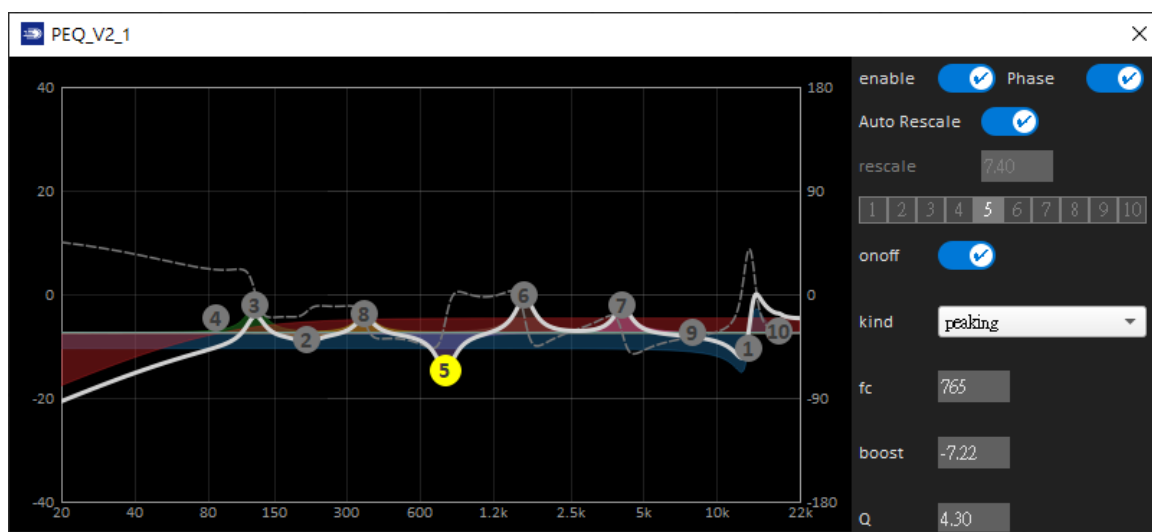


Fig.2 After Auto Rescale

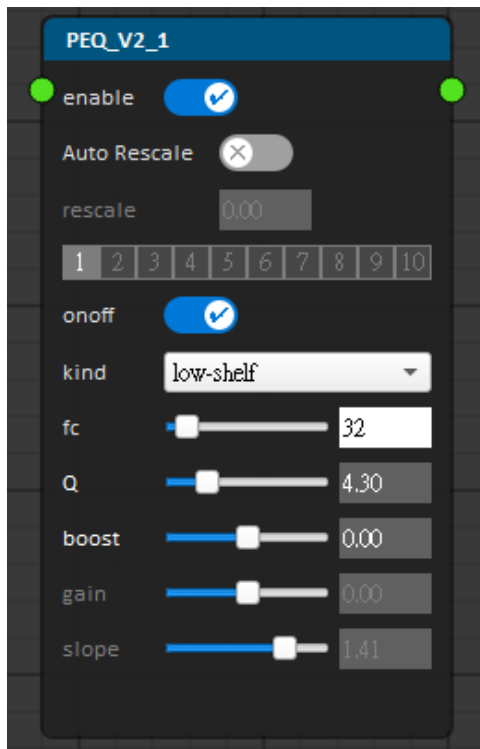
Input & Output Configuration of PEQ V2 object

- **Audio inputs:** 1~32
- **Audio Outputs:** 1~32
- **Band Counts:** 1~10

Adjustable Parameters of PEQ V2 object

- **enable:** Enable or bypass whole PEQ V2 audio object
- **onoff:** Enable or bypasses the corresponding EQ band
- **Auto Rescale:** Enables or bypasses the auto rescale function. The PEQ calculates the highest peaking gain as the rescale gain to reduce the overall gain and prevent headroom issues.
- **type:** Specifies the filter type for the Parametric EQ, including the following options:
 - **low-pass 1 / low-pass 2:** first-order/ second-order low pass filter
 - **high-pass 1 / high-pass 2:** first-order / second-order high pass filter
 - **all-pass 1 / all-pass 2:** first-order / second-order high pass filter
 - **low-shelf / high-shelf:** Boosts or attenuates frequencies below / above a specified cutoff point
 - **parametric:** A fully adjustable band filter for precise frequency control
 - **band-pass:** Allows frequencies within a certain range to pass while attenuating others
 - **peaking:** Boosts or attenuates a selected frequency band
 - **notch:** Attenuates a narrow frequency range to remove unwanted noise or interference
- **frequency:** Defines the cutoff or center frequency for the selected filter type.
- **boost:** Sets the boost or attenuation gain (in dB) at the specified frequency. *(Applicable only to shelving and peaking/parametric filters.)*
- **gain:** Controls the overall gain of the current EQ band.
- **Q:** Sets the Q factor for the filter.
- **slope:** Defines the slope factor for the shelving filter. *(Applicable only when the filter type is set to shelving.)*

Flow Studio GUI of PEQ V2 object



6.3.4 Generic IIR filter

Functional description of Generic IIR filter object

The Generic IIR filter is an IIR filter that directly executes the loaded coefficients, supporting an order of up to 10. Users can import IIR coefficients using .xls, .xlsx, or .csv files in the specified format shown in Fig.1. Column A is the IIR coefficient from b0 to b9 Column B is the IIR coefficient from a0 to a9 e.g. A3 denotes b2, B2 denotes B1. After importing the coefficient, the corresponding frequency and phase response will be displayed, allowing users to easily import their design data.

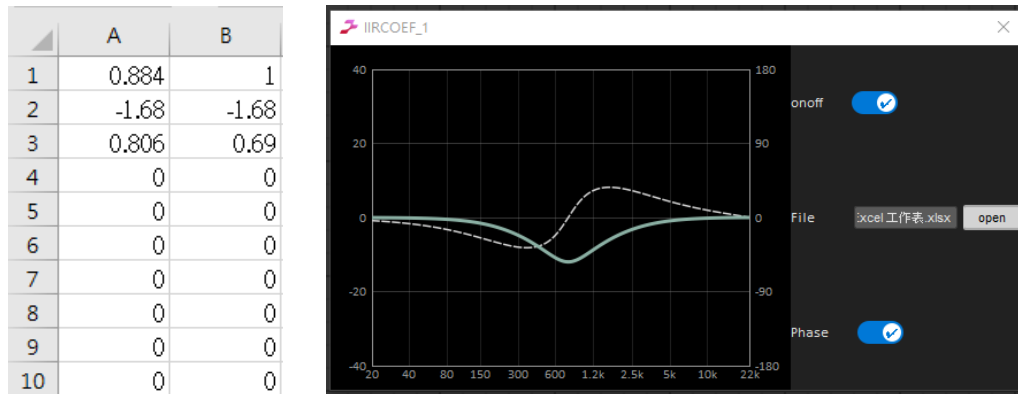


Fig.1 The specified format for IIR filter and the corresponding frequency and phase response.

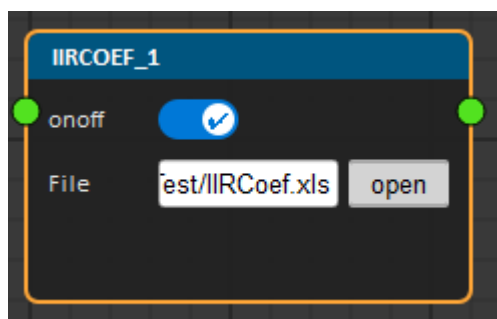
Input & Output Configuration of Generic IIR filter object

- Audio inputs: 1~32
- Audio Outputs: 1~32

Adjustable Parameters of AO

- IIR COEFS: b0~b9 and a0~a9, see Fig.1 for details

Flow Studio GUI of Generic IIR filter object



6.3.5 FIR filter, FIR filter (fixed-point)

Functional description of FIR filter object

As the name suggests, the **FIR filter** audio object (displayed as **FIR** or **FIR_FP** for the fixed-point version in the information tab) provides a Finite Impulse Response (FIR) filter, allowing users to apply precise frequency shaping through manually loaded FIR coefficients. FIR filter is a type of linear time-invariant (LTI) digital filter characterized by a finite-duration impulse response, meaning that the output depends only on a finite number of past input samples. Unlike IIR filters, FIR filters do not use feedback, ensuring inherent stability and linear-phase characteristics, making them ideal for high-fidelity audio processing and signal conditioning.

FIR Coefficient Configuration

- Users must manually input FIR coefficients into the coefficient space within the design window.
- Each coefficient should be placed on a separate line.
- The FIR filter supports a maximum of 1024 taps, providing high-resolution frequency response control. For example, in Fig. 2, *Column A is the FIR coefficient from b0 up to b1023. (e.g. A11 denotes the b10)*
- Once the coefficients are loaded, users can click the Save button to visualize the FIR filter's frequency response.

The transfer function $H(z)$ of the $N+1$ tap filter shown below:

$$H(z) = h_0 + h_1 * z^{-1} + h_2 * z^{-2} \dots h_n * z^{-n}$$

Fig. 1 FIR transform function

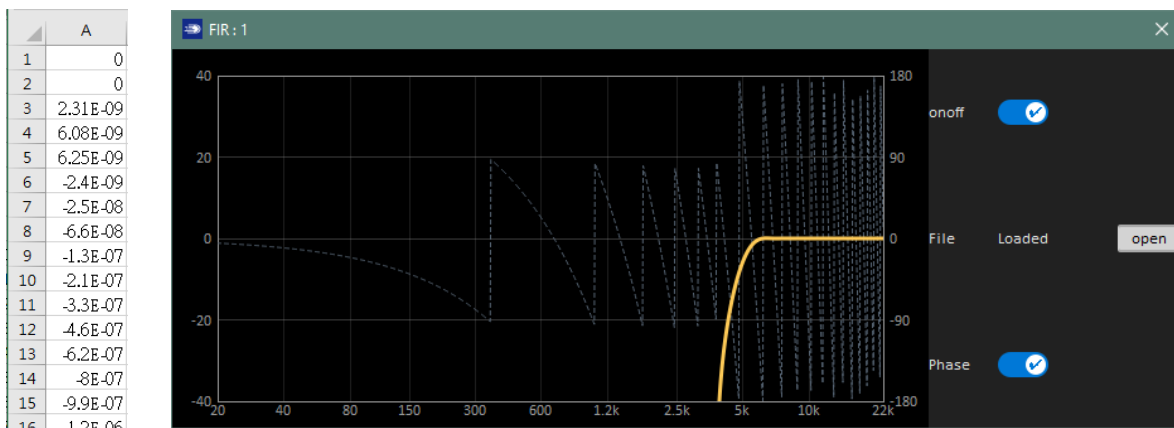


Fig. 2 The specified format for FIR filter and the corresponding frequency and phase response.

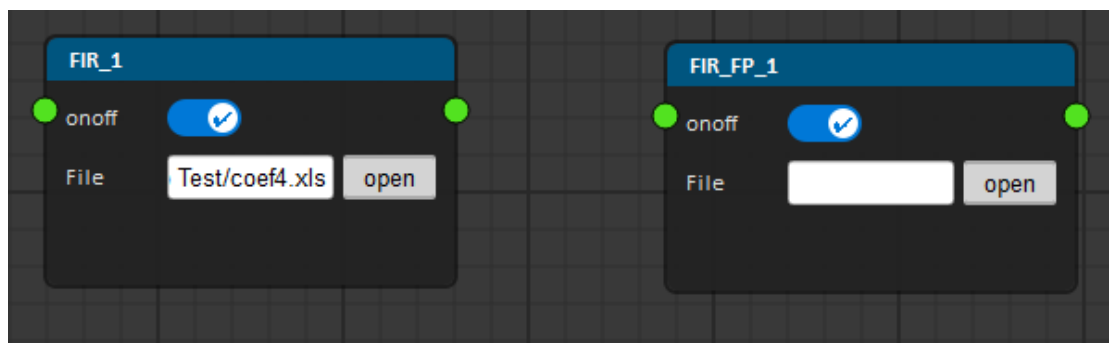
Input & Output Configuration of FIR filter object

- **Audio inputs:** 1~32
- **Audio Outputs:** 1~32

Adjustable Parameters of FIR filter object

- **onoff:** Enables or bypasses the FIR filter process. (Default: "on")
- **FIR:** File path of filter coefficients (b0,b1,b2..., b1023)

Flow Studio GUI of FIR filter object



6.3.6 Low Pass Filter/High Pass filter, Low Pass Filter/High Pass filter (fixed-point)

Functional description of LPF/HPF object

A **Low-Pass Filter (LPF)** object (displayed as **LPF** or **LPF_FP** for the fixed-point version in the information tab) is a filter that allows low-frequency signals to pass through while attenuating high-frequency signals. It is commonly used to remove high-frequency noise from a signal or to preserve the lower frequency components of an audio signal.

A **High-Pass Filter (HPF)** object (displayed as **HPF** or **HPF_FP** for the fixed-point version in the information tab) allows high-frequency signals to pass through while attenuating low-frequency signals. It is typically used to eliminate low-frequency interference or to enhance higher frequency components.

In summary, an LPF filters out high frequencies, while an HPF filters out low frequencies. These two audio objects provide the user with the option to select from a range of high-pass and low-pass filters, including Butterworth, Bessel, and Linkwitz-Riley filter types. The filter order is determined based on the selected filter type

Input & Output Configuration of LPF/HPF object

- **Audio inputs:** 1~32
- **Audio Outputs:** 1~32

Adjustable Parameters of LPF/HPF object

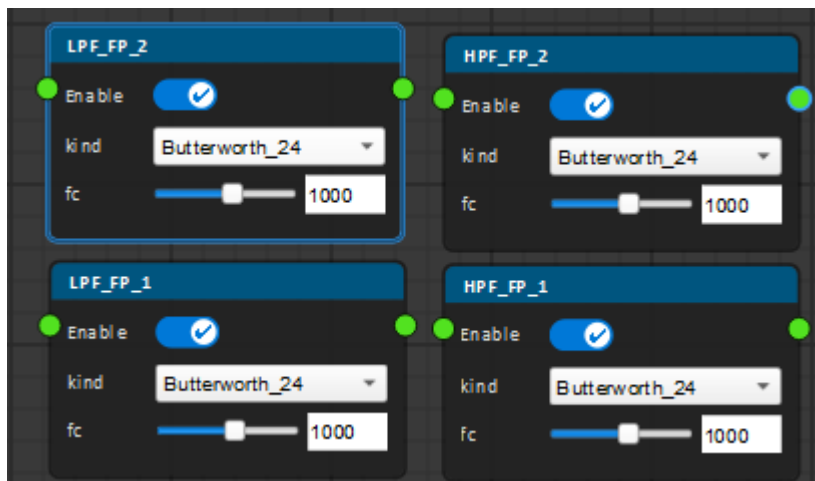
- **Enable:** Enables or bypasses the AO process. (Default: "Enable")
- **fc: cut off frequency of LPF/HPF**
- **type:** Specifies the filter type for the LPF/HPF, including the following options:
 - Butterworth (12, 18, 24, 30, 36 dB/octave)
 - Bessel (6, 12, 18, 24, 30, 36 dB/octave)
 - Linkwitz-Riley (12, 24, 36, 48 dB/octave)

Note: Choosing the Right Filter Type

- **Butterworth Filter** – Offers the smoothest frequency response in the passband, making it ideal for general audio applications where a natural sound is desired.
- **Bessel Filter** – Preserves waveform shape by minimizing phase distortion, making it suitable for time-sensitive applications like audio processing and control systems.
- **Linkwitz-Riley Filter** – Provides a steep roll-off with minimal phase distortion, commonly used in speaker crossovers for seamless frequency blending between drivers.

Each filter type offers a trade-off between roll-off steepness, frequency response smoothness, and phase linearity, allowing users to choose based on their specific audio processing needs.

Flow Studio GUI of LPF/HPF object



6.3.7 2-way crossover, 2-way crossover (fixed-point)

Functional description of 2-way crossover object

The **2-way crossover** audio object (displayed as **XOVER** or **XOVER_FP** for the fixed-point version in the information tab) implements a crossover filter. User can select a high-pass and a low-pass filter from a list of standard IIR high-pass and low-pass filters. The 2-way crossover object outputs two separate signals corresponding to the high-passed and low-passed versions of the input signal.

A crossover filter is a signal processing tool used to split an input signal into different frequency bands. It typically involves a high-pass filter to allow higher frequencies to pass and a low-pass filter to allow lower frequencies to pass. The crossover filter ensures that the appropriate frequency ranges are routed to the corresponding components, such as speakers in a multi-way audio system.

Some use cases of the 2-Way Crossover Audio Object

1. Loudspeaker Systems (Active Crossovers)

- In **high-fidelity audio systems**, crossovers are crucial for splitting signals between **woofers, mid-range drivers, and tweeters**. The **2-way crossover** ensures a clean transition between low and high frequencies, avoiding overlap that could cause phase issues or distortion.

2. Subwoofer Integration

- In home theaters or car audio systems, crossovers help blend the subwoofer with the main speakers by ensuring only low frequencies reach the subwoofer, preventing unwanted muddiness in the mix.

3. Audio Signal Processing & Effects

- Crossovers can be used in sound design and mixing, where different frequency bands are processed separately (e.g., applying reverb to highs while keeping lows dry).

4. Hearing Aids & Assistive Listening Devices

- Some hearing aids use crossover filtering to process different frequency ranges separately, ensuring clear speech intelligibility while reducing unwanted background noise.

By integrating this 2-way crossover audio object, users can design and fine-tune crossover responses efficiently, achieving optimal frequency separation and minimal phase distortion in their audio applications.

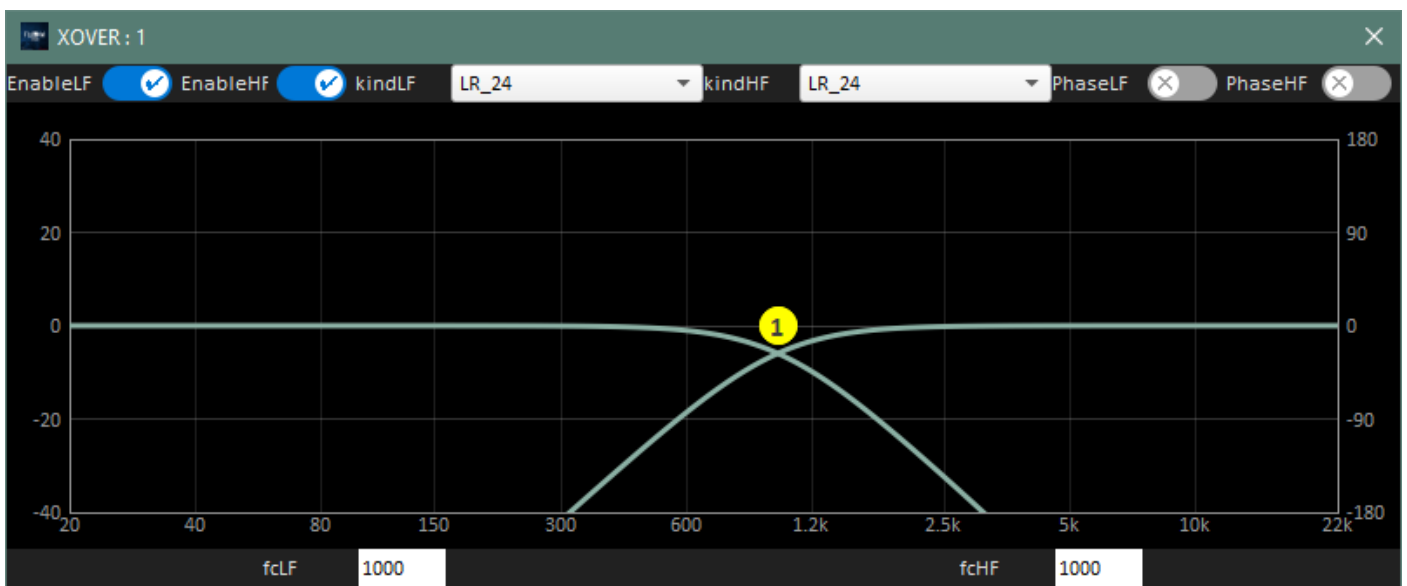
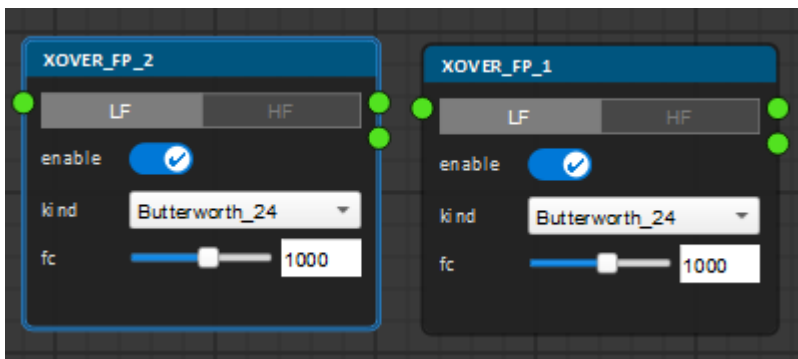
Input & Output Configuration of 2-way crossover object

- **Audio inputs:** 1
- **Audio Outputs:** 2

Adjustable Parameters of 2-way crossover object

- **Enable:** Enables or bypasses the crossover process. (Default: “Enable”)
- **fc:** cut off frequency of LPF/HPF
- **kind:** Specifies the filter type for the LPF/HPF, including the following options:
 - **Butterworth** (12, 18, 24, 30, 36 dB/octave)
 - **Bessel** (6, 12, 18, 24, 30, 36 dB/octave)
 - **Linkwitz-Riley** (12, 24, 36, 48 dB/octave)

Flow Studio GUI of 2-way crossover object



6.3.8 Game EQ

Functional description of Game EQ object

The Game EQ object (displayed as GAME_EQ in the information tab) is a tool that provides a collection of popular equalizer presets tailored for today's most popular games. Users can quickly switch between different EQ profiles to enhance their gaming experience according to the type of game they are playing. Game EQ enables **convenience**, helping users strike a balance between **ready-made presets** and optimized gaming audio.

Input & Output Configuration of Game EQ object

- **Audio inputs:** 1~32
- **Audio Outputs:** 1~32

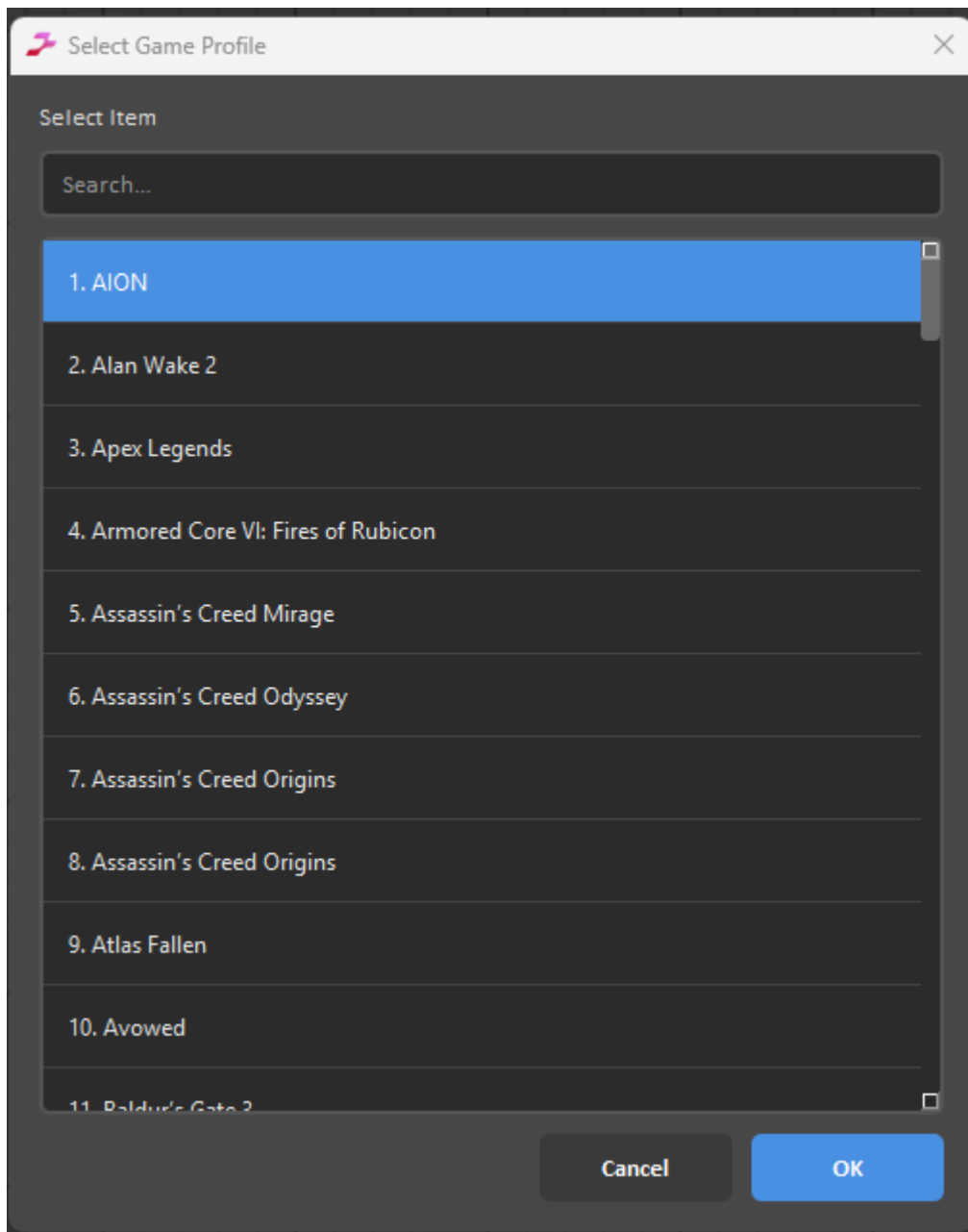
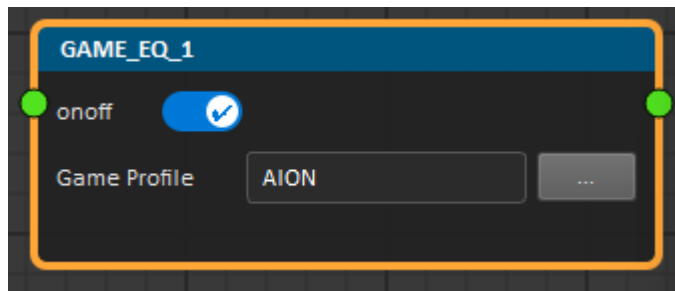
Adjustable Parameters of Game EQ object

- **enable:** Enable or bypass the Game EQ object.
- **Game Profile:** Provides dedicated EQ settings tailored for different games, enhancing immersion and delivering a more engaging in-game experience. Currently supports up to 90 popular titles.

GAME LIST

1	AION	46	Hell Let Loose
2	Alan Wake 2	47	Helldivers 2
3	Apex Legends	48	Hogwarts Legacy
4	Armored Core VI: Fires of Rubicon	49	League of Legends
5	Assassin's Creed Mirage	50	Left 4 Dead 2
6	Assassin's Creed Odyssey	51	Like a Dragon:Infinite Wealth
7	Assassin's Creed Origins	52	Like a Dragon:Pirate Yakuza in Hawaii
8	Assassin's Creed Valhalla	53	Marvel Rivals
9	Atlas Fallen	54	Marvel's Spider-Man 2
10	Avowed	55	Minecraft
11	Baldur's Gate 3	56	Monster Hunter World
12	Battlefield 2042	57	Mortal Kombat 1
13	Black Myth: Wukong	58	NBA 2K24
14	Call of Duty: Modern Warfare II	59	Once Human
15	Call of Duty: Modern Warfare III	60	Palworld
16	Call of Duty: Warzone	61	PEAK
17	CrossFire	62	PUBG:BATTLEGROUNDS
18	CS:GO	63	R.E.P.O.
19	Cyberpunk 2077	64	Raft
20	Darkest Dungeon II	65	Rainbow Six Siege
21	DayZ	66	REMATCH
22	Dead by Daylight	67	Resident Evil Village
23	Delta Force	68	Roblox

24	Destiny 2	69	Rust
25	Diablo IV	70	Silent Hill 2
26	Dota 2	71	Star Wars Battlefront II
27	Dragon Quest III HD-2D Remake	72	Star Wars Jedi: Survivor
28	Elden Ring	73	Star Wars Outlaws
29	Escape from Tarkov	74	Stellar Blade
30	Euro Truck Simulator 2	75	Street Fighter 6
31	Factorio	76	Team Fortress 2
32	Fall Guys	77	The Division 2
33	Fallout 76	78	The Elder Scrolls Online
34	Final Fantasy VII Rebirth	79	The Last Of Us
35	Final Fantasy XIV Online	80	The Last Of Us Part II
36	Final Fantasy XVI	81	The Witcher 3: Wild Hunt
37	Fortnite by FaZe	82	Valorant
38	Forza Horizon 5	83	Vampire: The Masquerade - Bloodhunt
39	Forza Motorsport	84	Warframe
40	Genshin Impact	85	Warhammer 40,000: Boltgun
41	Ghost of Tsushima	86	Warhammer 40,000: Space Marine 2
42	God of War Ragnarök	87	World of Tanks
43	GTA V	88	World of Warcraft
44	Halo Infinite	89	Xdefiant
45	Hearthstone	90	Zenless Zone Zero



6.4 Gains

Overview

Gain control is a fundamental component of audio processing, allowing users to adjust signal levels for optimal loudness, clarity, and balance. Flow Studio provides a variety of **Gains AOs**, enabling both simple volume modifications and more advanced dynamic adjustments. These AOs help prevent distortion, ensure consistent audio levels, and allow for seamless transitions within an audio workflow.

Available Gains AOs:

- **Gain & Gain (Fixed-Point):** The Gain AO adjusts the amplitude of an audio signal, allowing users to increase or decrease volume levels with precision. Fixed-point version is available.
- **Smart Gain:** Dynamically adjusts gain based on input signal characteristics, automatically optimizing loudness while preventing sudden level changes, which is useful for applications requiring adaptive volume control without manual intervention.
- **Mute & Mute (Fixed-Point):** The Mute AO instantly silences an audio signal without affecting its structure, making it useful for bypass controls, temporary signal suppression, or noise gating. Fixed-point version is available.
- **Attenuator:** Reduces signal amplitude without affecting tonal balance, preventing excessive loudness while maintaining audio integrity. Attenuator AO is commonly used for protecting downstream components from signal overload or gradual volume reduction in mixing environments.
- **Loudness Volume:** Designed to adjust volume while compensating for the human ear's sensitivity to different frequencies. Loudness Volume AO is useful in loudness normalization, broadcast audio processing, and adaptive playback systems.
- **Polarity:** Polarity AO inverts the phase of an audio signal by 180 degrees and is useful for phase correction, noise cancellation, and stereo imaging adjustments.
- **Quick Gain:** Quick Gain adjusts the input volume based on the recording result to ensure consistent loudness throughout the session. When you press the record button, the system analyzes the audio for a specified duration and applies the appropriate gain according to the target LUFS.

Gains AOs are essential for managing signal levels, dynamic control, and audio clarity within Flow Studio. Whether applying basic gain adjustments, automatic volume optimization, or phase corrections, these AOs provide precise, reliable, and efficient solutions. By leveraging these tools, users can enhance speech intelligibility, prevent clipping, balance multi-source audio, and create smooth transitions in their processing workflows. With both standard and adaptive gain controls, Flow Studio ensures flexible and high-quality audio management for any application.

6.4.1 Gain, Gain (fixed-point)

Functional description of Gain object

The **Gain** object (displayed as **GAIN** or **GAIN_FP** for the fix-point version in the information tab) is designed to amplify or attenuate an audio signal based on a specified gain value expressed in decibels (dB). It supports both positive and negative gain adjustments, allowing precise control over signal levels. A higher gain value increases the amplitude, while a lower or negative gain value attenuates the signal. To ensure smooth and natural level transitions, the gain adjustment is integrated with a **slew control** mechanism. This prevents abrupt changes that may introduce unwanted artifacts such as clicks, pops, or distortion, particularly when dynamically modifying the gain parameter in real-time. The Gain object is designed for flexible applications, supporting both floating-point and fixed-point arithmetic for optimized performance in resource-constrained environments and high-precision calculations to maintain audio fidelity.

The Gain object is commonly used in digital audio processing pipelines for applications such as volume control, signal normalization, and dynamic range adjustment. It is particularly useful in real-time audio systems where maintaining signal integrity and preventing unwanted artifacts is critical.

Input & Output Configuration of Gain object

- **Audio inputs:** 1~32
- **Audio Outputs:** 1~32

Adjustable Parameters of Gain object

- **gain (dB):** Gain value in dB scale ranging from -120 dB to 0 dB. (Default: 0 dB)

Flow Studio GUI of Gain object



6.4.2 Smart Gain

Functional description of Smart Gain object

The **Smart Gain** object (displayed as **SMART_GAIN** in the information tab) is designed to dynamically adjust the amplitude of an audio signal in real time. By leveraging **Voice Activity Detection (VAD)**, it ensures that only relevant speech or desired audio content is amplified, preventing unnecessary gain adjustments during silent or background noise segments. The Smart Gain object allows for a seamless and consistent audio experience by automatically increasing the signal level when it is too low while also applying compression to prevent excessive loudness. Smart Gain is particularly useful in applications such as voice communications, audio recording, and speech enhancement, where maintaining a clear and balanced output level is essential.

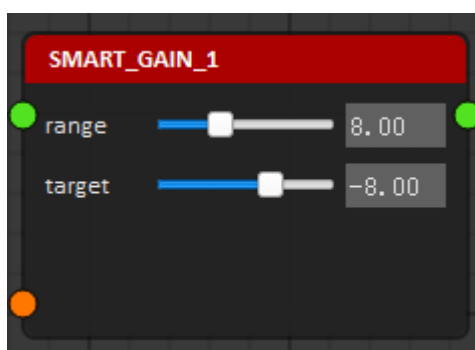
Input & Output Configuration of Smart Gain object

- **Audio inputs:** 1
- **Audio Outputs:** 1
- **Control Inputs:** 1 (external control input for VAD)

Adjustable Parameters of Smart Gain object

- **Range (dB):** Defines the linear gain adjustment area, specifying the maximum gain capability that Smart Gain can apply to boost the input signal. This parameter ensures controlled amplification without excessive signal distortion. Range: 0 ~ 24 dB (Default: 8.0 dB)
- **Target (dB):** Sets the target level for the gating process, ensuring that the maximum dBFS (decibels relative to full scale) value does not exceed a predefined threshold. If the signal amplitude surpasses this target, compression is applied to prevent distortion or clipping, maintaining a balanced and distortion-free audio output. Range: -24.0 ~ 0.0 dB. (Default: -8.0 dB)

Flow Studio GUI of Smart Gain object



6.4.3 Mute, Mute (fixed-point)

Functional description of Mute object

The **Mute** object (displayed as **MUTE** or **MUTE_FP** for the fix-point version in the information tab) is a simple yet effective block designed to enable or disable audio signal output with a smooth transition. It features a **soft ramping mechanism** that gradually attenuates or restores the signal level, preventing unwanted clicks, pops, or abrupt signal drops when toggling the mute state.

The Mute object is useful in various audio applications, including volume control systems, speech processing, and interactive audio environments where seamless muting and unmuting are required.

Input & Output Configuration of Mute object

- **Audio inputs:** 1~32
- **Audio Outputs:** 1~32

Adjustable Parameters of Mute object

- **Onoff:** Enables or bypasses the AO process. (Default: “Enable”)

Flow Studio GUI of Mute object



6.4.4 Attenuator

Functional description of Attenuator object

The **Attenuator** object (displayed as **ATTEN** in the information tab) is designed to reduce the amplitude of an audio signal based on a specified **linear gain value**. It ensures precise volume reduction while maintaining signal integrity. The attenuation process is controlled by a **slew control mechanism**, which smooths out gain transitions to prevent unwanted artifacts such as clicks or abrupt level changes.

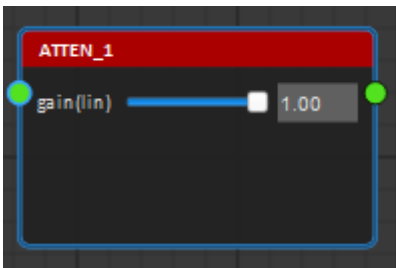
Input & Output Configuration of Attenuator object

- **Audio inputs:** 1~32
- **Audio Outputs:** 1~32

Adjustable Parameters of Attenuator object

- **gain (lin):** Defines the attenuation level in a linear scale, ranging from 0 to 1.0 (where 1.0 represents no attenuation, and 0 represents full attenuation). Notes: “lin” means linear (Default: 1.0)

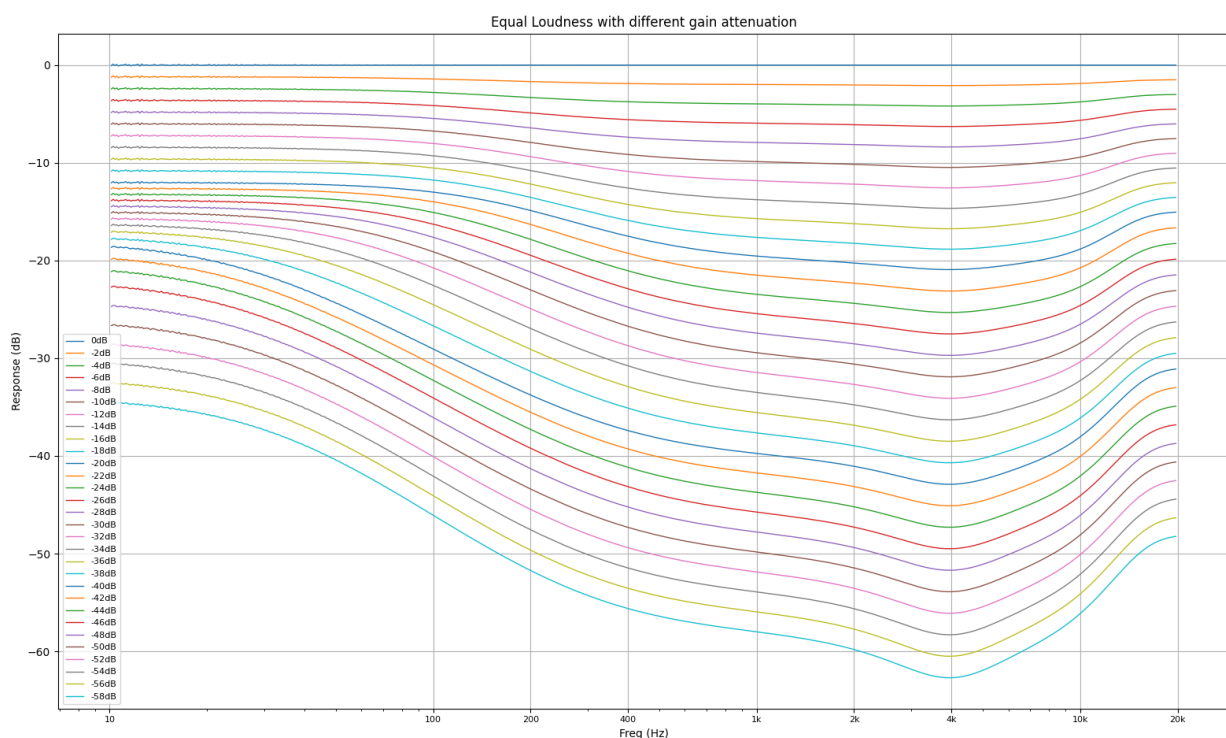
Flow Studio GUI of Attenuator object



6.4.5 Loudness Volume

Functional description of Loudness Volume object

The **Loudness Volume** object (displayed as **LOUDNESS** in the information tab) applies **gain-dependent equal-loudness control**, adjusting the frequency response to align with human auditory perception. This processing enhances the listening experience by compensating for the natural sensitivity variations of the human ear at different loudness levels. The **maxSPL** (Maximum Sound Pressure Level) parameter represents the **maximum measured SPL** when no attenuation is applied (gain = 0 dB). By setting an accurate maxSPL value, the module optimizes loudness perception based on the well-established equal-loudness contours, ensuring a more natural and consistent listening experience across different volume levels. Below is a reference loudness curve when maxSPL = 80 dB, demonstrating how frequency compensation is applied to maintain perceived loudness consistency.



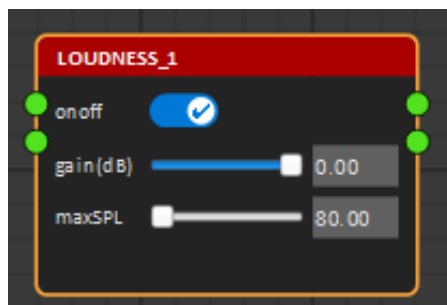
Input & Output Configuration of Loudness Volume object

- **Audio inputs:** 2
- **Audio Outputs:** 2

Adjustable Parameters of Loudness Volume object

- **Onoff:** Enables or bypasses the AO process. (Default: “Enable”)
- **gain (dB):** Adjusts the gain level, ranging from -120 dB to 0 dB. Lower values significantly attenuate the signal, while 0 dB represents the original unaltered level.
- **maxSPL:** Defines the maximum SPL reference for loudness compensation, with an adjustable range of 80 to 100 dB SPL. Proper calibration ensures optimal loudness adaptation based on human hearing characteristics.

Flow Studio GUI of Loudness Volume object



6.4.6 Polarity

Functional description of Polarity object

The **Polarity** object (displayed as **POLARITY** in the information tab) provides a controllable **phase inversion** function, allowing users to invert the phase of an audio signal by **180 degrees** when enabled. This feature is useful in various audio applications, such as phase correction, noise cancellation, and stereo field adjustments. When polarity is enabled, the module inverts the audio signal, effectively flipping its waveform. When disabled, the signal remains unchanged.

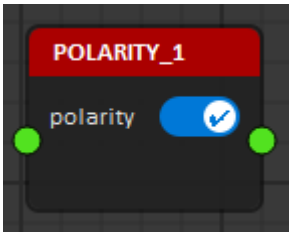
Input & Output Configuration of Polarity object

- **Audio inputs:** 1~32
- **Audio Outputs:** 1~32

Adjustable Parameters of Polarity object

- **Polarity:** Enables or bypasses the polarity inverting process. on: invert , off: normal (Default: “on”)

Flot Studio GUI of Polarity object



6.4.7 Quick Gain

Functional description of Quick Gain object

The **Quick Gain** object (displayed as QUICK_GAIN in the information tab) is a tool designed to detect and adjust input levels based on the LUFS (Loudness Units relative to Full Scale) standard. When the user presses the record button, the system captures the input signal for a predefined duration. Once recording is complete, it automatically calculates and applies the appropriate gain adjustment based on the difference between the measured loudness and the target level.

Quick Gain is especially useful in band rehearsal or multi-channel input scenarios, allowing each input track—such as different instruments—to be quickly leveled to a consistent loudness. Users can achieve an overall balance during the initial setup using Quick Gain, and then make fine adjustments according to personal preferences, significantly improving mixing efficiency and consistency.

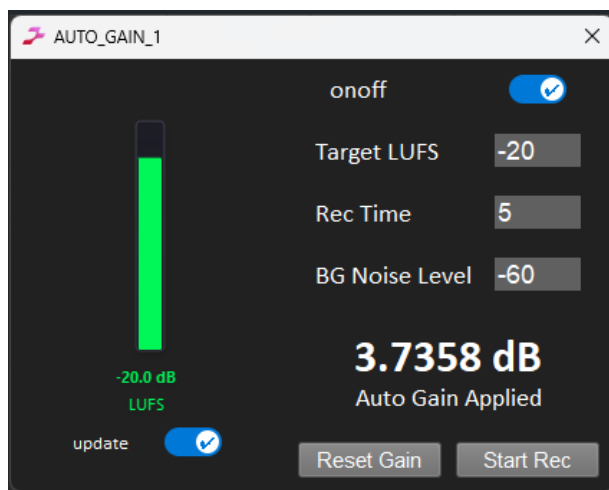
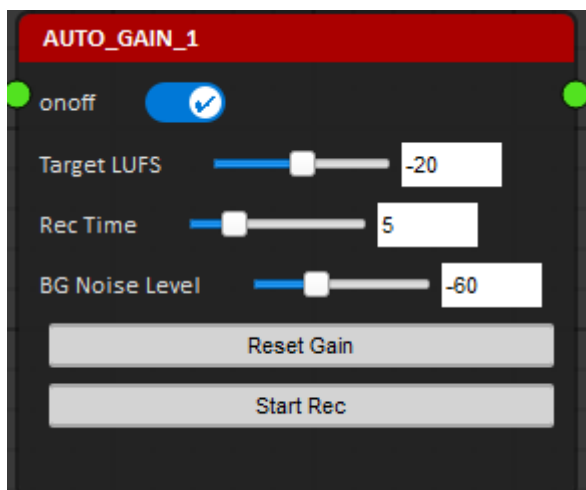
Input & Output Configuration of Quick Gain object

- **Audio inputs:** 1
- **Audio Outputs:** 1

Adjustable Parameters of Quick Gain object

- **onoff:** Enable or bypass the Quick Gain object.
- **Target LUFS:** Sets the target LUFS level. After recording, the system compares the measured loudness with the target and applies a corresponding gain to bring the output closer to the desired level. (min: -40dB, max: 0dB, default: -23dB)
- **Rec Time:** Sets the recording duration. The length of the recording depends on the performer's preference, and the final LUFS value is averaged over this time period, the system uses Auto Start — measurement begins from the first signal that exceeds the background noise threshold. However, if you don't start within the set Rec Time, or if the recorded sound is too short, the system will ask you to record again. (min: 1s, max: 20s, default: 5s)
- **BG Noise Level:** Set the background noise level to prevent amplifying background noise. (min: -70dB, max: -40dB, default: -60dB)
- **Reset Gain:** Resets the gain adjustment back to 0 dB.
- **Start Rec:** Button to start recording.

Flow Studio GUI of Quick Gain object



6.5 Dynamics

Overview

Dynamic processing is crucial for controlling the volume variations and overall sound dynamics of an audio signal. It is used to control loudness consistency, prevent distortion, and shape audio content. Flow Studio offers a range of **Dynamics AOs** that include limiters, compressors, gates, and clippers, each designed to address different aspects of dynamic control. These AOs enable users to manipulate the dynamic range of their signals, whether for subtle volume adjustments or more aggressive sound shaping.

Available Dynamics AOs:

- **Limiter & Limiter (Fixed-Point):** The Limiter AO is used to prevent signal peaks from exceeding a certain threshold, ensuring that the output does not distort. It's essential for maintaining clean, distortion-free audio in professional audio production. Fixed-point version is also provided.
- **2-Band Limiter:** A more specialized limiter that applies limiting to two separate frequency bands, allowing for more fine-grained control over low and high frequencies independently. This is useful in multi-band compression or for separating bass and treble dynamics.
- **3 Section Compressor & 3 Section Compressor (Fixed-Point):** The 3-Section Compressor applies compression across three distinct bands of the frequency spectrum, offering more targeted control over dynamic range. It is typically used in complex dynamic environments like multitrack mixing or broadcasting. Fixed-point version is also provided.
- **Compressor & Compressor (Fixed-Point):** The Compressor AO reduces the dynamic range of a signal by automatically lowering the volume of loud parts and boosting quieter parts. It's useful for balancing voice levels, taming peaks, and tightening up mixes. Fixed-point version is also provided.
- **Smart Compressor:** An intelligent version of the Compressor AO, the Smart Compressor automatically adjusts compression settings based on the input signal's characteristics. This makes it ideal for dynamic content that varies over time, such as speech, music, and live audio processing.
- **Clipper:** The Clipper AO limits the signal to a specified threshold by clipping off any peaks that exceed the level, creating a more aggressive form of dynamic control. This is often used for distortion effects or preventing overload in digital systems.
- **Gate:** The Gate AO allows an audio signal to pass only when it exceeds a certain threshold level, effectively silencing quiet sounds or background noise below the set threshold. It's particularly useful in noise reduction and isolating desired sound sources.
- **Smart Gate:** The Smart Gate functions similarly to the regular gate but uses intelligent algorithms to dynamically adjust the threshold based on the input signal's characteristics. It's ideal for adaptive noise reduction or controlling environmental sounds in live audio settings.

Dynamics AOs are essential tools for controlling the dynamic range of audio signals in Flow Studio, ensuring that signals remain balanced, clear, and free from distortion. By utilizing a combination of limiters, compressors, gates, and clippers, users can shape audio to meet the demands of various processing tasks.

6.5.1 Limiter, Limiter (fixed-point)

Functional description of Limiter object

The **Limiter** object (displayed as **LIMITER** or **LIMITER_FP** for the fixed-point version in the information tab) is a dynamic processor designed to prevent an audio signal from exceeding a predefined level. This implementation utilizes a gain-controlled method, as illustrated in Figure 1.

In essence, the Limiter functions by applying variable gain to a delayed input signal, with the gain value determined by a side-chain processing path that involves level detection and gain calculation. During operation, the level detector continuously monitors the input signal and calculates the required gain reduction when the signal surpasses the threshold. The computed gain is then applied to the delayed input signal, ensuring that the output remains within the desired level.

This architecture allows the Limiter to respond to signal peaks before they reach the output stage, thanks to the input delay. As a result, it achieves transparent limiting with minimal distortion or unwanted artifacts. The side-chain processing path precisely determines the necessary gain reduction, maintaining the maximum output level while preserving as much of the original signal dynamics as possible.

The operational diagram (Figure 2) visually represents how these components interact, providing precise control over signal peaks while minimizing distortion.

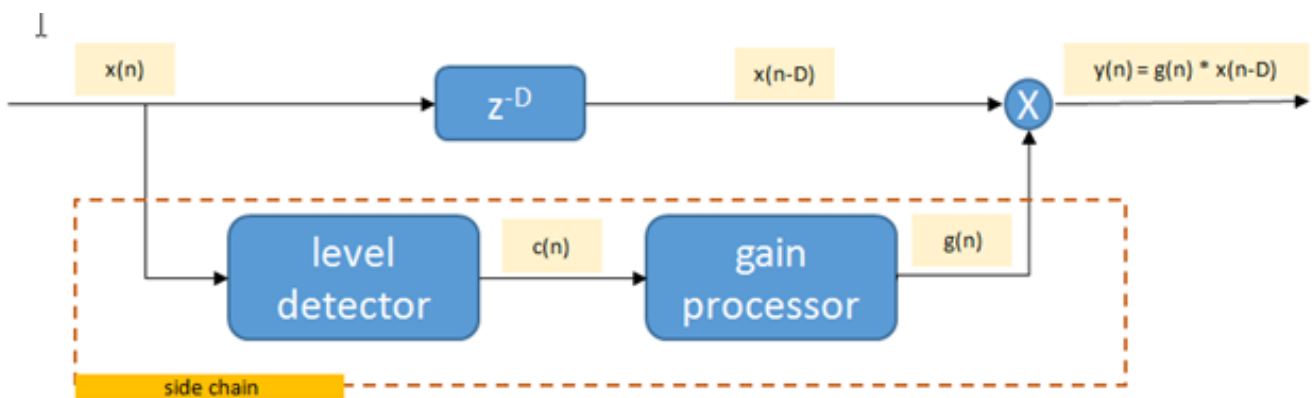


Fig.1 block diagram of limiter

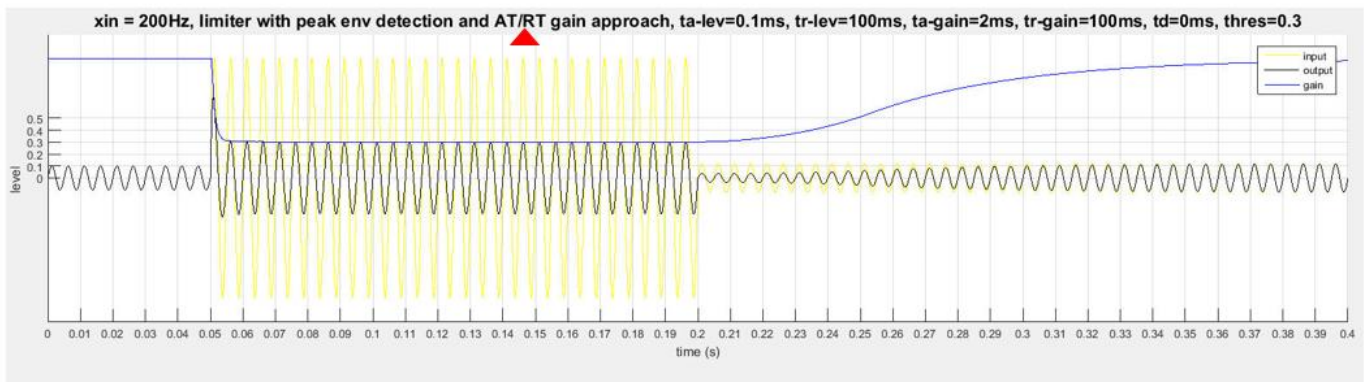


Fig.2 limiter working state diagram

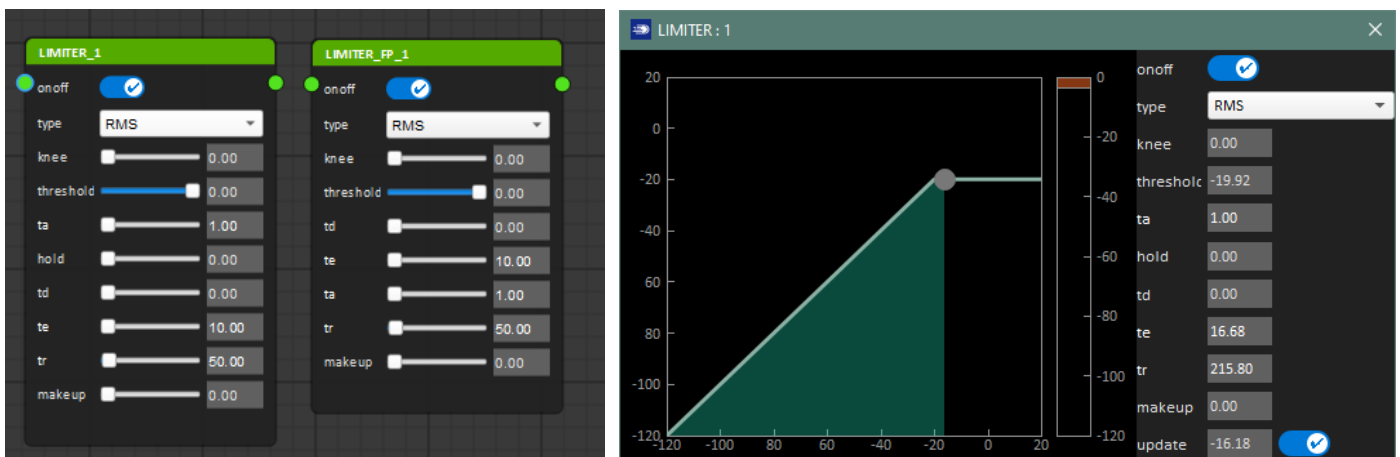
Input & Output Configuration Limiter object

- **Audio inputs:** 1~32
- **Audio Outputs:** 1~32

Adjustable Parameters of Limiter object

- **onoff:** Enables or bypasses the Limiter process. (Default: Enable)
- **type:** Type of level detector, RMS or Peak. (Default: RMS)
- **knee:** Knee parameter determines the smoothness of gain transition when signals cross the threshold. The value controls the width of this smoothing region, creating a more gradual response curve between unlimited and limited states. (Default: 0)
- **threshold:** The threshold is the level at which the limiter activates. When the input signal exceeds this threshold, the limiter applies gain reduction to prevent the output from surpassing the maximum allowed level. (Default: 0)
- **ta:** Attack time, specified in milliseconds, is the time the Limiter object takes to react to signals above the threshold and begin the limiting process. (Default: 1 ms)
- **hold (float version only):** The time (in milliseconds) hold the gain reduction, when signal below the threshold on release phase. (Default: 0 ms)
- **td:** The delay time (in milliseconds) in signal chain for look-ahead feature. (Default: 0 ms)
- **te:** The detecting period (in milliseconds) for RMS energy. (Default: 10 ms)
- **tr:** Release time, specified in milliseconds, is the time the Limiter object takes to restore the gain when the signal drops below the threshold. This parameter is only valid when using RMS detection type. (Default: 50 ms)
- **makeup:** The post gain (in dB) applied to compensate for overall gain reduction, restoring the signal's loudness after limiting. (Default: 0 dB)

Flow Studio GUI of Limiter object



6.5.2 2 Band Limiter

Functional description of 2 Band Limiter object

A **2-band limiter** object (displayed as **LIMITER_MB** in the information tab) splits the audio signal into high and low frequency bands using a built-in crossover, then processes each band through separate limiters. This design allows different attack/release times to be applied to each frequency range, enabling more tailored and responsive processing for each band. Fig.1 is the block diagram explains the above structure.

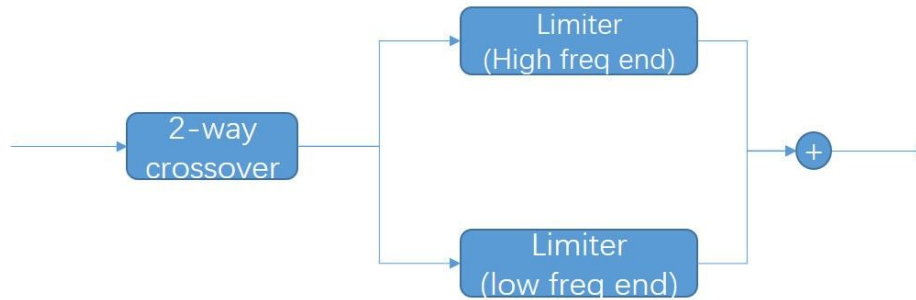


Fig 1. Block diagram of the 2-band limiter

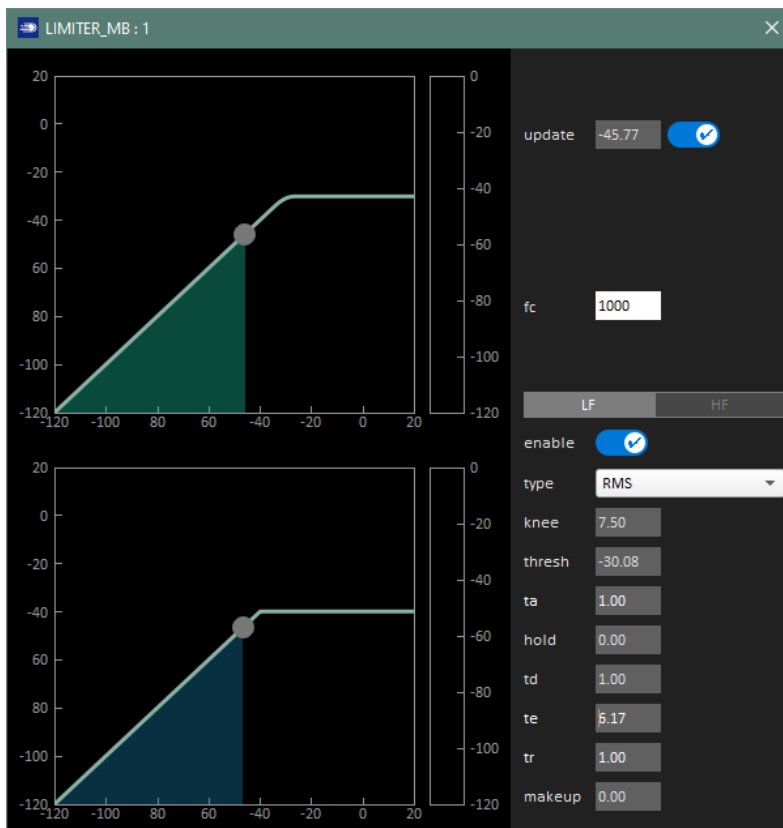
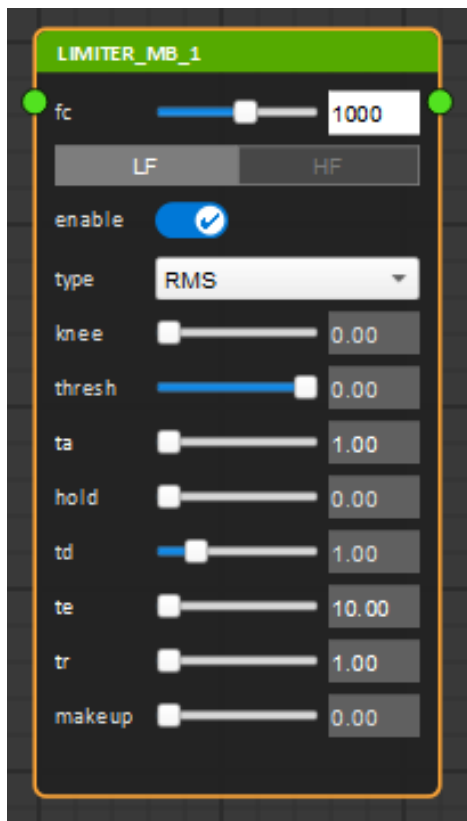
Input & Output Configuration of 2 Band Limiter object

- **Audio inputs:** 1
- **Audio Outputs:** 1

Adjustable Parameters of 2 Band Limiter object

- **fc:** Crossover frequency for each band division. (Default: 1000 Hz)
- **LF and HF tab:** switch between LF band and HF band for the following parameters.
- **enable:** Enables or bypasses the Limiter process of LF or HF band. (Default: Enable)
- **type:** Type of level detector, RMS or Peak. (Default: RMS)
- **knee:** Knee parameter determines the smoothness of gain transition when signals cross the threshold. The value controls the width of this smoothing region, creating a more gradual response curve between unlimited and limited states. (Default: 0)
- **thresh:** The threshold is the level at which the limiter activates. When the input signal exceeds this threshold, the limiter applies gain reduction to prevent the output from surpassing the maximum allowed level. (Default: 0 dB)
- **ta:** Attack time, specified in milliseconds, is the time the limiter takes to react to signals above the threshold and begin limiting. (Default: 1 ms)
- **hold:** The time hold the gain reduction, when signal below the threshold on release phase (Note: float version only) (Default: 0)
- **td:** The delay time in signal chain for look-ahead feature. (Default: 1 ms)
- **te:** The detecting period for RMS energy. (Default: 1 ms)
- **tr:** Release time, specified in milliseconds, is the time the limiter takes to restore the gain when the signal drops below the threshold. This parameter is only valid when using RMS detection type. (Default: 1 ms)
- **makeup:** The post gain applied to compensate for overall gain reduction, restoring the signal's loudness after limiting. (Default: 0 dB)

Flow Studio GUI of 2 Band Limiter object



6.5.3 3 section Compressor, 3 section Compressor (fixed-point)

Functional description of 3 section Compressor object

The **3 section Compressor** object (displayed as **COMP_Combo** or **COMP_Combo_FP** for the fixed-point version in the information tab) offers sophisticated three-section dynamic range control (DRC) with independently adjustable compression ratios for each section. Users can select their preferred level detection method—peak, RMS, or a customized blend of both—to precisely capture the signal characteristics.

At its core, this compressor applies a smoothed gain reduction to the output signal based on a logarithmic ratio of output to input levels. When this ratio exceeds 1, the device functions as a compressor, attenuating signals that surpass the threshold. Conversely, when the ratio falls below 1, it operates as an expander, enhancing dynamic range by reducing lower-level signals. At extreme settings with ratios of 10 or higher, the device effectively transforms into a limiter, preventing signals from exceeding a defined ceiling. Fig. 1 is an example of the gain response curve.

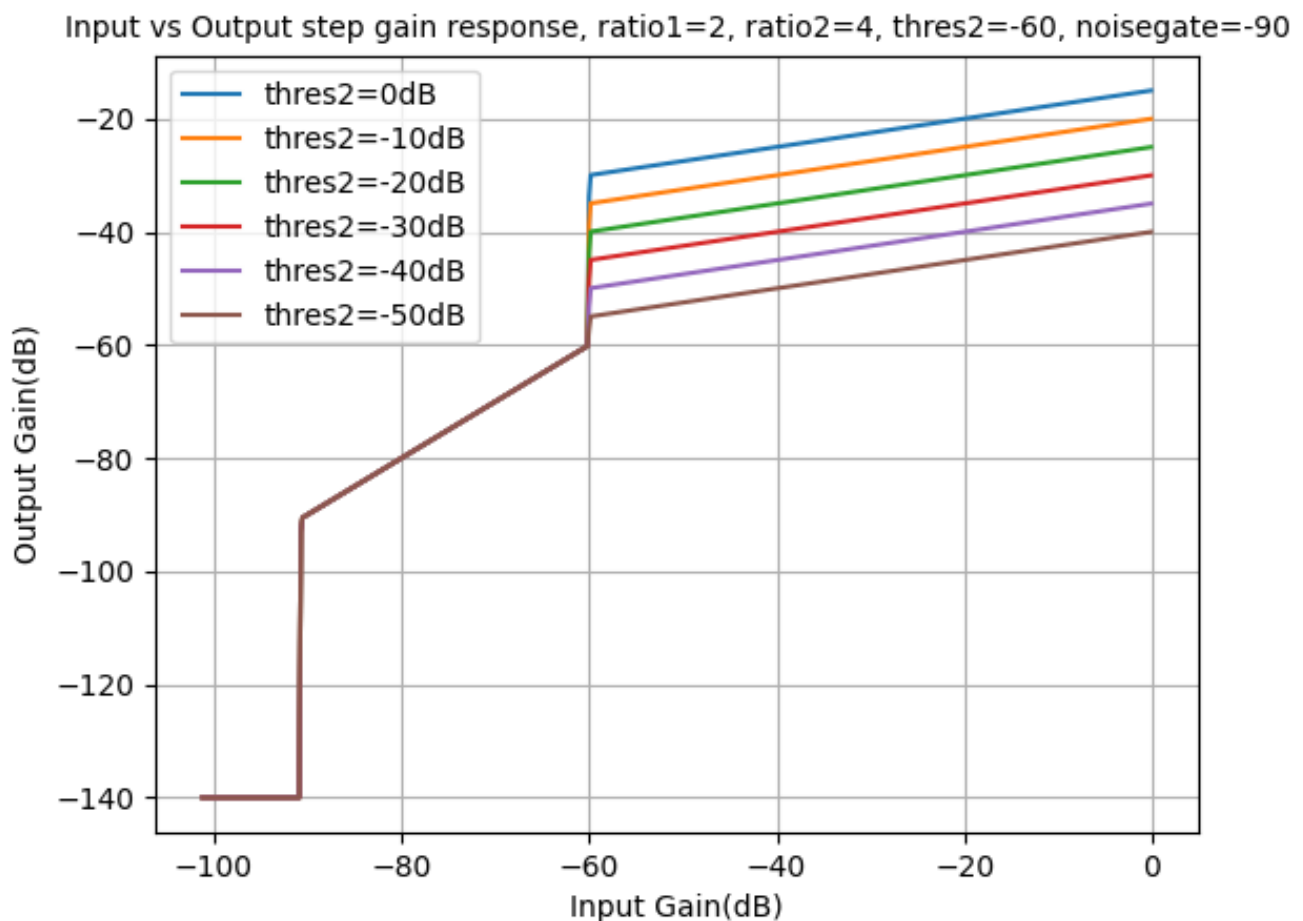


Fig.1 Gain response curve of 3-section compressor

Input & Output Configuration of 3 section Compressor object

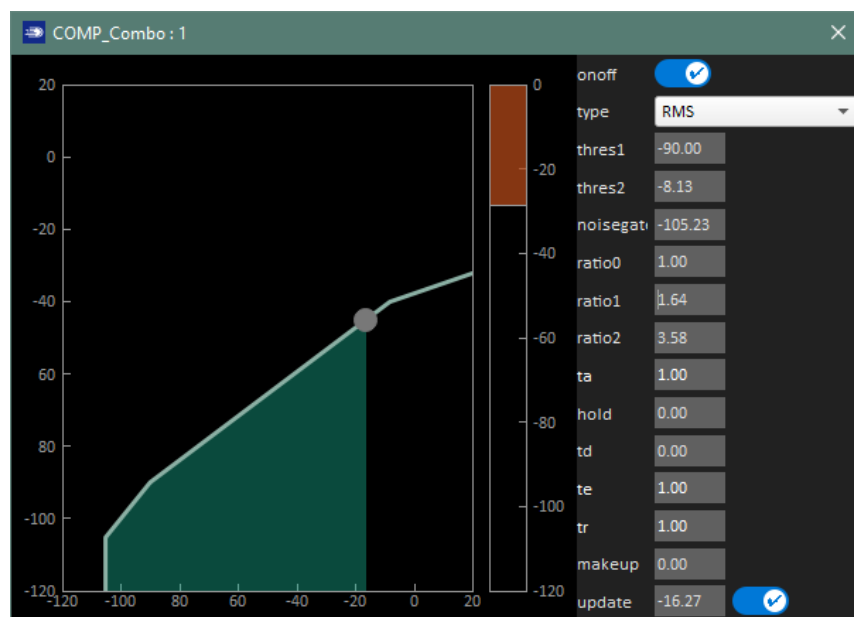
- Audio inputs: 1~32
- Audio Outputs: 1~32

Adjustable Parameters of 3 section Compressor object

- **onoff**: Enables or bypasses the compressor. (Default: "Enable")
- **level detection type**: Choose between RMS, peak, or a customized blend for signal level detection.
- **noise gate**: Sets the minimum level for signal detection. Signals below this threshold will be muted.
- **thres1**: Defines the first threshold level. Signals between the noise gate and Thres1 will be processed in section 1 with ratio0.
- **thres2**: Defines the second threshold level. Signals between thres1 and thres2 will be processed in section 2 with Ratio1. Signals above thres2 will be processed in section 3 with ratio2.
- **ratio0**: Compression ratio for section 1.
- **ratio1**: Compression ratio for section 2.
- **ratio2**: Compression ratio for section 3.
- **ta**: Attack time in milliseconds. Determines how quickly the compressor responds to signals exceeding the threshold.
- **tr**: Release time in milliseconds. Controls how quickly the compressor restores gain after the signal falls below the threshold.
- **te**: Detection period for RMS energy calculation.
- **hold**: Duration to maintain gain reduction after the signal drops below the threshold (during release phase).
- **makeup**: Post-compression gain control to compensate for overall volume reduction.
- **td**: Delay time in the signal chain for look-ahead functionality, allowing the compressor to react before transients occur.

Flow Studio GUI of 3 section Compressor object





6.5.4 Compressor, Compressor (fixed-point)

Functional description of Compressor object

The **Compressor** object (displayed as **COMP** or **COMP_FP** for the fixed-point version in the information tab) is a standard compressor that controls the output dynamic range by applying smoothed gain to the output signal with a ratio that follows an algorithmic relationship between output and input levels. When the ratio exceeds 1, it functions as a compressor, reducing the dynamic range by attenuating signals that surpass the threshold. When the ratio is less than 1, it operates as an expander, increasing dynamic range by further reducing lower-level signals. At extreme settings with ratios of 10 or higher, the device effectively becomes a limiter, preventing signals from exceeding a defined ceiling. Additionally, the compressor incorporates a knee parameter around the threshold for smooth, gradual transition into compression rather than abrupt gain reduction. This creates a more natural sound, especially for musical material. The threshold and ratio settings determine the specific gain response curve, as illustrated in Fig.1.

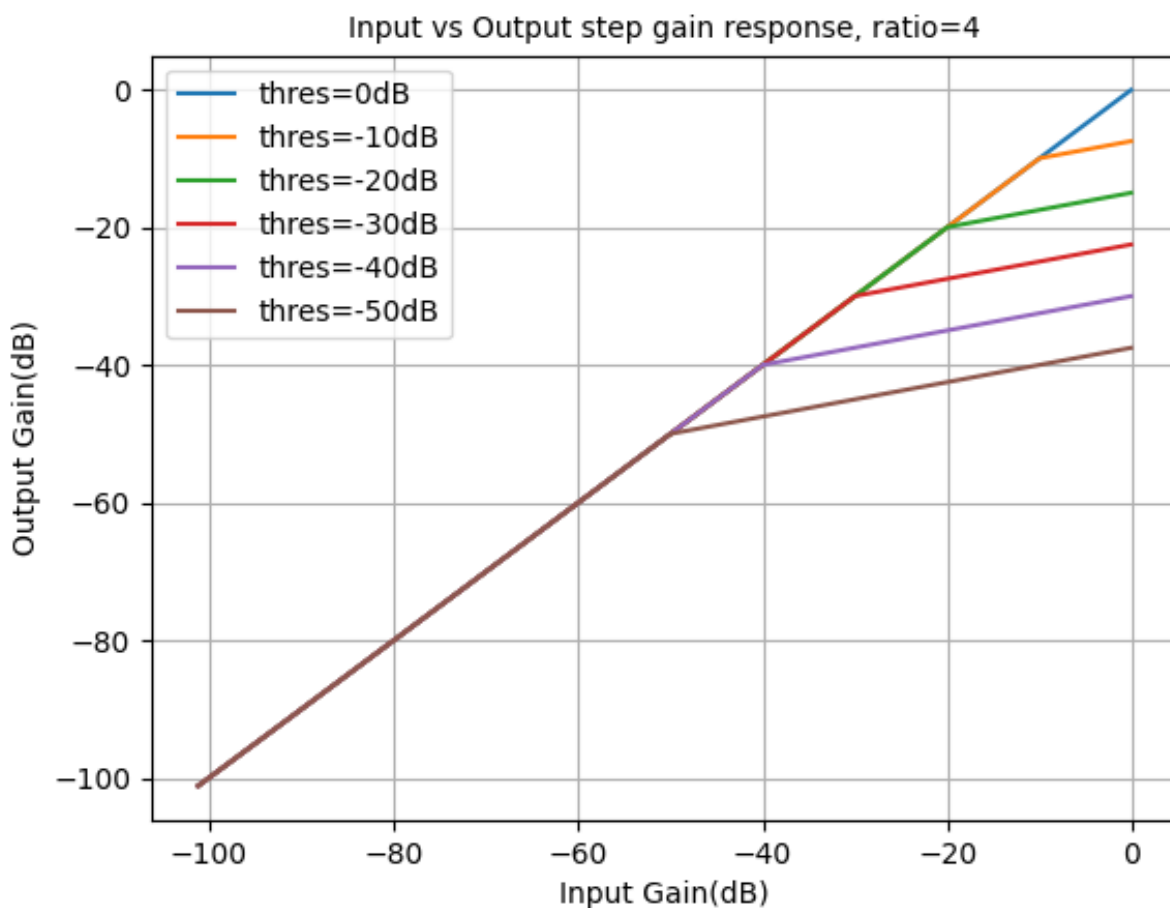


Fig.1 Gain response curve of compressor

Input & Output Configuration of Compressor object

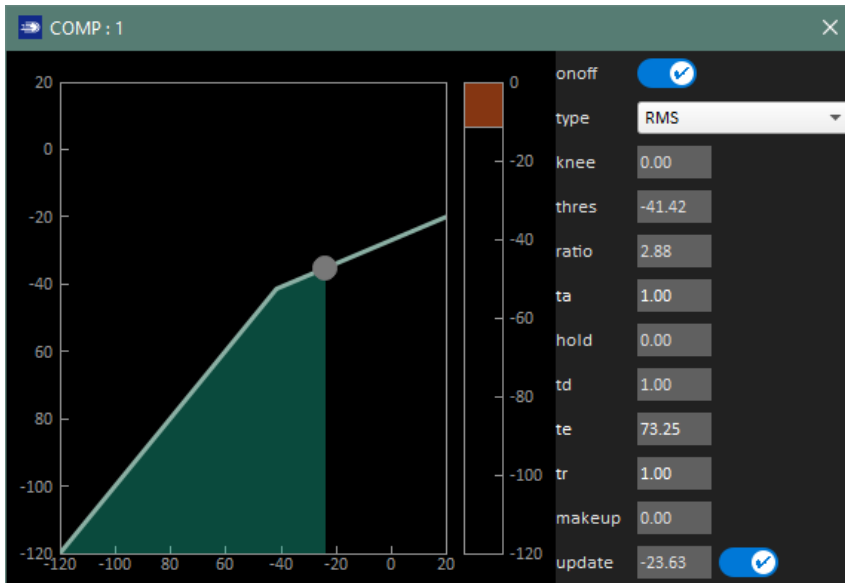
- Audio inputs: 1~32
- Audio Outputs: 1~32

Adjustable Parameters of Compressor object

- **onoff**: Enables or bypasses the compressor. (Default: "Enable")
- **type**: Choose between RMS, peak, or a combination of both for signal level detection.
- **thres**: Sets the level above which compression begins. The compressor activates when the detected signal exceeds this threshold.
- **Ratio**: Determines the amount of compression applied to signals above the threshold. Values greater than 1 compress the signal, less than 1 expand it, and values around 10 or higher effectively limit the signal.
- **ta**: Attack time in milliseconds. Controls how quickly the compressor responds when signals exceed the threshold.
- **tr**: Release time in milliseconds. Determines how quickly the compressor restores normal gain after the signal falls below the threshold.
- **te**: Defines the time period used for RMS energy detection.
- **td**: Delay time in the signal chain enabling look-ahead functionality, allowing the compressor to anticipate and react to transients before they occur.
- **knee**: Controls the smoothness of the transition at the threshold point. The value determines the width of the transition region, creating a more natural-sounding compression curve.
- **hold (float version only)**: Duration to maintain gain reduction after the signal drops below the threshold, before beginning the release phase.
- **makeup**: Post-compression gain control to compensate for volume reduction caused by compression, restoring overall signal level.

Flow Studio GUI of Compressor object





6.5.5 Smart Compressor

Functional description of Smart Compressor object

Based on the standard compressor, the **Smart Compressor** object (displayed as **AUTO_COMP** in the information tab) introduces an innovative auto mode for attack time, release time, and makeup gain. Instead of requiring fixed user settings or acoustic tuning, the auto attack and release time features implement adaptive timing that responds to the signal's characteristics. This approach produces more natural-sounding compression with fewer artifacts, addressing the limitation that fixed settings rarely suit all signal types effectively. The auto makeup gain feature intelligently restores perceptual loudness after compression. By performing real-time loudness estimation on both input and output signals, the Smart Compressor determines the precise makeup gain needed to compensate for loudness reduction, maintaining consistent perceived volume while preserving the benefits of dynamic range control.

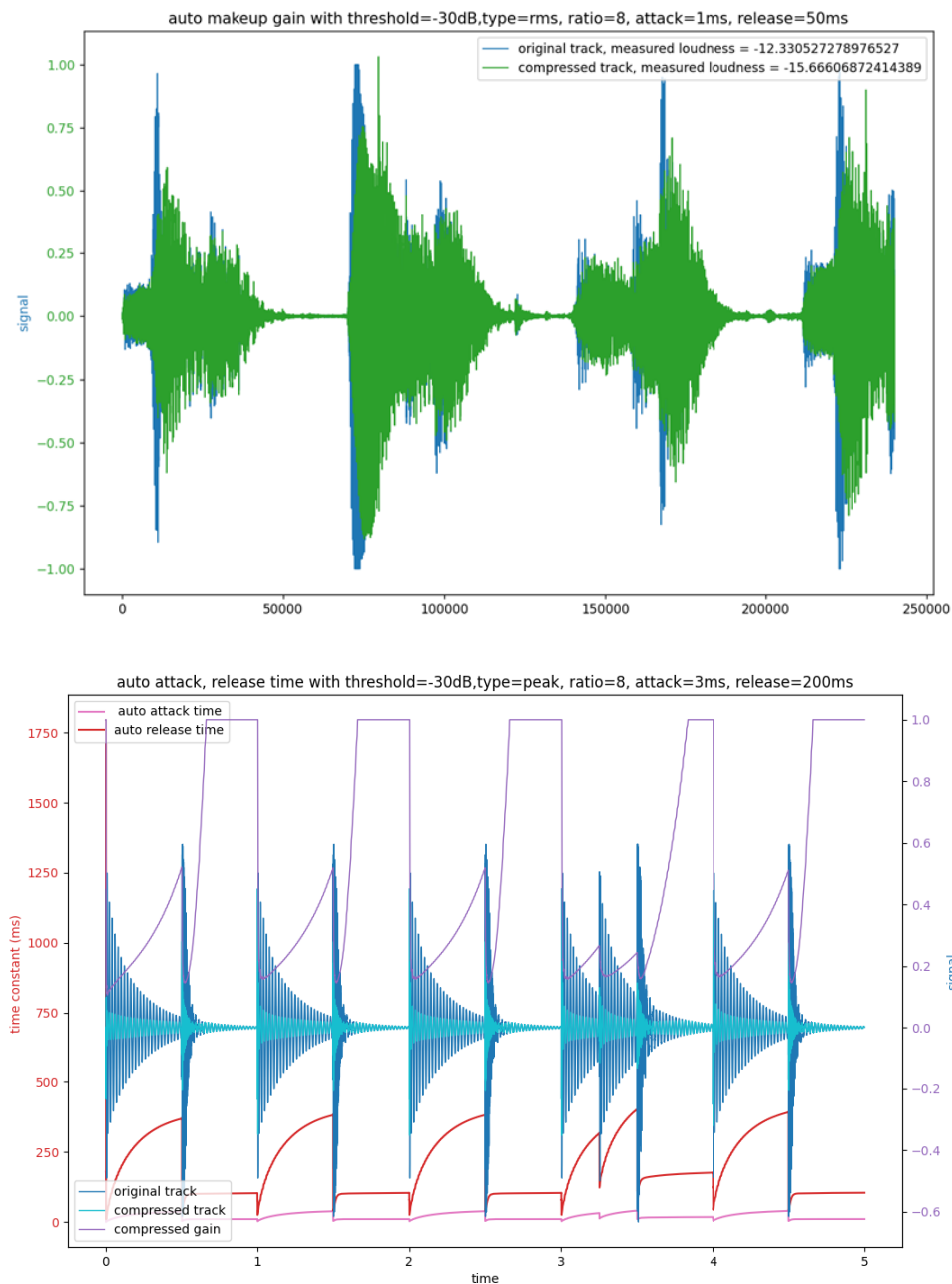


Fig1. Smart compressor response diagram

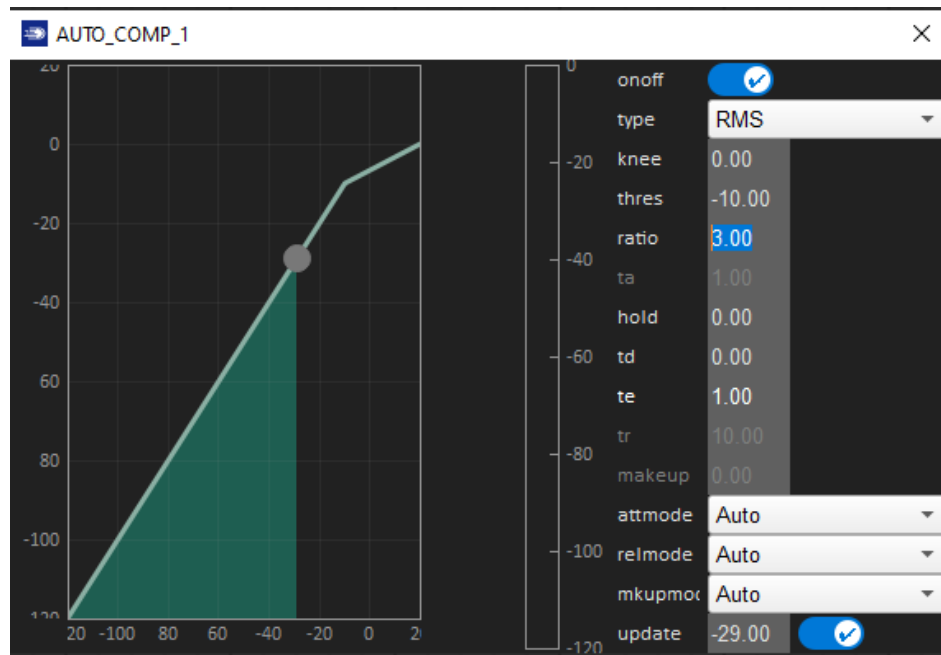
Input & Output Configuration of Smart Compressor object

- **Audio inputs:** 1~32
- **Audio Outputs:** 1~32

Adjustable Parameters of Smart Compressor object

- **onoff:** Enables or bypasses the smart compressor process. (Default: "Enable")
- **type:** choose between rms, peak, or a combination of both for signal level detection.
- **knee:** controls the smoothness of the transition at the threshold point. the value determines the width of the transition region, creating a more natural-sounding compression curve.
- **thres:** sets the level above which compression begins. the compressor activates when the detected signal exceeds this threshold.
- **ratio:** determines the amount of compression applied to signals above the threshold. values greater than 1 compress the signal, less than 1 expand it, and values around 10 or higher effectively limit the signal.
- **ta:** attack time in milliseconds. controls how quickly the compressor responds when signals exceed the threshold. Only available when attmode is set to Fixed mode. (Default: 1 ms)
- **hold:** duration to maintain gain reduction after the signal drops below the threshold, before beginning the release phase.
- **tr:** release time in milliseconds. determines how quickly the compressor restores normal gain after the signal falls below the threshold. Only available when relmode is set to Fixed mode. (Default: 10 ms)
- **te:** defines the time period used for rms energy detection.
- **td:** delay time in the signal chain enabling look-ahead functionality, allowing the compressor to anticipate and react to transients before they occur.
- **makeup:** post-compression gain control to compensate for volume reduction caused by compression. Only available when mkupmode is set to Fixed mode. (Default: 0 dB)
- **attmode:** select between fixed or auto attack time. auto mode adaptively controls how quickly compression is applied when signals exceed the threshold based on signal characteristics. (Default: Auto)
- **relmode:** select between fixed or auto release time. auto mode adaptively controls how quickly normal gain is restored when signals fall below the threshold based on signal characteristics. (Default: Auto)
- **mkupmode:** select between fixed or auto makeup gain. auto mode automatically restores gain to maintain equivalent perceived loudness between input and output signals. (Default: Auto)

Flow Studio GUI of Smart Compressor object



6.5.6 Clipper

Functional description of Clipper object

The **Clipper** object (displayed as **CLIPPER** in the information tab) creates a hard ceiling for the audio signal, preventing any samples from exceeding the defined maximum amplitude. Unlike soft clippers or limiters that may apply a gradual transition or ratio-based processing, hard clipping implements an immediate, non-negotiable amplitude ceiling, resulting in a characteristic distortion when driven hard.

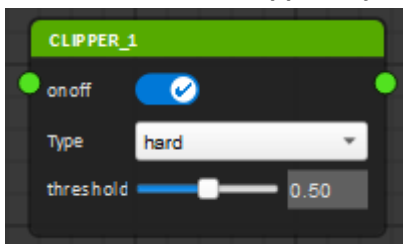
Input & Output Configuration of Clipper object

- **Audio inputs:** 1~32
- **Audio Outputs:** 1~32

Adjustable Parameters of Clipper object

- **onoff:** Enables or bypasses the Clipper process. (Default: on)
- **Type:** select clip type, soft or hard. (Default: hard)
- **threshold:** To set the signal level where it clips. When the absolute value of the input signal exceeds this level, the output is limited precisely to this threshold value. (Default: 0.5)

Flow Studio GUI of Clipper object



6.5.7 Gate

Functional description of Gate object

The **Gate** object (displayed as **GATE** in the information tab) attenuates the volume of quiet sounds that fall below a specified threshold. Unlike simple noise gates that create abrupt on/off transitions, this implementation applies carefully controlled attack, release, and hold times to generate a smooth gain curve. This results in natural-sounding volume reduction for lower-level signals while minimizing unwanted artifacts like pumping or clicking. The Gate object is particularly effective for cleaning up noisy recordings, isolating instruments in multi-track productions, or shaping the decay of percussive sounds.

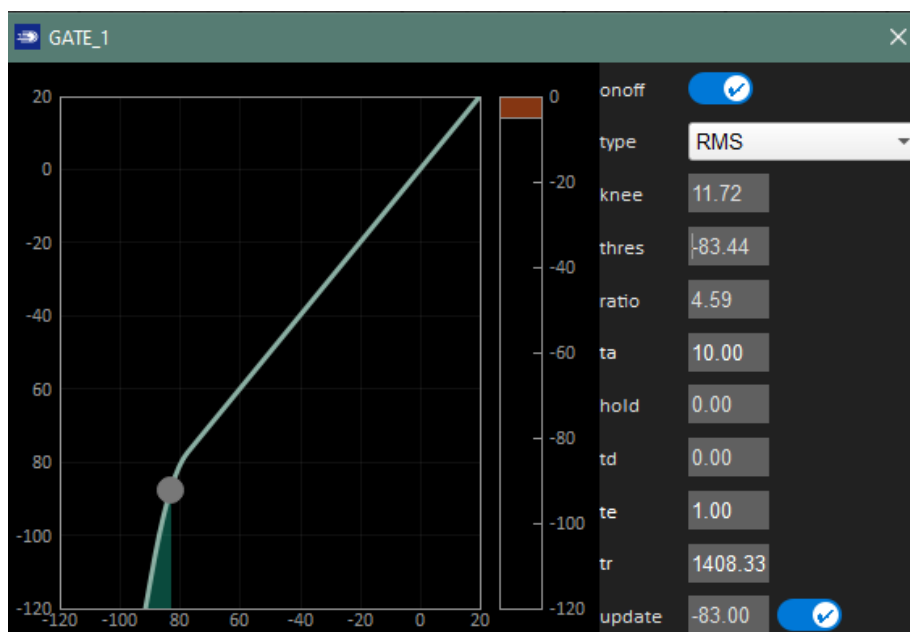
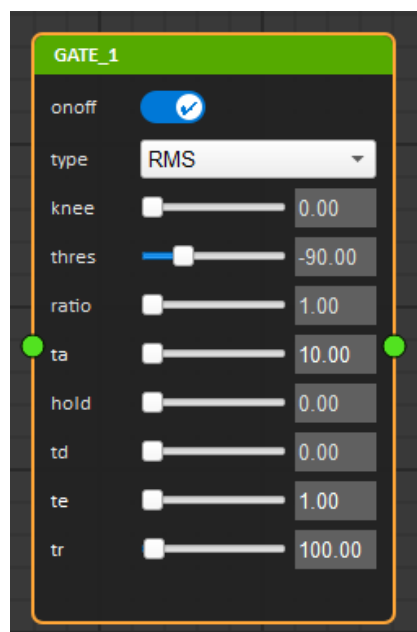
Input & Output Configuration of Gate object

- **Audio inputs:** 1~32
- **Audio Outputs:** 1~32

Adjustable Parameters of Gate object

- **onoff:** Enables or bypasses the Gate process. (Default: Enable)
- **type:** choose between RMS, Peak, or a combination of both for signal level detection. (Default: RMS)
- **knee:** controls the smoothness of the transition at the threshold point. the value determines the width of the transition region, creating more natural-sounding gating without abrupt changes. (Default: 0)
- **thres:** sets the level below which gating begins. the gate activates when the detected signal falls below this threshold. (Default: -90 dB)
- **ratio:** determines the expansion ratio applied to signals below the threshold, controlling how much quieter these signals become. (Default: 1)
- **ta:** attack time in milliseconds. controls how quickly the gate opens when signals exceed the threshold, allowing the signal to pass through. (Default: 10 ms)
- **hold:** duration to maintain the gate's state after the signal crosses the threshold, before beginning the release phase. prevents rapid gate flickering with signals hovering near the threshold. (Default: 0)
- **td:** delay time in the signal chain enabling look-ahead functionality, allowing the gate to anticipate and react to changes in signal level before they occur. (Default: 0 ms)
- **te:** defines the time period used for rms energy detection. (Default: 1 ms)
- **tr:** release time in milliseconds. determines how quickly the gate closes when signals fall below the threshold, attenuating the sound. (Default: 100 ms)

Flow Studio GUI of Gate object



6.5.8 Smart Gate

Functional description of Smart Gate object

Smart Gate object (displayed as **SMART_GATE** in the information tab) utilizes Voice Activity Detection (VAD) to identify vocal signals in the audio stream. During periods when non-vocal content is detected, it automatically adjusts the threshold level to optimize performance.

The gate attenuates the volume of sounds that fall below the specified threshold, applying the user-defined attack, release, and hold times to create a smooth gain curve. This approach ensures natural transitions rather than abrupt on/off switching, minimizing artifacts like clicking or pumping that can occur with conventional gates.

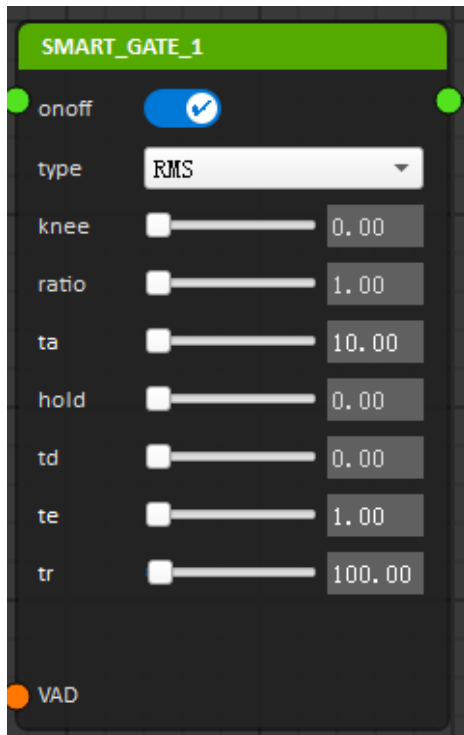
Input & Output Configuration of Smart Gate object

- **Audio inputs:** 1~32
- **Audio Outputs:** 1~32

Adjustable Parameters of Smart Gate object

- **onoff:** Enables or bypasses the Smart Gate process. (Default: Enable)
- **type:** choose between RMS, Peak, or a combination of both for signal level detection. (Default: RMS)
- **knee:** controls the smoothness of the transition at the threshold point. the value determines the width of the transition region, creating more natural-sounding gating without abrupt changes. (Default: 0)
- **ratio:** determines the expansion ratio applied to signals below the threshold, controlling how much quieter these signals become. (Default: 1)
- **ta:** attack time in milliseconds. controls how quickly the gate opens when signals exceed the threshold, allowing the signal to pass through. (Default: 10 ms)
- **hold:** duration to maintain the gate's state after the signal crosses the threshold, before beginning the release phase. prevents rapid gate flickering with signals hovering near the threshold. (Default: 0)
- **td:** delay time in the signal chain enabling look-ahead functionality, allowing the gate to anticipate and react to changes in signal level before they occur. (Default: 0 ms)
- **te:** defines the time period used for rms energy detection. (Default: 1 ms)
- **tr:** release time in milliseconds. determines how quickly the gate closes when signals fall below the threshold, attenuating the sound. (Default: 100 ms)

Flow Studio GUI of Smart Gate object



6.6 Routings & Mixing

Overview

Effective routing and mixing are essential for blending multiple audio signals and controlling how they interact with each other. These processes allow users to design complex audio flows, ensuring signals are directed and combined in precise ways. Flow Studio offers a set of **Routings & Mixing AOs** that facilitate efficient signal routing, blending, and volume control. These AOs allow users to create custom signal paths, manage the balance between different sources, and implement real-time adjustments to the mix.

Available Routings & Mixing AOs:

- **MERGER:** The **Merger AO** combines 2 input signals into a single output. This is useful for grouping signals, merging parallel audio paths, or combining different audio sources into one unified signal. It simplifies the process of combining outputs from multiple processing units into a final mixed signal.
- **Multiplexer & Multiplexer (Fixed-Point):** The Multiplexer AO selects one input from multiple possible sources based on a control signal, allowing users to route audio from different sources to the same destination. The fixed-point version is available.
- **Dry Wet Fader:** The Dry Wet Fader AO controls the balance between the original (dry) signal and the processed (wet) signal, often used in effects processing, creating a seamless blend between the two. It's widely used in reverb, delay, or modulation effects where the effect strength needs to be controlled in real time.
- **Mixer & Mixer (Fixed-Point):** The Mixer AO allows users to combine multiple input signals into one or more output signals, adjusting individual channel volumes, panning, and other mix parameters. It is the core tool for multi-source mixing and level balancing in any audio workflow. The fixed-point version is available.

The Routings & Mixing AOs are crucial for managing and blending multiple audio signals within Flow Studio. By using these AOs, users can design flexible signal paths, merge multiple sources, and control the relationship between dry and processed signals in real time. Whether users are mixing tracks, routing signals dynamically, or adjusting effect levels, these AOs provide the precision and flexibility needed to create custom audio workflows. With the ability to control complex signal interactions and adjust levels effectively, Flow Studio enables users to achieve professional-quality mixes and routing configurations in any audio application.

6.6.1 MERGER

Functional description of Merger object

The **Merger** object (displayed as **MERGER** in the information tab) combines two audio inputs by summing their signals and then **normalizing** the output by dividing the sum of the number of inputs, which prevents signal overflow and ensures that the output remains within a controlled amplitude range. Unlike a standard Adder object, which simply sums all inputs without scaling, the Merger object maintains a balanced output level, making it ideal for applications requiring signal blending while avoiding distortion or excessive gain increases.

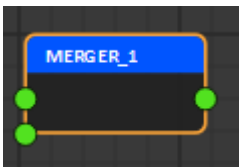
Input & Output Configuration of Merger object

- **Audio inputs:** 2
- **Audio Outputs:** 1

Adjustable Parameters of Merger object

- **None**

Flow Studio GUI of Merger object



6.6.2 Multiplexer, Multiplexer (fixed-point)

Functional description of Multiplexer object

The **Multiplexer** object (displayed as **MUX** or **MUX_FP** for the fix-point version in the information tab) functions as a signal routing tool that selects and forwards one specific input channel to the output. It allows dynamic switching between multiple input signals, making it useful for applications that require controlled signal selection.

The Multiplexer object includes an **editing button** on the right side, enabling users to customize the labels of the selection buttons below. These buttons provide an intuitive way to quickly choose the desired input channel for routing to the output. This feature is particularly useful in scenarios such as **audio source selection**, **dynamic signal switching**, and **routing control in digital audio processing systems**.

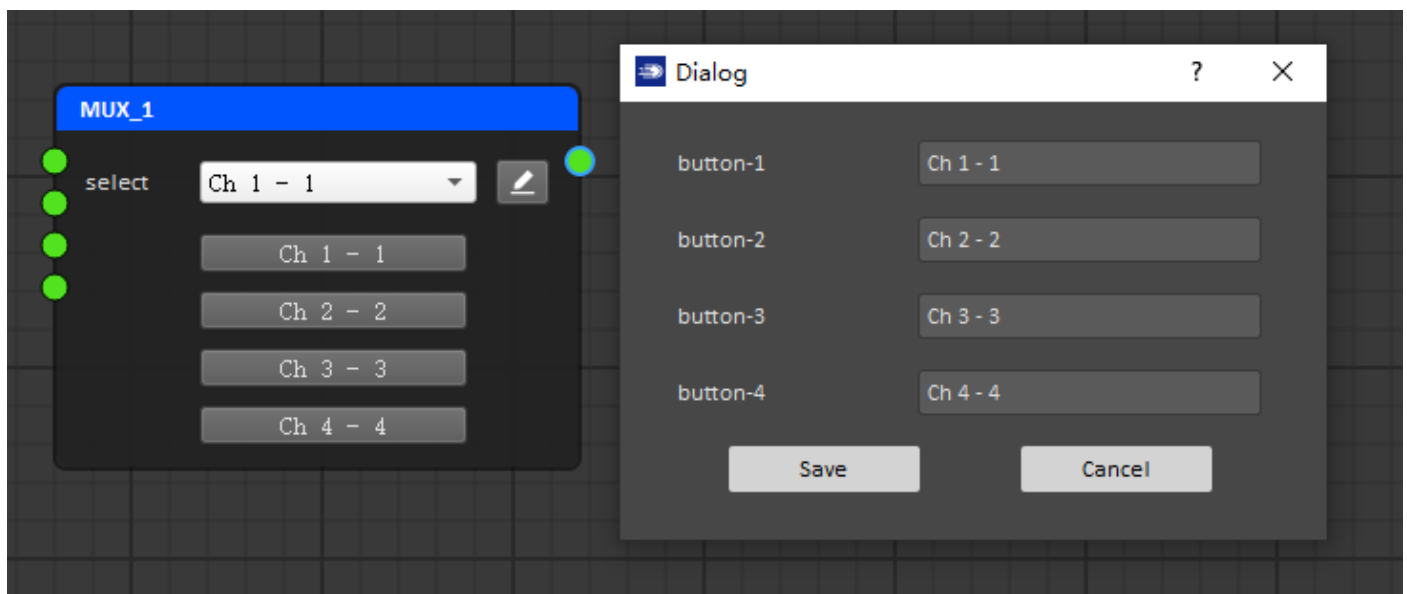
Input & Output Configuration of Multiplexer object

- **Audio inputs:** 4~16
- **Audio Outputs:** 1~4

Adjustable Parameters of Multiplexer object

- **select:** Select one of the input channel options to be routed to output

Flow Studio GUI of Multiplexer object



6.6.3 Dry wet fader

Functional description of Dry wet fader object

The **Dry wet fader** object (displayed as **DRYWET** in the information tab) provides a controllable **mixing ratio** between **two input signals**, typically representing the unprocessed ("dry") and processed ("wet") signals. This allows users to blend the original and processed signals seamlessly, enabling smooth transitions between effects and maintaining clarity in audio processing.

By adjusting the **mix** parameter, users can determine the balance between **Input 1 (dry signal)** and **Input 2 (wet signal)**. A value of **1** results in a fully dry signal, while **0** outputs only the wet signal. Intermediate values create a proportionate mix of both signals.

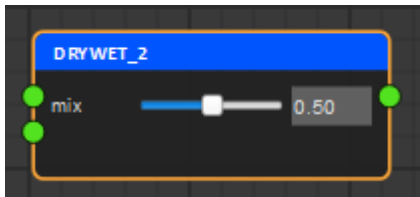
Input & Output Configuration of Dry wet fader object

- **Audio inputs:** 2
- **Audio Outputs:** 1

Adjustable Parameters of Dry wet fader object

- **mix:** Controls the **blend ratio** between Input 1 (dry) and Input 2 (wet). Range: 0~1 (Default: 0.5)

Flow Studio GUI of Dry wet fader object



6.6.4 Mixer, Mixer (fixed-point)

Functional description of Mixer object

The **Mixer** object (displayed as **MIXER** or **MIXER_FP** for the fix-point version in the information tab) is a **matrix mixer** that allows flexible routing and mixing of multiple audio input channels to multiple output channels. Each output channel can be formed by mixing any combination of the input channels, providing full control over the audio signal distribution. Additionally, each **input channel features individual gain control**, enabling precise level adjustments before being summed into the output. This functionality is essential for applications requiring dynamic signal balancing, multi-channel processing, and customized audio mixing.

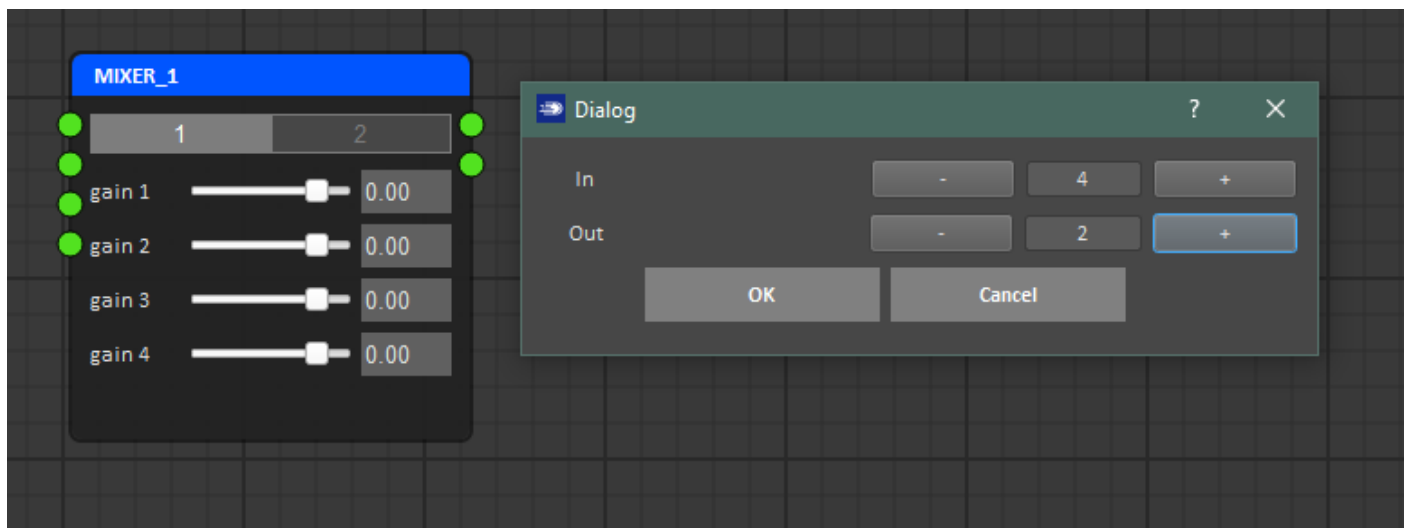
Input & Output Configuration of Mixer object

- **Audio inputs:** 1~8
- **Audio Outputs:** 1~8

Adjustable Parameters of Mixer object

- **gain:** Gain value in dB scale for each channel. Range: -120 dB ~ 24 dB. (Default: 0 dB)

Flow Studio GUI of Mixer object



6.7 Effects

Overview

In audio processing, effects are used to transform the characteristics of an audio signal, adding richness, space, depth, or unique tonal qualities. **Effects AOs** allow users to implement a wide range of creative and functional audio transformations.

Available Effects AOs:

- **Delay & Delay (Fixed-Point):** The Delay AO introduces a time-based echo to an audio signal, producing the effect of sound repetition after a set delay time. This is widely used in echo, slapback, or multi-tap effects. The fixed-point version is provided
- **Fractional Delay:** Fractional Delay allows for fine-grained control over the delay time, offering sub-sample resolution for precise echo effects. This is useful for advanced signal processing or creating modulated delay effects with high accuracy.
- **Multi-Tap Delay:** The Multi-Tap Delay AO generates multiple delayed versions of the input signal, each with different delay times. This can be used for creating complex echo effects, such as ping-pong delays or multiphonic echoes. It adds depth and movement to sound, often employed in sound design, music production, and specialized effects.
- **Low Pass Comb Filter:** A Low Pass Comb Filter applies a series of resonant peaks and dips in the frequency spectrum, creating a “comb-like” pattern of phase-shifted frequencies. This effect is used for creating vocal and tonal effects, phasing, and flanging, often used in psychedelic effects or synthesizer sound shaping.
- **Long All Pass Filter:** The Long All Pass Filter allows the phase manipulation of an audio signal without affecting its amplitude. It’s used for phase alignment and delayed feedback effects, where only the signal's timing is altered rather than its frequency content. This filter is useful in creating time-based effects and echo-based processing.
- **Reverb V2:** The Reverb V2 AO simulates the natural reflections of sound in various environments, creating a sense of space and depth. It’s used for adding room ambiance, echo, and environmental effects to an audio signal.
- **Chorus:** The Chorus AO creates a shimmering, spatial effect by slightly delaying and modulating a copy of the audio signal. This effect adds depth, warmth, and movement, often used in vocals, guitars, and synthesizer patches. It’s commonly used to create the illusion of multiple instruments playing the same part, resulting in a fuller, richer sound.
- **IR Convolver:** The IR Convolver AO applies impulse response (IR) convolution to an audio signal, simulating the characteristics of real-world spaces or equipment. This effect is primarily used for reverb simulations of physical spaces, such as rooms, halls, and cabinets. It’s also used for creating custom audio effects, where users can load specific IR files to emulate the tonal character of different environments or devices.

Effects AOs provide powerful tools for shaping sound and adding creative transformations to an audio signal. Whether you want to add depth with reverb, create complex delay patterns, or simulate real-world acoustics with convolution, these AOs offer the flexibility and precision needed for both musical and sound design applications.

6.7.1 Delay, Delay (fixed-point)

Functional description of Delay object

The **Delay** object (displayed as **DELAY** or **DELAY_FP** for the fix-point version in the information tab) is a fundamental signal processing component in Flow Studio, allowing users to introduce precise timing adjustments to audio signals. It provides sample-based delay control, making it useful for applications requiring phase alignment, spatial audio effects, and synchronization adjustments. Depending on the system configuration, the delay can be applied to signal Processing Chains to align multiple audio streams in a multi-channel system, spatial Audio Systems to create stereo widening or simulated echo effects or audio tuning and testing to fine-tune latency and compensate for system-induced timing variations. Like other signal processing objects, the Delay object is available in both floating-point and fixed-point versions, enabling users to select the format that best matches their system's processing architecture and precision requirements.

The delay time is set in samples, allowing for precise control over the timing offset. The maximum number of Delay objects depends on the system's available processing resources.

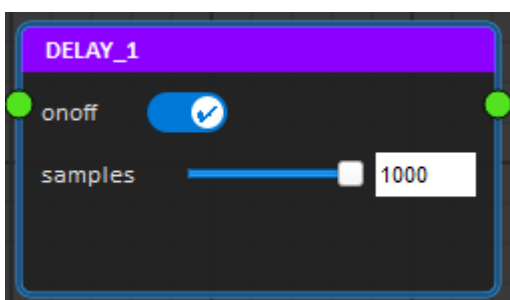
Input & Output Configuration of Delay object

- **Audio inputs:** 1-32
- **Audio Outputs:** 1-32

Adjustable Parameters of Delay object

- **Onoff:** Enables or bypasses the delay process. (Default: "Enable")
- **Samples:** The samples variable defines the delay time in sample units, determining how long an audio signal is delayed. For example, at a 48 kHz sampling rate, setting samples = 48 results in a 1-millisecond (ms) delay.

Flow Studio GUI of Delay object



6.7.2 Fractional Delay

Functional description of Fractional Delay object

The **Fractional Delay** object (displayed as **DELAY_Intp** in the information tab) is an advanced signal processing component in Flow Studio, designed to provide sub-sample delay precision for applications requiring high-accuracy time alignment, phase correction, and modulated delay effects. Unlike traditional integer-sample delays (just like the Delay object), Fractional Delay object allows for fine-grained delay adjustments by interpolating between samples, ensuring smooth and precise delay transitions.

Depending on the system configuration, the fractional delay can be applied to:

- Phase Alignment in multi-channel audio processing to ensure coherent signal timing.
- Modulated Delay Effects such as chorus, flanger, and phaser, where continuous delay variation enhances audio depth.
- High-Precision Synchronization in spatial audio processing to maintain accurate time relationships between signals.

The maximum number of Fractional Delay objects depends on the system's available processing resources.

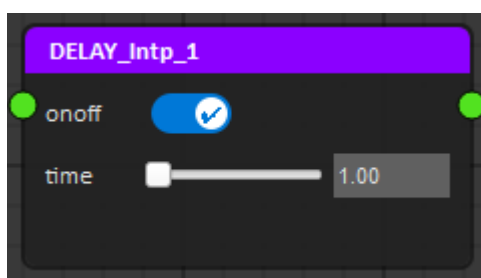
Input & Output Configuration of Fractional Delay object

- **Audio inputs: 1**
- **Audio Outputs: 1**

Adjustable Parameters of Fractional Delay object

- **Onoff:** Enables or bypasses the Fractional Delay process. (Default: "Enable")
- **Time:** Provides sub-sample resolution by interpolating between adjacent samples, enabling ultra-fine delay adjustments.

Flow Studio GUI of Fractional Delay object



6.7.3 Multitap Delay

Functional description of Multitap Delay object

The **Multitap Delay** object (displayed as **MULTITAP** in the information tab) is an advanced delay processing component in Flow Studio, designed to generate up to 8 independently controlled delayed signals (taps) from a single input. Each tap can be customized with unique delay parameters, allowing for rich spatial effects and complex echo patterns.

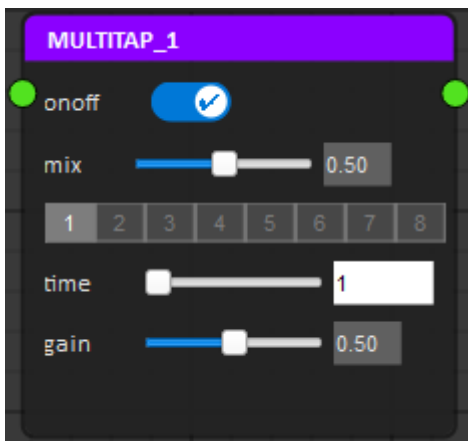
Input & Output Configuration of Multitap Delay object

- **Audio inputs:** 1
- **Audio Outputs:** 1

Adjustable Parameters of Multitap Delay object

- **Onoff:** Enables or bypasses the AO process. (Default: “Enable”)
- **mix:** Adjusts the blend between the dry (original) and wet (delayed)
- **time:** The samples variable defines the delay time in sample units.
- **gain:** Controls the individual tap volume. (Default: 0.5 dB)

Flow Studio GUI of Multitap Delay object



6.7.4 Low Pass Comb Filter

Functional description of Low Pass Comb Filter object

The **Low Pass Comb Filter** object (displayed as **LPF_COMB_FILTER** in the information tab) is a delay-based filtering component in Flow Studio, designed to generate resonant echoes while applying lowpass filtering to the delayed signal. This creates a series of resonant peaks and dips in the frequency spectrum, forming a comb-like pattern of phase-shifted frequencies. The Low Pass Comb Filter object is commonly used for vocal processing, tonal shaping, phasing, and flanging, making it particularly useful in psychedelic effects and synthesizer sound design. Figure 1 shows the reverb generation scheme of the Low Pass Comb Filtering process, which is used as a building block for artificial reverb, simulating early reflections.

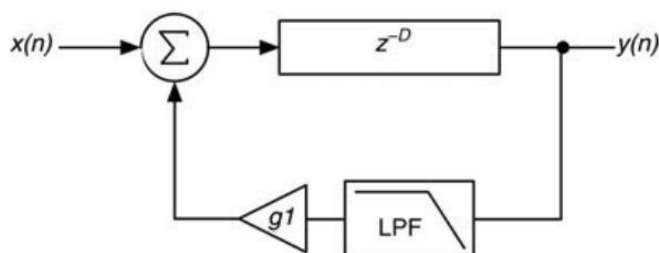


Fig. 1 Reverb Generation Scheme of The Low Pass Comb Filter object

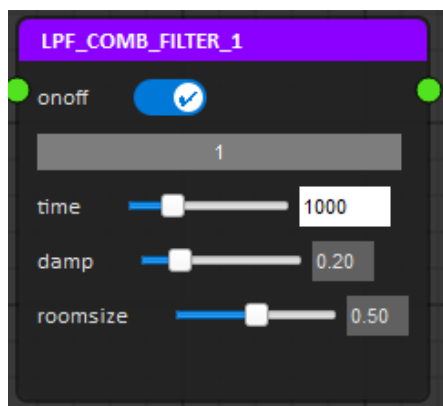
Input & Output Configuration of Low Pass Comb Filter object

- **Audio inputs:** 1
- **Audio Outputs:** 1

Adjustable Parameters of Low Pass Comb Filter object

- **onoff:** Enables or bypasses the Low Pass Comb Filter object. (Default: "Enable")
- **time:** The samples variable defines the delay time in sample units. (Default: 1000)
- **damp:** Set the feedback path 1-order low pass filter cutoff. (Default: 0.2)
- **roomsize:** Set the feedback amount of the comb filter. (Default: 0.5)

Flow Studio GUI of Low Pass Comb Filter object



6.7.5 Long All Pass Filter

Functional description of Long All Pass Filter object

The **Long All Pass Filter** object (displayed as **LONG_APF** in the information tab) is an advanced signal processing component in Flow Studio, designed to modify phase characteristics while maintaining a constant magnitude response across all frequencies. Unlike traditional filters that alter gain, the all-pass filter preserves amplitude (0 dB gain) while introducing frequency-dependent phase shifts, making it ideal for reverb processing, phase correction, echo simulation, and modulated effects. Figure 1 shows the reverb generation scheme of the Long All Pass Filtering process.

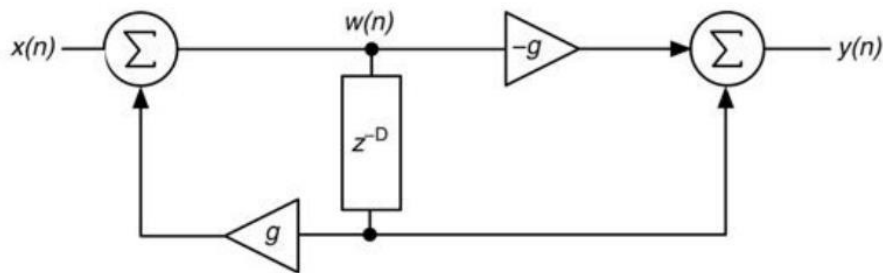


Fig. 1 Reverb Generation Scheme of The Long All Pass Filter object

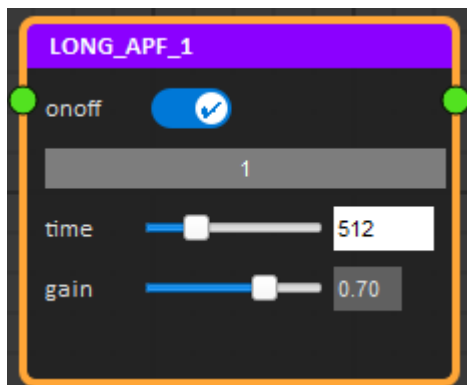
Input & Output Configuration of Long All Pass Filter object

- **Audio inputs:** 1
- **Audio Outputs:** 1

Adjustable Parameters of Long All Pass Filter object

- **onoff:** Enables or bypasses the Long All Pass Filter object. (Default: "Enable")
- **time:** The samples variable defines the delay time in sample units. (Default: 512)
- **gain:** Adjusts the feedback intensity, influencing the strength of the phase shift effect. (Default: 0.7 dB)

Flow Studio GUI of Long All Pass Filter object



6.7.6 Reverb V2

Functional description of REVERB V2 object

The **Reverb V2** object (displayed as **REVERB_V2** in the information tab) is a spatial audio processing component designed to simulate natural or artificial reverberation by replicating sound reflections in an acoustic space. This effect enhances the depth, warmth, and realism of an audio signal, making it an essential tool for music production, sound design, and immersive audio applications. For example, the Reverb V2 audio object can be used to emulate real-world environments, from small rooms to large concert halls (Natural Room Simulation) or it can be used in cinematic effects, and post-processing to shape the tonal characteristics of a sound (Creative Sound Design).

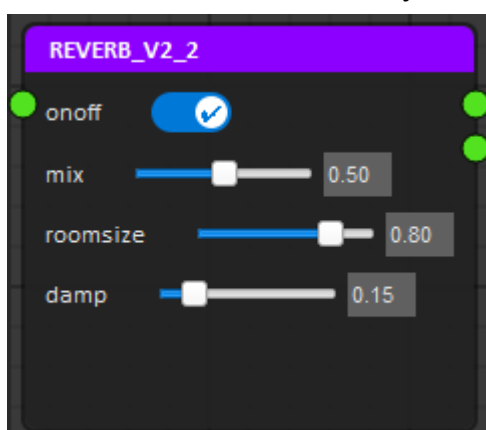
Input & Output Configuration of Reverb V2 object

- **Audio inputs:** 1
- **Audio Outputs:** 2

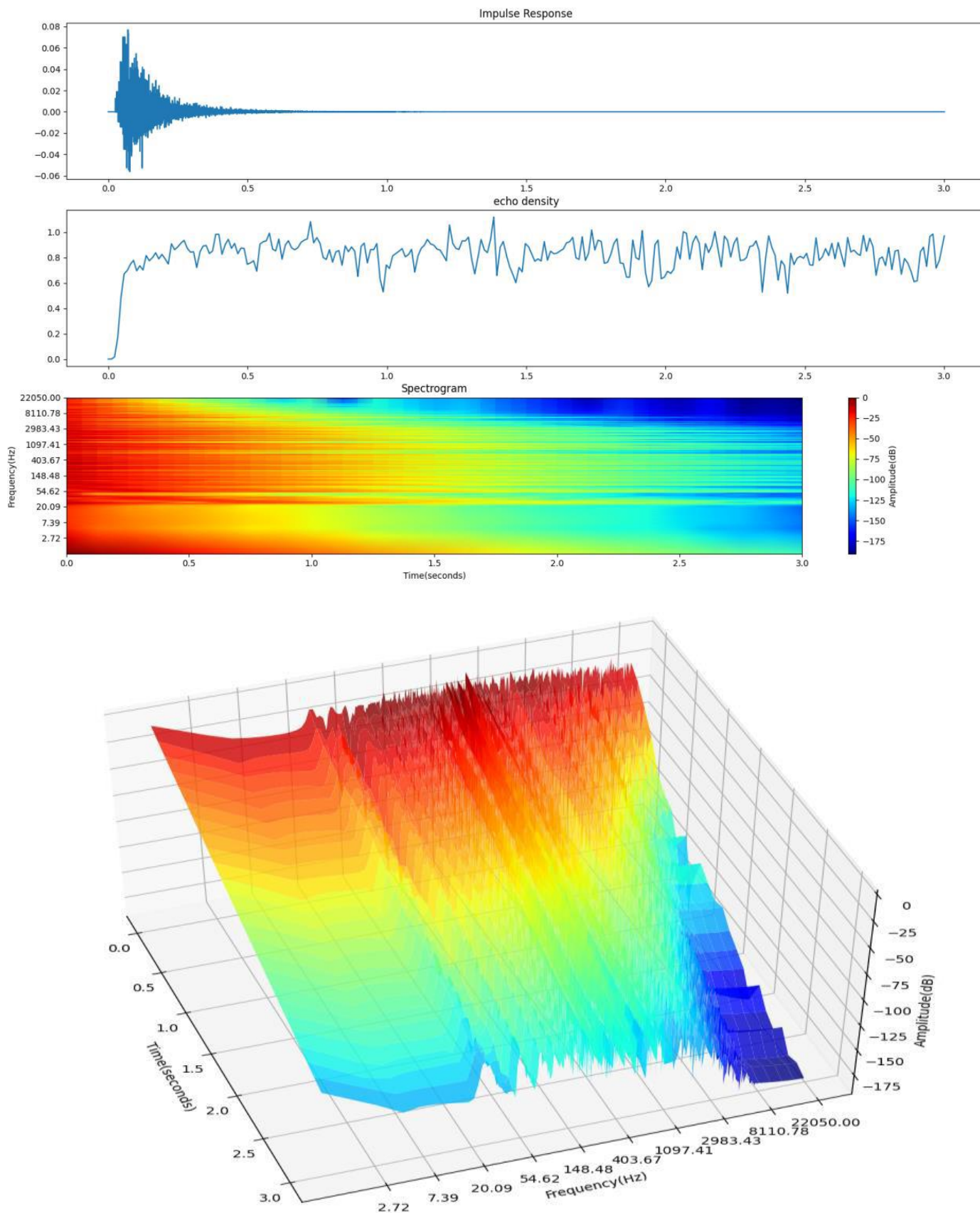
Adjustable Parameters of Reverb V2 object

- **onoff:** Enables or bypasses the AO process. (Default: “Enable”)
- **mix:** Controls the blend between the dry (original) and wet (processed) signal, adjusting the intensity of the reverb effect. (Default: 0.5)
- **roomsize:** Defines the perceived room size, where larger values increase the reverb tail length, simulating bigger acoustic spaces. (Default: 0.8)
- **damp:** Sets the high-frequency decay rate, where larger values result in faster attenuation of high frequencies. (Default: 0.15)

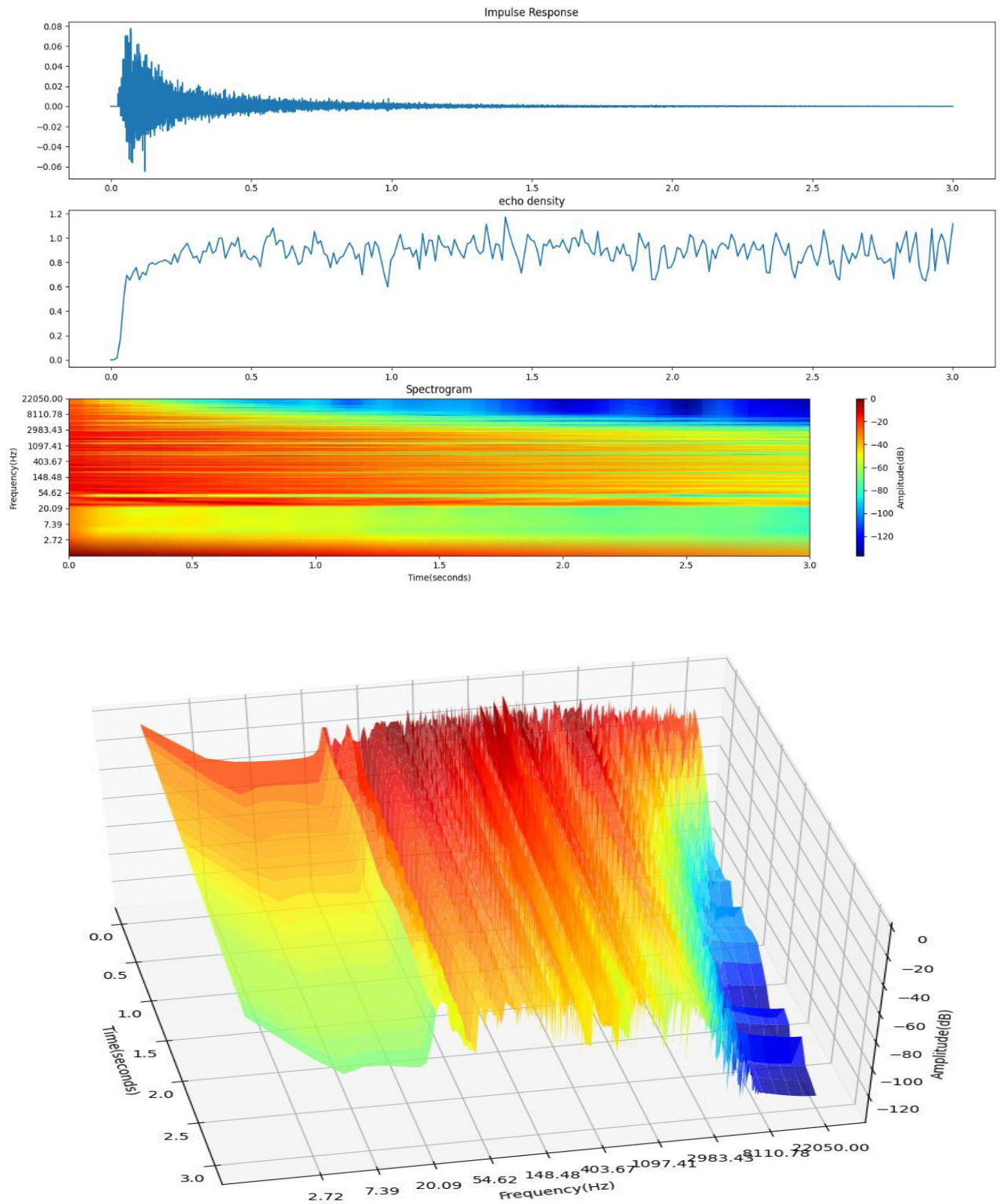
Flow Studio GUI of Reverb V2 object



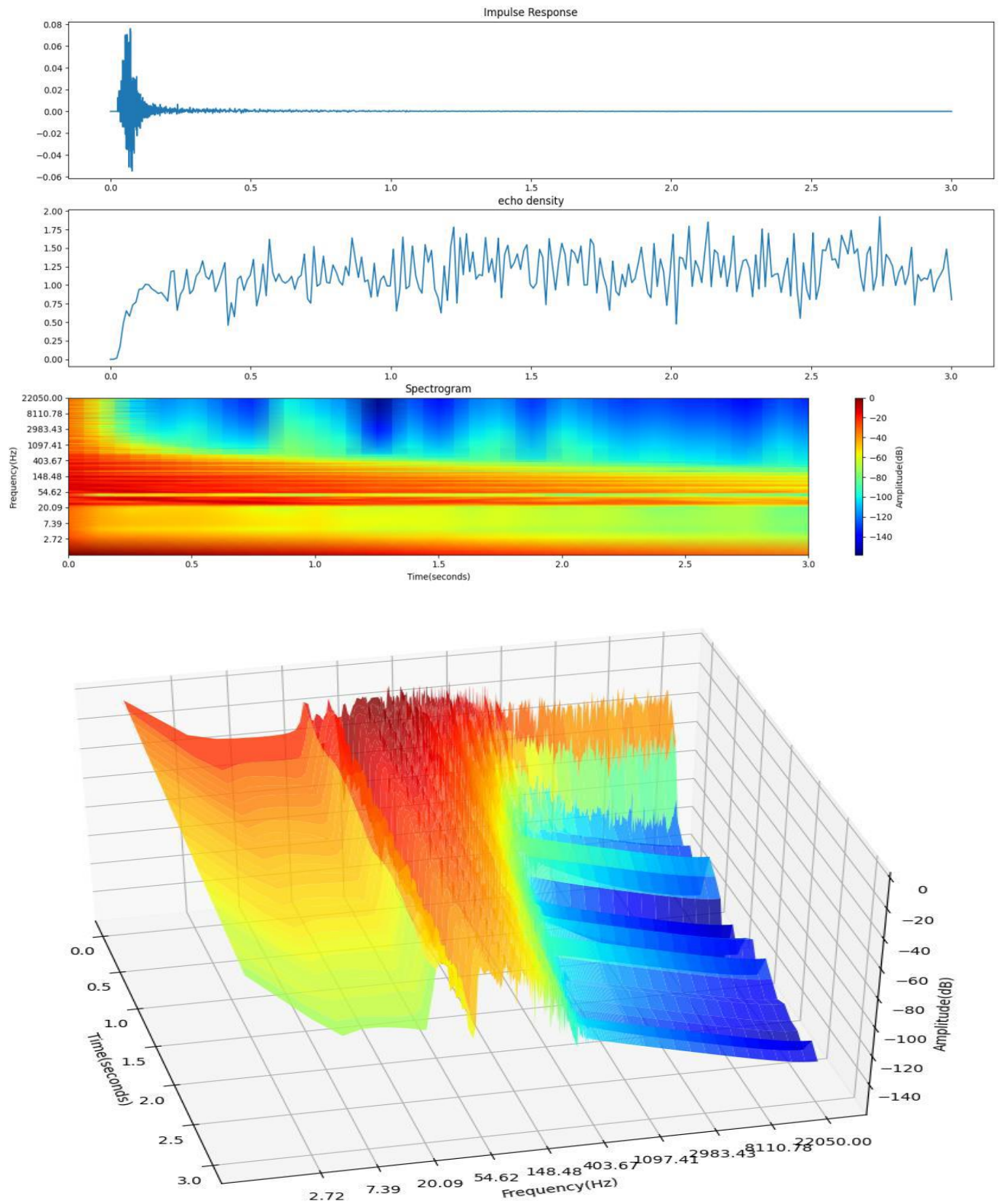
Supplementary figures of the Reverb V2 object



Roomsize =0.84, decay=0.2



Roomsize =0.95, decay=0.2



Roomsize=0.95, decay=0.9

6.7.7 Chorus

Functional description of Chorus object

The **Chorus** object (displayed as **CHORUS** in the information tab) is a time-based modulation effect designed to create a rich, shimmering sound. It achieves this by duplicating the input signal and introducing subtle variations in pitch and timing, resulting in a lush, layered effect. This technique closely mimics the natural detuning that occurs when multiple sound sources perform the same notes simultaneously, adding depth and movement to the audio. Widely used in **music production, vocal processing, and instrument enhancement**, the chorus object plays a crucial role in shaping a fuller and more immersive sound. It is often applied to guitars, keyboards, and synthesizers to enhance their tonal richness and provide a sense of spaciousness. Vocal tracks also benefit from chorus processing, as it can create a thicker, more harmonized sound that blends seamlessly into a mix.

Beyond traditional applications, chorus is a popular tool in **sound design and creative audio effects**, particularly in cinematic scoring, ambient soundscapes, and electronic music. By adding a dreamy, detuned character, it contributes to ethereal textures and evolving sonic landscapes, making it an essential effect for producing immersive and atmospheric compositions. Whether subtly thickening a track or creating otherworldly modulations, chorus remains a versatile and indispensable tool in modern audio production.

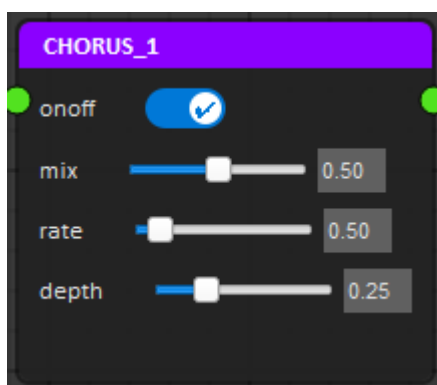
Input & Output Configuration of Chorus object

- **Audio inputs:** 1
- **Audio Outputs:** 1

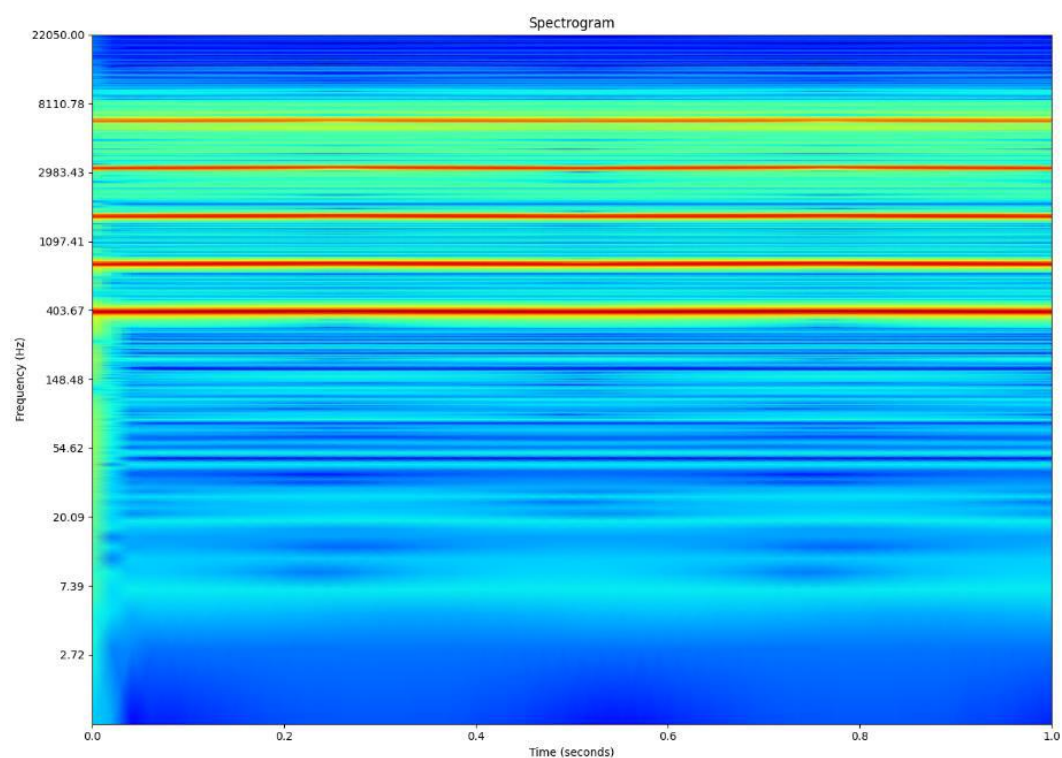
Adjustable Parameters of Chorus object

- **onoff:** Enables or bypasses the Chorus process. (Default: "Enable")
- **mix:** Sets the blend between the dry (original) and wet (processed) signal, adjusting the intensity of the chorus effect.
- **rate:** Controls the LFO (Low-Frequency Oscillator) sweep rate, where higher values result in a faster modulation speed, creating a more intense vibrato-like effect.
- **depth:** Adjusts the LFO amplitude, where larger values produce a stronger detuning effect, increasing the perceived richness and movement of the sound.

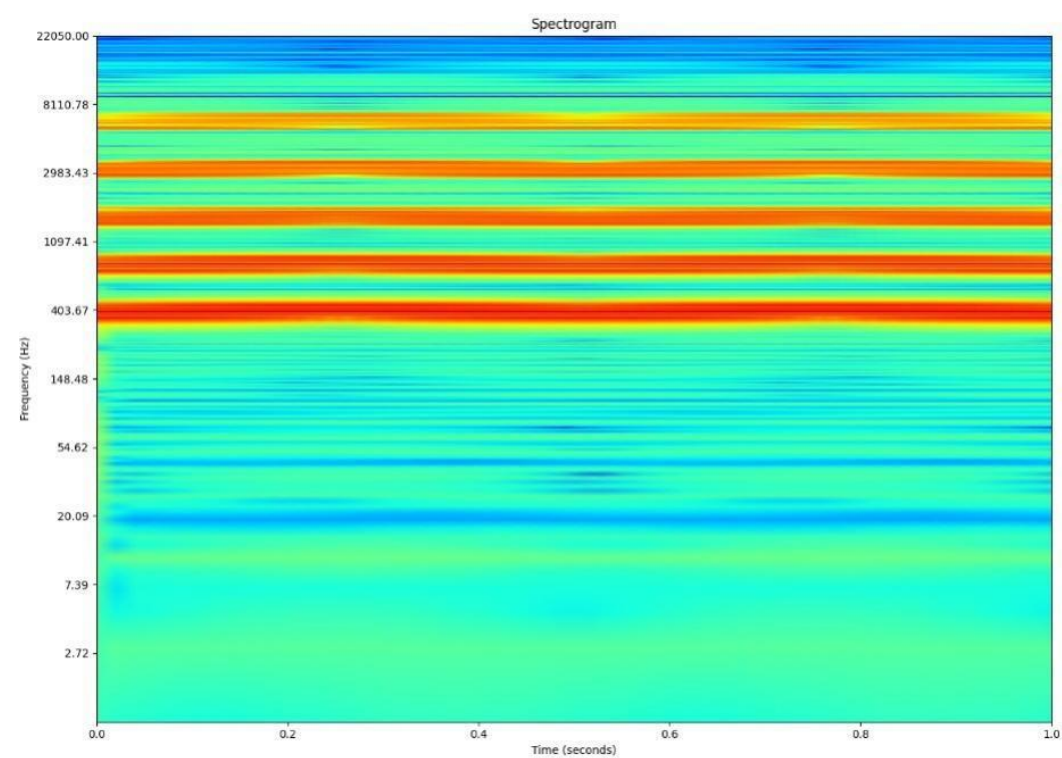
Flow Studio GUI:



Supplementary figures of Chorus object



Rate=4Hz, depth=0.1



Rate=4, depth=0.5

6.7.8 IR Convolver

Functional description of IR Convolver object

The **IR Convolver** object, displayed as **IR** in the information tab, is a fast impulse response (IR) processing effect designed to deliver **low-latency convolution** across a wide range of applications. By efficiently applying IR filtering, it allows users to **simulate real-world acoustic environments** and accurately **model speaker cabinet characteristics** with exceptional precision. This technology is particularly valuable in professional audio processing, where realistic sound reproduction and spatial accuracy are essential. One of the key strengths of the IR Convolver is its **optimized real-time performance**, ensuring fast processing with minimal delay. This makes it well-suited for live applications and interactive audio environments where responsiveness is critical. Additionally, the IR convolver offers **scalability**, allowing it to work efficiently with short impulse responses for speaker modeling while also supporting longer IRs for high-fidelity reverb simulation on powerful platforms. Whether used for small-scale applications or complex audio environments, the IR Convolver provides a flexible and efficient solution.

Another significant advantage is its **high accuracy in preserving the fine details** of the original impulse response. This level of precision enables **realistic tone shaping and spatial sound reproduction**, making it an essential tool for **music production, game audio, and immersive sound design**. Whether applied to room simulations, reverb modeling, or advanced audio effects, the IR Convolver ensures a **faithful and natural** representation of the desired acoustic space, offering unmatched realism and versatility in modern audio processing.

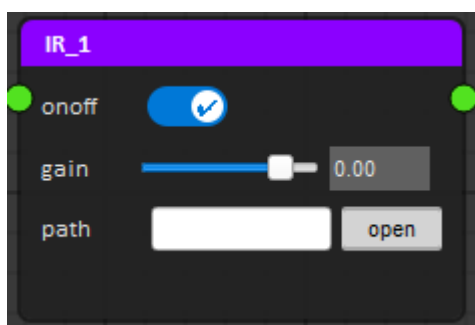
Input & Output Configuration of IR Convolver object

- **Audio inputs:** 1
- **Audio Outputs:** 1

Adjustable Parameters of IR Convolver object

- **onoff:** Enables or bypasses the AO process. (Default: “Enable”)
- **gain:** Input signal gain adjustment to prevent clipping
- **path:** User specific Impulse response in wav file format

Flow Studio GUI of IR Convolver object



6.8 Metering

Overview

Metering AOs are essential tools for analyzing and visualizing audio signals in real time. They provide valuable insights into signal levels, spectral distribution, and frequency response, helping users monitor and optimize their audio processing workflows. Flow Studio includes the following Metering AOs, each designed to serve a specific analysis function.

Available Metering AOs:

- **METER:** The Meter AO measures and displays the amplitude levels of an audio signal, allowing users to monitor signal strength and detect potential clipping or low-level issues. It is useful in gain staging, ensuring signals are at optimal levels before further processing.
- **SPECTRUM:** The Spectrum AO provides a frequency-domain representation of an audio signal, displaying its spectral energy distribution. This is beneficial for identifying tonal imbalances, troubleshooting unwanted noise, or analyzing the frequency response of an audio system.
- **RTA (Real-Time Analyzer):** The RTA AO offers a real-time frequency analysis of incoming audio, dividing the spectrum into multiple bands for detailed insight into the sound's frequency content. This tool is commonly used in room tuning, EQ adjustments, and live sound monitoring, helping engineers make precise frequency-based modifications.

Metering AOs play a crucial role in maintaining high-quality audio processing by providing real-time visual feedback on signal levels and frequency distribution. Whether users need to monitor signal amplitude, analyze spectral content, or fine-tune EQ settings, these AOs offer the essential tools for precise and effective audio management. By incorporating metering into their workflows, users can ensure that their audio signals remain balanced, clear, and optimized for any application.

6.8.1 Level Meter, Level Meter (fixed-point)

Functional description of Level Meter object

The **Level Meter** audio object (displayed as **METER** or **METER_FP** for the fix-point version in the information tab) is a signal level detector that monitors and measures the peak amplitude of an incoming audio signal. It operates using peak metering, which captures the highest signal level over time, ensuring accurate real-time level tracking. The metering function includes a **pre-defined attack and release time**, allowing for controlled responsiveness to sudden level changes.

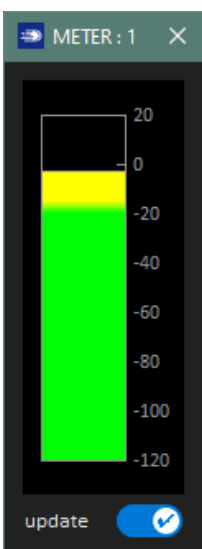
- **Attack Time:** Determines how quickly the meter responds to an increase in signal level. A shorter attack time results in a faster reaction to peaks, while a longer attack time smooths out rapid fluctuations.
- **Release Time:** Controls how quickly the meter returns to lower levels after a peak. A slow release creates a smoother display, while a fast release closely follows transient reductions in amplitude.

Since this Level Meter object is purely for analysis, it does not produce an audio output but instead provides real-time visual feedback, up to 16 channel. After wiring with any audio input (for example, sine wave or noise) and connected to the Flow Engine, user can see the variation of signal level over time when the “update” option is enabled.

Input & Output Configuration of Level Meter object

- **Audio inputs:** 1~16
- **Audio Outputs:** N/A

Flow Studio GUI of Level Meter object



6.8.2 Spectrum Analyzer

Functional description of Spectrum Analyzer object

The **Spectrum Analyzer** audio object (displayed as **SPECTRUM** in the information tab) is an **FFT-based spectrum analyzer** designed to analyze the frequency content of an input audio signal in real-time. It performs a **Fast Fourier Transform (FFT)** on incoming audio data, breaking it down into frequency bins to provide a visual representation of the spectral energy. This allows users to monitor, analyze, and optimize audio signals based on their frequency components. This analyzer supports **FFT sizes** of up to **16,384**, enabling high-resolution frequency analysis. Additionally, it features multiple **windowing functions** to reduce spectral leakage and provides adjustable **decay times (averaging)** for smoother spectrum visualization. Since the **Spectrum Analyzer** object is purely analytical, it does not produce an audio output but instead provides real-time visual feedback useful for audio diagnostics, mixing, and research applications.

After wiring with audio input and connected to the Flow Engine, user can see the variation of the frequency representation over time when the “update” option is enabled.

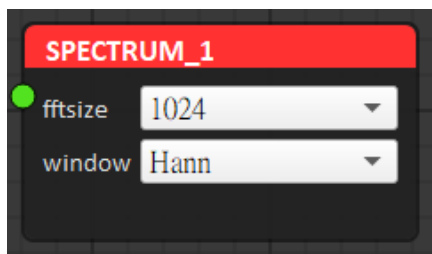
Input & Output Configuration of Spectrum Analyzer object

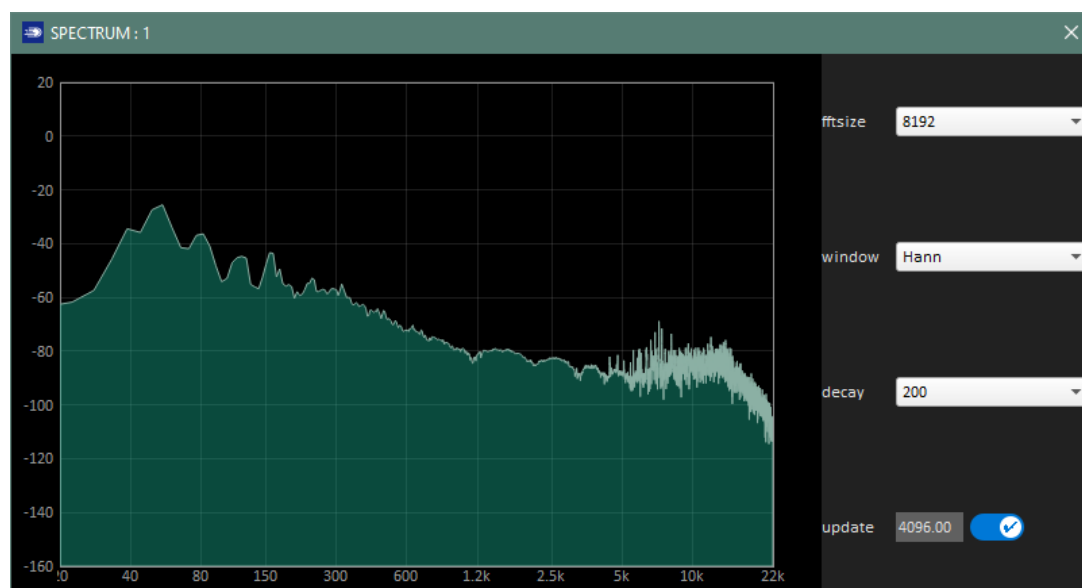
- **Audio inputs:** 1
- **Audio Outputs:** N/A

Adjustable Parameters of Spectrum Analyzer object

- **fftsize:** Determines the frequency resolution of the analysis. A larger FFT size provides finer frequency resolution but increases computational complexity. Options include 128, 256, 512, 1024, 2048, 4096, 8192, and 16384. (Default: 1024)
- **window:** Specify the window function that helps minimize spectral leakage, improving the accuracy of frequency analysis. Options include None, Hann, Blackmann-Harris, Rectangular, Hamming. (Default: Hann)
- **Decay Time:** Controls how quickly the spectral display updates, allowing for either fast transient detection or smoother long-term averaging. Options include 50, 100, 200, 500, 1000, 2000, and 5000 ms (Default: 200 ms)

Flow Studio GUI of Spectrum Analyzer object





6.8.3 Real Time Analyzer

Functional description of Real Time Analyzer object

The **Real Time Analyzer** (displayed as **RTA** in the information tab) is a **1/3-octave band spectrum analyzer** designed to measure and display the frequency response of an input audio signal. Unlike an FFT-based spectrum analyzer, which provides fine-grained frequency resolution, the **RTA** groups frequencies into standardized **1/3-octave bands**, aligning with human auditory perception and industry-standard measurement practices.

This analysis method is commonly used in **room acoustics, live sound tuning, and audio system calibration**, providing an easy-to-interpret visualization of an audio signal's spectral balance. The RTA object includes adjustable **decay times** to control how quickly the display updates, allowing either fast transient tracking or smoother long-term averaging.

After wiring with audio input and connected to the Flow Engine, user can see the variation of the different frequency bins over time when the “update” option is enabled.

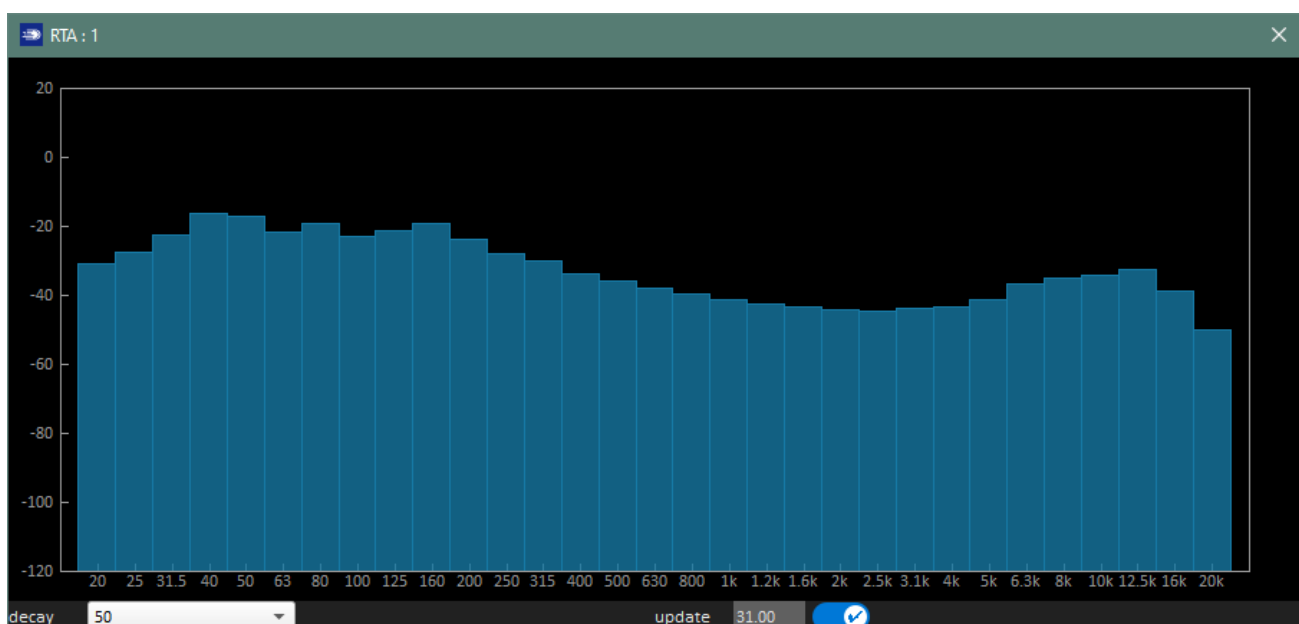
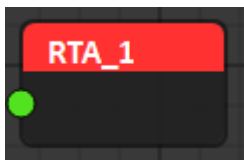
Input & Output Configuration of Real Time Analyzer object

- **Audio inputs:** 1
- **Audio Outputs:** N/A

Adjustable Parameters of Real Time Analyzer object

- **decay:** Averaging speed for display updates. Options include 50, 100, 200, 500, 1000, 2000, and 5000 ms (Default: 200 ms)

Flow Studio GUI of Real Time Analyzer object



6.9 Technology Provider

Overview

AOs in this category represent a collection of specialized audio processing objects developed in collaboration with leading third-party IP providers and audio technology companies. These AOs integrate proprietary signal processing techniques and patented algorithms, offering users cutting-edge audio enhancements for various applications. Flow Studio supports several third-party technology provider AOs, each delivering unique solutions for sound optimization and enhancement.

Available AOs from Technology Provider:

- **Fraunhofer:** Developed by Fraunhofer IIS, a world leader in audio coding and sound processing technologies, this AO incorporates high-performance audio algorithms for applications such as speech enhancement, perceptual audio coding, and spatial audio processing. Commonly used in broadcasting, telecommunications, and consumer electronics for delivering clear, high-quality sound.
- **NTT Sonority:** NTT Sonority specializes in advanced audio signal processing for noise suppression, voice clarity, and immersive sound experiences. This AO is particularly beneficial for telecommunications, conferencing systems, and automotive audio, where speech intelligibility and background noise reduction are critical.
- **Dirac:** Dirac's audio processing solutions are known for room correction, spatial audio enhancement, and phase correction. This AO enables high-fidelity sound reproduction by optimizing speaker and headphone output, making it ideal for home theater systems, automotive audio, and high-end consumer electronics.
- **Aizip:** Based in Silicon Valley, Aizip is a leader in model design of AI for IoT (AIoT). With its breakthrough neural network architecture and proprietary automated design tools, Aizip has demonstrated a wide range of deep neural networks (DNN) with superior performance. Since its founding in 2020, Aizip has designed and delivered TinyML models to its diverse customers worldwide.
- **Flow DSP:** Flow DSP AOs, developed in collaboration with TYMPHANY, provide customized audio processing solutions for specific hardware integrations and acoustic tuning. These AOs are tailored for speaker optimization, bass enhancement, and real-time sound adjustments, ensuring the best possible audio performance for Tymphany-designed acoustic systems.

By integrating these third-party IP-based AOs, Flow Studio enables developers to access premium, industry-proven audio processing capabilities. Whether improving speech intelligibility, refining spatial audio, or applying advanced noise reduction, these technology provider AOs empower users with state-of-the-art tools for superior audio performance.

6.9.1 Fraunhofer IIS

Fraunhofer Institute for Integrated Circuits IIS (**Fraunhofer IIS**) is a leading applied research institution based in Erlangen, Germany, and is part of the Fraunhofer-Gesellschaft, Europe's largest organization for applied research. Established in 1985, Fraunhofer IIS has played a pivotal role in developing innovative technologies across various domains. We and Fraunhofer have strong relationship for years and provide cutting-edge AOs as follows:

Available AOs from Fraunhofer:

- **Cingo**: Cingo brings life-like Dynamic Spatial Audio to headphones and creates enveloping 3D sound through stereo
- **uphear VQE**: uphear Voice Quality Enhancement (VQE) remove interfering sound captured by device's microphones, extracts the user's voice, and cancels out acoustical echoes. It includes features such as multichannel echo control, source localization, beamforming, noise reduction, dereverberation, automatic gain control.
- **uphear Virtualizer**: The audio enhancement technology transforms soundbars and smart speakers from mere playback devices into audio powerhouses. Listeners enjoy a wide sound field that extends beyond the loudspeakers with full immersion and presence. This provides the optimal basis for playback of Next Generation Audio formats.

We and Fraunhofer have partnered to provide state-of-the-art spatial audio solutions to audio manufacturers. We worked together to integrate Fraunhofer's audio processing suites and audio technologies into Flow DSP, our own DSP platform and design framework which allows creating immersive audio features in new audio products. With this new partnership with Fraunhofer IIS, we are now able to offer audio manufacturers something that is increasingly on demand, which are processing solutions for immersive sound, binaural spatialization, and audio enhancement. Fraunhofer's Cingo and uphear suites are now available in Flow Studio, being able to add breakthrough tuning features on target products with a simple drag-and-drop interface.

6.9.1.1 Cingo

Functional description of Cingo object

Cingo audio object (displayed as **CINGO** in the information tab) delivers immersive, dynamic spatial audio that transforms the listening experience on standard devices. This advanced audio solution creates a three-dimensional soundscape that feels remarkably natural and life-like, whether you're using ordinary headphones or stereo speakers. The technology enables sound to be perceived as coming from precise locations all around the listener—above, below, behind, or from any direction—creating a highly immersive audio environment. With spatial audio processing, Cingo significantly enhances the realism of music, movies, games, and virtual reality experiences without requiring specialized audio hardware. Developed by Fraunhofer IIS, the same research institute behind the MP3 format, Cingo represents a significant advancement in consumer audio technology that makes immersive sound accessible through everyday devices. Current setting of input channel configuration allows 2 and 7.1 surround.

Input & Output Configuration of Cingo object

- **Audio inputs:** 2, 7.1
- **Audio Outputs:** 2

Adjustable Parameters of Cingo object

Processing Mode

- **VSP Mode:** Set the virtual processing mode for Non-Spatial (Bypass) / Spatial (Headphone).
- **Preset:** Set the virtual environment Preset. Options include STUDIO, BOOTH, LECTURE_HALL, OFFICE, CINEMA, CHURCH, OUTDOOR, CAVE, CAR-PARK, ARENA, CONCERT_HALL, and BATHROOM (Default: STUDIO),
- **Front Angle:** Set the symmetrical angle for the front channels L & R, -180 ~ 180 (Default: 45) (in degree).
- **Surround Angle:** Set the symmetrical azimuth angle for the surround channels L & R, -180 ~ 180 (Default: 110) (in degree)
- **EQ Enable:** Turn on or turn off five band EQ. (Default: Off)
- **EQ Preset:** Set the music genre preset for five band equalizer. Options include Normal, Classical, Dance, Flat, Fol, Heavy metal, Hop hop, Jazz, Pop and Rock. (Default: Normal)
- **Smart Gain:** Set the smart gain to increase/decrease by dynamic range compression. Range: -20~20 (Default: 0.0)
- **DialogPlus Boost:** Set the level of dialog enhancement. Range: 0 ~ 100 (Default: 0),

Virtual Environment

- **Preset:** Set the virtual environment Preset. Options include STUDIO, BOOTH, LECTURE_HALL, OFFICE, CINEMA, CHURCH, OUTDOOR, CAVE, CAR-PARK, ARENA, CONCERT_HALL, and BATHROOM (Default: STUDIO),
- **Room Scale:** Virtual Space Size. Range: 0 ~ 100 (Default: 50)
- **Absorption Curve:** A sound reflection in virtual space. The harder input shows stronger reflection. Options include SOFT, MEDIUM, and HARD (Default: MEDIUM),
- **Reverb Mix:** Set the level of the reverb mix ratio for all channels. Changes the room impression from close virtual loudspeakers (non-reverberant) to distant virtual loudspeakers (reverberant). The energy is preserved. 0 ~ 100 (Default: 30).
- **Individual Factor:** Set HRTF adoption value based on model of different user head size. Propose preset value

(Small=35, Medium=50, Large=65). 0~100 (Default: 50).

Virtual Speaker Setting

- **Channel Spatial:** Turn on or turn off spatial effect of a channel (Default: On).
- **Channel Mute:** Set a channel to mute, On/Off (Default: Off).
- **Channel Azimuth Angle:** Set the azimuth angle of a channel, -180 ~ 180 (Default: ± 45) (in degree).
- **Channel Elevation Angle:** Set the elevation angle of a channels, -90 ~ 90 (Default: 0) (in degree).

Head Rotation Setting

- **Head Rotation:** Turn on the update of head rotation (Default: On).
- **Auto Euler Reset:** (Default: On).
- **Reset Cooldown:** 0 ~ 10,000 ms (Default: 5000ms), only available when Auto Euler Reset is in 'On' state.
- **Reset Apply Time:** 0 ~ 5500 ms (Default: 1200ms), only available when Auto Euler Reset is in 'On' state.
- **Reset Angle Threshold:** 20~90 degrees (Default: 30 degrees), only available when Auto Euler Reset is in 'On' state.. Defines the maximum angular deviation allowed to determine whether the user is “nearly stationary. A smaller value makes the detection stricter (less likely to trigger reset),

Flow Studio GUI of Cingo object



CINGO_1

Processing Mode

VSP: Spatial(Headphone)

Preset: MUSIC

Front Angle: 45

Surround Angle: 110

EQ Enable: ☒

EQ Preset: NORMAL

Smart Gain: 0

DialogPlus Boost: 0

Virtual Environment

Preset: STUDIO

Room Scale: 50

Absorption Curve: MEDIUM

Reverb Mix: 30

Individual Factor: 50

Virtual Speaker Settings

Left Channel

Spatial: ☒

Mute: ☒

Azimuth Angle: -45 degrees

Elevation Angle: 0 degrees

Right Channel

Spatial: ☒

Mute: ☒

Azimuth Angle: 45 degrees

Elevation Angle: 0 degrees

Head Rotation Visualizer

EulerUpdate: ☒

time: 50

HeadRotation Setting

Head Rotation: ☒

Auto Euler Reset: ☒

Reset Cooldown(ms): 5000

Reset Apply Time(ms): 1200

Reset Angle TH: 30

6.9.1.2 upHear VQE

Functional description of upHear VQE object

Fraunhofer **upHear Voice Quality Enhancement (VQE)** object (displayed as **upHear VQE** in the information tab) removes interfering sounds captured by a device's microphones, extracts the user's voice, and eliminates acoustical echoes. The system features multichannel echo control, source localization, beamforming, noise reduction, dereverberation, and automatic gain control. Input and output sample rate only support 16 kHz.

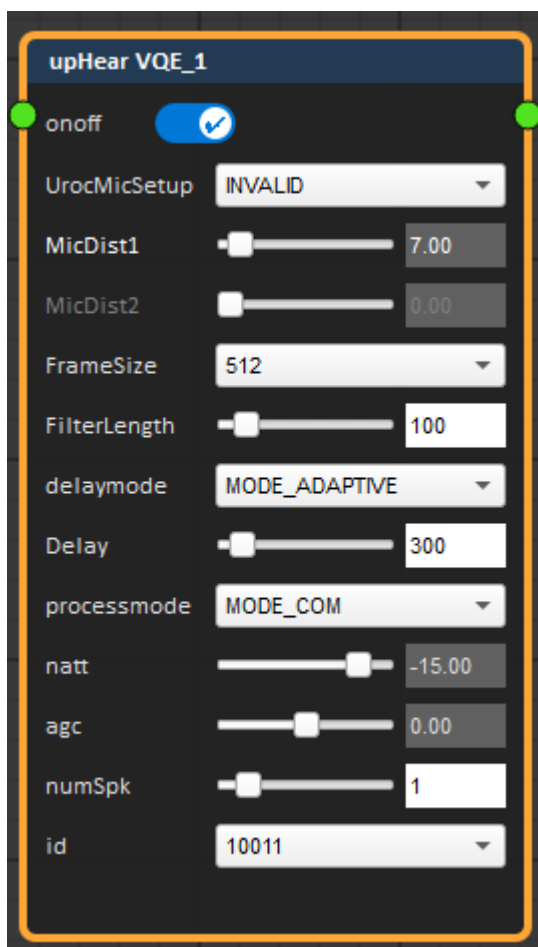
Input & Output Configuration of upHear VQE object

- **Microphone inputs (Near-end):** 2~4
- **Reference inputs (Far-end):** 0~8
- **Audio Outputs:** 1

Adjustable Parameters of upHear VQE object

- **UrocMicSetup:** Microphone array configuration. Options include *INVALID*, *SINGLE*, *DUAL*, *QUAD_ULA*, *QUAD_NONULA*, *OCTA_ULA*, *6_ULA*, *3_ULA*, *OCTA_CIRC_UNIFORM*, *6_PLUS_1_CIRC*, *QUAD_CIRC_UNIFORM*, *3_PLUS_1_CIRC*, *6_CIRC_UNIFORM*, *3_CIRC_UNIFORM*, *5_CIRC_UNIFORM*, and *CUSTOM_EL1_8M_2D*. (Default: *INVALID*) **Notes: ULA indicates Uniform Linear Array. QUAD_NONULA is a four mic array setup in a linear, non-uniform, and symmetrical manner. 6_PLUS_1_CIRC is a Seven mic array setup with 6 mics uniformly spaced on a circle, and a center microphone (has to be the last channel). CUSTOM_EL1_8M_2D is a Custom array setup.**
- **MicDist1:** Microphone distance(s), inner or radius. Range: 0~100cm (Default: 3 cm) This distance is dependent on UrocMicSetup. Note: For single channel microphone input, select a distance of zero.
- **MicDist2:** Secondary microphone distance parameter. Range: 0 ~ 100 cm (Default: 0 cm) Note: For QUAD_NONULA setups, exactly two distances are required. MicDist1 equals to the distance between the two innermost microphones, while MicDist2 indicates the distance between the two outermost microphones (Constraint: $\text{MicDist2} \neq 3 \times \text{MicDist1}$). For other UrocMicSetup types, set MicDist2 to zero.
- **FrameSize:** Size of processing frame. Options include 256, 512, and 1024 samples. (Default: 256 samples)
- **FilterLength:** Length of the processing filter. Range: 50ms~1000ms. (Default: 100ms)
- **delaymode:** Delay compensation method, with options including *MODE_INVALID*, *MODE_FIXED* and *MODE_ADAPTIVE*. (Default: *MODE_ADAPTIVE*)
- **Delay:** Delay parameter. Range: 150 ms ~ 200 ms. (Default: 300 ms)
- **processmode:** Processing mode selection. Options include *MODE_INVALID* *MODE_COM* (Communication mode, performs EC+SF+PF, configured for duplex communication) and *MODE_ASR* (Automatic speech recognition mode, performs EC+SF+PF, configured for ASR preprocessing) (Default: *MODE_COM*)
- **natt:** Noise attenuation level. Set to 0 to disable noise reduction. Range: -15dB ~ 0dB (Default: -10dB)
- **agc:** The target gain level for automatic gain control. (Default: 0 dB)
- **numSpk:** Specify the number of loudspeakers (Default: 0)
- **id:** Configuration identifier. Options: 10011 (Sampling rate 16 KHz, frameSize 256, 512, 1024; REF 1-8, in COM mode), 10012 (Sampling rate 16 KHz, frameSize 256, 512, 1024; REF 1-8, in ASR mode), and 10013 (Sampling rate 48 KHz, frameSize 512, 1024; REF 0, in COM mode) (Default: 10011)

Flow Studio GUI of uphear VQE object



The image shows the GUI for the 'upHear VQE_1' object in Flow Studio. The interface is dark-themed with a blue header bar. It features a series of controls for audio processing parameters, including a toggle switch, dropdown menus, and sliders. The parameters are listed on the left, and their current values or settings are shown on the right. The GUI is framed by a yellow border with green circular handles at the corners.

Parameter	Value/Setting
onoff	on (checked)
UrocMicSetup	INVALID
MicDist1	7.00
MicDist2	0.00
FrameSize	512
FilterLength	100
delaymode	MODE_ADAPTIVE
Delay	300
processmode	MODE_COM
natt	-15.00
agc	0.00
numSpk	1
id	10011

6.9.1.3 uphear Virtualizer

Functional description of upHear Virtualizer object

The **upHear Virtualizer** object (also displayed as **upHear Virtualizer** in the information tab) is compatible with all existing audio formats, ensuring no content is left behind. Legacy stereo and surround content is intelligently mixed to create authentic spatial impressions, delivering an immersive experience regardless of the original format. This universal compatibility ensures listeners can enjoy enhanced audio across their entire content library.

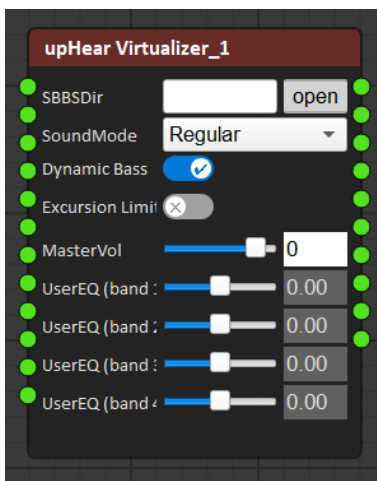
Input & Output Configuration of upHear Virtualizer object

- **Audio inputs:** 2~12
- **Audio Outputs:** 10

Adjustable Parameters of upHear Virtualizer object

- **SBBSDir:** Loads upHear Virtualizer parameters from internal buffers in SBBS files.
- **SoundMode:** Spatialization settings. Options: Off or Regular. (Default: Regular)
- **Dynamic Bass:** Controls dynamic bass enhancement. Options: ON/OFF (Default: ON)
- **Excursion Limiter:** Controls speaker excursion limiting function. Options: ON/OFF (Default: OFF)
- **MasterVol:** Master Volume control. Range: 0dB ~ -80dB (Default: 0 dB)
- **UserEQ (band 1 ~ band 4):** EQ band gain adjustment. Range: 9.5dB ~ -9.5dB. (Default: 0 dB)

Flow Studio GUI of upHear Virtualizer object



6.9.2 NTT Sonority

NTT Sonority, Inc. is a Japan-based audio technology company and a subsidiary of Nippon Telegraph and Telephone Corporation (NTT Group), one of the world's largest telecommunications companies. Headquartered in Japan, NTT Sonority specializes in cutting-edge sound solutions that integrate advanced acoustic engineering with innovative signal processing. With the partnership with NTT Sonority, we provide advanced AOs as follows:

Available AOs from NTT Sonority:

- **Intelligent Microphone Voice:** This algorithm is provided by NTT Sonority for intelligent microphone voice pick-up of headphone product.
- **Noise Level AGC:** This algorithm is provided by NTT Sonority for noise level auto gain control of headphone product. It adjusts its gain automatically in conjunction with the noise level of the surrounding environment.

NTT Sonority is redefining the future of audio with personalized listening experiences, advanced noise control, and immersive spatial sound technologies. By combining telecommunications expertise with audio engineering, the company continues to lead in innovative sound solutions for various industries, from consumer electronics to automotive and professional communication systems.

6.9.2.1 Intelligent Microphone Voice

Functional description of Intelligent Microphone Voice object

The **Intelligent Microphone Voice** object (displayed as **NTTS_IML** in the information tab) is provided by NTT Sonority for intelligent microphone voice pick-up of headphone product. It includes technologies such as beamforming and AEC for voice pick-up. The input signal data must be 16kHz sampling rate frequency and frame size of 64 samples (4ms) or 48 samples (3ms). Current setting is for 48 samples. The algorithm provides multiple parameter functions to optimize voice pickup performance based on the microphone layout of the headset. It includes volume and gain control, noise suppression, echo suppression, and beamforming parameters, allowing precise control over noise suppression levels across different frequency ranges and adjustment of volume levels for natural speech reproduction.

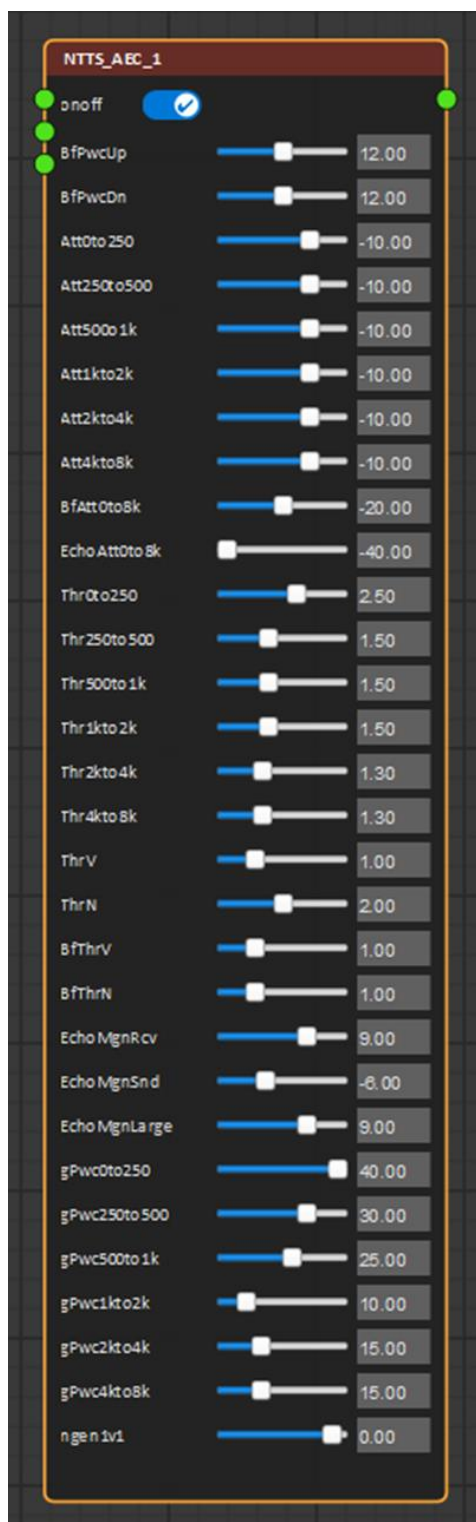
Input & Output Configuration of Intelligent Microphone Voice object

- **Audio inputs:** 3 (The first and second are microphone input, the third is reference input from Rx for AEC)
- **Audio Outputs:** 1

Adjustable Parameters of Intelligent Microphone Voice object

- **BfPwcUp / BfPwcDn:** Time constants for level calculation when the signal level is rising up or going down. Range: 4.0 to 20.0 ms (Default: 12.0 ms)
- **AttXtoY / ThrXtoY:** Sets the maximum suppression gain for stationary noise (AttXtoY) or non-stationary (BfAtt0to8k) noise within a specified frequency range. Available frequency ranges: 0–250 Hz, 250–500 Hz, 500–1000 Hz, 1000–2000 Hz, 2000–4000 Hz, and 4000–8000 Hz. Adjustable range: -40.0 to 0.0 dB.
- **BfAtt0to8k / EchoAtt0to8k:** Set maximum suppression gain for non-stationary noise (BfAtt0to8k) or echo (EchoAtt0to8k) within the frequency range of 0 to 8000Hz. Range: -40.0 to 0.0 dB. (Default: -20 dB/-40 dB)
- **ThrV / ThrN:** Set the coefficient for the stationary noise threshold (when identified as voice activity or noise). Range: 0.0 to 4.0 (Default:1.0/2.0)
- **BfThrV / BfThrN:** Set the coefficient for the beamforming threshold (when identified as voice activity or noise). Adjustable range: 0.0 to 4.0. (Default:1.0 for both)
- **EchoMgnRcv / EchoMgnSnd / EchoMgnLarge:** Set the coefficient applied to the echo level (margin), where EchoMgnRcv is for the general echo level, EchoMgnSnd is specifically applied during speech detection and EchoMgnLarge is for excessive echo levels. Adjustable range: -20.0 to 20.0 dB. Default: 9 / -6 / 9 dB)
- **gPwcXtoY:** Sets the time constant for time smoothing of gain within a specified frequency range. Available frequency ranges: 0–250 Hz, 250–500 Hz, 500–1000 Hz, 1000–2000 Hz, 2000–4000 Hz, and 4000–8000 Hz. Adjustable range: -4.0 to 40.0 ms
- **ngen1v1:** Set the CNG (known as Comfort Noise Generator) noise level. Adjustable range: -99.0 to 6 dB. (Default: 0 dB)

Flow Studio GUI of Intelligent Microphone Voice object



6.9.2.2 Noise Level AGC

Functional description of Noise Level AGC object

The noise level AGC (Automatic Gain Control) algorithm, provided by NTT Sonority, enables automatic gain adjustment for headphone products based on ambient noise levels. The algorithm dynamically increases the gain in response to higher noise levels and decreases it when the surrounding noise level is lower, ensuring optimal audio performance. It enhances the user's listening experience by preventing disruptions caused by excessive external noise. Through noise detection, it automatically adjusts the audio gain to maintain optimal sound quality. The input signal supports 16kHz/48kHz sampling rate frequency and frame size must be 48 samples.

The algorithm offers multiple parameter functions to optimize audio gain control based on ambient noise intensity for headsets. It incorporates level calculations using time constants, threshold tables, and gain tables to regulate gain adjustment intensity and noise detection thresholds, thereby determining the sensitivity of the noise assessment mechanism.

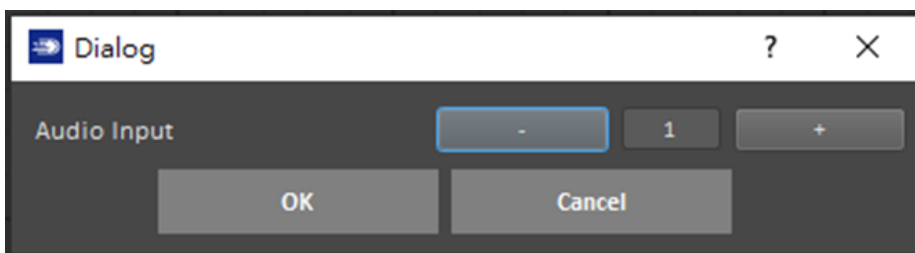
Input & Output Configuration of Noise Level AGC object

- **Audio inputs: 2 to 3** (The first is environmental noise input, the second or third is audio input depending on mono or stereo to adjust channel input number)
- **Audio Outputs: 1**

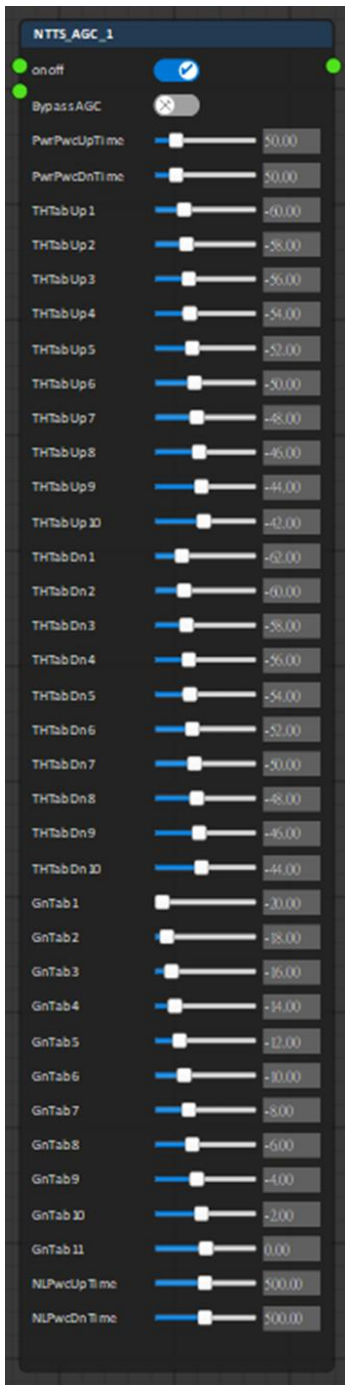
Adjustable Parameters of Noise Level AGC object

- **Onoff / BypassAGC:** Enable or bypass whole audio object / AGC function
- **PwrPwcUpTime / PwrPwcDnTime:** Set the time constant for level calculation when the signal level is increasing / decreasing. Adjust range: 4 to 300 ms
- **THTabUp1–10:** The threshold table with steps from 1 to 10, which is applied when the signal level is increasing. Range: -80 to 0 dB.
- **THTabDn1–10:** Threshold table with steps from 1 to 10, which is applied when the signal level is going down. Adjust range: -80 to 0 dB.
- **GnTab1–11:** Audio gain table with steps from 1 to 11. Range: -40 to 10 dB
- **NLPwcUpTime:** Time constant for noise level calculation (when level is rising). Range: 20 to 1000 ms
- **NLPwcDnTime:** Time constant for noise level calculation (when level is going down). Range: 20 to 1000 ms

Note: There is pop-up window to inquire about audio/voice input channel number to define audio object input total number is 2 or 3 (include first input port should be noise input)



Flow Studio GUI of Noise Level AGC object



6.9.3 Dirac

Dirac is a global audio tech company with headquarters in Uppsala, Sweden, and R&D facilities in Copenhagen, Denmark, and Bangalore, India, with representation in China, Germany, Japan, Korea, and the United States. With a strong focus on the automotive and home audio industries, Dirac collaborates with leading audio innovators and manufacturers to enhance dynamics, clarity, and immersive sound experiences in any environment. Peerless and Dirac Announce Strategic Partnership to Empower Home and Pro Audio Manufacturers to More Efficiently Create World-Class Room Correction Systems. The partnership integrates Dirac's award-winning Dirac Live solution into Peerless' Flow DSP audio development platform to enable home and pro audio companies to more seamlessly and efficiently integrate premium room correction into their product portfolio.

Available AOs from Dirac:

- **Dirac Live Processor:** Dirac Live is a revolutionary software designed to enhance the overall listening experience by significantly improving the sound quality. The software leverages advanced algorithms to precisely measure and optimize the audio output from speakers, tailoring it specifically to the unique acoustic characteristics of the listening environment.

The Flow Studio integration of Dirac Live accelerates manufacturers' time-to-market, improves their audio development efficiency, and optimizes sound quality per dollar. The partnership represents a significant milestone for both companies in their collective mission to scale premium sound reproduction across the home and pro audio markets.

6.9.3.1 Dirac Live Processor

Functional description of Dirac Live Processor object

Dirac Live Processor is a revolutionary object designed to enhance the overall listening experience by significantly improving sound quality. The Dirac Live Processor leverages advanced algorithms to precisely measure and optimize the audio output from speakers, tailoring it specifically to the unique acoustic characteristics of the listening environment. As a fully integrated Flow DSP module, the Dirac Live processor enables seamless customization of signal paths and effortless integration with other Flow DSP components. For developers, Flow Studio provides an intuitive configuration interface via SSH, streamlining the implementation of Dirac Live processing during product development.

Currently supported platforms: Raspberry Pi (2.0 configuration) and A113X2 (7.1.4, 5.1.2 configuration)

Input & Output Configuration of Dirac Live Processor object

- **Audio inputs:** 2(2.0), 8(5.1.2), 12(7.1.4)
- **Audio Outputs:** 2(2.0), 8(5.1.2), 12(7.1.4)

Adjustable Parameters of Dirac Live Processor object

- **configure:** 2.0 (for Raspberry Pi 2.0 audio system) or 5.1.2, 7.1.4 (for A113X2 atmos audio system)
- **username:** ssh login name.
- **password:** ssh login password.
- **device name:** name to be shown on Dirac Live.
- **input info:**
 - **name:** specifies the name of the input channel (e.g., left, right, or LFE (Low Frequency Effects))
 - **type (optional):** denotes special channel types (e.g., "channel_type_lfe" for low frequency effects channels)
- **speaker info:**
 - **Assoc in:** denotes which input the speaker is associated with.
 - **name:** the name of the speaker (e.g., "left", "right")
 - **type:** the type of speaker. Options include fullrange_speaker, midrange_speaker, woofer_speaker, subwoofer_speaker, tweeter_speaker, bass_managed_speaker, or dolby_atmos_speaker
 - **group id:** group id of related speakers (speakers with the same group_id will be grouped together)
- **update:** click on the update button to send the parameters to the concord server to take effect when you are sure the concord server's parameters are configured correctly

Note : If it is A113x2 platform please use "Flow APO" as remote target. both platforms' flw config path is /opt/dirac/concord/config/. On the A113X2 platform, an Attenuator AO should be concatenated after the Dirac Live Processor AO. See Fig.1

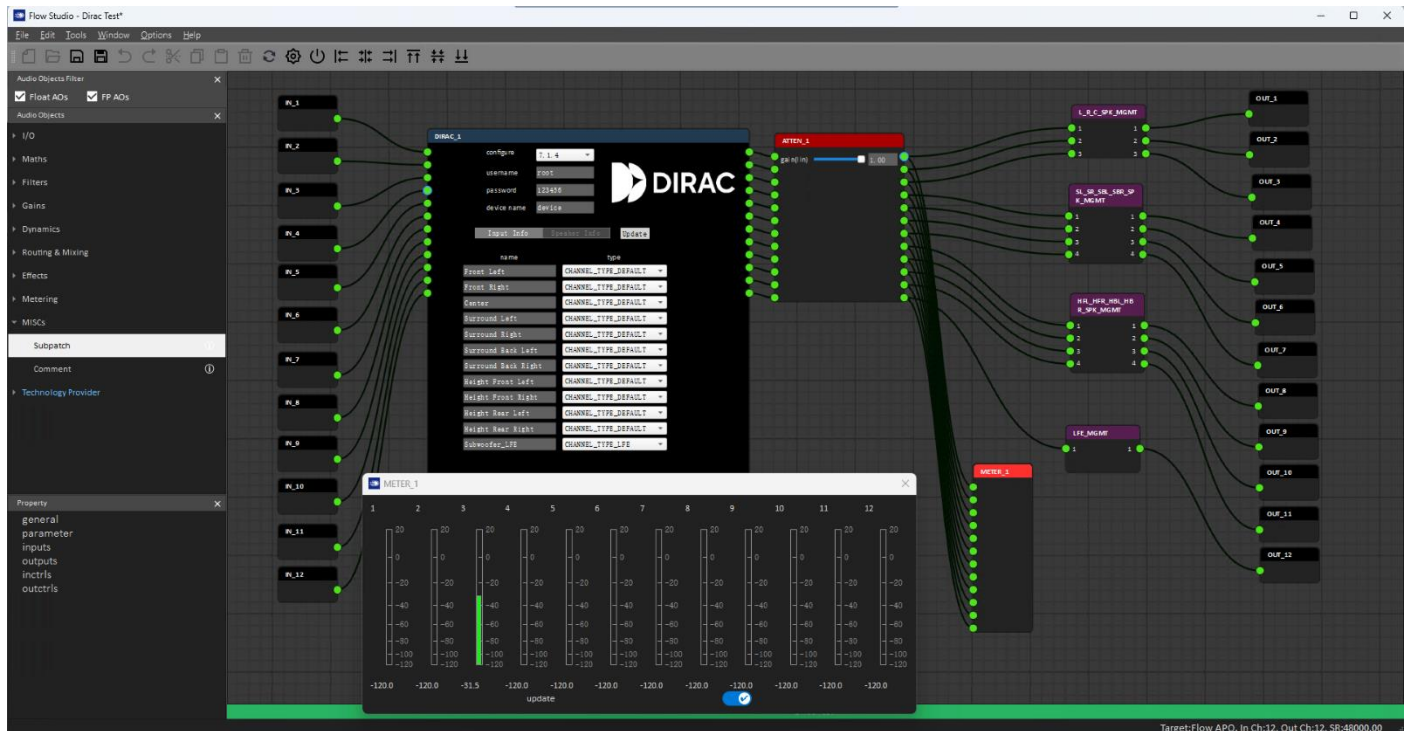


Fig.1 suggested signal flow arrangement with Dirac Live Processor

Flow Studio GUI of Dirac Live Processor object



6.9.4 Aizip

Aizip is a leader in model design of AI for IoT (AIoT). With its breakthrough neural network architecture and proprietary automated design tools, Aizip has demonstrated a wide range of deep neural networks (DNN) with superior performance. The company focuses on developing TinyML models for AIoT (Artificial Intelligence of Things) applications, providing powerful micro AI engines covering vision, audio, and time-series domains.

Aizip collaborated with Renesas to launch ultra-efficient small language models (SLMs) and compact AI agents based on ARM microprocessor units (MPUs). Additionally, the company has partnered with Ceva to expand the embedded AI NPU ecosystem by providing pre-optimized neural networks for wake word detection, facial recognition, and speaker identification in smart edge devices, reducing development time

Available AOs from Aizip:

- **Vocal Removal:** This algorithm removes vocal track from song in real time. Aizip achieves the separation of voice and music tracks from AI solution. It can be used in party box, karaoke and automobile devices. The algorithm has several outstanding technical features, tiny memory footprint, support all genres and languages and preserve high-quality

Aizip specializes in Tiny AI (micro artificial intelligence) and TinyML (micro machine learning), enabling efficient AI models to run on low-power, low-compute devices. These models are optimized for vision, audio, and time-series applications and can operate on various hardware architectures, including MCUs (Microcontroller Units), MPUs (Microprocessor Units), and NPUs (Neural Processing Units). The FlowStudio integration of Aizip AI vocal removal accelerates manufacturers' time-to-market, improves their audio development efficiency, and optimizes sound quality per dollar.

6.9.4.1 Vocal Removal

The Aizip Intelligent Audio (AIA) Algorithm enables real-time vocal track removal from songs. Utilizing advanced AI-driven signal separation techniques, Aizip effectively isolates vocal and instrumental components, making it ideal for applications such as **party speakers, karaoke systems, and automotive audio devices**.

Key technical features of the AIA algorithm include:

- **Minimal memory footprint:** Optimized for resource-constrained environments.
- **Broad compatibility:** Supports all music genres and languages.
- **High-fidelity output:** Maintains superior audio quality after separation.

This algorithm unlocks significant potential for AI-driven audio solutions on compact, versatile platforms, ensuring seamless integration across a wide range of embedded and consumer electronics devices.

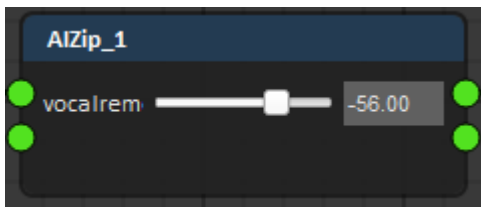
Input & Output Configuration of Aizip vocal removal object

- **Audio inputs:** 2
- **Audio Outputs:** 2

Adjustable Parameters of Aizip vocal removal object

- **onoff / Bypass:** Enable or bypass whole audio object / Vocal removal function
- **vocalremove:** Controls the intensity level of vocal removal. A lower intensity value results in stronger vocal suppression. The **minimum level is -200**, Complete vocal removal, leaving only the music track. Higher values retain more vocal content, while lower values increase suppression. **Adjustment range: -200 to 0.**

Flow Studio GUI of Dirac Live Processor object



6.9.4.2 Dialogue Enhancement

The Aizip Dialogue Enhancement solution is designed to significantly improve speech clarity in challenging acoustic environments. The algorithm enhances intelligibility while preserving the natural timbre of the speaker's voice.

This technology is ideal for applications such as **smart speakers, hearing aids, conference systems, and in-vehicle infotainment** where clear and uninterrupted communication is critical

Key technical features of the algorithm include:

- **Ultra-Low Latency:** Suitable for time-critical devices such as live communication systems and assistive hearing devices..
- **TinyML Optimization:** Low power consumption for battery-powered device.
- **Robust Across Environments:** Trained with large, diverse datasets to handle various noise types, accents, and languages.

This algorithm unlocks significant potential for AI-driven audio solutions on compact, versatile platforms, ensuring seamless integration across a wide range of embedded and consumer electronics devices.

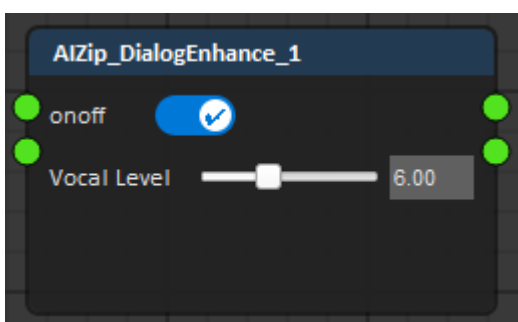
Input & Output Configuration of Aizip dialogue enhancement object

- **Audio inputs:** 2
- **Audio Outputs:** 2

Adjustable Parameters of Aizip dialogue enhancement object

- **onoff / Bypass:** Enable or bypass whole audio object / Dialogue enhancement function
- **Vocal Level :** Controls the intensity of the vocal gain. A higher value results in greater vocal emphasis. Maximum level is 12. Adjustment range of 1 to 12.

Flow Studio GUI of Dirac Live Processor object



6.9.5 Flow DSP

Flow DSP provides a suite of advanced audio objects (AOs) developed by TYMPHANY to enhance speech clarity, maintain consistent audio levels, and optimize overall sound quality in real time. These AOs leverage cutting-edge DSP (Digital Signal Processing) and AI-driven algorithms to provide users with professional-grade audio enhancements tailored for various applications.

Flow DSP AOs include two specialized AO groups and several individual AOs:

- **Advanced Playback** – A collection of playback enhancement tools that optimize audio output for different environments and devices, ensuring an immersive and high-quality listening experience.
- **AI DeepTalk** – A powerful AI-driven suite focused on speech-related solutions, utilizing deep learning techniques to enhance voice intelligibility, reduce noise, and improve communication clarity.
- **De-esser**: Effectively reduces excessive sibilance in speech, ensuring a smoother and more natural vocal tone.
- **Smart EQ (beta)**: Intelligently balances audio frequencies to enhance speech clarity while suppressing unwanted resonances.
- **CLIPFIX (beta)**: Repairs clipped or distorted audio signals, restoring lost details and improving overall sound quality.
- **AGC (Automatic Gain Control)** – Maintains consistent loudness by dynamically adjusting the input gain, preventing volume fluctuations.

By integrating Flow DSP AOs into your workflow, user can achieve superior audio quality with minimal effort, whether for broadcasting, voice communication, or media production.

6.9.5.1 Advanced Playback

Advanced Playback consists of a suite of powerful audio objects (AOs) designed to optimize playback quality, enhance spatial perception, and create a more immersive listening experience. These advanced processing tools allow users to fine-tune audio playback for various environments, speaker configurations, and listener preferences.

Available Advanced Playback AOs:

- **Dynamic Filter** – Adaptive filtering that responds to real-time audio dynamics, optimizing tonal balance and clarity.
- **Dynamic Bass** – Enhances low-frequency response while preserving balance and avoiding distortion, delivering deeper and richer bass.
- **Virtual Bass** – Psychoacoustic bass enhancement that extends perceived low-end frequencies, even on speakers with physical limitations.
- **Dynamic Loudness** – Intelligent loudness control that maintains clarity and presence across different playback levels.
- **Spatializer** – Creates a more immersive 3D sound experience by simulating spatial cues for enhanced depth and positioning.
- **Crossfeed** – Balances stereo image perception by blending channel signals for a more natural headphone listening experience.
- **Crosstalk Cancellation** – Reduces unwanted crosstalk effects, improving stereo separation and spatial accuracy.

With **Advanced Playback AOs**, users can seamlessly integrate them into their audio processing workflows, enabling precise customization and real-time adjustments. Whether enhancing bass depth, refining spatial audio, or optimizing playback for different devices, these tools empower developers to achieve superior audio performance with efficiency and ease.

6.9.5.1.1 Dynamic Filter

Functional description of Dynamic Filter object

The **Dynamic Filter** object (displayed as **DYNAMIC_FILTER** in the information tab) employs level-dependent filtering techniques to adapt its response based on the input signal's sound pressure level (SPL). It incorporates two distinct EQ blocks, each optimized for different signal levels—one tailored for low SPL and the other for high SPL. A mixer seamlessly blends between these EQ blocks, ensuring a smooth transition. The system is governed by a control algorithm that continuously analyzes the low-frequency energy of the input signal. Based on a predefined threshold, it maps this energy to a value between 0 and 1, which then dictates the mixer's crossfade behavior, dynamically adjusting the balance between the two EQ blocks.

This creates a smooth, dynamic transition between EQ settings as the input signal level changes. The algorithm is useful when the system needs to have different EQ setting on high and low level outputs. Fig.1 shows a high-level block diagram of Dynamic Filter algorithm.

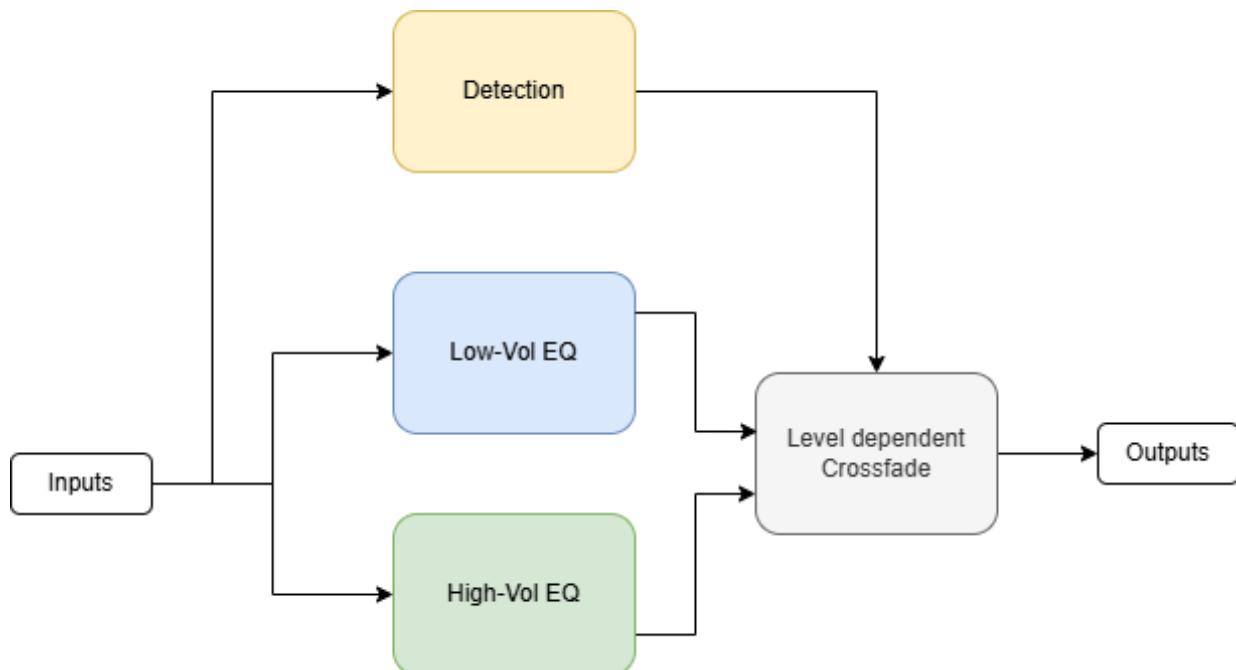


Fig.1 block diagram of Dynamic Filter

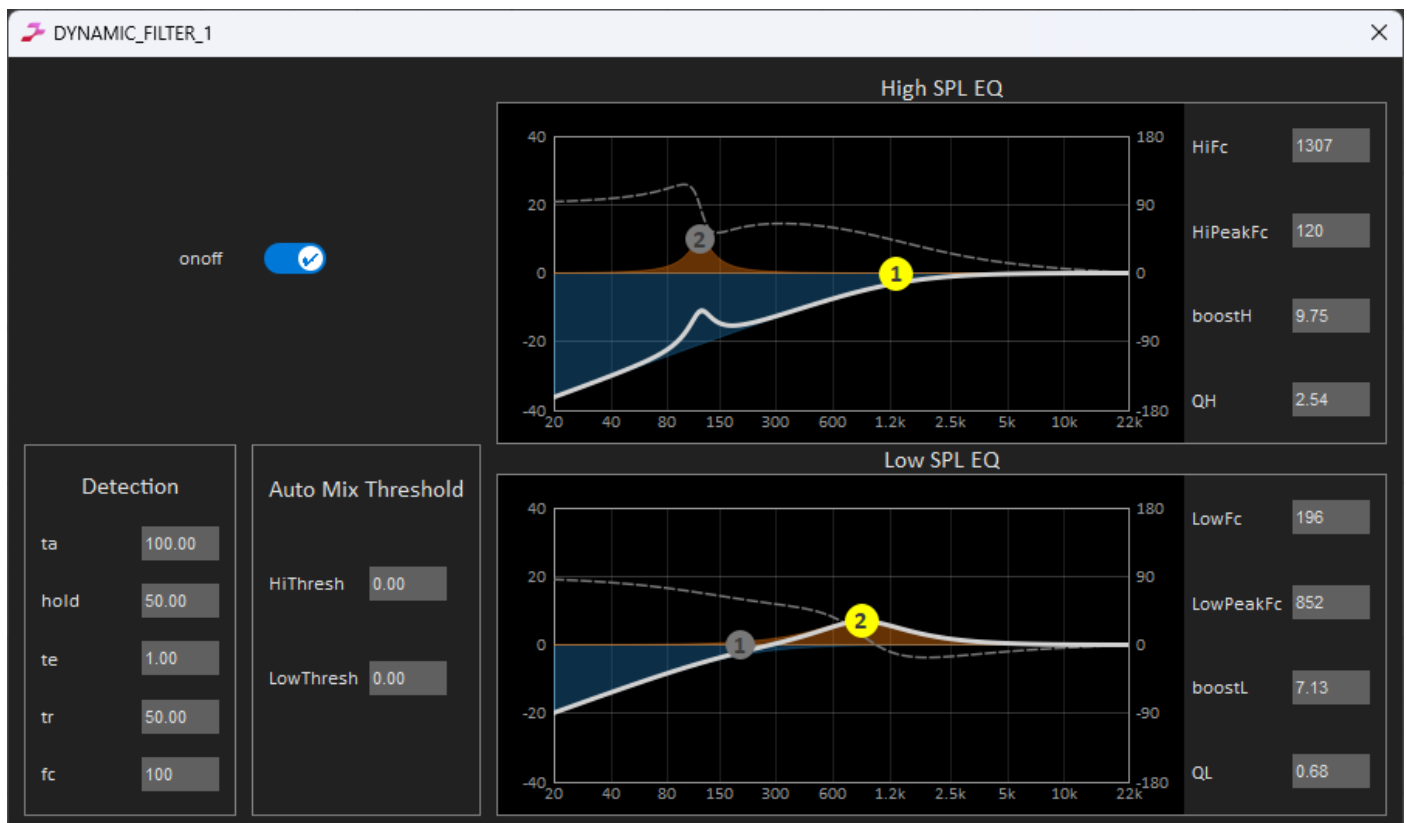
Input & Output Configuration of Dynamic Filter object

- Audio inputs: 2
- Audio Outputs: 2

Adjustable Parameters of Dynamic Filter object

- **Lowthresh / Hithresh:** Thresholds for the dynamic filter to start/end smooth transition between Low SPL EQ and High SPL EQ. The low SPL EQ is set below this threshold, and the High SPL EQ is set less than this threshold
- **ta:** Attack time (ms) for RMS envelope detection.
- **Hold:** Hold time (ms) RMS Envelop detection
- **te:** RMS Envelope detection period for RMS energy
- **tr:** RMS Envelope detection release time specified in ms.
- **fc:** Set low-pass filter's frequency before RMS envelope detection
- **LowFc:** Set low SPL EQ of high-pass filtering's frequency
- **LowPeakFc:** Set low SPL EQ of the peaking filtering's frequency
- **boostL:** Set the boost/attenuated gain in dB for low SPL EQ of the peaking filter
- **QL:** Set the Q factor for low SPL EQ of the peaking filter
- **HiFc:** Set high SPL EQ of high-pass filtering's frequency
- **HipeakFc:** Set high SPL EQ of the peaking filtering's frequency
- **boostH:** Set the boost/attenuated gain in dB for high SPL EQ of the peaking filter
- **QH:** Set the Q factor for high SPL EQ of the peaking filter

Flow Studio GUI of Dynamic Filter object



6.9.5.1.2 Dynamic Bass

Functional description of Dynamic Bass object

The **Dynamic Bass object** (displayed as **DBass** in the information tab) is a sophisticated level-dependent bass enhancement algorithm designed to optimize low-frequency response across varying playback volumes. By analyzing the input signal level in real-time, the algorithm applies carefully calibrated bass enhancement that inversely correlates with volume. At lower listening levels, where bass frequencies are naturally less perceptible to human hearing, the system provides greater low-frequency enhancement. As volume increases, the enhancement gradually reduces to maintain a balanced tonal presentation without overloading the system. Fig.1 shows the frequency response of the algorithm with different input level

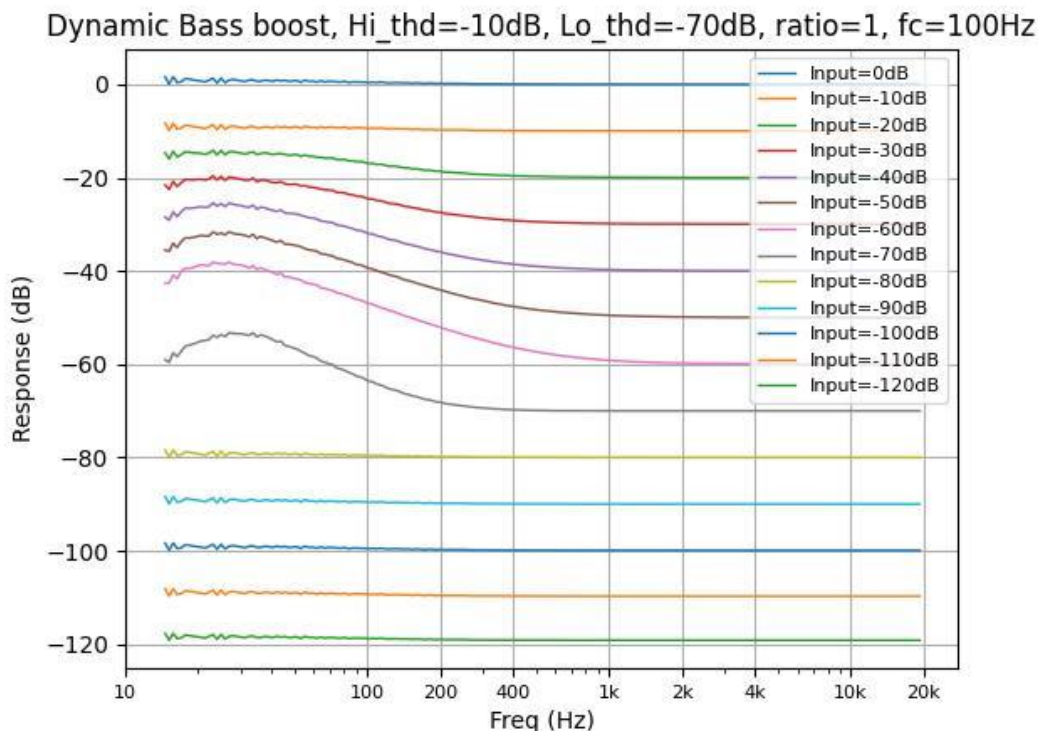


Fig.1 frequency response of the Dynamic Bass

Input & Output Configuration of Dynamic Bass object

- Audio inputs: 1~32
- Audio Outputs: 1~32

Adjustable Parameters of Dynamic Bass object

- **onoff**: Enable or bypass Dynamic Bass object.
- **HiThres**: High threshold, range from -30 to 0dB. Sets the upper limit for detector action. Signal inputs greater than this threshold receive no bass boost.
- **LowThres**: Low threshold, range from -70 to -30dB. Sets the lower limit for detector action. Signal inputs below this threshold receive no bass boost.
- **fc**: Cutoff frequency for the boosting filter. Range: 20 to 350 Hz.
- **ratio**: Boost up ratio to represent boost intensity. Range: 0.2 to 1.2.

Flow Studio GUI of Dynamic Bass object



6.9.5.1.3 Virtual Bass

Functional description of Virtual Bass object

The **Virtual Bass** object (displayed as **VBass** in the information tab) leverages advanced psychoacoustic principles to create the perception of low-frequency content beyond what small speakers can physically reproduce. Rather than attempting to force small drivers to generate true bass frequencies (which often results in distortion and inefficiency), this approach uses band extension methods to synthesize harmonically-related higher frequencies that trick the human auditory system into perceiving bass that isn't physically present.

This application is particularly valuable for small speaker systems, such as those found in mobile devices, portable speakers, slim TVs, and compact soundbars, where physical limitations prevent the reproduction of true deep bass. Virtual bass allows these devices to deliver a fuller, more satisfying audio experience without requiring larger drivers or enclosures.

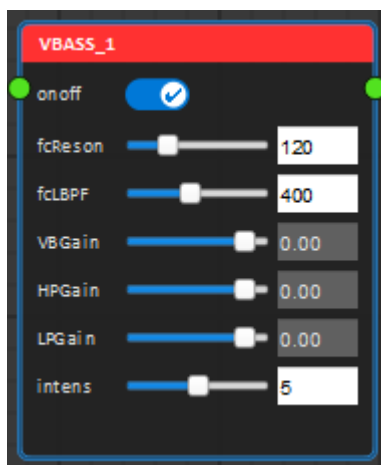
Input & Output Configuration of Virtual Bass object

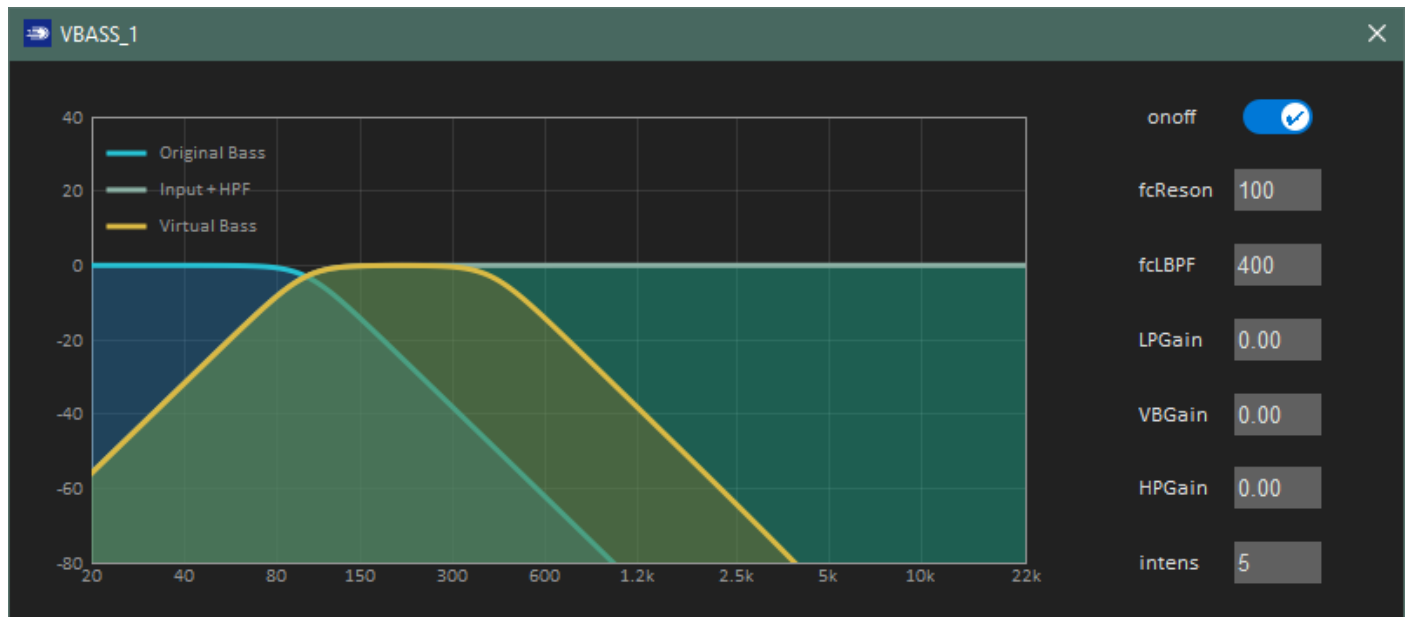
- **Audio inputs:** 1~32
- **Audio Outputs:** 1~32

Adjustable Parameters of Virtual Bass object

- **onoff:** Enable or bypass Virtual Bass object
- **fcReson (Hz):** Set the resonance frequency of the speaker
- **fcLBPF (Hz):** Set the low pass filter frequency to determine the higher limit of bass enhancement range.
- **VBGain (dB):** Set the gain of bass harmonic to control generated harmonic for bass.
- **HPGain (dB):** Set the gain of HFIL (High Pass) filter to control the original audio mid and treble.
- **LPGain (dB):** Set the gain of LFIL (Low Pass) filter to control the original audio bass.
- **Intens:** Set the intensity of NLD function to adjust harmonic strength.

Flow Studio GUI of Virtual Bass object





6.9.5.1.4 Dynamic Loudness

Functional description of Dynamic Loudness object

The **Dynamic Loudness** object (displayed as **DLOUDNESS** in the information tab) employs equal loudness curves to adaptively adjust frequency response based on varying input levels. This approach is rooted in psychoacoustic research showing that human hearing perceives different frequencies with varying sensitivity depending on volume. At lower listening levels, our ears are less sensitive to bass and treble frequencies compared to midrange sounds. Dynamic Loudness compensates for this natural hearing limitation by applying Fletcher-Munson or ISO 226:2003 equal loudness contours to intelligently boost low and high frequencies when volume decreases.

By analyzing input levels in real-time, the system automatically determines the appropriate frequency compensation needed. As volume increases, the bass and treble enhancement gradually reduces to maintain a natural tonal balance. This results in consistent perceived frequency response across all listening levels, ensuring full, rich sound even at quieter volumes where bass and treble would typically sound diminished. Fig.1 shows the frequency response of the algorithm with different input level

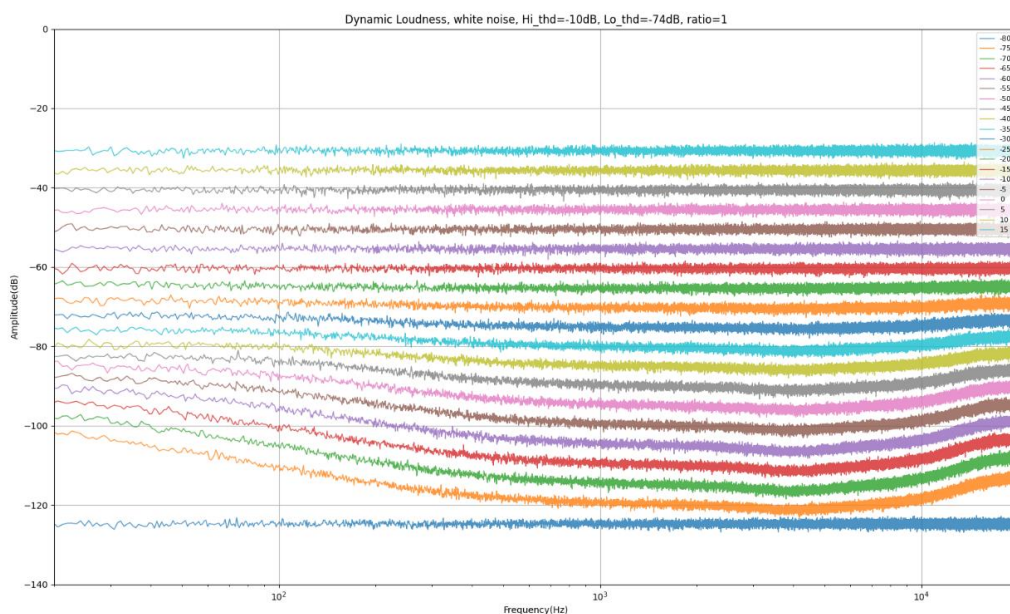


Fig.1 frequency response of the Dynamic Loudness

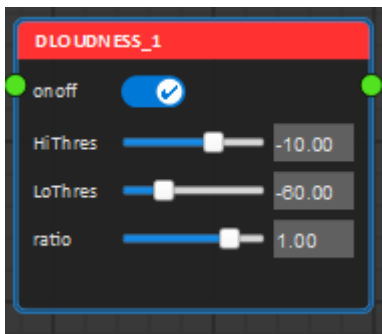
Input & Output Configuration of AO

- **Audio inputs:** 1~32
- **Audio Outputs:** 1~32

Adjustable Parameters of AO

- **onoff:** Enable or bypass Dynamic Loudness object
- **HiThres:** High threshold, range from -30 to 0dB. Sets the upper limit for detector action. Signal inputs greater than this threshold receive no bass boost.
- **LowThres:** Low threshold, range from -70 to -30dB. Sets the lower limit for detector action. Signal inputs below this threshold receive no bass boost.
- **ratio:** Boost up ratio to represent boost intensity. The range from 0.2 to 1.2.

Flow Studio GUI



•

6.9.5.1.5 Dynamic EQ

Functional description of Dynamic EQ object

The **Dynamic EQ** object (displayed as DYNAMIC_EQ in the information tab) employs equal loudness curves to adaptively adjust frequency response based on varying input levels. This approach is rooted in psychoacoustic research showing that human hearing perceives different frequencies with varying sensitivity depending on volume. At lower listening levels, our ears are less sensitive to bass and treble frequencies compared to midrange sounds. Dynamic Loudness compensates for this natural hearing limitation by applying Fletcher-Munson or ISO 226:2003 equal loudness contours to intelligently boost low and high frequencies when volume decreases.

By analyzing input levels in real-time, the system automatically determines the appropriate frequency compensation needed. As volume increases, the bass and treble enhancement gradually reduces to maintain a natural tonal balance. This results in consistent perceived frequency response across all listening levels, ensuring full, rich sound even at quieter volumes where bass and treble would typically sound diminished. Fig.1. 2.shows the frequency response of the algorithm with different input levels.

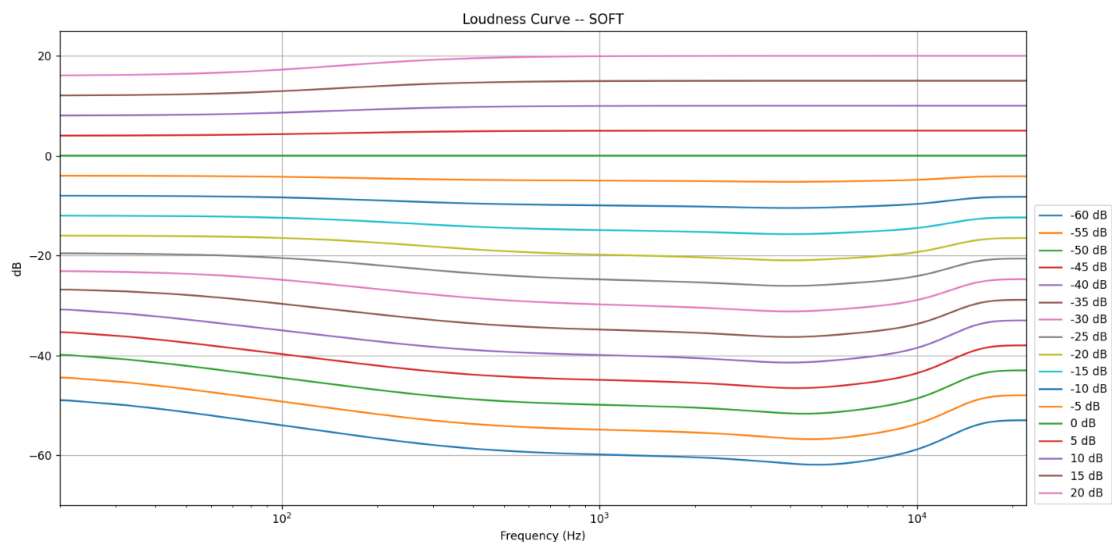


Fig.1 Dynamic EQ curve frequency response - SOFT

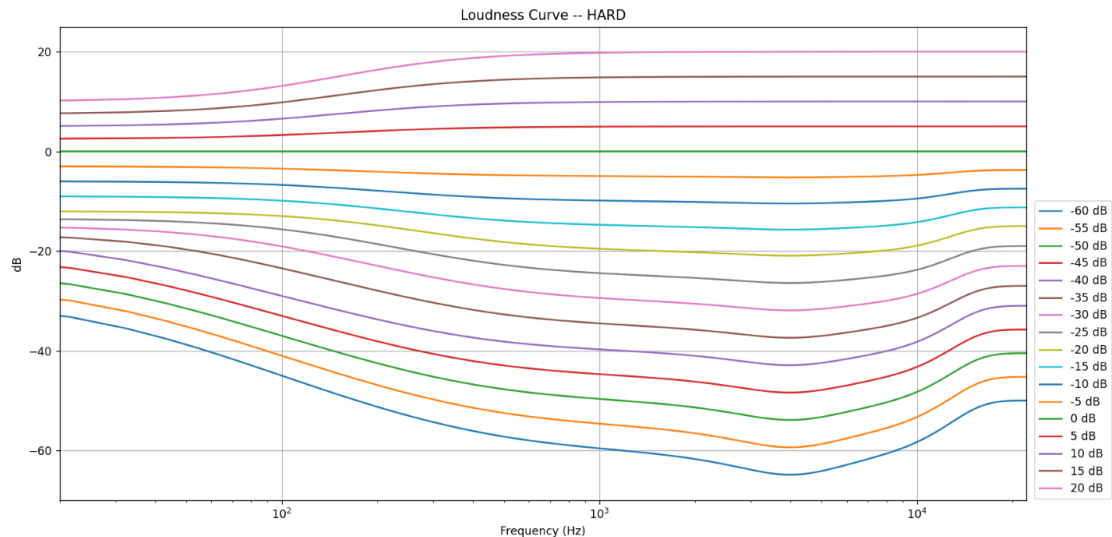


Fig.2 Dynamic EQ curve frequency response - HARD

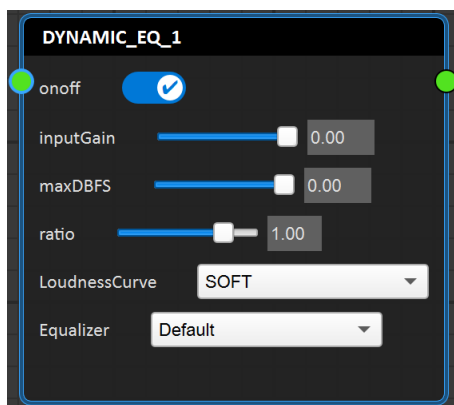
Input & Output Configuration of Dynamic EQ object

- **Audio inputs:** 1~32
- **Audio Outputs:** 1~32

Adjustable Parameters of Dynamic EQ object

- **onoff:** Enable or bypass the Dynamic EQ object.
- **inputGain:** Adjusts the output volume level. This gain is applied regardless of whether the object is enabled or bypassed. Range: -60 to 0.
- **max dBFS:** This parameter defines which input level (in dBFS) maps to 0 dB on the loudness curve, thereby determining the amount of frequency compensation applied at different volume levels. Range: -20 to 0.
- **ratio:** Boost-up ratio that determines the intensity of the bass enhancement. Range: 0.2 to 1.2.
- **Loudness Curve:** Two modes are available — SOFT and HARD.
- **Equalizer:** A predefined EQ that can be selected from four modes — default, game, music, and movie.

Flow Studio GUI of Dynamic EQ object



6.9.5.1.6 Spatializer

Functional description of Spatializer object

The **Spatializer** object, displayed as **SPATIALIZER** in the information tab, is an advanced sound image widening technique designed to enhance the **perceived spatial dimensions** of stereo audio. By manipulating **frequency-dependent delays** and adjusting the **channel differences** between the left and right signals, this effect creates a more immersive and expansive listening experience.

This technology is particularly useful in **music production, game audio, and virtual reality applications**, where a wider stereo image can add depth and realism to the sound. By carefully controlling phase relationships and spatial cues, the Spatializer can make a mix feel larger and more enveloping without introducing unwanted artifacts.

Whether used for subtle stereo enhancement or dramatic spatial effects, the Spatializer provides a versatile solution for **creating a sense of width, depth, and movement** in audio, making it a valuable tool for modern sound design and production.

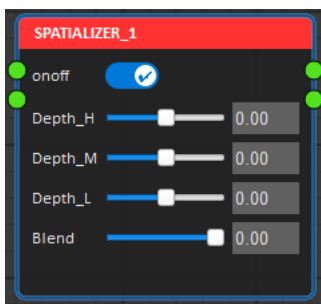
Input & Output Configuration of Spatializer object

- **Audio inputs:** 2
- **Audio Outputs:** 2

Adjustable Parameters of Spatializer object

- **onoff:** Enable or bypass Spatializer audio object
- **Depth_H:** Sound image widening control on high frequency band (dB)
- **Depth_M:** Sound image widening control on mid frequency band (dB)
- **Depth_L:** Sound image widening control on n low frequency band (dB)
- **Blend:** Mix between the original signal and sound image enhanced signal,

Flow Studio GUI of Spatializer object



6.9.5.1.7 Crossfeed

Functional description of Crossfeed object

The **Crossfeed** object, displayed as **CROSSFEED** in the information tab, is a binaural audio processing that simulates speaker listening experience on headphones. It blends a small portion of the left channel into the right ear and vice versa, mimicking how both ears hear sound being playback by real speakers. This reduces the extreme stereo separation of headphones, making the soundstage feel more natural and reducing listening fatigue. Fig.1 shows the Crossfeed Phenomenon when listening to real speakers versus simulated with headphones.



Fig.1 Crossfeed Phenomenon

Input & Output Configuration of Crossfeed object

- **Audio inputs:** 2
- **Audio Outputs:** 2

Adjustable Parameters of Crossfeed object

- **onoff:** Enables or bypasses the Crossfeed process. (Default: "Enable")
- **fftsize:** 512/1024, frame size of FFT process that determines the frequency resolution. The higher the resolution, the more precise the HRTF filters become, at the cost of higher latency and memory consumption. (Default: 512)
- **angle (degree):** Angle from the center of the head to a single sound source. 0 degrees represents directly in front. This angle also determines the orientation of both virtual speakers. (Fig. 2 shows the virtual speaker position in angle 45°)
- **distance (cm):** Distance from speaker to the center of head. This value also affects the placement distance of the speakers. (Fig. 2 shows the virtual speaker position in distance 25cm)
- **head (cm):** Head diameter. It influences the HRTF simulation and can be customized for user-specific personalization.

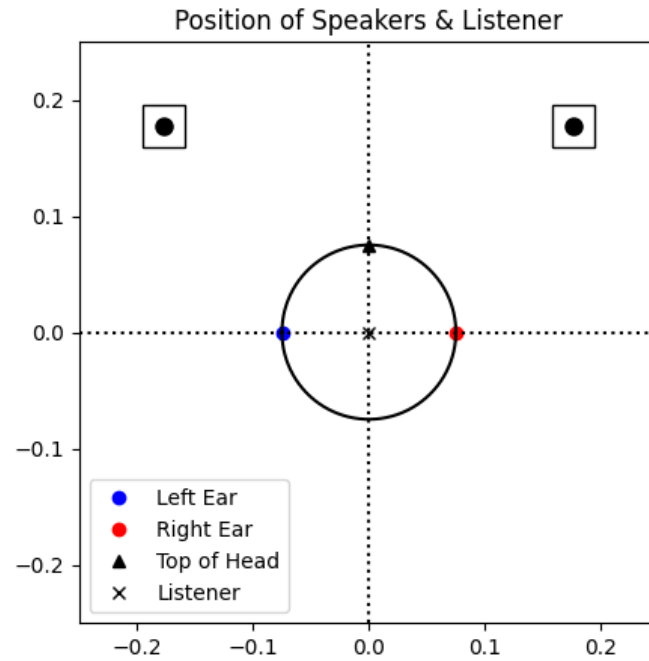
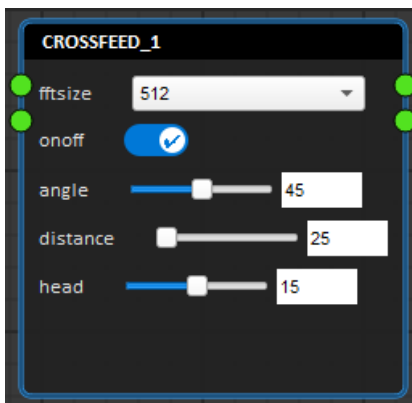


Fig.2 Position of virtual speaker and listener with angle: 45° and distance: 25cm.

Flow Studio GUI of Crossfeed object



6.9.5.1.8 CTC

Functional description of CTC object

The **CTC (Crosstalk Cancellation)** object, also displayed as **CTC** in the information tab, is designed to enable **binaural audio playback over speakers** by eliminating interference from the opposite audio channel. By applying precise cancellation techniques, CTC enhances the **stereo separation** of binaural recordings, resulting in a clearer and more immersive spatial experience. The CTC object operates within a **band-pass frequency range**, which can be adjusted based on the **acoustic response of the target device**. This flexibility allows for fine-tuning to optimize performance across different speaker setups. As illustrated in **Fig. 1**, CTC effectively cancels unwanted interference, ensuring that each ear receives a more accurate representation of the intended spatial sound.

It is important to note that **CTC should only be used for binaural audio playback**. To achieve a **spatial widening effect** that creates the illusion of listening to widely spaced virtual speakers on narrowly positioned physical speakers, additional **binaural processing techniques such as Crossfeed** should be applied before CTC in the signal flow. The correct method for implementing this process is demonstrated in **Fig. 2**, ensuring optimal audio spatialization for an immersive listening experience.

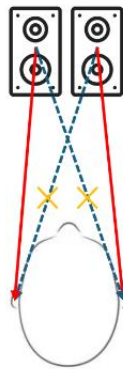


Fig.1 CTC Illustration

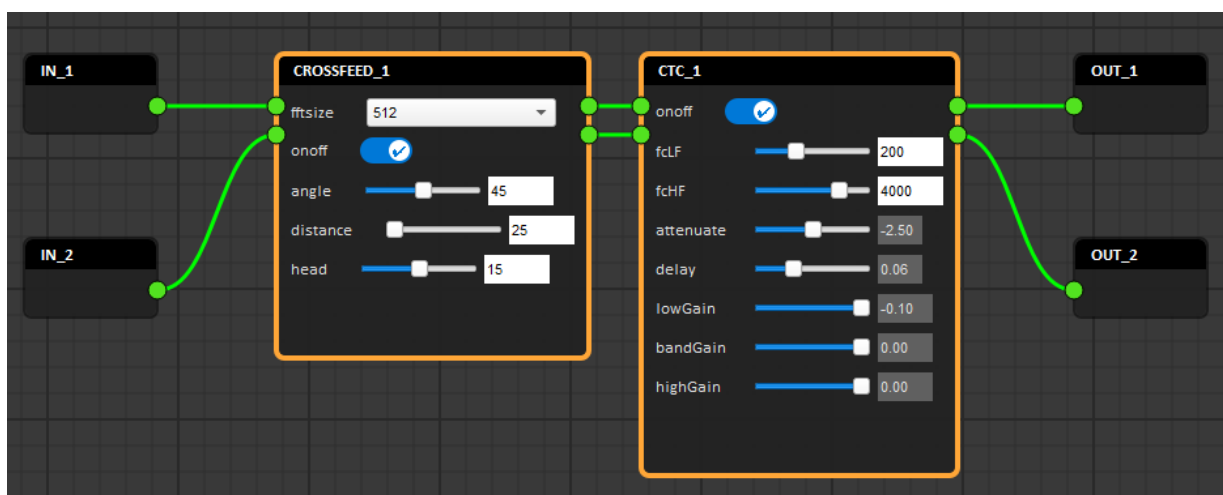


Fig.2 Correct usage diagram of Crossfeed + CTC

Input & Output Configuration of CTC object

- **Audio inputs: 2**
- **Audio Outputs: 2**

Adjustable Parameters of CTC object

- **onoff**: Enables or bypasses the AO process. (Default: “Enable”)
- **fcLF (Hz)**: Defines the low pass filter and the lower cutoff frequency of the band pass filter.
- **fcHF (Hz)**: Defines the high pass filter and the upper cutoff frequency of the band pass filter.
- **attenuate (dB)**: Adjust attenuation based on ILD (Interaural Level Difference). (Default: -2.5dB)
- **delay (ms)**: Adjust attenuation based on ITD (Interaural Time Difference). (Refer to Fig. 3 for the parameter settings)
- **lowGain (dB)**: Adjust the gain of low pass frequency content.
- **bandGain (dB)**: Adjust the gain of band pass frequency content.
- **highGain (dB)**: Adjust the gain of high pass frequency content.

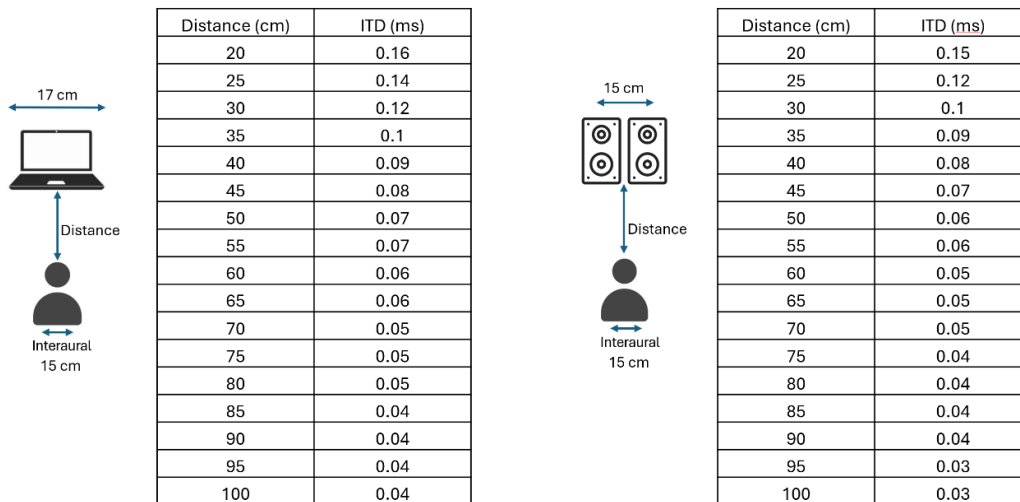
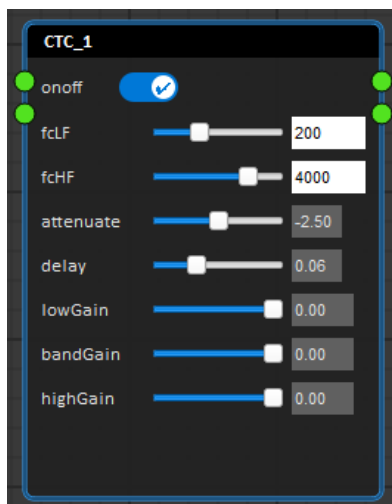


Fig.3 ITD Settings for Different Configurations

Flow Studio GUI



6.9.5.1.9 AFS

Functional description of AFS object

Acoustic Feedback Suppression is mainly used in party speakers to prevent the sharp screeching sound caused by microphones and speakers being too close together or improperly positioned. This kind of feedback negatively affects the user experience and is more likely to occur during outdoor singing or group events. Therefore, a real-time and automatic suppression mechanism is needed to improve overall audio stability and comfort.

The system works by analyzing the microphone input in real time to detect typical feedback characteristics and automatically applies notch filters to suppress the corresponding frequencies. The input consists of the speaker's intended input signal combined with the microphone signal, and the output is the final signal sent to the speaker. The system supports up to 24 notch bands and includes several key adjustable parameters:

There are still limits to feedback suppression. When the feedback is too strong or occurs across too many frequency bands at once, the system may not be able to fully eliminate it, even if all available notch filters are in use.

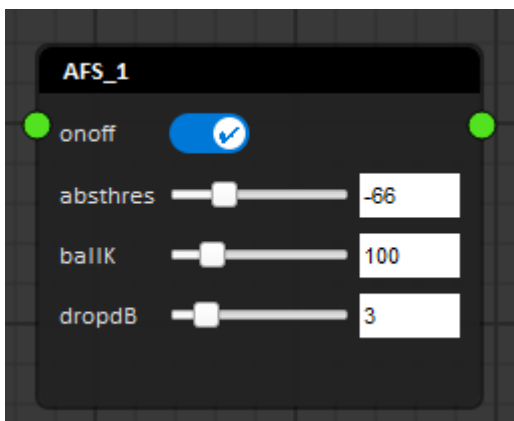
Input & Output Configuration of AFS object

- **Audio inputs: 1.** (The input consists of the speaker's intended input signal combined with the microphone signal)
- **Audio Outputs: 1** (The output is the final signal sent to the speaker.))

Adjustable Parameters of AFS object

- **onoff:** Enables or bypasses the AO process. (Default: "Enable")
- **absthres (dB):** Defines the activation energy threshold, with an origin of -66 dB. When the detected noise energy levels exceed this threshold, suppression is activated.
- **ballK:** Determines the duration of historical data included in the analysis, with a default value of 100 ms. A longer duration improves accuracy but reduces sensitivity, while a shorter duration increases sensitivity at the cost of accuracy.
- **dropdB (dB):** Determines the filter release condition. If the detected noise energy level falls below this threshold, the filter becomes inactive. The default value is 3 dB.

Flow Studio GUI



6.9.5.2 AI DeepTalk

AI DeepTalk is a collection of advanced audio objects (AOs) that leverage artificial intelligence and digital signal processing (DSP) techniques to enhance voice communication in real time. These tools are designed to improve speech clarity, suppress background noise, and optimize microphone array performance, making them ideal for applications such as conferencing, telephony, and voice-controlled systems.

Available AOs in AI DeepTalk

- **AI Noise Reduction (16k / 48k)** – AI-driven noise suppression optimized for different sampling rates, effectively reducing environmental noise while preserving speech quality.
- **Voice Enhancement Package** – A suite of algorithms that enhance voice intelligibility, improve clarity, and reduce distortions.
- **DSP Noise Reduction** – Traditional DSP-based noise suppression that efficiently removes background noise for clearer communication.
- **2-4 Mic Beamforming** – Multi-microphone processing that enhances directional voice capture while minimizing interference.
- **Coherence Beamforming** – Improves signal quality by leveraging coherence between multiple microphones, ensuring robust speech enhancement.
- **AI Beamforming** – AI-powered directional processing that dynamically enhances the target speaker's voice while reducing unwanted noise.
- **AEC (Acoustic Echo Cancellation)** – Removes echo artifacts caused by speaker-microphone feedback, ensuring clean and natural voice transmission.
- **AI VAD (Voice Activity Detection)** – AI-based detection that accurately differentiates speech from noise, improving system efficiency and responsiveness.
- **VAD (Voice Activity Detection)** – Traditional VAD that identifies speech presence, optimizing processing resources for real-time applications.

With Flow Studio, users can seamlessly integrate these AOs into their voice processing pipelines, achieving high-performance speech enhancement with minimal effort. Whether improving clarity in noisy environments, refining microphone array performance, or enabling intelligent voice interaction, these AI-driven tools empower developers to build cutting-edge voice applications with precision and efficiency.

6.9.5.2.1 AI Noise Reduction (16k)

Functional description of AI Noise Reduction (16k) object

The **AI Noise Reduction (16k)** object (displayed as **AI_NR** in the information tab) is an AI-driven speech noise reduction (NR) algorithm, trained on over 100 million data points to handle 73 different types of noise, covering both stationary and non-stationary noise—delivering far superior performance compared to traditional DSP-based noise reduction techniques. The key advantage of AI noise reduction lies in its ability to handle complex environmental noise, excelling in both outdoor and indoor scenarios:

- **Outdoor environments:** Effectively reduces road noise, traffic sounds, and railway crossing alarms, ensuring speech clarity even in bustling city streets or noisy outdoor settings.
- **Indoor environments:** Suppresses café background chatter, airplane cabin noise, indoor construction sounds, and household appliance noises (such as vacuum cleaners, washing machines, and refrigerator compressors), making speech clear even in highly noisy conditions.

Additionally, AINR is optimized for MCU-based processing, enabling low-power, high-efficiency noise suppression. Unlike traditional DSP noise reduction, AI-based noise suppression adapts dynamically to various noise environments without relying on predefined noise characteristics, providing more natural and intelligent noise reduction. This makes AINR particularly suitable for embedded systems, portable voice applications, smart voice assistants, and communication devices, ensuring speech clarity even in the most challenging noise environments.

Specifically optimized for voice communication, including phone calls, conference calls, and smart voice assistants. AINR 16k object/algorithm focuses on speech-band noise suppression, ensuring high intelligibility while effectively reducing background noise such as traffic sounds, office chatter, and crowd noise, providing a clearer conversation experience across various environments.

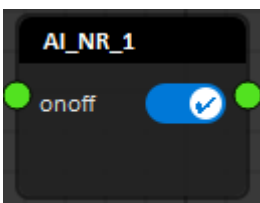
Input & Output Configuration of AI Noise Reduction (16k) object

- **Audio inputs:** 1
- **Audio Outputs:** 1

Adjustable Parameters of AI Noise Reduction (16k) object

- **Onoff:** Enables or bypasses the NR process. (Default: “Enable”)

Flow Studio GUI of AI Noise Reduction (16k) object



6.9.5.2.2 AI Noise Reduction (48k)

Functional description of AI Noise Reduction (48k) object

The **AI Noise Reduction (48k)** object (displayed as **AI_NR_48K** in the information tab) is an AI-driven speech noise reduction (NR) algorithm for 48k sample rate. Just like how AINR 16k model is trained, AINR 48k model is also trained on over 100 million data points to handle 73 different types of noise, covering both stationary and non-stationary noise—delivering far superior performance compared to traditional DSP-based noise reduction techniques.

Designed for live streaming and high-quality audio applications, including online broadcasting, video recording, podcasts, and content creation, AINR 48k processes the full frequency range, preserving more vocal details and naturalness. Whether in a studio setting, a recording environment, or a noisy outdoor scene, it ensures professional-grade speech clarity, delivering a more natural and immersive audio experience.

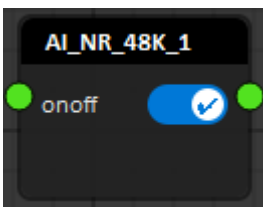
Input & Output Configuration of AI Noise Reduction (48k) object

- **Audio inputs:** 1
- **Audio Outputs:** 1

Adjustable Parameters of AI Noise Reduction (48k) object

- **Onoff:** Enables or bypasses the NR process. (Default: “Enable”)

Flow Studio GUI of AI Noise Reduction (48k) object



6.9.5.2.3 Voice Enhancement Package (VEP)

Functional description of Voice Enhancement Package (VEP) object

The **Voice Enhancement Package (VEP)** object (displayed as **VEP** in the information tab) is a comprehensive speech enhancement algorithm chain designed specifically for voice communication scenarios. It integrates Traditional Beamforming, Acoustic Echo Cancellation (AEC), and AI Noise Reduction (AI NR) to deliver high-quality, low-latency speech enhancement at 16kHz, which is ideal for applications such as video conferencing systems, smart voice communication, and far-field speech recognition, ensuring high-performance voice clarity in real-world environments.

Here are some key technologies & advantages:

- Integrated Speech Enhancement Algorithms:
 - Beamforming: Enhances target speech and suppresses background noise, supporting 2-4 microphone arrays. Users can input microphone coordinates and choose between fixed direction pickup or automatic direction tracking.
 - AEC (Acoustic Echo Cancellation): Effectively removes speaker echo, ensuring crystal-clear, full-duplex communication.
 - AI NR (AI Noise Reduction): AI-powered noise suppression trained to eliminate various background noises such as keyboard typing, office chatter, and household appliances, improving overall voice clarity.
- Optimized for Teams Certification with Low-Latency Processing:
 - The algorithm is specifically optimized for Teams certification, addressing signal delay issues by minimizing frequency-domain switching between algorithms, significantly reducing latency for real-time communication.
 - This optimization also reduces computational load, making it more efficient compared to traditional independently running speech enhancement modules.
- Flexible Microphone Configuration & Directional Control:
 - Users can input microphone coordinates to match different array structures.
 - Supports fixed direction pickup (ideal for conference rooms and desktop communication) or automatic direction tracking (suitable for dynamic speaker movement).

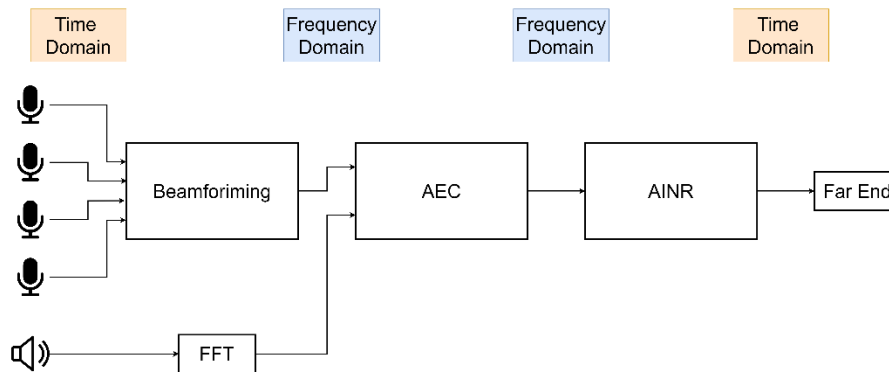
The Voice Enhancement Package (VEP) is ideal for applications such as video conferencing systems, smart voice communication, and far-field speech recognition, ensuring low-latency, high-performance voice clarity in real-world environments.

Input & Output Configuration of Voice Enhancement Package (VEP) object

- **Audio inputs:** 4
- **Audio Outputs:** 1

Adjustable Parameters of Voice Enhancement Package (VEP) object

- **onoff:** Enables or bypasses the AO process. (Default: “Enable”)
- **DOA:** Detects the direction of incoming sound sources and enables dynamic beam steering. If enabled, the MicDeg parameter will be ignored. (Default: “Unable”)
- **MicDeg:** User-specified angle.
- **Mic Direction Input:** The definition of the XY axes and angles follows the Polar Coordinates system. Currently, only 2D planar microphone array arrangements are supported, and the unit of the coordinate axes is millimeters (mm).



Flow Studio GUI of Voice Enhancement Package (VEP) object



6.9.5.2.4 DSP Noise Reduction

Functional description of DSP Noise Reduction object

The **DSP Noise Reduction** object (displayed as **NOISEREDUCTION** in the information tab) is designed specifically for stationary noise suppression, effectively reducing background noise and enhancing speech clarity. It is ideal for environments such as offices, meeting rooms, libraries, and open workspaces, where common types of stationary noise include air conditioning hum, computer fan noise, and background hums.

Additionally, this algorithm supports adjustable noise reduction levels, allowing users to customize the noise suppression intensity based on their needs. For instance, a strong noise reduction mode can be selected to minimize background noise in quiet meeting environments or voice recognition applications, whereas a weaker noise reduction mode can be chosen in open office spaces or scenarios where some ambient sound should be retained, ensuring a more natural auditory experience.

Leveraging advanced signal processing techniques, this algorithm achieves over 6dB of noise reduction while preserving speech clarity. Its flexibility and adaptability make it suitable for a variety of applications, including voice calls, video conferencing, and smart voice assistants.

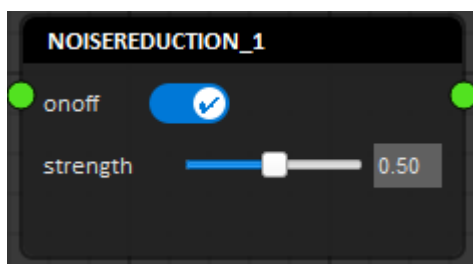
Input & Output Configuration of DSP Noise Reduction object

- **Audio inputs:** 1
- **Audio Outputs:** 1

Adjustable Parameters of DSP Noise Reduction object

- **Onoff:** Enables or bypasses the DSP NR process. (Default: "Enable")
- **Strength:** Strength parameter (range 0-1) to control the noise reduction level, where higher values apply stronger suppression. Users can adjust it as needed to balance speech clarity and noise suppression.

Flow Studio GUI of DSP Noise Reduction object



6.9.5.2.5 2-4 Mic Beamforming

Functional description of 2-4 Mic Beamforming object

The **2-4 Mic Beamforming** object (displayed as **BEAMFORMING_2ch/BEAMFORMING_3ch/BEAMFORMING** in the information tab) utilizes MVDR (Minimum Variance Distortionless Response) as its core computation, enabling directional audio capture based on a user-specified angle. It effectively enhances sound from the target direction while suppressing noise and interference from other directions. Additionally, the algorithm includes a built-in DOA (Direction of Arrival) detection function, which dynamically analyzes the sound source direction and automatically adjusts the beamforming. In this mode (When DOA is enabled), the algorithm is no longer restricted by the MicDeg.

This algorithm is suitable for voice calls and voice command recognition in smart devices and can be widely applied in mobile devices, smart speakers, conferencing systems, and more. By leveraging MVDR beamforming technology, the system can adapt to environmental changes in real time, ensuring clear voice capture while reducing background noise and echo, thereby improving voice recognition reliability. Additionally, beamforming requires integration with VAD (Voice Activity Detection) to ensure effective speech detection and improve audio capture accuracy.

Currently, the algorithm supports circular arrays and linear arrays, optimizing beamforming calculations based on the array geometry to enhance audio capture performance. Other array configurations (such as non-uniform or rectangular arrays) require practical testing to confirm performance and applicability. Additionally, the recommended angle spacing for different channel configurations is as follows:

- 2-channel (2ch): Recommended spacing of 90 degrees, DOA is not supported
- 3-channel (3ch): Recommended spacing of 45 degrees
- 4-channel (4ch): Recommended spacing of 30 degrees

Input & Output Configuration of 2-4 Mic Beamforming object

- **Audio inputs:** 2~4
- **VAD input:** only need 1-ch VAD input
- **Audio Outputs:** 1

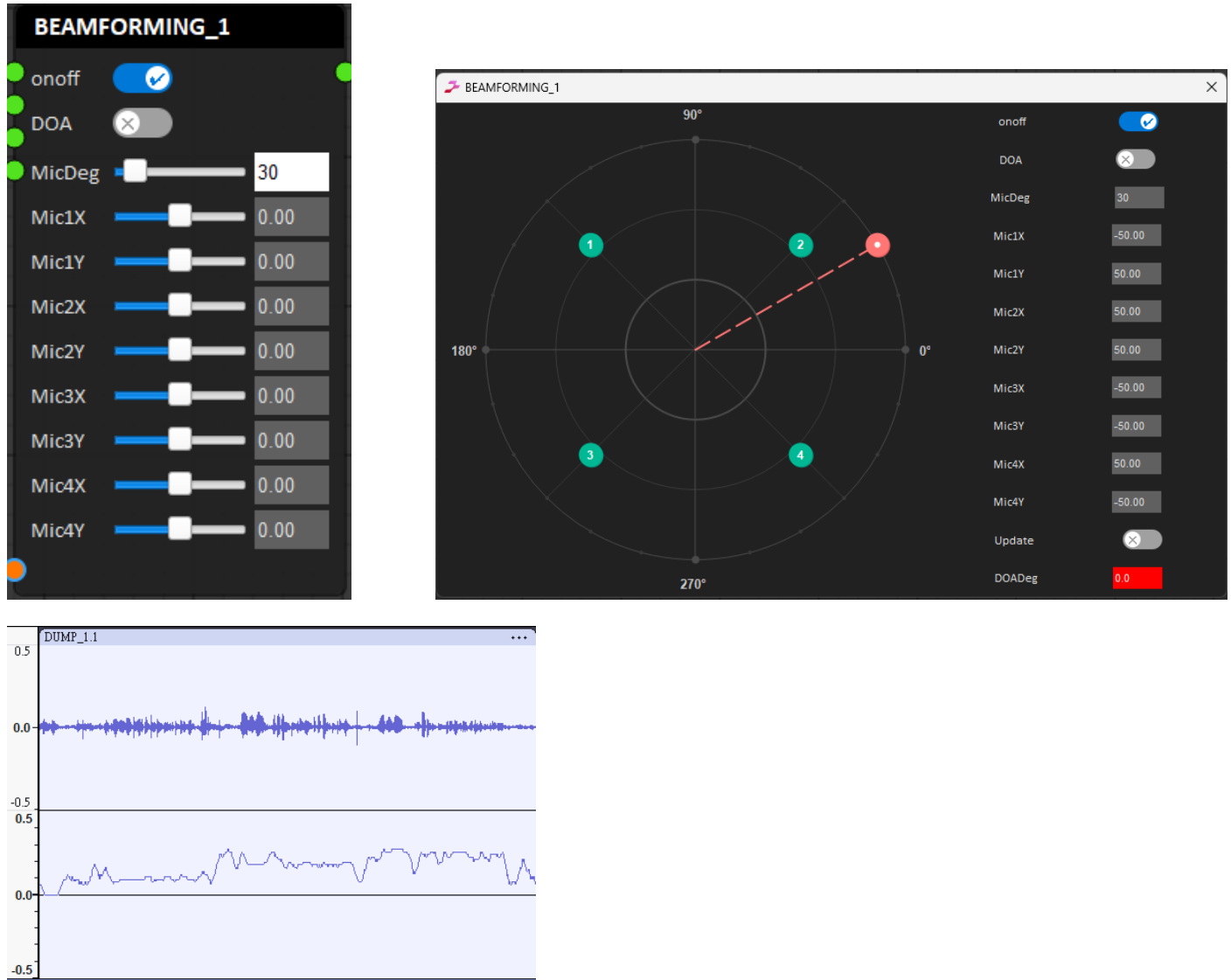
Adjustable Parameters of 2-4 Mic Beamforming object

- **onoff:** Enables or bypasses the beamforming process. (Default: "Enable")
- **DOA:** Detects the direction of incoming sound sources and enables dynamic beam steering. If DOA is enabled, the MicDeg parameter will be ignored. (Default: "Unable")
- **MicDeg:** User-specified angle for the signal of interest.
- **Mic Direction Input:** The XY axes and angles follows the Polar Coordinates system. Currently, only 2D planar microphone array arrangements are supported, and the unit of the coordinate axes is millimeters (mm).

DOA Angle Output on Second Audio Channel

- The DOA angle is encoded as a floating-point value mapped as follows:
0.0 → 0 degrees
0.360 → 360 degrees

Flow Studio GUI of 2-4 Mic Beamforming object



6.9.5.2.6 Coherence Beamforming

Functional description of Coherence Beamforming object

The **Coherence Beamforming** object (displayed as **COHBF** in the information tab) specifically designed for 2-Mic arrays. Unlike traditional beamforming methods, it does not rely on a fixed microphone spacing. Instead, it dynamically enhances target speech and suppresses background noise by analyzing the coherence properties of the input signals. By leveraging Coherence Detection, this technique effectively distinguishes speech from environmental noise without requiring a predefined microphone distance, making it adaptable to various hardware configurations. It is particularly suitable for voice calls, smart voice assistants, and conferencing systems. Even if the microphones are positioned irregularly, it can still provide stable noise suppression and speech enhancement.

It is recommended to set the microphone spacing between 10mm and 50mm, ensuring that speech originates from the area between the two microphones for optimal capture performance.

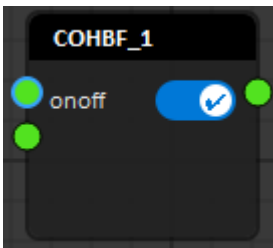
Input & Output Configuration of Coherence Beamforming object

- **Audio inputs: 2**
- **Audio Outputs: 1**

Adjustable Parameters of Coherence Beamforming object

- **Onoff:** Enables or bypasses the coherence beamforming process. (Default: “Enable”)

Flow Studio GUI of Coherence Beamforming object



6.9.5.2.7 AI BEAMFORMING (16k)

Functional description of AIBF object

The **AI BEAMFORMING (or AIBF)** object (displayed as **AI_BF** in the information tab) focuses the energy of a signal towards a specific direction, thereby enhancing signal strength and reducing the effects of multipath interference and background noise. AIBF outperforms traditional BF (2-4 Mic BEAMFORMING object), especially when SNR is low. During operation, AIBF object would first select and process a reference mic signal to obtain a spectral mask, which is further used to calculate the covariance matrix of speech and noise. These covariance matrices will then be used in later MVDR process to get single-channel clean speech signal. One of the key advantages of our AI-based approach is its lightweight computational design, making it highly efficient and suitable for deployment on resource-constrained edge devices, including MCU chips. By optimizing the algorithm for low-power processing, we enable real-time noise reduction with minimal latency, making AIBF an ideal solution for embedded speech applications such as wearable devices, IoT systems, and smart assistants. Notably, current AIBF is available only when sample rate is 16k Hz, due to the spectral mask generator being trained using 16k dataset.

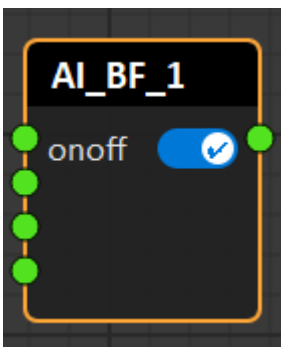
Input & Output Configuration of AIBF object

- **Audio inputs:** 4
- **Audio Outputs:** 1

Adjustable Parameters of AO

- **onoff:** Enable or bypass AI Beamforming object

Flow Studio GUI of AIBF object



6.9.5.2.8 AEC

Functional description of AEC object

The **AEC** object (also displayed as **AEC** in the information tab) adopts the **Speex AEC (Acoustic Echo Cancellation)** framework, a technology designed to suppress acoustic echoes caused by speaker output being picked up by the microphone, ensuring clear voice communication. By comparing far-end audio (speaker output) with near-end audio (microphone input), the system can automatically detect and remove echoes, allowing the other party to hear only the user's voice without interference from environmental echoes.

This technology is suitable for VoIP voice calls, video conferencing, smart voice devices, remote conferencing systems, and in-game voice chat. It is especially effective in scenarios where an open microphone and speaker are used simultaneously, significantly reducing echo interference and improving communication quality. With Speex AEC, smart speakers and voice assistants can more accurately recognize user commands, while multi-party communication systems can deliver a more natural conversational experience.

Using AEC requires both microphone input (including echoes) and far-end audio to enable the algorithm to compare and eliminate echoes. This technology is ideal for low-latency communication and performs best in stable audio paths. Additionally, adjusting the filter length allows the system to handle longer delay conditions, making it adaptable to various system environments.

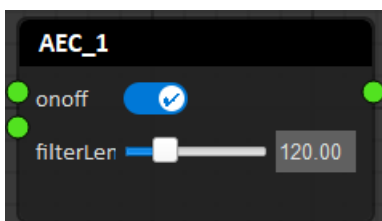
Input & Output Configuration of AEC object

- **Audio inputs:** 2, one is for microphone input, and the other is for far-end audio
- **Audio Outputs:** 1

Adjustable Parameters of AEC object

- **onoff:** Enables or bypasses the AEC process. (Default: "Enable")
- **filterLen:** Adjusts the filter length. A larger value allows adaptation to longer physical delays but also increases computational load.

Flow Studio GUI of AEC object



6.9.5.2.9 AI VAD

Functional description of AI VAD object

The **AI VAD (Voice Activity Detection)** object, displayed as **VAD** in the information tab, is an AI-driven technology optimized for indoor environments, offering highly accurate speech detection even in the presence of various background noises. By leveraging deep learning, AI VAD significantly improves noise resistance and speech detection accuracy, ensuring stable performance in challenging indoor settings. Compared to traditional VAD, which relies on fixed energy thresholds, AI VAD dynamically analyzes speech patterns, making it more adaptable to complex acoustic environments. AI VAD object includes two adjustable parameters: Threshold and Ratio, allowing users to fine-tune sensitivity and smoothing effects to meet different application needs. One of its key strengths is its ability to differentiate speech from common household noises, such as cat meows, dog barks, dishwashing sounds, running water, and vacuum cleaners, effectively preventing false activations and reducing missed speech events.

Designed with efficiency, AI VAD is optimized for low-power devices, making it particularly well-suited for MCU-based platforms used in smart voice assistants, voice-triggering applications, and noise reduction preprocessing. Its low-power consumption ensures reliable operation in embedded systems without compromising performance. With AI-powered processing, AI VAD delivers a more accurate and reliable speech recognition experience by minimizing false activations and speech loss, making it an essential tool for smart voice applications in modern indoor environments.

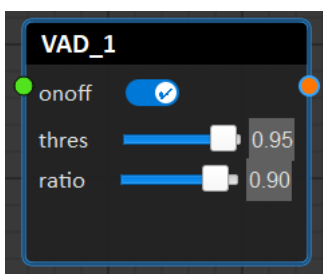
Input & Output Configuration of AI VAD object

- **Audio inputs: 1**
- **Audio Outputs: 0**

Adjustable Parameters of AI VAD object

- **onoff:** Enables or bypasses the AI VAD process. (Default: “Enable”)
- **thres:** Threshold (0-1) – Adjusts VAD detection sensitivity. When Threshold is set to 0, VAD is disabled, and no speech detection is performed. When Threshold is set to 1, VAD detects speech only when the AI model determines that the speech content is 100%.
- **ratio:** Smooths VAD detection output. Ratio is a first-order IIR filter, acting as a post-filter to smooth the VAD output.

Flow Studio GUI



6.9.5.2.10 VAD

Functional description of VAD object

The **VAD (Voice Activity Detection)** object, displayed as **VAD_IABSE** in the information tab, is used to distinguish human speech segments from digital signals. The VAD utilizes the IABSE (Improved Adaptive Band-Partitioning Spectral Entropy) algorithm to determine whether the input signal contains speech, enhancing detection accuracy through an improved entropy-based approach. This technology does not rely on AI but instead employs traditional methods to reduce memory usage. The VAD implementation includes an Improved Entropy-Based Endpoint Detection Algorithm, Adaptive Band-Partitioning Spectral Entropy, and the Zero-Crossing Rate method, effectively differentiating speech from background noise while ensuring low-latency and efficient voice detection. VAD is widely used in voice communication, smart voice assistants, noise reduction systems, and conferencing applications, improving speech recognition accuracy and optimizing audio processing performance. Due to its low memory requirements, it is particularly suitable for embedded devices and resource-constrained environments.

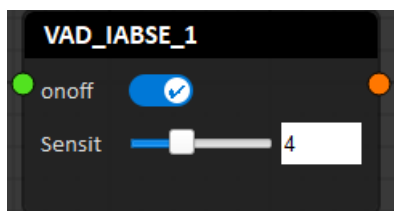
Input & Output Configuration of VAD object

- **Audio inputs:** 1
- **Audio Outputs:** 1

Adjustable Parameters of VAD object

- **onoff:** Enables or bypasses the VAD process. (Default: “Enable”)
- **Sensit:** Sensitivity of voice detection. The greater value results in more sensitive detection.

Flow Studio GUI of VAD object



6.9.5.3 De-esser

Functional description of De-esser object

The **De-esser** object (displayed as **DEESSER** in the information tab) is a specialized audio processing tool designed to reduce or attenuate sibilance—the harsh, piercing high-frequency sounds commonly found in dialogue or vocal recordings. These high-frequency sounds, typically produced by consonants such as "S," "F," "X," "SH," and soft "C," can be unpleasant and distracting in certain audio contexts. The De-esser works by selectively compressing these frequencies, offering a smoother and more balanced sound.

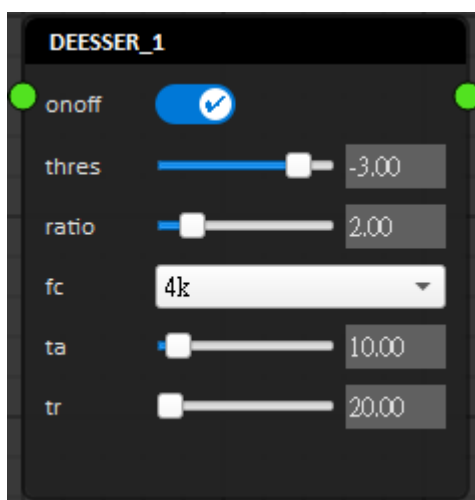
Input & Output Configuration of De-esser object

- **Audio inputs:** 1~32
- **Audio Outputs:** 1~32

Adjustable Parameters of De-esser object

- **onoff:** Enables or bypasses the de-essing process. (Default: Enable)
- **thres:** The threshold (dB) at which sibilance is detected and reduced. Range: -20 to 0 dB. (Default:-3 dB)
- **ratio:** Sets the compression ratio for the sibilance. Higher values result in more aggressive attenuation de-essing process. Range: 1 to 8. (Default:2)
- **fc:** The cutoff frequency (in Hertz) for the sibilance filter of the de-essing process, targeting the specific frequency range (4k, 5k, 6k) where sibilance is most pronounced. (Default:4 kHz)
- **ta:** Attack time (ms) for the de-essing process, controlling how quickly the compression is applied once sibilance is detected. Range: 0.1 to 200 ms. (Default: 10 ms)
- **tr:** Release time (ms) for the de-essing process, controlling how gradually the effect fades after the sibilance has been reduced. Range: 1 to 5000 ms. (Default: 20 ms)

Flow Studio GUI of De-esser object



6.9.5.4 Smart Equalizer

Functional description of Smart Equalizer object

The **Smart Equalizer** object (displayed as **SMART_EQ** in the information tab) is an advanced adaptive equalization system designed to dynamically correct and enhance audio playback based on environmental acoustics. By leveraging a built-in microphone, it automatically analyzes the surrounding space and adjusts the bass boost levels accordingly—without requiring user calibration. Also known as Smart EQ, it is particularly effective in compensating for wall reflections and corner loading, ensuring a more natural and balanced audio experience. However, while it can successfully correct overall tonal imbalances, note that it does not offer any precise treatment for higher-frequency discrepancies. Smart EQ is best suited for portable, carry-around speakers that wishes to maintain consistent playback experience when placed in different locations.

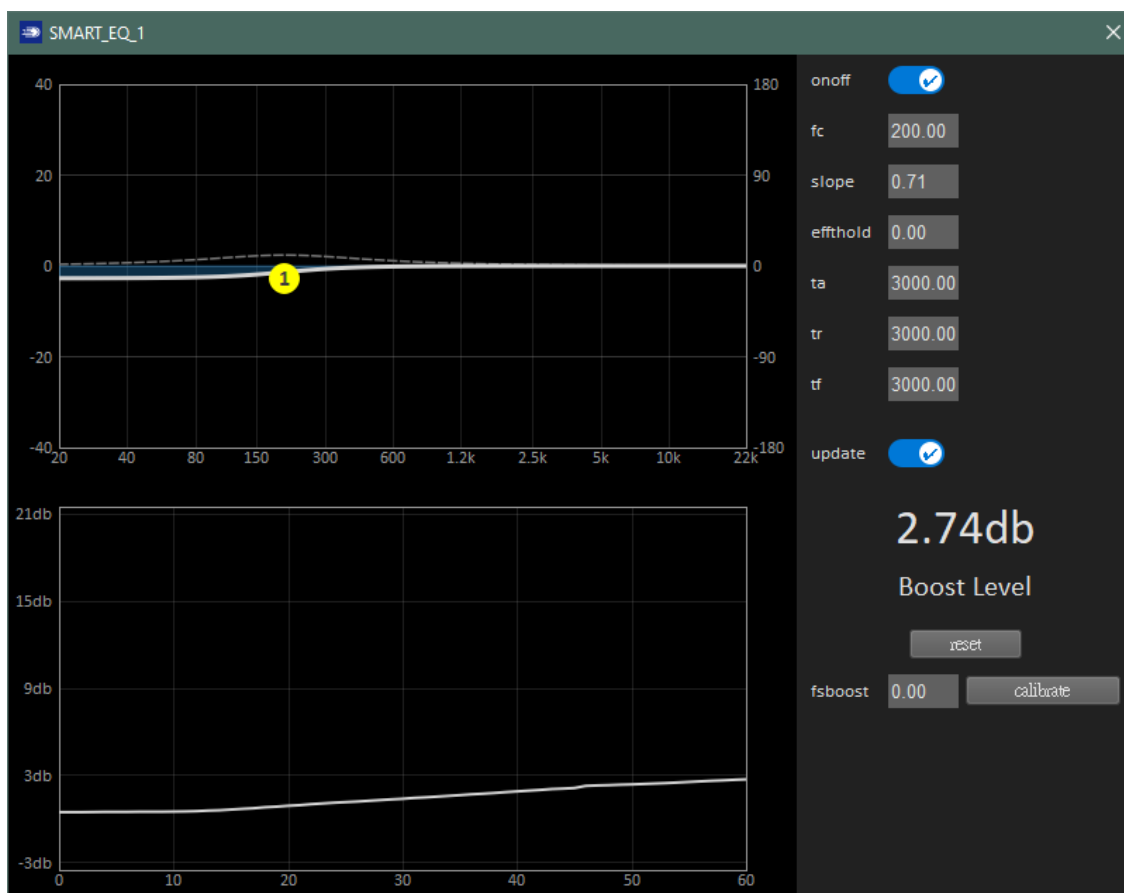
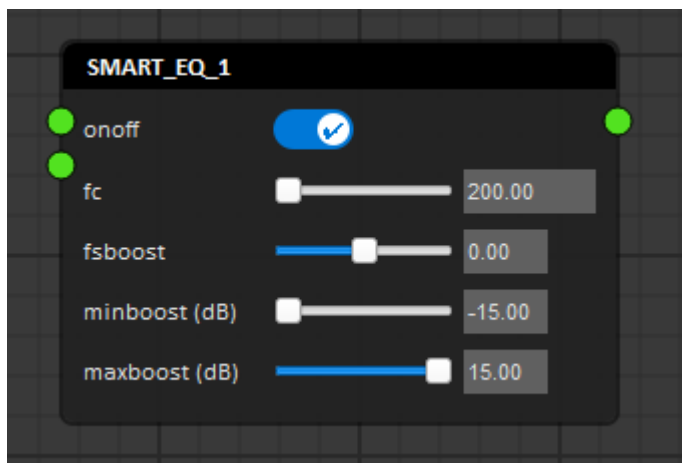
Input & Output Configuration of Smart Equalizer object

- **Audio inputs:** 2 (One input receives the intended mono playback signal, while the other captures environmental acoustics via a built-in microphone to calculate necessary bass boost adjustments.)
- **Audio Outputs:** 1 (optimized audio output as speaker signal)

Adjustable Parameters of Smart Equalizer object

- **onoff:** Turns the shelving filter in the Smart EQ on or off. When off, the compensation effect is disabled, and the output signal remains the same as the input playback signal. However, the system continues to update the current boost level in the background. (Default: Enable)
- **fc:** Defines the crossover frequency for the Low-Pass and High-Pass filters as well as the shelving filter's target frequency (Default: 200.00)
- **slop:** Determines the slope of the shelving filter, affecting how aggressively the compensation is applied. (Default: 0.71)
- **effthold:** Defines the effective threshold for frequency level calculations. Low or high-frequency content in the playback signal must exceed this threshold to be considered effective; otherwise, the system disregards the data. (Default: 0.00)
- **ta (Attack Time):** Controls the response time of the RMS level calculator for detecting signal changes. Default is 2266.8 seconds (AT = 0.99999), which ensures gradual adaptation rather than abrupt changes.
- **tr (Release Time):** Determines how long it takes for the RMS level calculator to return to a lower level after a peak in signal strength. Default is 2266.8 seconds (RT = 0.99999).
- **tf (Final Smoothing Stage):** Prevents truncation or loss of precision in the AT/RT calculations by adding an additional smoothing stage. Default is 22,668 seconds (FT = 0.999999), ensuring ultra-smooth adjustments.
- **minboost (dB):** The minimum boost level. (Default: -15 dB)
- **maxboost (dB):** The maximum boost level. (Default: 15 dB)
- **fsboost:** Represents the full-space boost level. Even in an ideal acoustic chamber, some minor bass boost may still exist. In such cases, this parameter allows users to calibrate and account for this effect. If optimal tuning has already been achieved, Smart EQ can be turned off, and pink noise can be used to measure the current boost level, which can then be set as the fsboost using the Calibrate function. (Default: 0.00)

Flow Studio GUI of Smart Equalizer object



6.9.5.5 CLIPFIX

Functional description of CLIPFIX object

The **CLIPFIX** object (also displayed as **CLIPFIX** in the information tab) prevents input audio clipping by dynamically adjusting gain levels to keep the signal within a safe range. It monitors the input levels and applies real-time gain reduction when the signal approaches the clipping threshold. This helps maintain audio clarity and prevents distortion caused by excessive signal levels. The CLIPFIX object requires a corresponding hardware design to function effectively. The concept is straightforward: the system utilizes an analog circuit that takes two analog audio signal inputs, each with different gain levels - one with a higher gain and the other with a lower gain. The CLIPFIX object dynamically switches between these two signal paths based on real-time analysis. When the high-gain signal path detects that the signal is approaching clipping, the algorithm seamlessly switches to the low-gain path. To ensure a smooth transition and prevent discontinuities in audio energy during switching, the algorithm applies signal energy compensation. This process effectively mitigates clipping and maintains consistent audio quality.

Note: It is recommended to execute the CLIPFIX audio object with a resolution of at least 24 bits to achieve optimal performance.

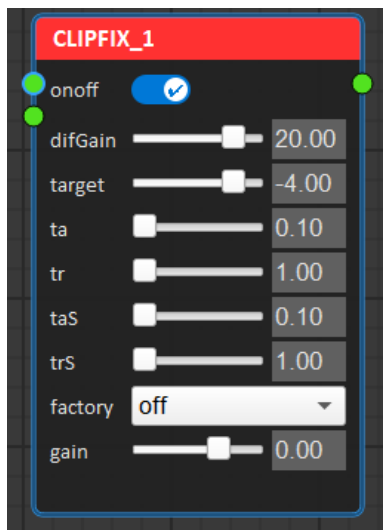
Input & Output Configuration of CLIPFIX object

- **Audio inputs:** 2
- **Audio Outputs:** 1

Adjustable Parameters of CLIPFIX object

- **Onoff:** Enables or bypasses the AO process. (Default: Enable)
- **difGain:** The gain difference, specified in dB, between the two signal paths in the hardware circuit. (Default: 20 dB)
- **target:** The saturation detection level, specified in dB, when the signal approaches this level, the algorithm initiates signal switching. (Default: -4 dB)
- **ta:** Attack time, specified in milliseconds, the CLIPFIX audio object takes to react to signals exceeding the threshold and begin signal switching. (Default 0.1 ms)
- **tr:** Release time, specified in milliseconds, the CLIPFIX audio takes to revert to the high-gain path when the signal is no longer approaching the target level. (Default: 1 ms)
- **taS:** Attack time for low gain path, specified in milliseconds, the response time for energy compensation in the low-gain path when switching occurs. (Default 0.1 ms)
- **trS:** Release time for low gain path, specified in milliseconds, the response time for restoring energy in the low-gain path after switching back to the high-gain path. (Default: 1 ms)
- **factory:** A factory test mode that bypasses all algorithm processing for signal measurement purposes. (Default: off)
- **gain:** A post-processing digital gain adjustment applied after signal switching. (Default: 0 dB)

Flow Studio GUI of CLIPFIX object



6.9.5.6 AGC

Functional description of AGC object

Being an abbreviation for **Automatic Gain Control**, the **AGC** object (also displayed as **AGC** in the information tab) is an adaptive audio processing algorithm designed to maintain a consistent output signal level by dynamically adjusting the gain of an incoming signal. It continuously analyzes the signal envelope and applies the appropriate gain adjustments to ensure the output remains close to a defined target level, regardless of fluctuations in input volume. The AGC object operates by calculating both a slow and fast envelope of the input signal. These envelopes are determined based on the maximum peak sample value within every 1 ms window of input data. The higher of the two envelope values is used for gain computation. By referencing a gain table, AGC updates the gain in real time to maintain an optimal signal level without excessive distortion or volume variation.

AGC is widely used in various audio applications to maintain consistent sound levels. In **automatic microphone gain adjustment**, AGC ensures that speech remains clear and balanced, whether in live settings or recorded content. This is particularly useful for public speaking, voiceovers, and conferencing systems. In **broadcast and streaming**, AGC prevents sudden fluctuations in volume, keeping the audio experience smooth and professional. Whether it's a podcast, a live news broadcast, or an online meeting, AGC helps maintain a steady output level without requiring manual adjustments.

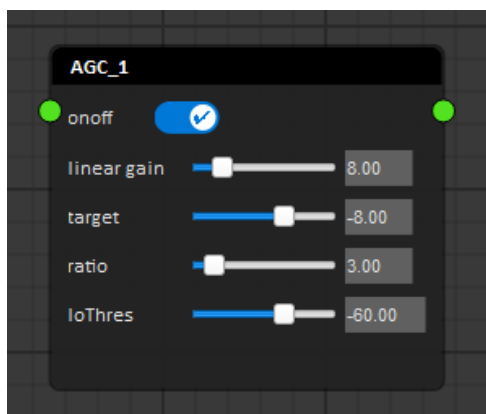
Input & Output Configuration of AGC object

- **Audio Inputs:** 1~32
- **Audio Outputs:** 1~32

Adjustable Parameters of AGC object

- **onoff:** Enables or bypasses AGC processing. (Default: on)
- **linear gain:** Sets the AGC gain in the linear range. Range: 0~50 dB. (Default: 8 dB)
- **target:** Defines the AGC output target for a 0 dBFS input. Range: -24~0 dB. (Default: -8 dB)
- **ratio:** Specifies the compression ratio above the upper threshold. Range: 1~24. (Default: 3)
- **loThres (dB):** Sets the minimum threshold for AGC activation. Range: -120~-30 dB. (Default: -60 dB)

Flow Studio GUI of AGC object



6.9.5.7 Sample Rate Converter (SRC)

Functional description of SRC object

The **Sample Rate Converter (SRC)** object enables seamless sample rate conversion between different audio objects (AOs) in Flow DSP. This allows for flexible signal flow design, particularly useful when working with AOs that operate at fixed sample rates. The SRC object eliminates the need for external software-based converters and allows for real-time sample rate adaptation within the audio engine.

The SRC supports converting any standard audio sample rate between **8kHz and 192kHz** to another target rate within the same range. This facilitates multi-rate signal processing and enables advanced audio use cases such as voice signal optimization and resampling.

The SRC AO reads the input sample rate from the preceding AO and provides the defined output sample rate to the next AO in the signal chain. Users can also configure the quality of conversion to optimize for CPU load, memory usage, or audio fidelity.

Input & Output Configuration of SRC object

- **Audio Inputs: 1**
- **Audio Outputs: 1**

Adjustable Parameters of SRC object

- **Input_Freq:** Sample rate of the incoming audio signal.
- **Output_Freq:** Target sample rate for the output audio.
- **Quality:** Defines the tradeoff between **audio conversion quality** and **CPU performance**. Three levels are available:
 - **Fastest:** Lowest CPU load, suitable for real-time or resource-constrained environments.
 - **Medium:** Balanced setting providing a compromise between performance and audio quality.
 - **Best:** Highest audio fidelity with the most accurate conversion, but requires more CPU processing power.

Flow Studio GUI of AGC object

